Symbolic AI - Definitions and Illustrations

Ibrahim Nasser*

Last updated on April 6, 2026

Contents

1 Introduction 1

2 Mathematical Language 3

3 Unary Natural Numbers 7

4 Sets 9

5 Relations 11

6 Functions 14

7 Equivalence Classes 20

8 Functional Programming using SML 22

9 Graphs and Trees 40

10 Mathematical Structures 45

11 Complexity Analysis in AI 47

12 Formal Languages and Grammars 51

13 Term Languages and Abstract Grammars 55

14 General Problem Solving 58

15 Constraint Satisfaction Problems 64

16 Formal Systems 71

17 Propositional Logic 73

18 Machine-Oriented Calculi for Propositional Logic 81

19 First-Order Predicate Logic 92

20 Automated Theorem Proving in First-Order Logic 104

21 Logic-Based Knowledge Representation 116

22 STRIPS Planning 126

*This document and its source files are licensed under Creative Commons Attribution–NonCommercial–NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).

Disclaimer

This document is an independent summary by Ibrahim Nasser as a personal learning and documentation effort for the courses Symbolic Methods for AI and AI 1: Artificial Intelligence 1 taught by Prof. Dr. Michael Kohlhase. It is not an official course document, and no guarantee is made for its correctness, completeness, or accuracy. The material here is meant as an auxiliary resource for understanding the subjects and should not be used as the sole basis for exams preparation. The official lecture notes and course materials remain the authoritative reference.

This summary is a living document, i.e. I may continue to revise, extend, and improve it over time as I refine my understanding or receive feedback.

If you find any mistakes, unclear explanations, or have suggestions for improvement, please contact me.

1 Introduction

Def 1.1. Artificial intelligence (AI): The capability of computational systems to perform tasks typically associated with human intelligence, such as learning, reasoning, problem-solving, perception, and decision-making

Def 1.2. Symbolic AI: A subfield of Artificial Intelligence (AI) based on the assumption that many aspects of intelligence can be achieved by the manipulation of symbols, combining them into meaningful structures (expressions) and manipulating them (using processes) to produce new expressions.

Def 1.3. Statisitical AI: Remedies the two shortcomings of symbolic AI approaches: that all concepts represented by symbols are crisply defined, and that all aspects of the world are knowable/representable in principle. Statistical AI adopts sophisticated mathematical models of uncertainty and uses them to create more accurate world models and reason about them.

Def 1.4. Subsymbolic AI (a.k.a connectionism/neural AI): A subfield of AI that posits that intelligence is inherently tied to brains, where information is represented by a simple sequence pulses that are processed in parallel via simple calculations realized by neurons, and thus concentrates on neural computing.

Def 1.5. Embodied AI: Posits that intelligence cannot be achieved by reasoning about the state of the world (symbolically, statistically, or sub-symbolically), but must be embodied i.e. situated in the world, equipped with a "body" that can interact with it via sensors and actuators. Here, the main method for realizing intelligent behavior is by learning from the world.

Def 1.6. Reasoning: The process of producing valid arguments and predictive world models. There are three forms of reasoning:

- **deductive reasoning** to produce new knowledge from existing knowledge. *Example:* All humans are mortal. Socrates is a human. Therefore, Socrates is mortal.
- **inductive reasoning** to produce knowledge from perception. *Example:* The sun has risen every morning in recorded history. Therefore, the sun will rise tomorrow.
- **abductive reasoning** to produce explanations for observations and given knowledge. *Example:* The grass is wet this morning. If it rained last night, that would explain it. Therefore, it probably rained.

Def 1.7. Inference: The act or process of reaching a **conclusion** about something from known facts or evidence (jointly called **premises**)

Def 1.8. Formal Logic: The science of deductively valid inferences, meaning arguments whose conclusions necessarily follow from their premises by virtue of their form (their structure), regardless of the topic.

Def 1.9. Agent: An agent is a structure $A := \langle \mathcal{P}, \mathcal{A}, f \rangle$ where:

- $\mathcal{P}$ is a set of percepts
- $\mathcal{A}$ is a set of actions
- f is a function $f : \mathcal{P}^* \to \mathcal{A}$ that maps from percepts to actions.

In other words, an agent is anything that perceives its environment via sensors and acts on it with actuators.

Def 1.10. Performance Measure: A function that evaluates a sequence of environments.

Def 1.11. Rational: An agent is called **rational**, if it chooses whichever action maximizes the expected value of the performance measure given the percept sequence to date.

Def 1.12. PEAS: To design rational agents, we must specify the PEAS components: Performance Measure, Environment, Actuators, and Sensors.

Def 1.13. Environment Types: For an agent a we classify the environment e of a by its **type**, which is one of the following. We call e

1. *fully observable*, iff the a's sensors give it access to the complete state of the environment at any point in time; otherwise, we call it *partially observable*.
2. *deterministic*, iff the next state of the environment is completely determined by the current state and a's actions; otherwise, *stochastic*.
3. *episodic*, iff a's experience is divided into atomic episodes, where it perceives and then performs a single action, and the next episode does not depend on previous ones. Otherwise, *sequential*.
4. *dynamic*, iff the environment can change without an action performed by a; otherwise, *static*. If the environment does not change but a's performance measure does, we call e *semidynamic*.

5. *discrete*, iff the sets of e's state and a's actions are couuntable; otherwise, *continuous*.
6. *single-agent*, iff only a acts on e; otherwise *multi-agent*.

Def 1.14. Reflex: An agent $\langle \mathcal{P}, \mathcal{A}, f \rangle$ is called a **reflex agent**, iff it only takes the last percept into account when choosing an action, i.e. $f(p_1, \cdots, p_k) = f(p_k)\ \forall p_1 \cdots p_k \in \mathcal{P}$

Def 1.15. Model-Based Agent: A model-based agent $\langle \mathcal{P}, \mathcal{A}, \mathcal{S}, \mathcal{T}, s_0, S, a \rangle$ is an agent $\langle \mathcal{P}, \mathcal{A}, f \rangle$ whose actions depend on:

- a **world model**: a set $\mathcal{S}$ of possible *states*, and a start state $s_0 \in \mathcal{S}$.
- a **transition model** $\mathcal{T}$ that predicts a new state $\mathcal{T}(s, a)$ from a state s and an action a.
- a **sensor model** S that given a state s and a percept p determines a new state $S(s, p)$.
- an **action function** $a : \mathcal{S} \to \mathcal{A}$ that given a state $s \in \mathcal{S}$ selects the next action $a \in \mathcal{A}$.

> **Note**
>
> If the agent is in state s then it took action a and now perceives p, then the agent state will become $s' = S(p, \mathcal{T}(s, a))$ and accordingly take action $a' = a(s')$.

Def 1.16. Goal Based Agent: A goal based agent $\langle \mathcal{P}, \mathcal{A}, \mathcal{S}, \mathcal{T}, s_0, S, a, \mathcal{G} \rangle$ is a model based agent with an explicit set of goals. It consists of:

- a set of internal states $\mathcal{S}$ and an initial state $s_0 \in \mathcal{S}$,
- a transition model $\mathcal{T} : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$,
- a state update function $S : \mathcal{P} \times \mathcal{S} \to \mathcal{S}$,
- a set of goals $\mathcal{G}$,
- a goal conditioned action function $a : \mathcal{S} \times \mathcal{G} \to \mathcal{A}$ selecting an action given a state and the goals.

Def 1.17. Utility-Based Agent: A utility-based agent uses a world model along with a utility function that models its preferences among the states of that world. It chooses the action that maximizes the expected utility.

Def 1.18. Learning Agent: A learning agent is an agent that can improve its own behavior through experience. It is composed of four main components:

- the performance element, which selects actions based on the agent's current percepts and represents its existing knowledge or behavior,
- the learning element, which improves the performance element over time by analyzing feedback and identifying how to make better decisions,
- the critic, which evaluates the agent's performance according to a given performance standard and provides feedback to the learning element,
- the problem generator, which suggests new and informative actions or experiences that help the agent explore and learn more effectively.

Def 1.19. State Representation: We call a state representation **atomic**, iff it has no internal structure (black box). However, iff each state is characterized by attributes and their values then the representation is **factored**. A **structured** state representation is when include representations of objects, their properties and relationships.

2 Mathematical Language

Def 2.1. Natural Language: A natural language is any form of spoken or signed means of communication that has evolved naturally in humans through use and repetition without conscious planning or premeditation.

Def 2.2. Domain of Discourse: The set of entities, objects, or concepts that a language, discussion, or formal system refers to. It specifies the scope within which terms, statements, and reasoning apply.

Def 2.3. Jargon (Terminology): A jargon (or terminology) is a set of specialized words or phrases (called technical terms or just terms) relating to concepts from a particular domain of discourse.

Def 2.4. Technical Language: A technical language is a natural language extended by a terminology and (possibly) special idioms, discourse markers, and notations.

Def 2.5. Stylized Language: Mathematicians use a stylized language that uses formulae to represent mathematical objects, uses math idioms for special situations (e.g. "iff", "hence", "let... be..., then..."), and classifies statements by role (e.g., definition, Lemma, Theorem, Proof, Example).

Def 2.6. Mathematical Vernacular: A technical language that you need to master to be successful when moving to a new environment (symbolic AI).

Def 2.7. Formulae: A mathematical formula can be a **mathematical statement** (clause) which can be true or false (e.g. $x > 5, 3 + 5 = 7$), or a **mathematical object** (e.g. $3, n, x^2 + y^2 + z^2, \int_1^0 x^{3/2} dx$)

Observation: mathematical vernacular loves to name objects/statements for precise references. For example: Let $p = 3x^2 + 7x$, then $p^3 + 17p + 1$ is $\cdots$

Moreover, mathematical vernacular aggregates objects/statements for cognitive efficiency, examples:

- $a \in S, b \in S,$ and $c \in S \rightsquigarrow a, b, c \in S$ (object aggregation)
- $i \geq 0$ and $i \leq n \rightsquigarrow 0 \leq i \leq n$ (statement aggregation)

Def 2.8. Sequence: mathematical vernacular uses the concept of sequences instead of lists. Sequences are usually finite, but can be infinite as well.

Def 2.9. Ellipses: mathematical vernacular uses ellipses $(\cdots)$ as a constructor for sequences. The meaning of an ellipsis is usually considered "obvious" and left for interpretation by the reader.

ellipses allow to write down large objects easily, examples:

- $1, \cdots, n \rightsquigarrow$ the sequence of natural numbers between 1 and n in order.
- $1, 4, 9, 16, \cdots \rightsquigarrow$ the sequence of squares in orders.
- $e_1, \cdots, e_n \rightsquigarrow$ a sequence of objects e_i for $1 \leq i \leq n$

Mathematical statements come in different epistemic varieties:

- **Definitions**: statements that introduce new global identifier for important objects
- **Assertions**: statements that state properties of mathematical objects
- **Examples**: statements that exhibit a witness for some property
- **Axioms**: statements that characterize the objects of a certain domain of discourse or theory. An axiom (postulate) is a statement about mathematical objects that we *assume* to be true.

Mathematical assertions are pragmatically classified into categories:

- **proofs** are arguments that justify the truth of statements beyond any doubt. Proofs are not really statements, but we sometimes treat them together.
- **proposition** is a statement which is interesting in its own right.
- **lemma** is an easily proved statement which is helpful for proving other propositions and theorems, but is usually not particularly interesting in its own right. A lemma is an intermediate theorem that serves as part of a proof of a larger theorem.
- **theorem** is a more important statement than a proposition which says something definitive on the subject, and often takes more effort to prove than a proposition or lemma. A theorem is a statement about mathematical object that we *know* to be true.

- **corollary** is a quick consequence of a proposition or theorem that was proven rececently. It is a theorem that follows directly from another theorem.
- **conjecture** is a statement that is thought to be provable, but has not been yet. A conjecture that is proven is a theorem

Mathematical statement use abbreviations, examples:

- $\wedge$ and $\vee$ for "and" and "or",
- $\neg$ for "not"
- $\forall x.P \quad (\forall x \in S.P)$ for "condition P holds for all x (in S)"
- $\exists x.P \quad (\exists x \in S.P)$ for "there exists an x (in S) such that the condition P holds"
- $\nexists x.P \quad (\nexists x \in S.P)$ for "there exists no x (in S) such that the condition P holds"
- $\exists^1 x.P(\exists^1 x \in S.P)$ for "there exists one and only x (in S) such that the condition P holds"
- "*iff*" for "if and only if", symbolized by $\Leftrightarrow$
- $\Rightarrow$ for "implies"; we can read $A \Rightarrow B$ as "if A then B"

Def 2.10. Declaration: In mathematical vernacular we call a clause that introduce new identifiers together with some properties a declaration. For example Let $\epsilon, \delta > 0 \cdots$

Def 2.11. Scope: The scope of an identifier is the part of an expression where the reference is valid; that is, where the identifier can be used to refer. In the other parts, the identifier may refer to a different entity, or to nothing at all (unbound)

Observations: definition have *global* scope, declarations have *local* scope.

Def 2.12. Definiendum: The definiendum is the symbol, expression, or term being introduced by a definition. It is the newly named object or concept.

Def 2.13. Definiens: The definiens is the symbol, expression, or construction that specifies the meaning of the defineiendum. It explains what the defineiendum denotes.

Mathematics has developed various forms of definition (definition schemata). A **simple definition** introduces a name (the defineiendum) for a compound object or concept (the definiens). The defineiendum must be new, i.e. may not have been used for anything else, in particular, the defineiendum may not occur in the definiens. We use the symbols := (and the inverse =:) to write simple definitions. For example, we can give the natural number 2 the name ϕ. In a formula we write this as $\phi := 2$ or $2 =: \phi$.

On the other hand, in a **pattern definition** the defineiendum is the operator or relation applied to n distinct variables -called pattern variables- $v_1, \cdots, v_n$ as arguments, and the definiens is an expression in these variables. When the new operator is applied to arguments $a_1, \cdots, a_n$, then its value is the definiens expression where the v_i are replaced by the a_i. We use the symbol := for operator definitions and :$\Leftrightarrow$ for relation definitions. For example, the following is a pattern definition for the set intersection operator $\cap$:

$$A \cap B := \{x \mid x \in A \wedge x \in B\}$$

The pattern variables are A and B, and with this definition we have e.g. $\Phi \cap \Phi = \{x \mid x \in \Phi \wedge x \in \Phi\}$.

An **implicit definition** (definition by description) is a clause or expression A, such that we can prove "there is exactly one n such that A" ($\exists^1 n.A$), where n is a new name - the defineiendum. If such a unique existence proof exists, we call n *well-defined*. For example, $\forall x.x \notin \Phi$ is an implicit definition for the empty set Φ (instead of explicitly saying $\Phi := \{\}$). We can prove unique existence of Φ by just exhibiting $\{\}$ and showing that any other set S with $\forall x.x \notin S$ we have $S \equiv \Phi$. S cannot have elements, so it has the same elements as Φ, and thus $S \equiv \Phi$.

Here is another implicit definition: The exponential function is that function $f : \mathbb{R} \to \mathbb{R}$ with $f' = f$ and $f(0) = 1$. Here A is the clause "$f' = f$ and $f(0) = 1$". Well-definedness is mathematically non-trivial.

Mathematics uses examples and counterexamples to support understanding a property P. Examples give us a sense of the extent of P and counterexamples help delineate the border of P.

Def 2.14. Example: An example E is a mathematical statement that consists of:

- symbol p for the **exemplandum** (plural: exemplanda): the property to be exemplified,
- the **exemplans** (plural: exemplantia): an expression A denoting a mathematical object that acts as witness object for the property p, and

- (optionally) a **justification** of E, i.e. a proof π that $p(A)$ holds in the current context.

Def 2.15. Counter Example: An example for the complement of the property p where π is a proof that $p(A)$ does not hold.

Def 2.16. Running Example: An example that is recalled time and again during an argument, applying the newly discovered knowledge to it, presumably to show how things work.

For instance, the following mathematical statement is a mathematical example:

The identity function on $\mathbb{R}$ is *continuous*.

- the exemplandum is "*continuous*",
- the exemplans A is "The identity function on $\mathbb{R}$", and
- the justification π, a proof of continuity $\mathrm{Id}_{\mathbb{R}}$: Let $\epsilon > 0$, then we choose $\delta := \epsilon \cdots$ is omitted.

We can use examples for $\forall$-statements as well. Let $B = \forall x.A$ be a mathematical statement, then we call an example/counter example for B whose exemplandum is the property "A holds for x" an example/counter example for B. For example, 3 is an example for the statement $B = \forall n.\mathrm{odd}(n)$.

> **Note**
>
> An example (or even several) for a mathematical statement B do not constitute a proof for B. On the other hand, one counter example for a mathematical statement B does show that $\neg B$ is true ($\exists x.\neg A \equiv \neg(\forall x.A)$).

Def 2.17. Syllogism: A syllogism (proof method) is an argument that applies deductive reasoning to arrive at a conclusion based on two (or more) propositions that are asserted or assumed to be true. A **syllogistic system** is simply a set of syllogisms

> **Note**
>
> Today we usually reserve the term syllogism to informal arguments, formal arguments are called inference rules.

In a **proof by contradiction**, we make an assumption ($\neg A$) to the contrary of what we want to prove (A), and then we show that the assumption leads to a contradiction and therefore must be false. That lets us to conclude that ($\neg\neg A$) must be true, and thus (A).

If we want to prove a statement of the form *"If A, then B"*, then we do that by:

- assuming A and proving B from that.
- after this subproof, we may not use A anymore.

We call this proof method a **proof by local hypothesis**

If we know a general rule of the form *"If A then B"*, and we also know a specific instance A', which arises from A by replacing its variables with concrete values. Then by **chaining** we can conclude the corresponding instance B', obtained from B by the same variable replacement.

Example.

1. We know: "Socrates is a human." This is A'.
2. We also know a general rule:
$$\forall x\ (\ x \text{ is human } \Rightarrow\ x \text{ is mortal }).$$
This is the "If A then B" statement, where
$$A(x) = (x \text{ is human}), \quad B(x) = (x \text{ is mortal}).$$
3. Substitute $x = \text{Socrates}$:
$$A(x) \to \text{Socrates is human}, \qquad B(x) \to \text{Socrates is mortal}.$$
4. Since "Socrates is human" is true, chaining gives us:
$$\text{Socrates is mortal}.$$

If we want to prove *"A for all x"*, then we prove A without regard for x, then argue something like *"as x was chosen arbitrarily when we proved A, we know A for every x"*

If we know that one of the cases $A_1, \cdots, A_k$ must always hold, then we use **proof by cases** to show that *"if A_1 then C"* and "if A_2 then C" and so on $\forall i : 1 \leq i \leq k$, hence we can conclude that C holds. The following is an example:

Lemma 2.1. For all rational numbers a and b, if $ab = 0$, then $a = 0$ or $b = 0$.

Proof.

1. Let $a, b \in \mathbb{Q}$ and assume $ab = 0$.
2. Obviously, either $a = 0$ or $a \neq 0$.
3. We prove that $a = 0$ or $b = 0$ by the two cases induced.
   4. Case $a = 0$:

      4.1 Then the conclusion of the lemma is trivially true, and there is nothing to prove.
   5. Case $a \neq 0$:

      5.1 We can multiply both sides of the equation $ab = 0$ by $\frac{1}{a}$ and obtain

      $$\tfrac{1}{a} \cdot ab = \tfrac{1}{a} \cdot 0.$$

      5.2 Reducing the fractions gives $b = 0$.
6. In both cases, either $a = 0$ or $b = 0$, so the statement holds. □

Def 2.18. Without Loss of Generality (WLOG): An inference principle used in proofs when a statement $Q(x)$ must be shown for all cases of x, but the different cases are equivalent by symmetry or trivial reduction. Formally, if the domain of x can be partitioned into cases $A_1, \ldots, A_k$ and it can be shown that:

- proving $Q(x)$ under one representative case A_i suffices, because
- for every other case A_j there is either a trivial proof of $Q(x)$ or a direct reduction to the representative case,

then we may assume A_i holds "without loss of generality". This allows us to simplify the argument by only proving the representative case.

For example, if we want to prove the property $Q(p)$ for all primes p. We know that every prime is either 2 (the only even prime) or an odd prime. If the case $p = 2$ is easy, and the general odd prime case is the only real work, then we can say:

> WLOG, assume p is odd

This does not exclude the even case; it means we have already checked that the even case adds no new difficulty.

In mathematical vernacular, different alphabets are used to convey precise meanings. Table 1 presents the Greek letters that you are expected to read, recognize, and write. In addition, table 2 shows the curly Roman and Fraktur letters, which are also standard in mathematical writing. Becoming fluent with these alphabets is an essential part of mathematical literacy.

α	A	alpha	β	B	beta	γ	Γ	gamma
δ	Δ	delta	ϵ	E	epsilon	ζ	Z	zeta
η	H	eta	θ, ϑ	Θ	theta	ι	I	iota
κ	K	kappa	λ	Λ	lambda	μ	M	mu
ν	N	nu	ξ	Ξ	xi	o	O	omicron
π, ϖ	Π	pi	ρ	P	rho	σ	Σ	sigma
τ	T	tau	υ	Υ	upsilon	φ, ϕ	Φ	phi
χ	X	chi	ψ	Ψ	psi	ω	Ω	omega

Table 1: Greek Alphabet

Roman	$\mathcal{A}, \mathcal{B}, \mathcal{C}, \mathcal{D}, \mathcal{E}, \mathcal{F}, \mathcal{G}, \mathcal{H}, \mathcal{I}, \mathcal{J}, \mathcal{K}, \mathcal{L}, \mathcal{M}, \mathcal{N}, \mathcal{O}, \mathcal{P}, \mathcal{Q}, \mathcal{R}, \mathcal{S}, \mathcal{T}, \mathcal{U}, \mathcal{V}, \mathcal{W}, \mathcal{X}, \mathcal{Y}, \mathcal{Z}$
Fraktur	$\mathfrak{A}, \mathfrak{B}, \mathfrak{C}, \mathfrak{D}, \mathfrak{E}, \mathfrak{F}, \mathfrak{G}, \mathfrak{H}, \mathfrak{I}, \mathfrak{J}, \mathfrak{K}, \mathfrak{L}, \mathfrak{M}, \mathfrak{N}, \mathfrak{O}, \mathfrak{P}, \mathfrak{Q}, \mathfrak{R}, \mathfrak{S}, \mathfrak{T}, \mathfrak{U}, \mathfrak{V}, \mathfrak{W}, \mathfrak{X}, \mathfrak{Y}, \mathfrak{Z}$

Table 2: Roman and Fraktur Letters

3 Unary Natural Numbers

Def 3.1. Unary Natural Numbers: Terms generated by the following: $u ::= 0 \mid s(u)$ where 0 denotes zero and $s(u)$ denotes the successor of u. i.e. $\{0, s(0), s(s(0)), s(s(s(0))), \cdots\}$.

Def 3.2. Piano Axioms: The following set of axioms are called the Piano axioms. They can be used to *reason* about natural numbers:

- **P1.** 0 is a unary natural number.
- **P2.** Every unary natural number has a successor that is unary natural number and that is different from it.
- **P3.** 0 is not a successor of any unary natural number.
- **P4.** Different unary natural numbers have different successors.
- **P5. Induction Axiom**. Every unary natural number possess a property P if
  - 0 has the property P, and
  - the successor of every unary natural number that has the property P also possesses property P

More formally, we call the set of unary natural numbers $\mathbb{N}_1$, and we say $P(n)$ if a unary natural number $n \in \mathbb{N}_1$ has a property P. Therefore, we express the axioms as follows:

- $0 \in \mathbb{N}_1$
- $\forall n \in \mathbb{N}_1 . s(n) \in \mathbb{N}_1 \wedge n \neq s(n)$
- $\neg(\exists n \in \mathbb{N}_1 . 0 = s(n))$
- $\forall n \in \mathbb{N}_1 . \forall m \in \mathbb{N}_1 . n \neq m \Rightarrow s(n) \neq s(m)$
- $\forall P . P(0) \wedge (\forall n \in \mathbb{N}_1 . P(n) \Rightarrow P(s(n))) \Rightarrow (\forall m \in \mathbb{N}_1 . P(m))$

Theorem 3.1. Every unary natural number is either zero (0) or the successor of a unary natural number.

Proof. We make use of the induction axiom P5: We use the property P of "being zero or a successor" and prove the statement by convincing ourselves of the prerequisites of:

1. 0 is zero so 0 is "zero or a successor".
2. Let n be arbitrary unary natural number that is "zero or a successor".
3. Then its successor is a "successor", so the successor of n is also "zero or a successor".
4. Since we have taken n arbitrary we have shown that for any n, its successor has property P.
5. Property P holds for all unary natural number by P5, so we have proven the assertion.

□

Def 3.3. Mathematical Induction: A method for proving a statement $P(n)$ is true for every natural number n. A proof by mathematical induction consists of two steps:

- The **base case**: Prove that $P(0)$ is true.
- The **induction step**: Prove that for every n, if $P(n)$ is true then $P(n+1)$ is also true.

The hypothesis in the induction step, that $P(n)$ is true for a particular n, is called the **induction hypothesis**

Def 3.4. The addition "function": We define the addition operation + procedurally:

- Adding 0 to a number does not change it, expressed as: $n + 0 = n$.
- Adding m to the successor of n yields the successor of $m + n$, expressed as: $m + s(n) = s(m + n)$.

Theorem 3.2. For all unary natural number n, m, l, we have $(n + m) + l = n + (m + l)$.

Proof. We prove this by induction on l.

Let $P(l)$ be the property:

$$(n + m) + l = n + (m + l).$$

- base case:

$$(n + m) + 0 = (n + m) = n + (m + 0)$$

- step case: Assume $(n+m)+l = n+(m+l)$ for an arbitrary l. We must show

$$(n+m)+s(l) = n+(m+s(l)).$$

$$\begin{aligned}(n+m)+s(l) &= s\big((n+m)+l\big)\\ &\overset{\text{IH}}{=} s\big(n+(m+l)\big)\\ &= n+s(m+l)\\ &= n+(m+s(l)).\end{aligned}$$

  Hence the property holds for $s(l)$.

$\square$

Def 3.5. Multiplication: The **unary multiplication** operation can be defined by the equations:

- $n \cdot 0 = 0$, and
- $n \cdot s(m) = n + (n \cdot m)$

Def 3.6. Exponentiation: The **unary exponentiation** operation can be defined by the equations:

- $exp(n, 0) = s(0)$, and
- $exp(n, s(m)) = n \cdot exp(n, m)$

Def 3.7. Summation: The **unary summation** operation can be defined by the equations:

- $\bigoplus_{i=0}^{0}(n_i) = 0$, and
- $\bigoplus_{i=0}^{s(m)}(n_i) = n_{s(m)} \bigoplus \bigoplus_{i=0}^{m}(n_i)$

Example:

$$\begin{aligned}\bigoplus_{i=0}^{s(s(0))}(n_i) &= n_{s(s(0))} \bigoplus \bigoplus_{i=0}^{s(0)}(n_i)\\ &= n_{s(s(0))} \bigoplus n_{s(0)} \bigoplus \bigoplus_{i=0}^{0}(n_i)\\ &= n_{s(s(0))} \bigoplus n_{s(0)} \bigoplus 0\end{aligned}$$

Def 3.8. Product: The **unary product** operation can be defined by the equations:

- $\bigodot_{i=0}^{0}(n_i) = s(0)$, and
- $\bigodot_{i=0}^{s(m)}(n_i) = n_{s(m)} \bigodot \bigodot_{i=0}^{m}(n_i)$

4 Sets

Def 4.1. Set: A collection of different things (called **elements** or **members** of the set).

We can represent sets by listing the elements within curly brackets, e.g. $\{a, b, c\}$ or by describing the elements via a property, e.g. $\{x \mid x \mod 2 = 0\}$ (the set of even numbers). We can state element-hood ($a \in S$) or not ($b \notin S$)

> **Note**
>
> Learn to distinguish between objects and their representations. ($\{a, b, c\}$ and $\{b, a, a, c\}$ are different representations of the same set)

Def 4.2. Set Equality: $A \equiv B :\equiv \forall x. x \in A \Leftrightarrow x \in B$

Def 4.3. Subset: $A \subseteq B :\equiv \forall x. x \in A \Rightarrow x \in B$

Def 4.4. Proper Subset: $A \subset B :\equiv (A \subseteq B) \wedge (A \not\equiv B)$

Def 4.5. Super Set: $A \supseteq B :\equiv \forall x. x \in B \Rightarrow x \in A$

Def 4.6. Proper Superset: $A \supset B :\equiv (A \supseteq B) \wedge (A \not\equiv B)$

Def 4.7. Set Union: $A \cup B := \{x \mid x \in A \vee x \in B\}$

Def 4.8. Set Intersection: $A \cap B := \{x \mid x \in A \wedge x \in B\}$

Def 4.9. Disjoint: Two sets A, B are **disjoint** iff $A \cap B = \Phi$

Def 4.10. Set Difference: $A \setminus B := \{x \mid x \in A \wedge x \notin B\}$

Def 4.11. Power Set: $\mathcal{P}(A) := \{S \mid S \subseteq A\}$ (the set of all subsets)

> **Note**
>
> Set operators such as union ($\cup$), intersection ($\cap$), and set difference ($\setminus$) always return a *set*.
> For example, consider
> $$\{a, b\} \cap \{b, c\} = \{b\}.$$
> The result $\{b\}$ is a set.
> Now observe:
> - $\{b\} \notin \{a, b\}$, since the elements of $\{a, b\}$ are a and b, not the set $\{b\}$.
> - $\{b\} \notin \{b, c\}$, since the elements of $\{b, c\}$ are b and c, not the set $\{b\}$.
> - $\{b\} \subseteq \{a, b\}$, because every element of $\{b\}$ (namely b) is also in $\{a, b\}$.
> - $\{b\} \subseteq \{b, c\}$, for the same reason.
>
> Thus, the outcome of a set operation should always be compared to other sets using $\subseteq$ (subset relation), not $\in$ (element relation).

Def 4.12. Cartesian Product: $A \times B := \{(a, b) \mid a \in A \wedge b \in B\}$, (a, b) is a *pair*

Def 4.13. Family: A set of sets

Def 4.14. Topology: A family of open sets

Def 4.15. Indexing Function: Let I and X be sets. A function $f : I \to X$ is called an indexing function. The set I is called the index set. For each $i \in I$, we define $x_i := f(i)$, here i is called the index (or parameter) of x_i.

Def 4.16. Indexed Family: Given an indexing function $f : I \to X$, the set of values $f(I) = \{f(i) : i \in I\}$ is called an indexed family. It is often written as $(x_i)_{i \in I}$, $\langle x_i \rangle_{i \in I}$, or $\{x_i\}_{i \in I}$.

Example: Consider index set $I := \{1, 2, 3\}$. Consider indexing function f defined as:

$$f(1) = \{a, b\} \quad f(2) = \{c\} \quad f(3) = \{d, f\}$$

the set of values $f(I) = \{f(1), f(2), f(3)\}$ is the indexed family:

$$(X_i)_{i \in I} = f(I) = \{\{a, b\}, \{c\}, \{d, f\}\}$$

where $x_i = f(i)$, i.e. $x_1 = f(1) = \{a, b\}, \cdots$

Def 4.17. Union over a Family: Let $(X_i)_{i \in I}$ be an indexed family, then $\bigcup_{i \in I} X_i := \{x \mid \exists i \in I. x \in X_i\}$

Def 4.18. Intersection over a Family: Let $(X_i)_{i \in I}$ be an indexed family, then $\bigcap_{i \in I} X_i := \{x \mid \forall i \in I. x \in X_i\}$

Def 4.19. n-fold Cartesian Product: $\prod_{i=1}^{n} X_i := X_1 \times \cdots \times X_n := \{\langle x_1, \cdots, x_n \rangle \mid \forall i. 1 \leq i \leq n \Rightarrow x_i \in X_i\}$ where $\langle x_1, \cdots, x_n \rangle$ is called an n-tuple.

Example: consider the indexed family $(X_i)_{i \in \{1,2,3\}} = \{\{a, b\}, \{c\}, \{d, f\}\}$. $X_1 \times X_2 \times X_3 = \{\langle a, c, d \rangle, \langle a, c, f \rangle, \langle b, c, d \rangle, \langle b, c, f \rangle\}$.

Def 4.20. n-dim Cartesian Space: $X^n := \{\langle x_1, \cdots, x_n \rangle \mid 1 \leq i \leq n \Rightarrow x_i \in X\}$ where $\langle x_1, \cdots, x_n \rangle$ is called a vector.

Example: Let $X = \{1, 2\}$, then $X^2 = \{\langle 1, 1 \rangle, \langle 1, 2 \rangle, \langle 2, 1 \rangle, \langle 2, 2 \rangle\}$. Another example is the usual 2D plane: $\mathbb{R}^2 := \{\langle x, y \rangle \mid x \in \mathbb{R}, y \in \mathbb{R}\}$

Def 4.21. Size of a Set: The size $\Gamma(A)$ of a set A is the number of elements in A.

Note

- $\Gamma(A \cup B) \leq \Gamma(A) + \Gamma(B)$
- $\Gamma(A \cap B) \leq \min(\Gamma(A), \Gamma(B))$
- $\Gamma(A \times B) = \Gamma(A) \cdot \Gamma(B)$

Def 4.22. Pairwise Disjoint: A family of sets is called **pairwise disjoint** or **mutually disjoint**, if any two of those sets are disjoint.

Def 4.23. Multiset: A **multiset** (or **bag**) is a generalization of a set where elements can appear more than once. Formally, a multiset $\mathcal{M}$ over a domain D is a pair (D, m), where $m : D \to \mathbb{N}$ is a function mapping each element $x \in D$ to its **multiplicity** (the number of occurrences).

Def 4.24. Multiset Membership: $x \in \mathcal{M} :\equiv m(x) > 0$

Def 4.25. Multiset Size: The size of a multiset $\mathcal{M} = (D, m)$, denoted $\Gamma(\mathcal{M})$, is the sum of the multiplicities of all its elements: $\Gamma(\mathcal{M}) = \sum_{x \in D} m(x)$.

Def 4.26. Multiset Sum (Union): The sum of two multisets $\mathcal{M}_1 = (D, m_1)$ and $\mathcal{M}_2 = (D, m_2)$, denoted $\mathcal{M}_1 \uplus \mathcal{M}_2$, is a multiset where the multiplicity of each element is the sum of its multiplicities: $m_{\uplus}(x) = m_1(x) + m_2(x)$.

Def 4.27. Multiset Intersection: The intersection of two multisets $\mathcal{M}_1 = (D, m_1)$ and $\mathcal{M}_2 = (D, m_2)$, denoted $\mathcal{M}_1 \cap \mathcal{M}_2$, is a multiset where the multiplicity of each element is the minimum of its multiplicities in the two multisets: $m_{\cap}(x) = \min(m_1(x), m_2(x))$.

Def 4.28. Multiset Equality: $\mathcal{M}_1 \equiv \mathcal{M}_2 :\equiv \forall x \in D. m_1(x) = m_2(x)$

5 Relations

Def 5.1. Relation: $R \subseteq A \times B$ is a (binary) relation between A and B

Def 5.2. Relation on: a relation $R \subseteq A \times B$ where $A = B$ is called a *relation on* A

Def 5.3. Total: A relation $R \subseteq A \times B$ is called *total* (*left total*) iff $\forall x \in A. \exists y \in B. (x, y) \in R$

Def 5.4. Converse Relation: $R^{-1} \subseteq B \times A := \{(y, x) \mid (x, y) \in R\}$ is the converse relation of R

Def 5.5. Relation Composition: The composition of $R \subseteq A \times B$ and $S \subseteq B \times C$ is defined as $R \circ S := \{(a, c) \in A \times C \mid \exists b \in B. (a, b)$

A relation on A: $R \subseteq A \times A$ is called:

- **reflexive** on A, iff $\forall a \in A. (a, a) \in R$
- **irreflexive** on A, iff $\forall a \in A. (a, a) \notin R$
- **symmetric** on A, iff $\forall a, b \in A. (a, b) \in R \Rightarrow (b, a) \in R$
- **asymmetric** on A, iff $\forall a, b \in A. (a, b) \in R \Rightarrow (b, a) \notin R$
- **antisymmetric** on A, iff $\forall a, b \in A. (a, b) \in R \wedge (b, a) \in R \Rightarrow a = b$
- **transitive** on A, iff $\forall a, b, c \in A. (a, b) \in R \wedge (b, c) \in R \Rightarrow (a, c) \in R$
- **equivalence relation** on A, iff R is reflexive symmetric, and transitive

Def 5.6. Equality Relation: The equality relation is an equivalence relation on any set.

Def 5.7. Divides Relation: The *divides* relation on the integers is defined as $\mid \subseteq \mathbb{Z} \times \mathbb{Z}$, where $\mid = \{(a, b) \in \mathbb{Z} \times \mathbb{Z} \mid \exists k \in \mathbb{Z} : b = a \cdot k\}$. We write $a \mid b$ to denote $(a, b) \in \mid$ (read as *"a divides b"*).

For example: $5 \mid 10$ because $\exists k : 10 = 5 \cdot k$ where $k = 2$.

Def 5.8. Congruence Modulo: For a fixed $n \in \mathbb{N}$ with $n \geq 1$, the *congruence modulo n* relation on the integers is defined as

$$\equiv_n \subseteq \mathbb{Z} \times \mathbb{Z}, \qquad \equiv_n = \{(a, b) \in \mathbb{Z} \times \mathbb{Z} \mid n \mid (a - b)\}.$$

We write $a \equiv b \pmod{n}$ to denote $(a, b) \in \equiv_n$ (read as *"a is congruent to b modulo n"*).

Examples:

- $12 \equiv 2 \pmod 5$ because $5 \mid 10$
- $7 \equiv 22 \pmod 5$ because $5 \mid 15$
- $9 \equiv 9 \pmod 5$ because $5 \mid 0$

Note

In the definition of congruence modulo n, we do *not* take the absolute value of the difference:

$$a \equiv b \pmod{n} \iff n \mid (a - b).$$

Since the divisor n can be multiplied by a negative integer, the sign of $(a - b)$ does not matter. For example, $2 \equiv 12 \pmod 5$ because $2 - 12 = -10$ and $-10 = 5 \cdot (-2)$, so $5 \mid (2 - 12)$.

Exercise: Which of the following binary relations on integers are equivalence relations?

1. Φ. It is not reflexive because we must have $(a, a) \in R$ for all integers a. But Φ has no pairs at all. $\rightarrow$ NO.
2. $\mathbb{Z} \times \mathbb{Z}$
   - It is reflexive since $(a, a) \in R \quad \forall a$
   - It is symmetric because if $(a, b) \in R$ then (b, a) also is
   - It is transitive because every possible pair is in it anyway
   - $\rightarrow$ YES.
3. $m \leq n$

- It is reflexive since $m \leq m$
- It is not symmetric, e.g. $2 \leq 3$ but not $3 \leq 2$
- $\rightarrow$ NO

4. $\exists z \in \mathbb{Z} : m + z = n$
   - This is actually the same as $\mathbb{Z} \times \mathbb{Z}$ because for any integers m, n, we can always pick $z = n - m$. $\rightarrow$ YES.
5. $\exists z \in \mathbb{Z} : m \cdot z = n$
   - This is the same as $m \mid n$. It is reflexive ($m \cdot 1 = 1$). However, it is not symmetric ($2 \mid 4$ but $4 \nmid 2$). $\rightarrow$ NO.
6. $5 \mid (m - 2)$
   - This means m is congruent to 2 modulo 5.
   - It is reflexive ($5 \mid 0$)
   - It is symmetric (negative or positive does not matter)
   - It is transitive, if $5 \mid m - n$ and $5 \mid n - k$, then $5 \mid (m - n) + (n - k) = m - k$
   - $\rightarrow$ YES.
7. $m \mid n$. It is reflexive but not symmetric. $\rightarrow$ NO.

Def 5.9. Parital Ordering (non-strict): A relation $R \subseteq A \times A$ is called a **partial ordering** on A, iff R is reflexive, antisymmetric, and transitive on A.

Def 5.10. Strict Partial Ordering: A relation $R \subseteq A \times A$ is called a **strict partial ordering** on A, iff R is irreflexive, and transitive on A.

> **Note**
>
> We usually write strict partial ordering with asymmetric symbols like $\prec$ and non-strict ones by adding a line that reminds us of equality ($\preceq$).

Def 5.11. Comparable: In a non-strict partial ordering, two elements a, b are comparable if either $a \preceq b$ or $b \preceq a$. If neither holds, they are incomparable.

Def 5.12. Linear Order: A partial ordering is called linear (or total) on A, iff all elements in A are comparable, i.e. if $(x, y) \in R$ or $(y, x) \in R$ for all $x, y \in A$.

For example, the $\leq$ relation is a linear order on $\mathbb{N}$. However, the "divides" ($|$) relation is a non-strict partial ordering on $\mathbb{N}$ that is not linear (e.g., neither $2 \mid 3$ nor $3 \mid 2$).

Lemma 5.1. Strict partial orderings are asymmetric.

Proof. By contradiction: if $(a, b) \in R$ and $(b, a) \in R$, then $(a, a) \in R$ by transitivity, and that can't be because its irreflexive. $\square$

Lemma 5.2. If $\preceq$ is a non-strict partial order, then $\prec := \{(a, b) \mid a \preceq b \wedge a \neq b\}$ is a strict partial order. Conversely, if $\prec$ is a strict partial order, then $\preceq := \{(a, b) \mid a \prec b \ \vee \ a = b\}$ is a non-strict partial order.

Proof. **(1) Non-strict $\rightarrow$ Strict:** Assume $\preceq$ is reflexive, antisymmetric, and transitive. Define $a \prec b \iff (a \preceq b \wedge a \neq b)$.

- *Irreflexive:* $a \prec a$ would require $a \neq a$, impossible.
- *Transitive:* If $a \prec b$ and $b \prec c$, then $a \preceq b$, $b \preceq c$, and $a \neq b$, $b \neq c$. By transitivity of $\preceq$, $a \preceq c$. If $a = c$, antisymmetry would force $a = b$, contradiction. So $a \neq c$, hence $a \prec c$.

Thus $\prec$ is a strict partial order.

(2) Strict $\rightarrow$ Non-strict: Assume $\prec$ is irreflexive and transitive. Define $a \preceq b \iff (a = b \vee a \prec b)$.

- *Reflexive:* $a = a$ gives $a \preceq a$.
- *Antisymmetric:* If $a \preceq b$ and $b \preceq a$, then either $a = b$ or we would have $a \prec b$ and $b \prec a$, contradicting asymmetry.
- *Transitive:* If $a \preceq b$ and $b \preceq c$, then combining cases ($=$ or $\prec$) always gives $a \preceq c$.

Thus $\preceq$ is a non-strict partial order. $\square$

Examples:

- On $\mathbb{N}$, the non-strict partial order $\leq$ induces the strict order $<$ by removing equality. Conversely, $<$ induces $\leq$ by adding equality.
- On sets, the non-strict partial order $\subseteq$ induces the strict order $\subset$. Conversely, $\subset$ induces $\subseteq$ by adding equality.

Exercise: which of the following binary relation on integers are partial ordering:

1. Φ. It is not reflexive because we must have $(a, a) \in R$ for all integers a. But Φ has no pairs at all. $\rightarrow$ NO.
2. $\mathbb{Z} \times \mathbb{Z}$
   - It is reflexive since $(a, a) \in R \quad \forall a$
   - It is symmetric because if $(a, b) \in R$ then (b, a) also is, $\rightarrow$ it is not antisymmetric, $\rightarrow$ NO
3. $m \leq n$
   - It is reflexive since $m \leq m$
   - It is asymmetric, e.g. $2 \leq 3$ but not $3 \leq 2$
   - It is transitive
   - $\rightarrow$ YES
4. $\exists z \in \mathbb{Z} : m + z = n$
   - This is actually the same as $\mathbb{Z} \times \mathbb{Z}$ because for any integers m, n, we can always pick $z = n - m$. $\rightarrow$ NO.
5. $\exists z \in \mathbb{Z} : m \cdot z = n$
   - This is the same as $m \mid n$. It is reflexive ($m \cdot 1 = 1$). It is asymmetric on $\mathbb{N}$ but not on $\mathbb{Z}$ ($2 \mid -2$ and $-2 \mid 2$ but $2 \neq -2$). $\rightarrow$ NO.
6. $5 \mid (m - 2)$
   - This means m is congruent to n modulo 5.
   - It is reflexive ($5 \mid 0$)
   - It is symmetric (not asymmetric)
   - It is transitive, if $5 \mid m - n$ and $5 \mid n - k$, then $5 \mid (m - n) + (n - k) = m - k$
   - $\rightarrow$ NO.
7. $m \mid n$. It is reflexive but not asymmetric. $\rightarrow$ NO.

Def 5.13. Partially Ordered Set (Poset): A **Partially Ordered Set** (or **poset**) is a pair $\langle G, \preceq \rangle$ where G is a set and $\preceq$ is a non-strict partial ordering on G.

Def 5.14. Least Element: Let $\langle G, \preceq \rangle$ be a poset and $T \subseteq G$. An element $t \in T$ is the **least** (or **smallest**, **minimal**, or $\preceq$**-minimal**) element of T iff $t \preceq t'$ for all $t' \in T$.

Def 5.15. Greatest Element: Let $\langle G, \preceq \rangle$ be a poset and $T \subseteq G$. An element $t \in T$ is the **greatest** (or **largest**, **maximal**, or $\preceq$**-maximal**) element of T iff $t' \preceq t$ for all $t' \in T$.

Def 5.16. Supremum and Infimum: Let $\langle G, \preceq \rangle$ be a non-strict partial ordering and $T \subseteq G$, then we call the **least upper bound** ($\sup(T) \in G$) of T the **supremum** of T and the **greatest lower bound** ($\inf(T) \in G$) of T the **infimum** of T (if they exist).

Example: Consider the non-strict partial ordering $\langle \mathbb{R}, \preceq \rangle$. Let T be the open interval $(0, 1) \subseteq \mathbb{R}$, then $\sup(T) = 1$, and $\inf(T) = 0$. Note that $0, 1 \in \mathbb{R}$ but $\notin T$. (its different from `min` and `max` of T).

6 Functions

Def 6.1. Partial function: A relation $R \subseteq X \times Y$ is a *partial function* from X to Y ($f : X \rightharpoonup Y$) iff $\forall x \in X, \forall y_1, y_2 \in Y.\big((x, y_1) \in R \wedge (x, y_2) \in R \Rightarrow y_1 = y_2\big)$ (i.e. there is at most one $y \in Y$ with $(x, y) \in f$)

Def 6.2. dom, codom: We call X the domain ($\mathrm{dom}(f)$), and Y the codomain ($\mathrm{codom}(f)$)

Def 6.3. Function Application: Instead of writing $(x, y) \in f$ we write $f(x) = y$

Def 6.4. Undefined: we call a partial function $f : X \rightharpoonup Y$ undefined at $x \in X$, iff $\forall y \in Y.(x, y) \notin f$ ($f(x) = \perp$).

Def 6.5. Function: A partial function $f : X \rightharpoonup Y$ is called a *function* (or *total function*) from X to Y iff it is a total relation. ($\forall x \in X.\exists^1 y \in Y : (x, y) \in f$)

> **Note**
>
> A partial function guarantees uniqueness, but not existence. So each x has at most one y (some might have none). A total function guarantees both uniqueness and existence (at most and at least $\rightarrow$ exactly one)

Def 6.6. Partial Function Space: Given two sets X and Y, the set of all possible partial functions from X to Y is called the partial function space, denoted as $X \rightharpoonup Y$

> **Note**
>
> If X and Y are finite, then for each element $x \in X$ we have $\Gamma(Y) + 1$ choices (either pick one of the $\Gamma(Y)$ values in Y, or leave it undefined). Therefore, $\Gamma(X \rightharpoonup Y) = (\Gamma(Y) + 1)^{\Gamma(X)}$.

Example, $X = Y = \{0, 1\}$

$$X \rightharpoonup Y = \{\Phi, \{(0,0)\}, \{(0,1)\}, \{(1,0)\}, \{(1,1)\}, \{(0,0),(1,0)\}, \{(0,1),(1,1)\}, \{(0,1),(1,0)\}, \{(0,0),(1,1)\}\}$$

Notice that $\Gamma(X \rightharpoonup Y) = 9 = (\Gamma(Y) + 1)^{\Gamma(X)} = (2 + 1)^2$

Def 6.7. Function Space: Given two sets X and Y, the set of all possible functions from X to Y is called the partial function space, denoted as $X \rightarrow Y$

> **Note**
>
> If X and Y are finite, then for each element $x \in X$ we must pick exactly one of the $\Gamma(Y)$ values in Y. Therefore, $\Gamma(X \rightarrow Y) = (\Gamma(Y))^{\Gamma(X)}$.

Example, $X = Y = \{0, 1\}$

$$X \rightarrow Y = \{\{(0,0),(1,0)\}, \{(0,1),(1,1)\}, \{(0,1),(1,0)\}, \{(0,0),(1,1)\}\}$$

Notice that $\Gamma(X \rightarrow Y) = 4 = (\Gamma(Y))^{\Gamma(X)} = (2)^2$

A function $f : X \rightarrow Y$ is called

- **injective** iff $\forall x_1, x_2 \in X.f(x_1) = f(x_2) \Rightarrow x_1 = x_2$
- **surjective** iff $\forall y \in Y.\exists x \in X.f(x) = y$
- **bijective** iff f is injective and surjective

Let's look for some examples to make the previous definitions clear.

Assume $X = \{1, 2, 3\}$ $\quad Y = \{a, b\}$ The relation $R = \{(1, a), (1, b)\}$ is just a relation, it is not a partial function nor a total function.

If $R = \{(1, a)\}$ it becomes a partial function but not a total one.

If $R = \{(1, a), (2, a), (3, a)\}$ it becomes a total function because each $x \in X$ has exactly one $y \in Y$. However, it is not injective ($1, 2, 3$ all map to the same value) and not surjective (b is not reached).

IF $R = \{(1, a), (2, a), (3, b)\}$ then this is a surjective function but not an injective one (since $1, 2$ map to the same value).

Assume $X = \{1, 2\}$ $\quad Y = \{a, b, c\}$ $\quad f = \{(1, a), (2, b)\}$ then this is an injective function (different inputs map to different output) but not surjective since there is no x such that $f(x) = c$.

And finally, assume $X = \{1, 2\} \quad Y = \{a, b\} \quad f = \{(1, a), (2, b)\}$ then this is a total function that is both injective and surjective, i.e. a bijective function.

Def 6.8. Function Composition: If $f : A \to B$ and $g : B \to C$ are functions, then we call $g \circ f : A \to C; x \longmapsto g(f(x))$ the composition of g and f (read g after f).

> **Note**
>
> Given $f, g : \mathbb{N} \to \mathbb{N}$, if f and g both are injective/bijective, so is $g \circ f$

Observations:

- If f is injective, then the converse relation f^{-1} is a partial function.

  For example: $X = \{1, 2, 3\}, Y = \{a, b, c, d\}, f : X \to Y = \{(1, a), (2, b), (3, c)\}$. f is clearly injective since different inputs map to different outputs. $\mathrm{dom}(f^{-1}) = \mathrm{codom}(f)$ and vice versa. $f^{-1} = \{(a, 1), (b, 2), (c, 3)\}$. It is a partial function because for $d \in \mathrm{dom}(f^{-1})$ there is no $f^{-1}(d)$

- If f is surjective, then the converse relation f^{-1} is a total relation.

  For example: $X = \{1, 2, 3\}, Y = \{a, b\}, f : X \to Y = \{(1, a), (2, b), (3, b)\}$. f is clearly surjective because both a, b are hit by some x. $f^{-1} = \{(a, 1), (b, 2), (b, 3)\}$. It is a total function.

- IF f is bijective, call the converse relation **inverse function**, we also write it as f^{-1}.

- If f is bijective, then the inverse function f^{-1} is a total function.

- If $f : X \to Y$ is bijective, then $f \circ f^{-1} = \mathrm{Id}_Y$.

  For example: $X = \{1, 2, 3\}, Y = \{a, b, c\}, f = \{(1, a), (2, b), (3, c)\}, f^{-1} = \{(a, 1), (b, 2), (c, 3)\}$. $(f \circ f^{-1})(y) = f(f^{-1}(y))$

  - $y = a : f^{-1}(a) = 1, f(1) = a$
  - $y = b : f^{-1}(b) = 2, f(2) = b$
  - $y = c : f^{-1}(c) = 3, f(3) = c$

  $\to \quad f \circ f^{-1} = \mathrm{Id}_Y$

Def 6.9. Image and Preimage: Let $f : X \to Y$ be a function, $X' \subseteq X$ and $Y' \subseteq Y$, then we call

- $f(X') := \{y \in Y \mid \exists x \in X'.(x, y) \in f\}$ the **image** of X' under f,
- $\mathrm{Im}(f) := f(X)$ the **image** of f, and
- $f^{-1}(Y') := \{x \in X \mid \exists y \in Y'.(x, y) \in f\}$ the **preimage** of Y' under f.

> **Note**
>
> Let $f : \mathbb{N} \to \mathbb{N}$ be injective and let its partial inverse be
>
> $$f^{-1} : \mathbb{N} \rightharpoonup \mathbb{N}, \qquad \mathrm{dom}(f^{-1}) = \mathrm{im}(f), \quad f^{-1}(y) = x \text{ iff } f(x) = y.$$
>
> Then $f^{-1} \circ f = \mathrm{Id}_{\mathbb{N}}$ as a total function $\mathbb{N} \to \mathbb{N}$.
> *Proof.* For any $x \in \mathbb{N}$ we have $f(x) \in \mathrm{im}(f) = \mathrm{dom}(f^{-1})$, hence $(f^{-1} \circ f)(x) = f^{-1}(f(x)) = x$. $\square$

For example: Define $f : \mathbb{N} \to \mathbb{N}, \quad f(x) = 2x$. Then f is injective with image $\mathrm{im}(f) = \{0, 2, 4, 6, \dots\}$. Its partial inverse is

$$f^{-1} : \mathbb{N} \rightharpoonup \mathbb{N}, \quad f^{-1}(y) = \tfrac{y}{2} \text{ if } y \text{ is even}, \quad \text{undefined if } y \text{ is odd}.$$

For all $x \in \mathbb{N}$ we have

$$(f^{-1} \circ f)(x) = f^{-1}(f(x)) = f^{-1}(2x) = \tfrac{2x}{2} = x,$$

hence $f^{-1} \circ f = \mathrm{Id}_{\mathbb{N}}$. By contrast, $f \circ f^{-1}$ is only the identity on the even numbers.

Another example: Define $f : \mathbb{N} \to \mathbb{N}, \quad f(x) = x + 1$. Then f is injective with image $\mathrm{im}(f) = \{1, 2, 3, \dots\}$. Its partial inverse is

$$f^{-1} : \mathbb{N} \rightharpoonup \mathbb{N}, \quad f^{-1}(y) = y - 1 \text{ if } y \geq 1, \quad \text{undefined if } y = 0.$$

For all $x \in \mathbb{N}$ we have

$$(f^{-1} \circ f)(x) = f^{-1}(f(x)) = f^{-1}(x+1) = (x+1) - 1 = x,$$

hence again $f^{-1} \circ f = \mathrm{Id}_{\mathbb{N}}$. Here $f \circ f^{-1}$ is only the identity on $\{1, 2, 3, \dots\}$.

Def 6.10. Isomorphic: We call two sets A and B isomorphic iff there is a bijection $f : A \to B$.

Def 6.11. Cardinality: We say that a set A is **finite** and has **cardinality** $\Gamma(A) \in \mathbb{N}$, iff there is a bijective function $f : A \to \{n \in \mathbb{N} \mid n < \Gamma(A)\}$.

Def 6.12. Countably Infinite: We say that a set A is countably infinite, iff there is a bijective function $f : A \to \mathbb{N}$. Examples: $\mathbb{N}, \mathbb{Z}, \cdots$

Def 6.13. Countable: A set A is called countable, iff it is finite or countably infinite

> **Note**
>
> The set of real numbers $\mathbb{R}$ is not countable (there is no bijection) $\mathbb{R} \to \mathbb{N}$

Def 6.14. Restriction: Let $f : A \to B$ and $C \subseteq A$, then we call the function $f\restriction_C := \{(c, b) \in f \mid c \in C\}$ the restriction of f to C.

Def 6.15. Lambda-Notation: Let X be a set and E an expression depending on a variable $x \in X$. The expression $\lambda x \in X.\, E$ denotes the (partial) function f from X to Y with $f(x) = E \quad$ for all $x \in X$.

Examples:

- the identity function is written in lambda notation: $\lambda n \in \mathbb{N}.n = \{(n, n) \mid n \in \mathbb{N}\} = \mathrm{Id}_{\mathbb{N}} = \{(0, 0), (1, 1), (2, 2), \cdots\}$
- the square function: $\lambda x \in \mathbb{N}.x^2 = \left\{(x, x^2) \mid x \in \mathbb{N}\right\} = \{(0, 0), (1, 1), (2, 4), (3, 9), \cdots\}$
- the "add" operation: $\lambda(x, y) \in \mathbb{N} \times \mathbb{N}.x + y = \{((0, 0), 0), ((1, 0), 1), \cdots\}$

Def 6.16. Curried Function Type: Let X, Y, Z be sets. An *uncurried* function is written as

$$f : X \times Y \to Z, \quad f(x, y) = E(x, y).$$

The same function can equivalently be written in *curried* form as

$$f : X \to Y \to Z, \quad f(x)(y) = E(x, y).$$

Thus in the curried notation the function takes one argument at a time: for $x \in X$, the value $f(x)$ is itself a function $Y \to Z$, and applying it to $y \in Y$ yields $f(x)(y) \in Z$.

Example. The addition function can be written in uncurried or curried form:

$$\mathrm{add} : \mathbb{N} \times \mathbb{N} \to \mathbb{N}, \quad \mathrm{add}(m, n) = m + n,$$

equivalently: $\mathrm{add} : \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad \mathrm{add}(m)(n) = m + n.$

In the curried form, for fixed $m \in \mathbb{N}$, the expression $\mathrm{add}(m) : \mathbb{N} \to \mathbb{N}$ is the function $n \mapsto m + n$, i.e. the function that adds m to its argument.

Another Example (Trinary)

$$h : \mathbb{N} \times \mathbb{N} \times \mathbb{N} \to \mathbb{N}, \quad h(m, n, o) = m + n + o.$$

Equivalently, in curried form we can write

$$h : \mathbb{N} \to \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h(m)(n)(o) = m + n + o.$$

In this form, $h(m)$ is a function

$$h(m) : \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h(m)(n)(o) = m + n + o,$$

which for fixed m returns the function "add m to the sum of two further arguments."

Def 6.17. Well-Defined Function: Let X, Y be sets. A function declaration $f : X \to Y$ together with a defining expression $E(x)$ is called *well-defined* iff for every $x \in X$:

- the expression $E(x)$ is meaningful (all variables are bound and the operations are valid), and
- the result $E(x)$ lies in Y.

Equivalently: a function is well-defined if its type annotation $f : X \to Y$ is consistent with the expression that defines it.

Checking well-definedness in practice:

The general definition above says that a function declaration must be consistent with its defining expression. Typical ways in which this can fail are:

- **Type mismatch:** the declared type $X \to Y$ does not agree with the actual input/output structure of the definition.
- **Free variables:** the definition uses symbols that are not bound by the input.
- **Partiality:** the definition does not provide an output for all inputs in the domain.

To illustrate, let us fix two functions

$$f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}, \qquad g : \mathbb{N} \to \mathbb{N},$$

and examine a series of possible definitions of functions h. Each case shows whether the definition is well-defined and why.

A correct curried definition

$$h : \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h(m)(n) = f(n, g(m)).$$

This is well-defined: for each $m \in \mathbb{N}$ we obtain $g(m) \in \mathbb{N}$, so $(n, g(m)) \in \mathbb{N} \times \mathbb{N}$ and $f(n, g(m)) \in \mathbb{N}$.

Incorrect composition

$$g \circ f : \mathbb{N} \to \mathbb{N}.$$

This is not well-defined with the given type. Since $f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$, the composition $g \circ f$ has domain $\mathbb{N} \times \mathbb{N}$, hence $g \circ f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$, not $\mathbb{N} \to \mathbb{N}$.

A correct nested definition

$$h : \mathbb{N} \to \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h(m)(n, o) = f(n, f(m, o)).$$

This is well-defined: $f(m, o) \in \mathbb{N}$, so $(n, f(m, o)) \in \mathbb{N} \times \mathbb{N}$, and thus $f(n, f(m, o)) \in \mathbb{N}$.

Type mismatch between curried and uncurried

$$h : \mathbb{N} \to \mathbb{N} \to \mathbb{N}, \quad h = f.$$

This is not well-defined, since $f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ has a different type from the declared type of h. Mathematically $f(x, y)$ and $f(x)(y)$ can be identified via currying, but in type-theoretic terms they are distinct unless an explicit curry operation is applied.

Free variable problem

$$h : \mathbb{N} \to \mathbb{N}, \quad h(x) = f(x, y).$$

This is not well-defined, since y is not bound. Without fixing y in advance, the right-hand side is not a function of x alone.

Partial definition problem

$$h : \mathbb{N} \times \mathbb{N} \to \mathbb{N}, \quad h(x, 1) = x.$$

This is not a total function, because the definition only specifies values when the second argument is 1. A function of type $\mathbb{N} \times \mathbb{N} \to \mathbb{N}$ must assign a value for every pair (x, y).

Def 6.18. Invertible: Let A and B be sets and $f : A \to B$ a function, then f is called invertible, iff there is a function $g : B \to A$, such that $f \circ g = \mathrm{Id}_B$. g is called the inverse function (or just inverse) of f and is written as f^{-1}.

Theorem 6.1. A function $f : A \to B$ is invertible, iff it is bijective.

Proof.

1. If f is bijective, then it is invertible:

- Assume $f : A \to B$ is bijective
- for each $b \in B$ there exists a unique $a \in A$ such that $f(a) = b$. Define $g : B \to A$ by sending b to this unique a. Therefore:
  1. $g \circ f = \mathrm{Id}_A$, because if you start with $a \in A$, apply f, then g, you get back a.
  2. $f \circ g = \mathrm{Id}_B$, because if you start with $b \in B$, apply g, then f, you get back b.
- Hence, g is the inverse of f. So f is invertible

2. If f is invertible, then it is bijective:

Suppose there exists a function $g : B \to A$ such that $g \circ f = \mathrm{Id}_A$ and $f \circ g = \mathrm{Id}_B$. We want to show f is bijective.

1. injectivity
   (a) Assume $f(a) = f(a')$.
   (b) Apply g to both sides:
   $$a = \mathrm{Id}_A(a) = g(f(a)) = g(f(a')) = a'$$
   (c) So, $a = a'$. Hence f is injective.
2. surjectivity
   (a) Take any $b \in B$. Then
   $$f(g(b)) = (f \circ g)(b) = \mathrm{Id}_B(b) = b$$
   (b) So b is indeed an image of f. Hence f is surjective.

□

Lemma 6.2. Let A and B be sets, then the set $\mathcal{R} = \mathcal{P}(A \times B)$ of all relations between A and B and the function space $A \to \mathcal{P}(B)$ are isomorphic.

Proof. For each $R \in \mathcal{R}$ and each $x \in A$ we define: $S_x^R := \{ y \in B \mid (x, y) \in R \}$ (the set of all y in B that are related to x by R). Example:

$$A = \{1, 2\} \quad B = \{a, b, c\} \quad R = \{(1, a), (1, c), (2, b)\}$$

Then for $x = 1 : S_1^R = \{a, c\}$ and for $x = 2 : S_2^R = \{b\}$.

We define a the following mapping:

$$\varphi : \mathcal{R} \to (A \to \mathcal{P}(B)), \quad \varphi(R) = f_R \qquad \text{where } f_R(x) = S_x^R.$$

For our example, this would be $\varphi(R) = f_R = \{1 \mapsto \{a, c\}, 2 \mapsto \{b\}\}$

Then, for any $f \in A \to \mathcal{P}(B)$, we compute the inverse as $\varphi^{-1}(f) = \{(x, y) \mid y \in f(x)\}$. This is exactly the relation corresponding to f.

Since both directions match up perfectly, φ is a bijection, hence an isomorphism. □

Example: the $>$ relation on $\mathbb{N}$ can also be represented as the function $f_>$ that maps a number n to the set $\{n + 1, \cdots\}$, i.e. $f_> : \mathbb{N} \to \mathcal{P}(\mathbb{N})\ ;\ n \mapsto \{ m \mid m > n \}$

Another example: let $A = \{x, y\}, B = \{1\}$, the cartesian product $A \times B = \{(x, 1), (y, 1)\}$. Hence, the power set (all possible relations) is:

$$\mathcal{P}(A \times B) = \{\{\phi\}, \{(x, 1)\}, \{(y, 1)\}, \{(x, 1), (y, 1)\}\}$$

Note: the cardinality of the power set of the cartesian product of two sets A, B is $\Gamma(\mathcal{P}(A \times B)) = 2^{\Gamma(A)\Gamma(B)}$.

$\mathcal{P}(B) = \{\{\phi\}, \{1\}\}$, function space $A \to \mathcal{P}(B)$ is the set of all functions $f : A \to \mathcal{P}(B)$:

$$A \to \mathcal{P}(B) = \{f_1, f_2, f_3, f_4\}$$

where:

- $f_1 = \{(x, \{\phi\}), (y, \{\phi\})\}$
- $f_2 = \{(x, \{\phi\}), (y, \{1\})\}$
- $f_3 = \{(x, \{1\}), (y, \{\phi\})\}$
- $f_4 = \{(x, (1)), (y, \{1\})\}$

Since $\Gamma(A \to \mathcal{P}(B)) = \Gamma(\mathcal{P}(A \times B))$, we can define a bijection:

- $\varphi(\{\phi\}) \mapsto f_1$
- $\varphi(\{(x, 1)\}) \mapsto f_3$
- $\varphi(\{(y, 1)\}) \mapsto f_2$
- $\varphi(\{(x, 1), (y, 1)\}) \mapsto f_4$

$\rightsquigarrow$ isomorphism.

Note

The bijection $\phi(R) = f$ entails the following properties of R:

- R is a total relation, iff $\phi \notin \mathrm{Im}(f)$
- R is a partial function, iff $\Gamma(S) \leq 1 \quad \forall S \in \mathrm{Im}(f)$
- $R = \phi$, iff $\mathrm{Im}(f) = \{\phi\}$
- R is reflexive, iff $x \in f(x) \quad \forall x \in A$

We have defined equivalence relations as reflexive, symmetric, and transitive relations, e.g., equality is an equivalence relation.

Let S be a set of persons, and $=_A \subseteq S^2$, such that $s =_A t$, iff s and t have the same age, then $=_A$ is an equivalence relation on S.

Let S and T be sets and $S \sim_\Gamma T$, iff there is a bijection $f : S \to T$, then $\sim_\Gamma$ is an equivalence relation. For example, let $S = \{1, 2, 3\} \quad T = \{a, b, c\}$. We can define a bijection $f = \{1 \mapsto a, 2 \mapsto b, 3 \mapsto c\}$, so, $S \sim_\Gamma T$ (they have the same size).

Lemma 6.3. If S is a set and $R \subseteq S^2$ is an equivalence relation, then $= \subseteq R$.

Proof Sketch: because of reflexivity ($\forall x \in S, (x, x) \in R$), the equality relation $=$ is exactly $\{(x, x) \mid x \in S\}$. Therefore, $= \subseteq R$.

7 Equivalence Classes

Def 7.1. Equivalence Classes and Quotients: Let S be a set and R be an equivalence relation on S, then for any $x \in S$ we call the set $[x]_R := \{ y \in S \mid R(x,y) \}$ the **equivalence class** of x under R, and the set $S/R := \{ [x]_R \mid x \in S \}$ the **quotient space** of S under R, it is often read as S *modulo* R. The element x is called the **representative** of $[x]_R \in S/R$.

Example: Let $S = \mathbb{Z}$ and R is the equivalence relation on S that describes the same parity (congruence modulo 2) $x \equiv y \pmod 2 \Leftrightarrow 2 \mid (x-y)$:

$$R = \{ (x,y) \in \mathbb{Z} \times \mathbb{Z} \mid x \equiv y \pmod 2 \}$$

- $(x,y) \in R$ if the difference $x - y$ is divisable by 2.
- i.e. either both are even, or both are odd.

All odd-odd pairs:

$$\{ (x,y) \mid x, y \in \{\cdots, -3, -1, 1, 1, 3, 5, \cdots\} \}$$

All even-even pairs:

$$\{ (x,y) \mid x, y \in \{\cdots, -4, -2, 0, 2, 4, 6, \cdots\} \}$$

Example pairs in R: $(1,3), (5,-1), (7,7), (2,4), (0,-6), (-8,10)$ Then, we have the following two distinct classes:

- $[1]_R = \{\cdots, -3, -1, 1, 1, 3, 5, \cdots\}$
- $[2]_R = \{\cdots, -4, -2, 0, 2, 4, 6, \cdots\}$

For each $x \in \mathbb{Z}$ we have:

- the set $[1]_R$ as the equivalence class if x is odd
- the set $[2]_R$ as the equivalence class if x is even
- 1 and 2 are the representatives

And the quotient space set of $\mathbb{Z}$ under R is:

$$\mathbb{Z}/R = \{[1]_R, [2]_R\} = \{\text{odds}, \text{evens}\}$$

Def 7.2. Canonical Projection: The mapping $\pi_R : S \to S/R \quad ; \quad x \mapsto [x]_R$ is called the **canonical projection** or **canonical surjection** of S to S/R.

Given our example, we define the following canonical projection:

$$\pi_R : \mathbb{Z} \to \{[1]_R, [2]_R\}, \quad x \mapsto [1]_R \text{ if odd, and } x \mapsto [2]_R \text{ if even.}$$

Def 7.3. System of Representatives: Let R be an equivalence relation on S, a subset $M \subseteq S$ is called a system of representatives, iff M contains exactly one representative for each equivalence class of R.

Given our example, a system of representatives is any subset of $\mathbb{Z}$ that picks exactly one element from each equivalence class, e.g.

- $M = \{1, 2\}$ is a system of representatives
- $M = \{-3, 0\}$ is a system of representatives
- $M = \{5, -8\}$ is a system of representatives

Observation: Often a quotient space S/R behaves similarly to the original set S.

Example: $n \equiv_k m \Leftrightarrow k \mid (n-m)$ is an equivalence relation and $\pi_{\equiv_k} : \mathbb{Z}/\equiv_k \to \{ n \mid 0 \leq n \leq (k-1) \}$ is bijective.

Another example, the general congruence modulo $n \equiv_k m \Leftrightarrow k \mid (n-m)$ partitions $\mathbb{Z}$ into exactly k equivalence classes (all numbers that leave the same remainder when divided by k). e.g. $k = 3$:

- $[0] = \{\cdots, -6, -3, 0, 3, 6, \cdots\}$
- $[1] = \{\cdots, -5, -2, 1, 4, 7, \cdots\}$
- $[2] = \{\cdots, -4, -1, 2, 5, 8, \cdots\}$

We have the canonical projection $\pi_{\equiv_k} : \mathbb{Z} \to \mathbb{Z}/\equiv_k, \quad n \mapsto [n]$ that sends each $n \in \mathbb{Z}$ to its equivalence class.

Notice that the quotient space $\mathbb{Z}/\equiv_k$ has exactly k classes, so, we have the following bijection:

$$\varphi : \mathbb{Z}/\equiv_k \to \{0, 1, \cdots, k-1\}, \quad [n] \mapsto \text{ the remainder of } n \text{ mod } k.$$

If we compose the canonical projection with the bijection we get the remainder, e.g. for $k = 3$:

$$-6 \xmapsto{\pi_{\equiv_3}} [0] \xmapsto{\varphi} 0.$$

$$7 \xmapsto{\pi_{\equiv_3}} [1] \xmapsto{\varphi} 1.$$

Lemma 7.1. Let $= \subseteq S \times S$ be the equality relation on a set S, then $[x]_= \in S/=$ is always a **singleton** and thus $S \sim_\Gamma S/=$.

Proof. If we take the equality relation on a set S, then for any $x \in S$, the equivalence class is:

$$[x]_= = \{\, y \in S \mid y = x \,\}$$

But the only y that satisfies $y = x$ is $y = x$ itself. So we get $[x]_= = \{x\}$. This is a **singleton** set. So every equivalence class under $=$ has exactly one element.

The quotient space is $S/= \; := \{\, [x]_= \mid x \in S \,\}$ and each $[x]_=$ is the singleton set $\{x\}$, so the quotient space becomes $S/= := \{\, \{x\} \mid x \in S \,\}$. In other words, the quotient space under the equality relation just replaces each element x with one-element set $\{x\}$.

That means the set S is in bijection with its quotient space under $=$ via the following bijective map:

$$\phi : S \to S/=, \quad x \mapsto [x]_= = \{x\}$$

Therefore, we express the bijection via same cardinality:

$$S \sim_\Gamma S/=$$

□

Fun Example: Let S be the set of all cities worldwide and $\sim_c \in S^2$ defined by $a \sim_c b$, iff a and b are in the same country. Then $\sim_c$ is an equivalence relation on S, and the quotient space $S/\sim_c$ of S under $\sim_c$ consists of the equivalence class $[x]_{\sim_c}$ where $x \in S$. The capital of a country is a good representative for an equivalence class.

For instance: let's take a subset of all worldwide cities for simplicity, consider domain of discourse:

$$S = \{\text{Berlin}, \text{Munich}, \text{Hamburg}, \text{Paris}, \text{Lyon}, \text{Marseille}, \text{Rome}, \text{Milan}\}.$$

For $a, b \in S$ define the relation:

$$a \sim_c b \iff a \text{ and } b \text{ are in the same country.}$$

This relation is:

- reflexive: any city is in the same country as itself.
- symmetric: if Berlin is in the same country as Munich, then Munich is in the same country as Berlin.
- transitive: if Berlin and Munich are in the same country, and Munich and Hamburg are in the same country, then Berlin and Hamburg are in the same country.

Hence, $\sim_c$ is an equivalence relation.

The quotient space $S/\sim_c$ is the set of equivalence classes, i.e. the partition of S into countries:

$$[\text{Berlin}]_{\sim_c} = \{\text{Berlin}, \text{Munich}, \text{Hamburg}\}, \quad [\text{Paris}]_{\sim_c} = \{\text{Paris}, \text{Lyon}, \text{Marseille}\}, \quad [\text{Rome}]_{\sim_c} = \{\text{Rome}, \text{Milan}\}.$$

So

$$S/\sim_c = \Big\{\{\text{Berlin}, \text{Munich}, \text{Hamburg}\},\ \{\text{Paris}, \text{Lyon}, \text{Marseille}\},\ \{\text{Rome}, \text{Milan}\}\Big\}.$$

The set of chosen representatives is: $\{\text{Berlin}, \text{Paris}, \text{Rome}\}$.

8 Functional Programming using SML

Def 8.1. Functional Programming: We call programming languages where procedures can be fully describes in terms of their input/output behabior **functional**.

> Note
>
> There are two main programming paradigms: **imperative** and **declarative**. Imperative programming is where the programmer instructs the machine how to change its state, it can be:
> - structured programming: makes extensive use of block structures and subroutines
> - procedural programming: groups instructions into procedures
> - object-oriented programming: groups instructions together with the part of the state they operate on
>
> On the other hands, declarative programming is where the programmer merely declares properties of the desired result, but not how to compute it. Examples:
> - functional programming: the desired result is declared as the value of a series of function calls
> - logic programming: the desired result is declared as the answer to a question about a system of facts and rules
> - mathematical programming: the desired result is declared as the solution of an optimization problem

Def 8.2. Standard ML (SML): SML is general-purpose, high-level, modular, function programming language with compile-time type checking and type inference.

Def 8.3. Predefined objects in SML:

- **basic types**, e.g. `int`, `real`, `bool`, `string`, ...
- **basic type constructors**, e.g. `->`, `*`, ...
- **basic operators**, e.g. `numbers`, `true`, `false`, `+`, `-`, `*`, `-`, `>`, `^` ...
- **control structures**, e.g. `if` Φ `then` E_1 `else` E_2;
- **comments**, e.g. `(* this is a comment *)`

A simple example, if we write `2;` we get back `val it = 2 : int`. and if we write `4.711;` we get back `val it = 4.711 : real`. In other words, SML interpreter has determined that the input has a value and that value has a specific type.

Def 8.4. Declarations bind variables:

- **value declarations**, e.g. `val pi = 3.1415;`
- **type declarations**, e.g. `type twovec = int * int;`
- **function declarations**, e.g. `fun square (x:real) = x * x;`

A function declaration only declares the function name as a globally visible name. The **formal parameters** in brackets are only visible in the **function body**.

Alternatively, we can just declare a function object, e.g. via `val foo = fn x => x + x;`.

SML functions that have been declared can be applied to arguments of the right type, e.g. `square 4.0` which evaluates to `4.0 * 4.0` and this to `16.0`.

```
- fun square(x:real) = x * x; (*function declaration*)
```

```
val square = fn : real -> real (*interpreter response*)
```

```
- square 4.0; (*function application*)
```

```
val it = 16 : real (*interpreter response*)
```

Def 8.5. Local Declaration: A local declaration uses `let` to bind variables in its scope (delineated by `in` and `end`).

> Note
>
> Local declarations can **shadow** existing variables. i.e. it hides the old binding in its scope.

Example:

```
- val test = 4;
```

`val test = 4 : int`

`- let val test = 7 in test * test end;`

`val it = 49 : int`

`- test;`

`val it = 4 : int`

> **Note**
>
> In imperative programming, variable are identifiers for memory locations that can be changed at will. However, in functional programming, variables are bound to values in a given scope, that value cannot be changed in the scope. i.e. variable binding in functional programming is a dynamic mapping from variable names to immutable memory locations, which can be locally overwritten (shadowing).

Def 8.6. Pattern Matching: Pattern matching is using special expressions with unbound variables (we call them structured patterns) to declare more than one variable.

For example:

`- val unitvector = (1,1);`

`val unitvector = (1,1): int * int`

`- val (x,y) = unitvector;`

`val x = 1 : int`

`val y = 1 : int`

Def 8.7. Anonymous Variables: Variables without a name, denoted by "underscore" (_).

e.g. if we are not interested in one if the values: `- val (x,_) = unitvector` $\rightsquigarrow$ `val x = 1 : int`. Similarly, we can define a function that takes a pair and select the first element:

`- fun first(p) = let val (x,_) = p in x end;` $\rightsquigarrow$ `val first = fn : 'a * 'b -> 'a`. Example usage:

```
- first (3,"hello");
val it = 3 : int
```

A selector of the second element:

```
- fun second(p) = let val (_,x) = p in x end;
val second = fn : 'a * 'b -> 'b
- second (3,"Welcome to SML");
val it = "Welcome to SML" : string
```

> **Note**
>
> `'a`, `'b`, ... are called type variables (SML supports universal types).

So the `first` function signature (`fn : 'a * 'b -> 'a`) means the function takes a pair of type `'a * 'b` as input and returns an object of type `'a`.

SML has a built-in "list type". Given a type ⟨⟨ `ty` ⟩⟩, `list` ⟨⟨ `ty` ⟩⟩ is also a type (a list of objects of type ⟨⟨ `ty` ⟩⟩). For example, `list int` is a type for lists of integers.

```
- [1,2,3];
val it = [1,2,3] : int list
```

Def 8.8. Lists Constructors: Lists have two constructors: `nil` (empty list) and `::` (called `cons`).

```
- nil;
val it = [] : 'a list
- 9::nil;
val it = [9] : int list
```

Def 8.9. Recursion: We call an SML function recursive, iff the function is called in the body of the function declaration.

Recursion example:

```
- fun upto(m,n) = if m>n then nil else m::upto(m+1,n);
vap upto = fn : int * int -> int list
upto (2,5);
val it = [2,3,4,5] : int list
```

Let's trace that function:

```
upto (2,5) = if 2>5 then nil else 2::upto(3,5)
upto(3,5) = if 3>5 then nil else 3::upto(4,5)
upto(4,5) = if 4>5 then nil else 4::upto(5,5)
upto(5,5) = if 5>5 then nil else 5::upto(6,5)
upto(6,5) = if 6>5 then nil else 6::upto(7,5)
```

so it will evaluate to: `2::3::4::5::nil`

Another Example, consider we want a function that sums all the elements in a list. We also do that via recursion, if we get an empty list, we return 0, if we get a list with elements (a head element, and tail of the rest) then we sum the head to the sum of the tail.

```
- fun sumlist nil = 0 | sumlist (h::t) = h + sumlist t;
val sumlist = fn : int list -> int
- sumlist ([1,4,6]);
val it = 11 : int
- sumlist ([]);
val it = 0 : int
```

> **Note**
>
> Every *infix* operator in SML (+,-,...) has a *function style* (`op+`, `op-`,...) notation.

For example, we can write the summation using that notation in our `sumlist` function:

```
- fun sumlist nil = 0 | sumlist (h::t) = op+ (h, sumlist t);
val sumlist = fn : int list -> int

- sumlist ([1,2,3]);
val it = 6 : int
```

SML have other *function style* operations, e.g.:

```
(op-) (7, 4);            (* 3 *)
(op*) (6, 5);            (* 30 *)
(op div) (10, 3);        (* 3 *)
(op mod) (10, 3);        (* 1 *)
(op^) ("hi", "there");   (* "hithere" *)
(op::) (1, [2,3]);       (* [1,2,3] *)
(op@) ([1,2], [3,4]);    (* [1,2,3,4] *)
(op=) (3, 3);            (* true *)
(op=) ("a", "b");        (* false *)
(op<>) (3, 4);           (* true *)
(op<>) (5, 5);           (* false *)
(op<)  (2, 5);           (* true  *)
(op<=) (5, 5);           (* true  *)
(op>)  (7, 3);           (* true  *)
(op>=) (3, 4);           (* false *)
```

Evaluation in SML is **call-by-value** i.e. to whenever we encounter a function applied to arguments, we compute the value of the argument first. We write $t_1 \rightsquigarrow t_2 \rightsquigarrow \cdots \rightsquigarrow t_n$ for tracing the recursive arguments t_i through a recursive computation. For example:

$$\text{upto}\,(2,4) \rightsquigarrow 2 :: \text{upto}\,(3,4) \rightsquigarrow 2 :: (3 :: \text{upto}\,(4,4)) \rightsquigarrow 2 :: (3 :: (4 :: \text{upto}\,(5,4))) \rightsquigarrow 2 :: (3 :: (4 :: \text{nil}))$$

Note

Recursive functions need not to terminate. For example: `fun diverges (n) = n + diverges (n+1)` which evaluates to: $\text{diverges}(1) \rightsquigarrow 1 + \text{diverges}(2) \rightsquigarrow 1 + (2 + \text{diverges}(3)) \rightsquigarrow \cdots$

Examples:

Length of a list:

```
- fun length nil = 0 | length (_::t) = 1 + length t;
val length = fn : 'a list -> int
- length ([1,2,3,4,5]);
val it = 5 : int
```

Fibonacci:

```
- fun fib 0 = 0
= | fib 1 = 1
= | fib n = fib (n-1) + fib (n-2);
val fib = fn : int -> int

- fib 6;
val it = 8 : int
```

Factorial:

```
- fun fact 0 = 1 | fact n = n * fact(n-1);
val fact = fn : int -> int
- fact 3
val it = 6 : int
```

Note that each recursive call has to remember the multiplication to do later. The final result waits until `fact(0)` is evaluated. However, there is another recursion technique called **tail recursion** where we accumulate the result as we go from recursive to another using a helper variable (accumulator) and function. For example, here is the same factorial function using tail recursion:

```
- fun facthelper (0, acc) = acc | facthelper (k,acc) = facthelper (k-1, k * acc);
val facthelper = fn : int * int -> int
- fun fact n = facthelper (n,1);
val fact = fn : int -> int
- fact 3;
val it = 6 : int
```

Let's track that: $\text{fact}(3) \rightsquigarrow \text{facthelper}(3,1) \rightsquigarrow \text{facthelper}(2,3) \rightsquigarrow \text{facthelper}(1,6) \rightsquigarrow \text{facthelper}(0,6) \rightsquigarrow 6$

Def 8.10. Tail Recursion: A tail call is a subroutine call performed as the final action of a routine R. If the call is a recursive call, R is said to be tail recursive.

Def 8.11. Append `@`: The operator `@` appends two lists together, e.g. `- [1]@[2];` gives us `val it = [1,2] : int list`

But we can define our own append recursively:

```
- fun append (nil,y) = y
    | append (h::t, y) = h::append (t,y);

val append = fn : 'a list * 'a list -> 'a list

- append ([1,2,3], [4,5,6]);
val it = [1,2,3,4,5,6] : int list
```

Reversing a list:

```
- fun reverse nil = nil
  |   reverse (h::t) = reverse t @ [h];

val reverse = fn : 'a list -> 'a list
```

```
- reverse ([1,2,3,4,5,6,7]);
val it = [7,6,5,4,3,2,1] : int list
```

Membership Check:

```
- fun member (_, nil) = false
    | member (x, h::t) =
      if x = h then true
      else member (x,t);

val member = fn : ''a * ''a list -> bool

- member (4,[1,2,3,4,5]);
val it = true : bool

- member (4,[1,2,3,5]);
val it = false : bool
```

Reverse a list and append it to another list:

```
- fun reverse (nil,y) = y
    | reverse (h::t, y) = reverse (t, h::y);

val reverse = fn : 'a list * 'a list -> 'a list

- reverse ([4,5,6], [3,2,1]);
val it = [6,5,4,3,2,1] : int list
```

Let's trace that with `[2,3], [1]`: $\text{reverse}([2,3],[1]) \rightsquigarrow \text{reverse}([3],[2,1]) \rightsquigarrow \text{reverse}([],[3,2,1]) \rightsquigarrow [3,2,1]$

Flattening a list of lists to one list

```
- fun flat nil = nil
    | flat (h::t) = h @ flat t;

val flat = fn : 'a list list -> 'a list

- flat ([["when", "shall"], ["we"], ["meet", "again"]]);
val it = ["when","shall","we","meet","again"] : string list
```

Remove First Occurrence:

Here is a function that takes an element and a list and give the list without the first occurrence of that element:

```
- fun remove_first (x,[]) = []
    | remove_first (x, h::t) =
        if x = h then t else h::remove_first (x,t);

val remove_first = fn : ''a * ''a list -> ''a list

- remove_first (1,[2,3,1,4,1]);
val it = [2,3,4,1] : int list
```

Remove All Occurrences:

```
- fun remove_all (x,[]) = []
    | remove_all (x, h::t) =
        if x = h then remove_all (x,t) else h::remove_all(x,t);

val remove_all = fn : ''a * ''a list -> ''a list

- remove_all (1,[2,3,1,4,1]);
val it = [2,3,4] : int list
```

Count All Occurrences

```
- fun count _ [] = 0
    | count x (h::t) = (if x = h then 1 else 0) + count x t;

val count = fn : ''a -> ''a list -> int

- count 1 [1,2,3,1,4,1,1,5];
val it = 4 : int
```

Sort List:

```
- fun sort [] = []
    | sort (h::t) =
        let
            fun place (x, []) = [x]
              | place (x, h::t) =
                if x <= h then x :: h :: t else h :: place (x, t)
        in
            place (h, sort t)
        end;

val sort = fn : int list -> int list

- sort [4,5,2,3,8,7,6,1,0];
val it = [0,1,2,3,4,5,6,7,8] : int list
```

Let's trace that on [2,1,3]:

sort [2,1,3] $\rightsquigarrow$ place (2, sort [1,3]) $\rightsquigarrow$ place (2, place (1, sort [3])) $\rightsquigarrow$ place (2, place (1, place (3, sort []))) $\rightsquigarrow$

place (2, place (1, place (3, []))) $\rightsquigarrow$ place (2, place (1, [3])) $\rightsquigarrow$ place (2, [1,3]) $\rightsquigarrow$ 1::place(2, [3]) $\rightsquigarrow$ 1::[2,3] $\rightsquigarrow$ [1,2,3]

Merge two Sorted Lists:

```
- fun merge ([],b) = b
    | merge (a, []) = a
    | merge (h1::t1, h2::t2) =
        if h1 <= h2 then h1::merge (t1, h2::t2)
        else h2::merge (h1::t1, t2);

val merge = fn : int list * int list -> int list

- merge ([1,3,5], [2,4,6]);
val it = [1,2,3,4,5,6] : int list
```

Def 8.12. SML Strings: SML treats strings as lists of characters. We can go from `string` to `chat list` and vice versa via `explode` and `implode`.

For example:

```
- explode "Ibrahim";
val it = [#"I",#"b",#"r",#"a",#"h",#"i",#"m"] : char list
- implode it;
val it = "Ibrahim" : string
```

Using that and our reverse function, we can define a function that checks if a string is *palindrome* or not (read the same from either side, e.g. otto, anna, etc.)

```
- fun reverse nil = nil
    | reverse (h::t) = reverse t @ [h];

val reverse = fn : 'a list -> 'a list

- fun palindrome s =
    let
        val cs = explode s
    in
```

```
        cs = reverse cs
    end;

val palindrome = fn : string -> bool

- palindrome "apa";
val it = true : bool
- palindrome "apaa";
val it = false : bool
```

Def 8.13. Short-Circuit Operators: SML has built-in short-circuit operators. `andalso` is the logical AND and `orelse` is the logical OR. These behave like `&&` and `||` in C/Java.

```
- true andalso false;
val it = false : bool

- true orelse false;
val it = true : bool
```

Def 8.14. Higher-Order Functions: In SML, we call functions that take functions as arguments **higher-order functions** and those that do not **first-order functions**

For example, the built-in higher-order function `map` that takes a functions and a list and returns a list with the passed functions applied to each element of the list.

```
- fun f x = x + 1;
val f = fn : int -> int
- map f [1,2,3];
val it = [2,3,4] : int list
```

Note: we can program our own `map` function recursively:

```
- fun mymap (f, nil) = nil
    | mymap (f, h::t) = f(h)::mymap (f,t);
val mymap = fn : ('a -> 'b) * 'a list -> 'b list

- mymap (f,[1,2,3]);
val it = [2,3,4] : int list
```

Selective Copy:

We would like to make a function that takes a list and give us copy of the list with, for instance, only the even numbers. First, we define an `is_odd` function:

```
- fun is_odd x = (x mod 2) = 1;
val is_odd = fn : int -> bool
- is_odd 3;
val it = true : bool
- is_odd 2;
val it = false : bool
```

Then we define a function `remove_if` that takes two arguments, one is the filter (a function as an argument) (here it is our `is_odd`), and a list which we hope to filter out.

```
- fun remove_if filter [] = []
    | remove_if filter (h::t) =
        if filter(h) then remove_if filter t
        else h::(remove_if filter t);

val remove_if = fn : ('a -> bool) -> 'a list -> 'a list

- remove_if is_odd [1,2,3,4,5,6,7,8,9];
val it = [2,4,6,8] : int list
```

Def 8.15. Function Literal: A function literal (also called **anonymous function**) is a function definition that is not bound to an identifier.

For example, here we pass to the `map` function an anonymous function and a list:

```
- map (fn x => x * x) [2,3,4];
val it = [4,9,16] : int list
```

Note: we can also give an anonymous function a name (treated as value declaration) and call it, e.g.:

```
- val identity = fn x => x;
val identity = fn : 'a -> 'a
- identity (5);
val it = 5 : int
```

Given that functions can be passed as arguments, they can be returned as well, e.g., here is a functions `addconst` that takes an argument k and returns another functions that adds k to its argument.

```
- val addconst = fn k => (fn x => x + k);
val addconst = fn : int -> int -> int
- (addconst 4) 5;
val it = 9 : int
```

Def 8.16. Cartesian vs Cascaded Function: We call a function **cartesian**, iff the *argument* type is a *product* type, we call it **cascaded**, iff the *result* type is a *function* type. (uncurried vs curried)

For example, a cartesian plus function would look like:

```
- fun cartesian_plus (x:int, y:int) = x + y;
val cartesian_plus = fn : int * int -> int
- cartesian_plus (3,2);
val it = 5 : int
```

However, a cascaded plus function would look like:

```
- fun cascaded_plus (x:int) = (fn y:int => x + y);
val cascaded_plus = fn : int -> int -> int
- (cascaded_plus 3) 2;
val it = 5 : int
```

The following function is both cascaded and cartesian:

```
- fun both_plus (x:int, y:int) = fn (z:int) => x+y+z;
val both_plus = fn : int * int -> int -> int
- (both_plus(2,3)) 5;
val it = 10 : int
```

Note

Function application always associates to the left, and function types always associate to the right. Usually we omit brackets for convenience, hence associativity must be implied.

- $\text{both_plus}(2,3)5 \rightsquigarrow (\text{both_plus}(2,3))5$
- $\text{int} \to \text{int} \to \text{int} \rightsquigarrow \text{int} \to (\text{int} \to \text{int})$

Def 8.17. Left Folding Operator: SML provides the left folding operator to realize a recurrent computation schema. It is a higher-order function that:

- takes a function `f : 'a * 'b -> 'b`
- takes a starting value `s : 'b`
- takes a list `[x1, ... xn] : 'a list`
- and reduces that list into a single value by applying `f` from the left

The general form of the foldl operator:

```
foldl f s [x1, x2, x3, ..., xn]
= f(xn, f(x(n-1), ... f(x1, s)...))
```

We can think of the foldl operator as recursive list processing. For example, remember our `sumlist` that we wrote recursively:

```
- fun sumlist nil = 0 | sumlist (h::t) = h + sumlist t;
```

instead of implementing a recursive function, we can foldl the summation operator (`op+`) and give it `s=0`, and our list to sum:

```
- foldl op+ 0 [1,2,3];
val it = 6 : int
```

This is what is going on under the hood:

```
- op+ (3, op+ (2, op+ (1, 0)));
val it = 6 : int
```

We also can use value declaration to declare a general summation function that works for any list using the form `val n = foldl f s` where `n` is the name that we can call. For example:

```
- val sum = foldl op+ 0;
val sum = fn : int list -> int
- sum [1,2,3,4];
val it = 10 : int
- sum [];
val it = 0 : int
```

Another Example: Remember the recursive function we wrote to get the length of a list:

```
- fun length nil = 0 | length (_::t) = 1 + length t;
```

With foldl, we can just write the following:

```
foldl (fn (_, acc) => acc + 1) 0 [10,20,30]
```

It will evaluate to: `fn (30,fn (20,fn (10,0)))` $\rightsquigarrow$ `fn (30,fn (20,1))` $\rightsquigarrow$ `fn (30,2)` $\rightsquigarrow$ `3`.

The type of foldl is: `foldl : ('a * 'b -> 'b) -> 'b -> 'a list -> 'b`. i.e. First argument is a function `f` of type `'a * 'b -> 'b`. In the previous example, that function was `fn (_, acc) => acc + 1`:

```
- fn (_, acc) => acc + 1;
val it = fn : 'a * int -> int
```

Second argument is a start value of type `'b`, in our case it is `0` of type `int`. The third argument is a list of type `'a list`. Finally, it returns a result of type `'b`.

Another Example: remember how we wrote a recursive function to reverse a list:

```
- fun reverse nil = nil | reverse (h::t) = reverse t @ [h];
val reverse = fn : 'a list -> 'a list
- reverse [1,2,3,4,5];
val it = [5,4,3,2,1] : int list
```

We can instead foldl the `op::` (cons) operator:

```
- foldl op:: nil [1,2,3,4,5];
val it = [5,4,3,2,1] : int list
```

It will evaluate to: `op::(5,op::(4,op::(3,op::(2,op::(1,nil)))))` $\rightsquigarrow$

- `op::(5,op::(4,op::(3,op::(2,[1]))))` $\rightsquigarrow$
- `op::(5,op::(4,op::(3,[2,1])))` $\rightsquigarrow$
- `op::(5,op::(4,[3,2,1]))` $\rightsquigarrow$
- `op::(5,[4,3,2,1])` $\rightsquigarrow$
- `[5,4,3,2,1]`

Check whether a number is prime

Def 8.18. Prime Numbers: An integer $p \in \mathbb{Z}$ is prime if $p > 1$ and the only positive divisors of p are 1 and p itself. Examples:

- 2 is prime because its divisors are $\{1, 2\}$
- 7 is prime because its divisors are $\{1, 7\}$
- 9 is not prime because its divisors are $\{1, 3, 9\}$

The infinite set of primes is $\{2, 3, 5, 7, 11, 13, \cdots\}$

Def 8.19. Composite Numbers: A positive number $n > 1$ is called composite iff it is not prime. 1 is neither prime nor composite by defintiion.

To check if a number n is prime we need to

- check if $n >= 2$, otherwise it is 1 which is not prime.
- if $n = 2$, then it is prime.
- for all $n > 2$ we need to see if there exists a number d between 2 and $n - 1$ that divides n, if we find one, then n is not prime. If we don't find any, then n is prime.

For example, consider $n = 5$, it does not divide any number in $[2 - 4]$ meaning the only divisors are 1 and 5, $\rightsquigarrow$ prime. However, 6 divides $3, 2 \in [2 - 5] \rightsquigarrow$ not prime.

To implement that recursively, we need a main function (`prime n`) and a helper one (`PrimeHelper (n,d)`). `prime n` returns `false` if $n = 1$, otherwise it calls `PrimeHelper (n,2)`. `PrimeHelper (n,d)` has two base cases:

- if $n = d$ (i.e. $n = 2$) $\rightsquigarrow$ `true`
- if $n \bmod d = 0 \rightsquigarrow$ `false` (we found a d such that, n is divisible by d)

otherwise, we recursively call `PrimeHelper (n,d+1)`.

```
- fun PrimeHelper (n,d) =
    if d = n then true
    else if n mod d = 0 then false
    else PrimeHelper (n,d+1);

val PrimeHelper = fn : int * int -> bool

- fun prime n =
    if n<2 then false
    else PrimeHelper (n,2);

val prime = fn : int -> bool

- prime 3;
val it = true : bool
- prime 4;
val it = false : bool
- prime 9;
val it = false : bool
- prime 11;
val it = true : bool
```

Let's trace: $\text{prime}\, 9 \rightsquigarrow \text{PrimeHelper}(9, 2) \rightsquigarrow \text{PrimeHelper}(9, 3) \rightsquigarrow \text{false}$

Check if a list has a prime number:

We use our `prime n` function and the `foldl` operator. We will foldl the list with the function `fn (a,b) => (prime a) orelse b` starting with `false` as the starting value.

```
- fun foo l = foldl (fn (a,b) => (prime a) orelse b) false l;
val foo = fn : int list -> bool
- foo [4,2,6,8];
val it = true : bool
- foo [4,6,8,10];
val it = false : bool
```

Let's trace:

```
foldl f false [4,2,6,8]
= f(8, f(6, f(2, f(4, false))))
= f(8, f(6, f(2, ((prime 4) orelse false))))
= f(8, f(6, f(2, false)))
```

```
= f(8, f(6, ((prime 2) orelse false)))
= f(8, f(6, true))
= f(8, ((prime 6) orelse true))
= f(8, true)
= (prime 8) orelse true
= true
```

Def 8.20. Right Folding Operator: SML provides the right folding operator to realize a recurrent computation schema. It is a higher-order function that:

- takes a function `f : 'a * 'b -> 'b`
- takes a starting value `s : 'b`
- takes a list `[x1, ... xn] : 'a list`
- and reduces that list into a single value by applying `f` from the right

The general form of the foldr operator:

```
foldl f s [x1, x2, x3, ..., xn]
= f(x1, f(x2, ... f(xn, s)...))
```

For example: `foldr f s [x1, x2, x3, x4] = f(x1, f(x2, f(x3, f(x4, s))))`.

A useful usecase would be to foldr the cons operator (`op::`) which gives us an append function of two lists. We previously wrote an append function recursively:

```
- fun append (nil,y) = y | append (h::t, y) = h::append(t,y);
val append = fn : 'a list * 'a list -> 'a list
- append ([1,2],[3,4]);
val it = [1,2,3,4] : int list
```

We give the recursive function a list x and a list y and we get back a new list wih y appended to x. If we foldr the cons operator, then we give first the list y, then x to get our desired result:

```
- foldr op:: [3,4] [1,2];
val it = [1,2,3,4] : int list
```

It will evaluate to: `op::(1,op::(2, [3,4]))`

Def 8.21. Defining Equations: A set of equations $F(x) = \Phi_i(F, x)$ where F and x are variables and $\Phi_i(F, x)$ an expression in F and x is called defining for a function f if f solves them simultaneously for all x.

For example, here is a defining equation system for unary addition:

```
+(n, 0)      = n
+(m, s(n))   = s(+(m, n))
```

Theorem 8.1. $+ \subseteq \mathbb{N}_1 \times \mathbb{N}_1 \times \mathbb{N}_1$ is a total function.

Lemma 8.2. $\forall (n, m) \in \mathbb{N}_1 \times \mathbb{N}_1$ there is exactly one $l \in \mathbb{N}_1$ with $+(n, m) = l$.

Proof. By induction on m. We have two cases:

1. Base case ($m = 0$)
   - Choose $l := n$, so we have $+(n, 0) = n = l$
   - For any $l' = +(n, 0)$ we have $l' = n = l$
2. Induction step ($m = s(k)$)
   - Assume that there is a unique $r = +(n, k)$, choose $l := s(r)$, so we have $+(n, s(k)) = s(+(n, k)) = s(r)$
   - Again, for any $l' = +(n, s(k))$ we have $l' = l$

□

Corollary 8.3. $+ : \mathbb{N}_1 \times \mathbb{N}_1 \to \mathbb{N}_1$ is a total function

More Examples:

- the equations: $\delta(0) = 0$ and $\delta(s(n)) = s(s(\delta(n)))$ are defining equations for the doubling function.

- the equations: $\mu(l,0) = 0$ and $\mu(l, s(r)) = +(\mu(l,r), l)$ are defining equations for the multiplication function.
- the equations; $i(a, A, B) = A$ and $i(b, A, B) = B$ where A, B are unary natural number are defining equations for the `if-then-else` function on unary natural number where a represents *true* and b is *false*.
- the equations: $\mu(0) = 0$, $\mu(s(0)) = 0$, and $\mu(s(s(n))) = s(n)$ are defining equations for the function μ on unary natural number where μ is the `lower halving` function $\mu : n \mapsto \lfloor \frac{n}{2} \rfloor$.

Def 8.22. Inductively Defined Sets: An inductively defined set $\langle S, C \rangle$ is a set S together with a finite set $C := \{ c_i \mid 1 \leq i \leq n \}$ of k_i-ary constructors $c_i : S^{k_i} \to S$ with $k_i \geq 0$, such that:

- if $s_i \in S \quad \forall 1 \leq i \leq k_i$, then $c_i(s_1, \cdots, s_{k_i}) \in S$ (*generated by constructors*)
- all constructors are injective (*no internal confusion*)
- $\mathrm{Im}(c_i) \cap \mathrm{Im}(c_j) = \phi$ for $i \neq j$ (*no inter-constructor confusion*)
- for every $s \in S$ there is a constructor $c \in C$ with $s \in \mathrm{Im}(c)$ (*no junk*)
- If $k_i = 0$, it is a nullary constructor (a constant element).

Example: $\langle \mathbb{N}_1, \{s, 0\} \rangle$ is an inductively defined set.

Proof.

1. Generated by constructors:

This is guranteed by $P1$ and $P2$. $0 \in \mathbb{N}_1$ and if $n \in \mathbb{N}_1$ then $s(n) \in \mathbb{N}_1$.

2. No internal confusion: $c_i(s_1, \cdots, s_{k_i}) = c_i(t_1, \cdots, t_{k_i}) \Rightarrow s_j = t_j \forall j$.

$P4$ gurantees this. If $s(n) = s(m)$ then $n = m$

3. No inter-constructor confusion:

$P3$ asserts that. Different constructors never produce the same element. The element 0 is not the image s, nothing is both a zero and a successor.

4. No junk is assured by $P5$: $\forall s \in S, \exists c \in C$ such that $s \in \mathrm{Im}(c)$. i.e. every element of S comes from some constructor. There are no extra elements that didn't come from constructors. □

We can define Lists of natural numbers $\mathcal{L}[\mathbb{N}]$ inductively as well. Let the `nil`-rule be starting with the empty list $[]$, and the `cons`-rule extends the list by adding a number $n \in \mathbb{N}$ at the front. For example $\mathrm{cons}(3, \mathrm{cons}(2, \mathrm{cons}(1, \mathrm{nil}))) = [3, 2, 1]$ and $[] = \mathrm{nil}$

Def 8.23. List Piano Axioms: We call the following set of axioms *list Peano axioms* for $\mathcal{L}[\mathbb{N}]$ in analogy to the Peano Axioms in Def 3.2.

- **LP1**: $\mathrm{nil} \in \mathcal{L}[\mathbb{N}]$.
- **LP2**: $\mathrm{cons} : \mathbb{N} \times \mathcal{L}[\mathbb{N}] \to \mathcal{L}[\mathbb{N}]$.
- **LP3**: `nil` is not a `cons`-value.
- **LP4**: `cons` is injective.
- **LP5**:
  1. **Base Case**: `nil` has property P. ($P(\mathrm{nil})$)
  2. **Inductive Step**: If a list l has property P, then for every natural number $n \in \mathbb{N}$, the list $\mathrm{cons}(n, l)$ also has property P. i.e.
  $$P(l) \Rightarrow P(\mathrm{cons}(n, l)) \quad \forall n \in \mathbb{N}, \forall l \in \mathcal{L}[\mathbb{N}]$$
  3. **Conclusion**: $P(l) \quad \forall l \in \mathcal{L}[\mathbb{N}]$
  $$(P(\mathrm{nil}) \wedge \forall l \in \mathcal{L}[\mathbb{N}], \forall n \in \mathbb{N}, (P(l) \Rightarrow P(\mathrm{cons}(n, l)))) \Rightarrow \forall l \in \mathcal{L}[\mathbb{N}], P(l)$$

Example (Length of a List). We define the length function on lists as follows:

$$\mathrm{len}(nil) = 0, \qquad \mathrm{len}(\mathrm{cons}(n, l)) = 1 + \mathrm{len}(l).$$

We want to prove

$$\forall l \in \mathcal{L}[\mathbb{N}],\ \mathrm{len}(l) \in \mathbb{N}.$$

Proof. We proceed by list induction (Axiom LP5).

Base Case: Let $l = nil$. Then

$$\text{len}(nil) = 0 \in \mathbb{N}.$$

Thus the base case holds.

Inductive Step: Assume $\text{len}(l) \in \mathbb{N}$ for some $l \in \mathcal{L}[\mathbb{N}]$ (induction hypothesis). We need to show that $\text{len}(\text{cons}(n, l)) \in \mathbb{N}$ for any $n \in \mathbb{N}$. Indeed,

$$\text{len}(\text{cons}(n, l)) = 1 + \text{len}(l).$$

By the induction hypothesis, $\text{len}(l) \in \mathbb{N}$. Since $\mathbb{N}$ is closed under addition, we have $1 + \text{len}(l) \in \mathbb{N}$.

Conclusion: By the principle of list induction, it follows that

$$\forall l \in \mathcal{L}[\mathbb{N}], \quad \text{len}(l) \in \mathbb{N}.$$

□

Example (Double Reversal). We define the reverse function on lists as follows:

$$\begin{aligned} \text{rev}(nil) &= nil, \\ \text{rev}(\text{cons}(n, l)) &= \text{append}(\text{rev}(l), \text{cons}(n, nil)), \end{aligned}$$

where append is the usual concatenation of two lists.

We want to prove

$$\forall l \in \mathcal{L}[\mathbb{N}], \quad \text{rev}(\text{rev}(l)) = l.$$

Proof. We proceed by list induction (Axiom LP5).

Base Case: Let $l = nil$. Then

$$\text{rev}(\text{rev}(nil)) = \text{rev}(nil) = nil,$$

so the base case holds.

Inductive Step: Assume $\text{rev}(\text{rev}(l)) = l$ for some $l \in \mathcal{L}[\mathbb{N}]$ (induction hypothesis). We need to show that

$$\text{rev}(\text{rev}(\text{cons}(n, l))) = \text{cons}(n, l), \qquad \forall n \in \mathbb{N}.$$

We compute:

$$\begin{aligned} \text{rev}(\text{rev}(\text{cons}(n, l))) &= \text{rev}(\text{rev}(\text{append}(\text{rev}(l), \text{cons}(n, nil)))) \\ &= \text{rev}\Big(\text{append}(\text{rev}(\text{cons}(n, nil)), \text{rev}(\text{rev}(l)))\Big). \end{aligned}$$

By the induction hypothesis, $\text{rev}(\text{rev}(l)) = l$. Also, $\text{rev}(\text{cons}(n, nil)) = \text{append}(\text{rev}(nil), \text{cons}(n, nil)) = \text{cons}(n, nil)$. So:

$$\text{rev}(\text{rev}(\text{cons}(n, l))) = \text{rev}(\text{append}(\text{cons}(n, nil), l)).$$

Now apply the auxiliary lemma that

$$\text{rev}(\text{append}(x, y)) = \text{append}(\text{rev}(y), \text{rev}(x)).$$

Using this with $x = \text{cons}(n, nil)$ and $y = l$, we get:

$$\text{rev}(\text{append}(\text{cons}(n, nil), l)) = \text{append}(\text{rev}(l), \text{rev}(\text{cons}(n, nil))).$$

But $\text{rev}(\text{cons}(n, nil)) = \text{cons}(n, nil)$, hence

$$\text{rev}(\text{rev}(\text{cons}(n, l))) = \text{append}(l, \text{cons}(n, nil)) = \text{cons}(n, l).$$

Thus the inductive step is proved.

Conclusion: By the principle of list induction, it follows that

$$\forall l \in \mathcal{L}[\mathbb{N}], \quad \text{rev}(\text{rev}(l)) = l.$$

□

Def 8.24. Datatype Declaration: SML datatype declarations provide a concerete syntax for inductively defined sets (called (abstract) data types in programming) via the keyword `datatype` followed by a sequence of **constructor declarations**

Example: We can declare a datatype for unary natural numbers by:

```
- datatype mynat = zero | suc of mynat;
datatype mynat = suc of mynat | zero

- val n0 = zero;
val n0 = zero : mynat

- val n1 = suc zero;
val n1 = suc zero : mynat

- val n2 = suc (suc zero);
val n2 = suc (suc zero) : mynat

- val n3 = suc (suc (suc zero));
val n3 = suc (suc (suc zero)) : mynat
```

Therefore, we can implement functions on our datatype `mynat`, e.g. here is unary addition:

```
- fun add (zero, m) = m | add (suc n,m) = suc (add (n,m));
val add = fn : mynat * mynat -> mynat

- add (suc zero, suc (suc zero));
val it = suc (suc (suc zero)) : mynat
```

Converting unary to int:

```
- fun num (zero) = 0 | num (suc n) = num (n) + 1;
val num = fn : mynat -> int

- num (suc (suc (suc zero)));
val it = 3 : int
```

Check even-odd unary using Mutual Recursion:

```
- fun even (zero) = true | even (suc n) = odd (n)
      and
      odd (zero) = false | odd (suc n) = even (n);

val even = fn : mynat -> bool
val odd = fn : mynat -> bool

- odd (suc (suc (suc zero)));
val it = true : bool

- even (suc (suc (suc (suc zero))));
val it = true : bool
```

Trace:

```
even (suc (suc (suc (suc zero)))) = odd (suc (suc (suc zero))) = even(suc (suc zero))
= odd (suc zero) = even (zero) = true
```

We also can declare **Enumeration Types**, e.g. here is a type for weekdays:

```
- datatype day = mon | tue | wed | thu | fri | sat | sun;
datatype day = fri | mon | sat | sun | thu | tue | wed
```

And here is a function that check if a given weekday is weekend or not:

```
- fun weekend sat = true | weekend sun = true | weekend _ = false;
val weekend = fn : day -> bool

- weekend fri;
val it = false : bool

- weekend sat;
val it = true : bool
```

Binary Tree datatype (only leafs are valued):

```
datatype tree = leaf of int | node of tree * tree;

- val t = node (leaf 1, leaf 2);
val t = node (leaf 1,leaf 2) : tree
```

Binary Tree datatype (every node is valued):

```
- datatype tree = leaf of int | node of int * tree * tree;

- val t1 = node (1, leaf 2, leaf 3);
val t1 = node (1,leaf 2,leaf 3) : tree
```

A function to sum node values in a binary tree:

```
- fun sum (leaf n) = n
    | sum (node (v, l, r)) = v + sum l + sum r;

val sum = fn : tree -> int

- sum t1;
val it = 6 : int
```

Number of leaves:

```
- datatype tree = leaf of int | node of int * tree * tree;

- fun leavescount (leaf x) = 1 | leavescount (node (l, r)) = leavescount l + leavescount r;
val leavescount = fn : tree -> int

- val t1 = node (leaf 1, node (leaf 2, leaf 3));

- leavescount t1;
val it = 3 : int
```

Multiway tree:

```
datatype t = a of int | c of t list

- val v = a 1;
val v = a 1 : t

- val v = c [a 1, c [a 2]];
val v = c [a 1,c [a 2]] : t
```

Sum all nodes in the tree:

```
- fun sum (a n) = n
    | sum (c children) = foldl (fn (sub, acc) => sum sub + acc) 0 children;

val sum = fn : t -> int
```

```
- sum v;
val it = 3 : int
```

Another Example: We can declare types for geometric shapes:

- A circle is defined by only one value $r \in \mathbb{R}^+$ (its radius)
- A square is defined by one value $a \in \mathbb{R}^+$ (side length)
- A triangle is defined by the three values $a, b, c \in \mathbb{R}^+ \times \mathbb{R}^+ \times \mathbb{R}^+$ (two sides and base)

```
- datatype shape =
      Circle of real
    | Square of real
    | Triangle of real * real * real;

- Circle 4.0;
val it = Circle 4 : shape

- Square 3.0;
val it = Square 3 : shape

- Triangle (4.0,3.0,5.0);
val it = Triangle (4,3,5) : shape
```

And here is a function that compute the area of a shape:

```
- fun area (Circle r) = Math.pi * r * r
    | area (Square a) = a * a
    | area (Triangle (a,b,c)) =
        let
            val s = (a + b + c)/2.0
        in
            Math.sqrt(s * (s-a) * (s-b) * (s-c))
        end;

val area = fn : shape -> real

- area (Square 3.0);
val it = 9 : real

- area (Circle 4.0);
val it = 50.2654824574 : real

- area (Triangle (4.0, 3.0, 5.0));
val it = 6 : real
```

Until now, our functions have been characterized entirely by their values on their arguments (as mathematical functions behave). However, SML also considers effects, e.g. for

- **input/output**: the interesting bit about a `print` statement is the effect,
- **mutation**: allocation and modification of storage during evaluation,
- **communication**: data may be send and received over channels, and
- **exceptions**: abort evaluation by signaling an exceptional condition.

> **Note**
>
> An effect is any action resulting from an evaluation that is not returning a value.

Input and Output is handled via *streams*. There are two predefined streams `TextIO.stdIn` and `TextIO.stdOut`.

Example: Input via `TextIO.inputLine`: `TextIO.instream` → `string`

```
- TextIO.inputLine(TextIO.stdIn);
```

```
ibrahim
val it = SOME "ibrahim\n" : string option
```

Example: Printing to Standard Output. `TextIO.print` prints its arguments to `stdIn`. `print` also returns a value `()` of type `unit`:

```
- print "Hello World! \n";
Hello World!
val it = () : unit
```

> **Note**
>
> Users can create streams as files using `TextIO.openIn` and `TextIO.openOut`. However, streams should be closed when no longer needed via `TextIO.closeIn` and `TextIO.closeOut`.

Def 8.25. Exception: An **exception** is a special SML object. **Raising** an exception e in a function aborts functional computation and returns e to the next level.

Example: Predefined exceptions:

```
- 3 div 0;
uncaught exception Div [divide by zero]
  raised at: stdIn:88.3-88.6

- fib 100;
uncaught exception Overflow [overflow]
  raised at: <file stdIn>
```

Example: User-defined exceptions:

```
- exception Empty;
exception Empty

- Empty;
val it = Empty(-) : exn
```

Example: Exception constructors:

```
- exception SysError of int;
exception SysError of int

- SysError;
val it = fn : int -> exn
```

Example: Programming with exceptions (factorial):

```
- exception Factorial;
exception Factorial

- local
    fun fact 0 = 1 | fact n = n * fact (n-1)
  in
    fun safe_factorial n =
        if n >= 0 then fact n else raise Factorial
  end;

val safe_factorial = fn : int -> int

- safe_factorial(~1);
uncaught exception Factorial
  raised at: stdIn:112.34-112.43
```

Note

`local` act just like `let`, only that is also that the body (between `in` and `end`) contains sequence of declarations and not an expression whose value is returned as the value of the overall expression. Here the identifier `fact` is bound to the recursive function in the definition of `safe_factorial` but unbound outside. This way we avoid polluting the name space with functions that are only used locally.

Exception Handling:

```
 exception NaN;
 fun read_integer () =
    let
        val intstring = case TextIO.inputLine(TextIO.stdIn) of
                            NONE => raise NaN
                          | SOME(s) => s
    in
        case Int.fromString intstring of
             NONE => raise NaN
           | SOME i => i
    end;
```

```
fun factorial_driver () =
    let val input = read_integer ()
        val result = Int.toString (safe_factorial input)
    in
        print result
    end
    handle Factorial => print "Out of range.\n"
         | NaN => print "Not a Number!\n";
```

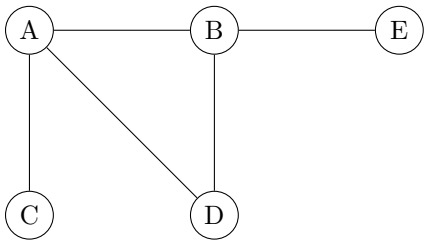
```
- factorial_driver ();
3
6
val it = () : unit

- factorial_driver ();
-3
Out of range.
val it = () : unit

- factorial_driver ();
hi
Not a Number!
val it = () : unit
```

9 Graphs and Trees

Def 9.1. Undirected Graph: An undirected graph is a pair $\langle V, E\rangle$ such that

- V is a set of **vertices (nodes)**, and
- $E \subseteq \{\, \{v, \bar{v}\} \mid v, \bar{v} \in V \wedge v \neq \bar{v} \,\}$ is the set of its **undirected edges**.

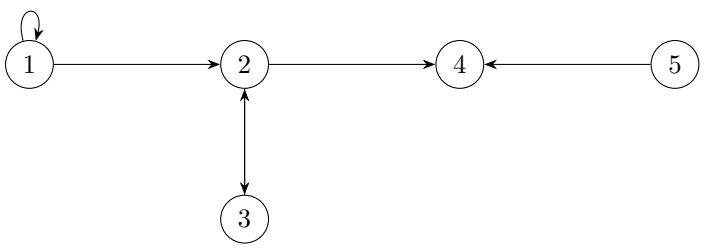
Example: An undirected graph $G_1 = \langle V_1, E_1\rangle$, where $V_1 = \{A, B, C, D, E\}$ and $E_1 = \{\{A, B\}, \{A, C\}, \{A, D\}, \{B, D\}, \{B, E\}\}$



Def 9.2. Directed Graph: A directed graph (digraph) is a pair $\langle V, E\rangle$ such that

- V is a set of **vertices (nodes)**, and
- $E \subseteq V \times V$ is the set of its **directed edges**.

Example: A directed graph $G_2 = \langle V_2, E_2\rangle$, where $V_2 = \{1, 2, 3, 4, 5\}$ and $E_2 = \{(1, 1), (1, 2), (2, 3), (3, 2), (2, 4), (5, 4)\}$



Def 9.3. Indegree: Given a directed graph $\langle V, E\rangle$, the indegree $\text{indeg}(v)$ of a vertex $v \in V$ is defined as $\text{indeg}(v) = \Gamma(\{\, w \mid (w, v) \in E \,\})$.

Example: $\text{indeg}(2) = 2$

Def 9.4. Outdegree: Given a directed graph $\langle V, E\rangle$, the outdegree $\text{outdeg}(v)$ (branching factor) of a vertex $v \in V$ is defined as $\text{outdeg}(v) = \Gamma(\{\, w \mid (v, w) \in E \,\})$.

Example: $\text{outdeg}(5) = 1$

> **Note**
>
> For an undirected graph, $\text{indeg}(v) = \text{outdeg}(v)$ for all $v \in V$.

Observation: directed graphs are nothing else than relations.

Def 9.5. Initial vs Terminal Node: Let $G = \langle V, E\rangle$ be a directed graph, then we call a node $v \in V$

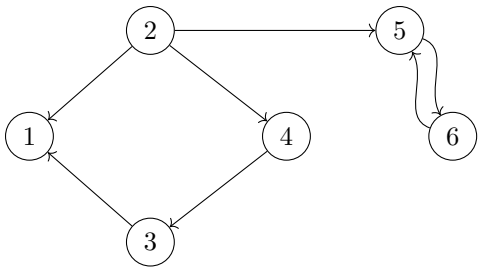
- **initial** (source of G), iff there is no $w \in V$ such that $(w, v) \in E$ (no predecessor)
- **terminal** (sink of G), iff there is no $w \in V$ such that $(v, w) \in E$ (no successor)

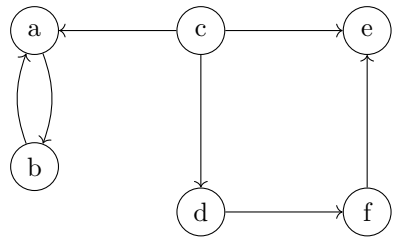
Def 9.6. Graph Isomorphism: Iff we can find a bijection $\psi : V \to \bar{V}$ between two graphs $G = \langle V, E\rangle$ and $\bar{G} = \langle \bar{V}, \bar{E}\rangle$ then we call them isomorphic. The bijection $\psi : V \to \bar{V}$ is defined as $(a, b) \in E \Leftrightarrow (\psi(a), \psi(b)) \in \bar{E}$ for directed graph. And for undirected graph it is defined as $\{a, b\} \in E \Leftrightarrow \{\psi(a), \psi(b)\} \in \bar{E}$

Def 9.7. Equivalent Graphs: Two graphs G and $\bar{G}$ are **equivalent** iff there is an isomorphism ψ between G and $\bar{G}$.

Example: the following two graphs are equivalent as there exists an isomorphism between them given by the bijection:

$$\psi := \{a \mapsto 5, b \mapsto 6, c \mapsto 2, d \mapsto 4, e \mapsto 1, f \mapsto 3\}$$

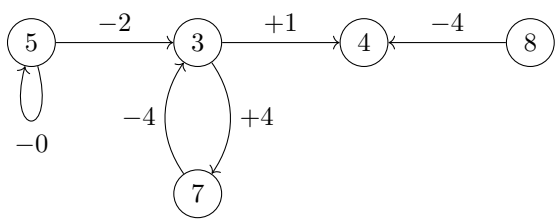




Def 9.8. Labeled Graph: A labeled graph G is a quadruple $\langle V, E, L, l\rangle$ where $\langle V, E\rangle$ is a graph and $l : V \cup E \rightharpoonup L$ is a partial function into a set L of labels.

Example: $G = \langle V, E, L, l\rangle$ with $V = \{A, B, C, D, E\}$, where

- $E = \{(A, A), (A, B), (B, C), (C, B), (B, D), (E, D)\}$
- $l : V \cup E \rightharpoonup \{+, -, \phi\} \times \{1, \cdots, 9\}$ with
  - $l(A) = 5 \quad l(B) = 3 \quad l(C) = 7 \quad l(D) = 4 \quad l(E) = 8,$
  - $l((A, A)) = -0 \quad l((A, B)) = -2 \quad l((B, C)) = +4,$
  - $l((C, B)) = -4 \quad l((B, D)) = +1 \quad l((E, D)) = -4$



Def 9.9. Paths in Graphs: Given a graph $G := \langle V, E\rangle$ we call a $n + 1$-tuple $p = \langle v_0, \cdots, v_n\rangle \in V^{n+1}$ a **path** in G iff $(v_{i-1}, v_i) \in E$ for all $1 \leq i \leq n$ and $n > 0$.

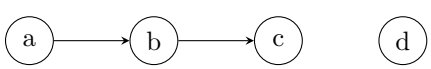
- We say that v_i are nodes on p and that v_0 and v_n are **linked** by p.
- v_0 and v_n are called the **start** and **end** of p (write $\text{start}(p)$ and $\text{end}(p)$), the other v_i are called **inner nodes** of p.
- n is called the **length** of p (write $\text{len}(p)$).
- We denote the set of paths in G with $\Pi(G)$.

> **Note**
>
> Not all v_i-s in a path are necessarily different.

For a graph $G = \langle V, E\rangle$ and a path $p = \langle v_1, \cdots, v_n\rangle \in V^{n+1}$, write

- $v \in p$, iff $v \in V$ is a vertex on the path ($\exists i.v_i = v$)
- $e \in p$, iff $e = (v, \bar{v}) \in E$ is an edge on the path ($\exists i.v_i = v \wedge v_{i+1} = \bar{v}$)

Example: Consider $G = \langle V, E\rangle$ with $V = \{a, b, c, d\}$ and $E = \{(a, b), (b, c)\}$:



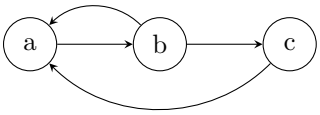
An example path would be $p = \langle a, b, c\rangle$. Here $n = 2$ and $p \in V^3 = \{(a, a, a), (a, b, d), \cdots\}$. Obviously, since $n = 2$ so we have 2 edges and 3 nodes.

- nodes a, b, c are on the path
- $\text{start}(p) = a$ and $\text{end}(p) = c$
- b is an inner node, and $\text{len}(p) = 2$
- $e = (a, b), \bar{e} = (b, c) \in p$
- $\Pi(G) = \{(a, b), (b, c), (a, b, c)\}$

Def 9.10. Cyclic Graphs: Given a directed graph $G = \langle V, E\rangle$, a path p is called **cyclic** iff $\mathrm{start}(p) = \mathrm{end}(p)$. A cycle $\langle v_0, \cdots, v_n\rangle$ is called **simple**, iff $v_i \neq v_j$ for $1 \leq i, j \leq n$ with $i \neq j$ (all inner nodes are distinct).

Def 9.11. DAG: A directed graph with no cycles is called **directed acyclic graph (DAG)**.

Example: $p = \langle a, b, c, a\rangle$ is simple cyclic path. $p = \langle a, b, a, b, a\rangle$ is a cyclic path that is not simple.



Def 9.12. Node Depth: Let $G = \langle V, E\rangle$ be a directed graph, then the depth $\mathrm{dp}(v)$ of a vertex $v \in V$ is defined to be 0, iff v is a source of G and the supremum $\sup(\{\,\mathrm{len}(p) \mid \mathrm{indeg}(\mathrm{start}(p)) = 0 \wedge \mathrm{end}(p) = v\,\})$ otherwise, i.e. the length of the longest path from a source of G to v (can be infinite).

Def 9.13. Graph Depth: Given a directed graph $G = \langle V, E\rangle$, the **depth** ($\mathrm{dp}(G)$) of G is defined as the supremum $\sup(\{\,\mathrm{len}(p) \mid p \in \Pi(G)\,\})$, i.e. the maximal path length in G.

Def 9.14. Tree: A tree is a DAG $G = \langle V, E\rangle$ such that

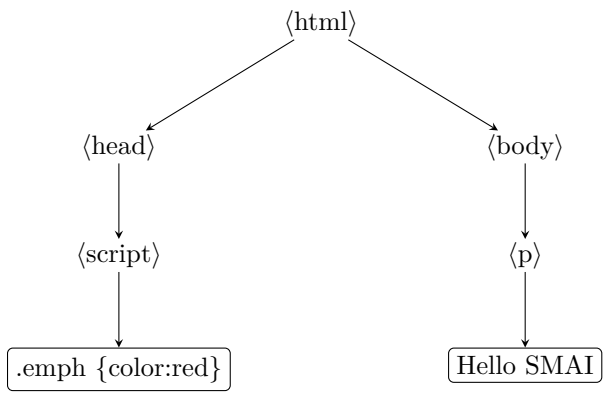
- There is exactly one initial node $v_r \in V$ (called the **root**)
- All nodes but the root have indegree of 1.

We call v the **parent** of w, iff $(v, w) \in E$ (w is a child of v). We call a node v a **leaf** of G, iff it is terminal, i.e. if it does not have children.

Lemma 9.1. For any node $v \in V$ except the root v_r, there is exactly one path $p \in \Pi(G)$ with $\mathrm{start}(p) = v_r$ and $\mathrm{end}(p) = v$.

Observation: We often deal with **well-bracketed structures** in Computer Science, e.g. mathematical expressions, Markup languages like `HTML`, programming languages like `python`, etc. A common approach to scaling in computer science is to come up with a common data structure that allows to program the same algorithm for all of these structures, that where trees come in. For example (`HTML`):

```
<html>
  <head>
    <script>.emph {color:red}</script>
  </head>
  <body>
    <p>Hello SMAI</p>
  </body>
</html>
```



Observation: functional programming languages like SML are ideal for computing with labeled trees.

Example: data types for node labeled trees (binary and general):

```
datatype lbt = leaf of int |t of int * lbt * lbt
```

```
datatype ltree = leaf of int | t of int * ltree list
```

We can compute e.g. the weight (sum of all labels) recursively:

```
fun wt (leaf n) = n | wt (t n l r) = n + wt l + wt r
```

```
fun wt (leaf n) = n
  | wt (t n ts) = n + wtl ts
and wtl (nil) = 0
  | wtl (h::ts) = wt h + wtl ts
```

Observation: often, DAGs are better representations of trees (trees with sharing):

```
 f
 |
( )
 |
 f
 |
( )
 |
 f
 |
( )
 |
 a
```

```
                f
              /   \
            f       f
          /  \    /   \
         f    f  f     f
        / \  / \ / \   / \
       a  a a  a a  a a   a
```

Here, we assume that the constant symbol a represent a leaf and the function symbol f represents a parent node with two children which can be f or a. Hence, the full tree is represented as:

$$f(f(f(a,a), f(a,a)), f(f(a,a), f(a,a)))$$

The DAG compresses the same structure by reusing subterms. It internally does not duplicate all the $f(a,a)$ pieces, it stores them once and reuses them by reference. For example, define a base piece: $t_1 = f(a,a)$, then reuse it: $t_2 = f(t_1, t_1)$ and finally $t_3 = f(t_2, t_2)$

Idea: We want to represent a DAG in SML, and then be able to *explode* it into a tree. A DAG can share subtrees via references and save memory, but a tree has no sharing and every reference is copied. *Exploding* is the process of removing and duplicating the structure.

Datatype Definition

```
type id = int
type label = string

datatype DAG =
   leaf of id * label
 | ref of id
 | dag of id * label * DAG list
```

Example:

```
ex1 =
  dag 1 "f" [
    dag 2 "f" [
      dag 3 "f" [leaf 4 "a", ref 4], ref 3
    ],
    ref 2
  ]
```

Notice that the innermost part is `dag 3 "f" [leaf 4 "a", ref 4]`. That is a node with `id=3`, label "f", and two children: `leaf 4 "a"` (a leaf node with `id=4`, label = "a"), and `ref 4` (a reference back to the same leaf id (4)). So this subtree is $f(a,a)$ but written with sharing.

Next level, we see `dag 2 "f" [(dag 3 ...), ref 3]`. That is a node with id=2, label "f", and two children: one is the whole `dag 3` subtree ($f(a,a)$), the other one is a reference back to it. So this is $f(f(a,a), f(a,a))$.

Finally, we have `dag 1 "f" [(dag 2 ...), ref 2]`. This is the root node with id=1, label "f", and two children. One is the whole `dag 2` ($f(f(f(a,a), f(a,a)))$, the other one is a reference back to that node. So this is the full

$$f(f(f(a,a), f(a,a)), f(f(a,a), f(a,a)))$$

Now, we want a function `fun explode (dag, dict) = tree` where `dict` is a dictionary mapping identifiers to already-exploded subtrees, so that references can be looked up.

Case 1: `explode (leaf (_, l), _) = leaf l`, i.e. if we see a leaf, ignore the id, just return a tree leaf with the label.

Case 2: `explode (ref i, dict) = lookup dict i`, i.e. if we see a reference, we just look up the corresponding tree in the dictionary.

Case 3: internal DAG node:

```
explode (dag (_, l) ds, dict) =
  let val (ts, _) = explist ds dict
  in tree l ts
  end
```

i.e. for a node, explode the children `ds` using `explist` and build a new tree node with label `l` and explode children `ts`

`explist` is a helper function defined as:

```
and explist [] dict = ([], dict)
  | explist (h::t) dict =
      let val (th, dict1) = explode h dict
          val (tt, dict2) = explist t dict1
      in (th::tt, dict2)
      end
```

It explodes a list of DAG children into a list of tree children.

And here are some helper functions for completeness: a function that looks up identifiers:

```
fun did (leaf i _) = i | did (ref _ j) = j | did (dag i _ _) = i
```

A dictionary type and a lookup function:

```
type dict = (id * DAG) list

exception NotFound

fun lookup i [] = raise NotFound
  | lookup i (h::t) =
      let
        val (k,d) = t
      in
        if i = k then d else lookup i t
      end
```

10 Mathematical Structures

Def 10.1. Mathematical Structure: A mathematical structure combines multiple mathematical objects (the components) into a new object. The components usually have names by which they can be referenced. Given a definition of a mathematical structure S, we say that any object that conforms to that is an instance of S.

Example: A graph $G = \langle V, E \rangle$ is a mathematical structure where the components are: V (the set of vertices) and E (the set of edges). A graph G where $V = \{1, 2, 3\}$ and $E = \{(1, 2), (2, 3)\}$ is one instance of a graph.

Def 10.2. Structure Signature: A structure signature combines the components, their types, and accessor names of a mathematical structure in a tabular overview.

Example:

$$\left\langle \begin{array}{lll} V & \text{Set} & \text{vertices (nodes)} \\ E & \{\,(u, v) \mid u, v \in V\,\} & \text{edges} \end{array} \right\rangle$$

> **Note**
>
> Most programming languages have some way of creating *"named structures"* and referencing components is usually done via *"dot notation"* (e.g. `person.name`).

Def 10.3. Group: A group is a mathematical structure $\langle G, \circ, e, .^{-1} \rangle$ that consists of:

- a base set G of objects,
- an operation $\circ : G \times G \to G$, such that $\forall a, b \in G. a \circ b \in G$ and $\forall a, b, c \in G. (a \circ b) \circ c = a \circ (b \circ c)$,
- a unit $e \in G$, such that $\forall a \in G. e \circ a = a \circ e = a$, and
- the inverse function $.^{-1} : G \to G$, such that $\forall a \in G. a \circ a^{-1} = a^{-1} \circ a = e$

An example of a group is the set $G = \mathbb{Z}$, with the operation $+ : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$; then $e = 0$ and $.^{-1}$ is $\lambda x \in \mathbb{Z}. -x$. We write it as $\langle \mathbb{Z}, +, 0, - \rangle$.

Question: can we do the same for multiplication where $e = 1$? If we think about it, we would be stuck on finding the $.^{-1}$ component of the group. What can we always multiply with any integer and get $e = 1$ back? Take for example 1, we can multiply it with $\frac{1}{1}$ (great!). Take -1, we can multiply it with $\frac{1}{-1}$ (also great!). But take any other integer, e.g. 2, we must multiply it with $\frac{1}{2}$ to get 1 back, however, $\frac{1}{2} \notin \mathbb{Z}$, it is however $\in \mathbb{Q}$! Moreover, 0 never has a multiplicative inverse in any field! The multiplicative group would only make sense iff $G = \mathbb{Q} \setminus \{0\}$. We write that as $\langle \mathbb{Q} \setminus \{0\}, *, 1, a \mapsto \frac{1}{a} \rangle$.

But we need some multiplication structure that works for $\mathbb{Z}$!

Def 10.4. Monoid: A monoid is a mathematical structure $\langle M, \circ, e \rangle$ that consists of:

- A base set M of objects,
- An operation $\circ : M \times M \to M$, such that for all $a, b \in M$, we have $a \circ b \in M$ (closure),
- Associativity: for all $a, b, c \in M$, $(a \circ b) \circ c = a \circ (b \circ c)$, and
- A unit element $e \in M$, such that for all $a \in M. e \circ a = a \circ e = a$.

Hence, the mathematical structure $\langle \mathbb{Z}, *, 1 \rangle$ is a monoid (a group missing an inverse function).

We saw that $\langle \mathbb{Z}, +, 0, - \rangle$ is a group under addition. And $\langle \mathbb{Z}, *, 1 \rangle$ is a monoid under multiplication. Now, we want to do everyday arithmetics with integers; we want expressions where we have additions and multiplication (e.g. $a * (b + c)$) which evaluates to $(a * b) + (a * c)$.

For that, we need a mathematical structure where both addition and multiplication live together.

Def 10.5. Ring: A ring is a mathematical structure $\langle R, +, 0, -, *, 1 \rangle$ consisting of:

- A base set R.
- An addition operation $+ : R \times R \to R$ such that $\langle R, +, 0, - \rangle$ is a group (called abelian group).
- A multiplication operation $* : R \times R \to R$ such that $\langle R, *, 1 \rangle$ is a monoid.
- Distributivity: for all $a, b, c \in R, a * (b + c) = (a * b) + (a * c), (a + b) * c = (a * c) + (b * c)$

Hence, mixing components from the group $\langle \mathbb{Z}, +, 0, - \rangle$ and the monoid $\langle \mathbb{Z}, *, 1 \rangle$ leaves us with a ring $\langle \mathbb{Z}, +, 0, -, *, 1 \rangle$

Def 10.6. Magma: A magma is a mathematical structure $\langle M, \circ \rangle$ consisting of:

- A base set M, and
- A binary operation $\circ : M \times M \to M$.

Example: $\langle \mathbb{Z}, - \rangle$ (integers under subtraction). $\langle \mathbb{N}, + \rangle$ (natural numbres under addition)

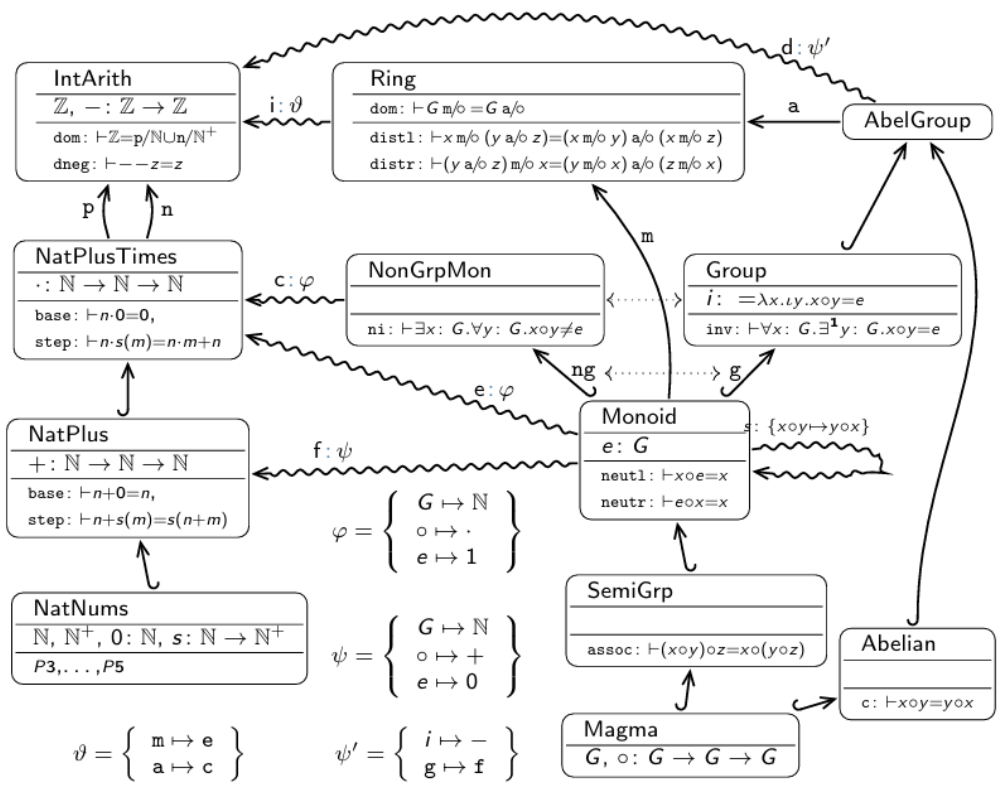
Def 10.7. Semigroup: A semigroup (magma with associativity) is a mathematical structure $\langle S, \circ \rangle$ consisting of:

- A base set S,
- A binary opertation $\circ : S \times S \to S$, and
- Associativity: for all $a, b, c \in S, (a \circ b) \circ c = a \circ (b \circ c)$

Example: $\langle \mathbb{Z}, + \rangle$ (integers under addition).

> **Note**
>
> Given collections M and S of instances of magmas and semigroups, $S \subseteq M$ as there are magmas whose operation is not associative.

Mathematical Structures in a Graph:



- nodes are structures with objects ($\circ : \tau$) and properties ($n :\vdash P$).
- edges represent inheritance
  - regular inheritance: $\hookrightarrow$
  - copying substructures: $\overset{n}{\to}$
  - interpretation via φ: $\overset{n}{\rightsquigarrow}$
- Examples are just interpretations ($\rightsquigarrow$) read backwards
- Any object / property / theorem can be copied along any edge

11 Complexity Analysis in AI

Def 11.1. Algorithm: An algorithm is a series of instructions to control a (computation) process.

Def 11.2. Computational Problem: A computational problem is a triple $\langle I, S, \sigma \rangle$, where I is a set of inputs, S is a set of solutions, and $\sigma : I \to \mathcal{P}(S)$ asigns to each input the set of valid solutions. The problem is called **computable** if there exists an algorithm $\alpha : I \to S$ such that for every $i \in I, \alpha(i) \in \sigma(i)$ and $\alpha(i)$ is defined, i.e. the algorithm always **halts**.

Def 11.3. Partially Computable Problem: A computational problem $\langle I, S, \sigma \rangle$ is called **partially computable** if there exists a *partial* algorithm $\alpha : I \rightharpoonup S$ such that for every $i \in I$, if $\alpha(i)$ is defined then $\alpha(i) \in \sigma(i)$. In this case α may diverge on some inputs.

Def 11.4. Resource: A resource is a measurable quantity consumed by an algorithm during execution. The most common resources are **time** (number of computation steps) and **space** (amount of memory used).

Def 11.5. Complexity Theory: (Computational) complexity theory focuses on classifying computational problems according to their resource usage, and relating these classes to each other.

Def 11.6. Complexity Measure: A complexity measure is a mapping $m : I \to \mathbb{N}$ that assigns to each input $i \in I$ a natural number describing the amount of some resource consumed by an algorithm on input i.

Def 11.7. Input Size: The size of an input $i \in I$, denoted $|i|$, is a natural number measuring the length of the encoding of i (for instance, the number of bits required to represent i).

Def 11.8. Asymptotic Analysis: Asymptotic analysis studies the growth of a complexity measure as the input size n tends to infinity. It abstracts from constant factors and lower-order terms to capture the long-term behavior of an algorithm.

Def 11.9. Running Time: If an algorithm α terminates within $t(n)$ steps on every input of size n, then we say that α has **running time** $T(\alpha) := t$.

Def 11.10. Time Complexity: Let $S \subseteq \mathbb{N} \to \mathbb{N}$ be a set of natural number functions. An algorithm α with running time $T(\alpha) = t$ is said to have **time complexity** in S (written $T(\alpha) \in S$, or colloquially $T(\alpha) = S$) if $t \in S$.

Def 11.11. Space Complexity: Let $S \subseteq \mathbb{N} \to \mathbb{N}$ be a set of natural number functions. An algorithm α is said to have **space complexity** in S if, on every input of size n, α uses at most $s(n)$ units of memory for some $s \in S$.

Def 11.12. Best-Case Complexity: The best-case complexity of an algorithm α is the function

$$T_\alpha^{\text{best}}(n) = \min\{\text{running time of } \alpha(i) \mid i \in I, |i| = n\}.$$

It describes the minimal resource usage among all inputs of size n.

Def 11.13. Average-Case Complexity: The average-case complexity of an algorithm α is the expected resource usage over all inputs of size n, with respect to some probability distribution D on I. Formally,

$$T_\alpha^{\text{avg}}(n) = \mathbb{E}_{i \sim D, |i|=n}[\text{running time of } \alpha(i)].$$

Def 11.14. Worst-Case Complexity: The worst-case complexity of an algorithm α is the function

$$T_\alpha^{\text{worst}}(n) = \max\{\text{running time of } \alpha(i) \mid i \in I, |i| = n\}.$$

It describes the maximal resource usage among all inputs of size n.

> **Note**
>
> We are mostly interested in worst-case complexity.

Def 11.15. Big-$\mathcal{O}$ Notation: In practice, asymptotic analysis is most often applied to the worst-case complexity. Let $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We say that f is **asymptotically bounded** by g, written $f \in \mathcal{O}(g)$ or $f \leq_a g$, if there exist constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$\forall n \geq n_0 : \quad f(n) \leq c \cdot g(n).$$

Equivalently, the **Landau set** $\mathcal{O}(g)$ is the set of all such functions:

$$\mathcal{O}(g) = \{f \mid \exists c > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0 : f(n) \leq c \cdot g(n)\}.$$

Example: Let $f(n) = 3n^2 + 2n + 7$ and $g(n) = n^2$. We want to show that $f \in \mathcal{O}(g)$.

By the definition of Big-$\mathcal{O}$, this means we must find constants $c > 0$ and $n_0 \in \mathbb{N}$ such that

$$\forall n \geq n_0 : \quad f(n) \leq c \cdot g(n).$$

Let's take $n_0 = 1$, then $\forall n \geq 1$ we have:

$$2n \leq 2n^2 \quad \text{and} \quad 7 \leq 7n^2.$$

Therefore,

$$f(n) = 3n^2 + 2n + 7 \quad \leq \quad 3n^2 + 2n^2 + 7n^2 = 12n^2.$$

Here we see that $f(n) \leq 12 \cdot g(n)$ for all $n \geq 1$. Thus the definition holds with $c = 12$ and $n_0 = 1$. We conclude that $f(n) \in \mathcal{O}(n^2)$.

Lemma 11.1. Growth Ranking. For $\bar{k} > 2$ and $k > 1$ we have:

$$\mathcal{O}(1) \subset \mathcal{O}(\log_2(n)) \subset \mathcal{O}(n) \subset \mathcal{O}(n^2) \subset \mathcal{O}(n^{\bar{k}}) \subset \mathcal{O}(k^n)$$

The following sets are often used for S in $T(\alpha)$:

Landau set	class name	rank	Landau set	class name	rank
$\mathcal{O}(1)$	constant	1	$\mathcal{O}(n^2)$	quadratic	4
$\mathcal{O}(\log_2 n)$	logarithmic	2	$\mathcal{O}(n^k)$	polynomial	5
$\mathcal{O}(n)$	linear	3	$\mathcal{O}(k^n)$	exponential	6

Table 3: Common complexity classes

Computing with Landau sets is quite simple:

- $\mathcal{O}(c \cdot f) = \mathcal{O}(f)$ for any constant $c \in \mathbb{N}$ (*drop constant factors*)
- $\mathcal{O}(f) \subseteq \mathcal{O}(g) \Rightarrow \mathcal{O}(f + g) = \mathcal{O}(g)$ (*drop low-complexity summands*)
- $\mathcal{O}(f \cdot g) = \mathcal{O}(f) \cdot \mathcal{O}(g)$ (*distribute over products*)

Example: $\mathcal{O}(4n^3 + 3n + 7^{1000n}) = \mathcal{O}(2^n)$ (*exponential*)

Note

$\mathcal{O}(n) \subset \mathcal{O}(n \cdot log_2(n)) \subset \mathcal{O}(n^2)$ (*linearithmic*)

Note

Updated Growth Ranking & Common Classes
To account for all algorithmic patterns discussed (such as nested loops with logarithmic guards), we refine the growth hierarchy. For $a > 2$ and $b > 1$, the asymptotic ranking is :

$$\mathcal{O}(1) \subset \mathcal{O}(\log_2 n) \subset \mathcal{O}(n) \subset \mathcal{O}(\mathbf{n} \log_{\mathbf{2}} \mathbf{n}) \subset \mathcal{O}(n^2) \subset \mathcal{O}(n^a) \subset \mathcal{O}(b^n)$$

Determining the Time/Space Complexity of Algorithms

Assume we have

- Γ: a mapping that tells us the complexity of variables.
- α: a program fragment.
- $T_\Gamma(\alpha)$: the time complexity of α under Γ.
- $C_\Gamma(\alpha)$: the context update, i.e. what new cost α introduces.

We use induction on program strcuture:

1. Constant

If $\alpha = \delta$ (where δ is a literal, like 5 or `true`), then $T_\Gamma(\alpha) \in \mathcal{O}(1)$

2. Variable

If $\alpha = v$ and $v \in \text{dom}(\Gamma)$, then $T_\Gamma(\alpha) \in \mathcal{O}(\Gamma(v))$

3. Function Call

If $\alpha = \varphi(\psi)$, with $T_\Gamma(\alpha) \in \mathcal{O}(f)$, and $T_{\Gamma \cup C_\Gamma(\varphi) \in \mathcal{O}(g)}$, then $T_\Gamma(\alpha) \in \mathcal{O}(f \circ g)$

4. Assignment

If $\alpha = v := \varphi$, with $T_\Gamma(\varphi) \in S$, then $T_\Gamma(\alpha) \in S$, $C_\Gamma(\alpha) = \Gamma \cup (v, S)$

5. Composition (sequence)

If $\alpha = \varphi; \psi$, with $T_\Gamma(\varphi) \in P$, and $T_{\Gamma \cup C_\Gamma(\varphi)}(\psi) \in Q$, then $T_\Gamma(\alpha) \in \max\{P, Q\}$

6. Branching

If $\alpha =$ `if` γ `then` φ `else` ψ `end`, with $T_\Gamma(\gamma) \in C$, $T_{\Gamma \cup C_\Gamma(\gamma)}(\varphi) \in P$, and $T_{\Gamma \cup C_\Gamma(\gamma)}(\psi) \in Q$, then $T_\Gamma(\alpha) \in \max\{C, P, Q\}$

7. Loop

If $\alpha =$ `while` γ `do` φ `end`, with $T_\Gamma(\gamma) \in \mathcal{O}(f)$, and $T_{\Gamma \cup C_\Gamma(\gamma)}(\varphi) \in \mathcal{O}(g)$, then $T_\Gamma(\alpha) \in \mathcal{O}(f(n) \cdot g(n))$

> **Note**
>
> By default, the overall time complextiy is $T(\alpha) = T_\phi(\alpha)$

Sequence Example

```
x := readArray(n)   -- O(n)
y := sum(x)         -- O(n)
```

$\max(\mathcal{O}(n), \mathcal{O}(n)) = \mathcal{O}(n)$

Conditional Example

```
if isSorted(x) then     -- O(n)
    binarySearch(x,k)   -- O(log n)
else
    linearSearch(x,k)   -- O(n)
end
```

$\max(\mathcal{O}(n), \mathcal{O}(\log(n), \mathcal{O}(n))) = \mathcal{O}(n)$

While loop example

```
i := 1
while i <= n do
    i := i + 1
end
```

Guard (loop condition check) runs $\mathcal{O}(n)$ times, body is $\mathcal{O}(1)$. Total $\mathcal{O}(n \cdot 1) = \mathcal{O}(n)$

Function Call Example

Suppose `buildArray(n)` $\in \mathcal{O}(n^2)$ returns an array of size n^2 and `sort(m)` $\in \mathcal{O}(m \cdot \log(m))$

`sort(buildArray(n))`

$\mathcal{O}((n^2) \cdot \log(n^2)) = \mathcal{O}(n^2 \cdot \log(n))$

Nested For Loop Example

```
for i = 1 to n do        -- O(n)
    for j = 1 to n do    -- O(n)
        x := x + 1       -- O(1)
```

Inner loop: $\mathcal{O}(n \cdot 1)$, outer loop: $\mathcal{O}(n \cdot n \cdot 1)$, total: $\mathcal{O}(n^2)$

Nested (one logarthmic loop)

```
for i = 1 to n do
    k := 1
    while k < n do
        k := 2 * k
```

Inner loop starts with $k = 1$, each iteration k doubles and the loop stops when $k \geq n$. How many times can we double before reaching n? Solve $2^t \geq n \rightsquigarrow t = \lceil \log_2(n) \rceil$. So the inner loop runs $\mathcal{O}(\log(n))$ iterations. Each iteration is $\mathcal{O}(1)$, therefore total $\mathcal{O}(\log(n))$.

Outer loop runs exactly n times with each time the cost is $\mathcal{O}(\log(n))$. Total cost is $n \cdot \mathcal{O}(\log(n)) = \mathcal{O}(n \cdot \log(n))$

Note

Take home message: not every time you see two nested loops you assume $\mathcal{O}(n^2)$

Here is another example: assume `<<body>>` $\in \mathcal{O}(1)$:

```
i, j := 0
while i <= n-1 do
    i := i+1
    while j <= 10 do
        j := j+1
        <<body>>
    end
end
```

Outer Loop: i increments by 1 each time untill $i > n - 1$ we stop, so it runs n times.

Inner Loop: the condition is $j \leq 10$, but note that j is initialized once before the loop and not reset for each i! That means, first outer iteration $j = 0$, runs until $j = 11$, i.e. 11 iterations. After that, $j = 11$ always and the condition $j \leq 10$ is false, so the inner loop never runs again. Inner loop total cost is $\mathcal{O}(11) = \mathcal{O}(1)$

Therefore, all in all we have linear time ($\mathcal{O}(n)$).

12 Formal Languages and Grammars

Def 12.1. Alphabet: An alphabet is a finite set; we call each element $a \in A$ a **character**, and an n-tuple $s \in A^n$ a **string** (of *length* n over A).

> **Note**
>
> We often write a string $\langle c_1, \cdots, c_n \rangle$ as "$c_1 \cdots c_n$", for instance `"abc"` for $\langle a, b, c \rangle$

Example: Let $A = \{0, 1\}$ be an alphabet

- `"0", "1"` are strings of length 1 ($A^1 = \{0, 1\}$)
- `"00","01","10","11"` are strings of length 2 in (A^2)
- `"001"` $\in A^3$

Def 12.2. Empty String: Let A be an alphabet, then $A^0 = \{\langle\rangle\}$, where $\langle\rangle$ is the unique 0-tuple. We consider $\langle\rangle$ as the string of length 0 and call it the **empty string** and denote it with ϵ.

> **Note**
>
> $\{1, 2, 3\} = \{3, 2, 1\}$ BUT $\langle 1, 2, 3 \rangle \neq \langle 3, 2, 1 \rangle$ (sets vs strings)

Def 12.3. String Length: Given a string s, we denote its length with $|s|$.

Def 12.4. String Concatenation: The concatenation $\text{conc}(s, t)$ of two strings $s = \langle s_1, \cdots, s_n \rangle \in A^n$ and $t = \langle t_1, \cdots, t_m \rangle \in A^m$ is defined as $s + t$ or simply `st`

Example: conc("text", "book") = "text" + "book" = "textbook".

Def 12.5. Kleene Plus and Kleene Star: Let A be an alphabet, then we define the sets:

- $A^+ := \bigcup_{i \in \mathbb{N}^+} A^i$ (*nonempty strings*), and
- $A^* := A^+ \cup \{\epsilon\}$ (*strings*).

Def 12.6. Formal Language: A set $L \subseteq A^*$ is called a *formal language* over A.

Example: If $A = \{a, b\}$ then:

$$A^* = \bigcup_{n=0}^{\infty} A^n$$

where $A^0 = \epsilon$, $A^1 = \{a, b\}$, $A^2 = \{a, b, ab, ba\}$, and so on ...

$$A^* = \{\epsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, \cdots\}$$
$$A^+ = \{a, b, aa, ab, ba, bb, aaa, aab, aba, abb, \cdots\}$$

A formal language might be $L = \{a, b, aab, bbb, \cdots\}$

Def 12.7. Repeating Chars: We use $c^{[n]}$ for the string that consists of the character `c` repeated n times.

Examples:

- Let $A = \{a, b\}$, $a^{[5]} =$ `"aaaaa"`.
- The set $M := \left\{ \text{ba}^{[n]} \mid n \in \mathbb{N} \right\}$ of strings that start with character b followd by an arbitrary numbers of a's is a formal language over $A = \{a, b\}$

> **Note**
>
> A formal language might be infinite and even undecidable even if the alphabet A is finite. For example, let $A = \{a, b\}$ then the formal language $L = \{a, ab, bba\}$ is finite but the formal language $L = \{\, a^{[n]} \mid n \in \mathbb{N} \,\}$ is infinite.

Since we cannot list all strings in a formal language L because there are infinitely many, we need a **finite description** that can generate all strings in L. We need **Grammars**.

Def 12.8. Phrase Structure Grammar: A phrase structure grammar (*type 0 grammar, unrestricted grammar, grammar*) is a tuple $\langle N, \Sigma, P, S\rangle$ where

- N is a finite set of **nonterminal symbols**,
- Σ is a finite set of **terminal symbols**. (members of $N \cup \Sigma$ are called symbols).
- P is a finite set of **production rules**: pairs $p := h \to b$ (also written as $h \Rightarrow b$), where $h \in (\Sigma \cup N)^* N (\Sigma \cup N)^*$ and $b \in (\Sigma \cup N)^*$. The string h is called the **head** of p and b the **body**.
- $S \in N$ is a distinguished symbol called the **start symbol** (also **sentence symbol**).

The sets N and Σ are assumed to be disjoint. Any word $w \in \Sigma^*$ is called a **terminal word**.

> Note
>
> Production rules map strings with at least one nonterminal to arbitrary other strings

Notation: If we have n production rule $h \to b_i$ sharing a head, we often write $h \to b_1 \mid \cdots \mid b_n$ instead.

Example: A simple grammar for english sentences:

$$\begin{aligned}
\text{S} &::= \text{NP Vi} \\
\text{NP} &::= \text{Article N} \\
\text{Article} &::= \text{the} \mid \text{a} \mid \text{an} \\
\text{N} &::= \text{dog} \mid \text{teacher} \mid \cdots \\
\text{Vi} &::= \text{sleeps} \mid \text{smells} \mid \cdots
\end{aligned}$$

Here S is the start symbol, NP, Article, N, and Vi are nonterminals, the, a, dog, smells, ... are terminals. Let's generate a valid sentence from the grammar:

```
NP           Vi
Article N    Vi
the     N    Vi
the    dog   Vi
the    dog sleeps
```

Def 12.9. Lexical: A production rule whose head is a single nonterminal and whose body consists of a single terminal is called **lexical** or a **lexical insertion rule**.

Example: Both of the following rules are lexical

$$\begin{aligned}
\text{Noun} &\to \text{"dog"} \\
\text{Verb} &\to \text{"run"}
\end{aligned}$$

Def 12.10. Lexicon of a Grammar: The subset of lexical rules of a grammar G is called the **lexicon** of G

Def 12.11. Vocabulary: The set of body symbols are called the vocabulary (or the alphabet).

Def 12.12. Lexical Categories of a Grammar: The nonterminals that appear in the heads of lexical rules are called lexical categories of the grammar.

Def 12.13. Structural Rules: The non lexicon production rules are called structural.

Def 12.14. Phrasal categories: The nonterminals in the heads of production rules that expand into other nonterminals are called phrasal (syntactic) categories.

Def 12.15. G-Derivation: Given a phrase structure grammar $G := \{N, \Sigma, P, S\}$, we say G **derives** $t \in (\Sigma \cup N)^*$ from $s \in (\Sigma \cup N)^*$ in one step, iff there is a production rule $p \in P$ with $p = h \to b$ and there are $u, v \in (\Sigma \cup N)^*$, such that $s = uhv$ and $t = ubv$. We write $s \to_G^p t$ (or $s \to_G t$ if p is clear from the context) and use $\to_G^*$ for the reflexive transitive closure of $\to_G$. We call $s \to_G^* t$ a G-derivation of t from s.

Let's unravel that. Take the following grammar:

- $N = \{S\}$

- $\Sigma = \{a, b\}$
- $P = \{S \to \text{aSb}, S \to \text{ab}\}$
- $S = S$

The following derivations holds:

- $S \to_G^p \text{aSb}$ where $p : S \to \text{aSb}$ (*one step*)
- $\text{aSb} \to_G^q \text{aabb}$ where $q : S \to \text{ab}$ (*one step*)
- $S \to_G^{S \to \text{aSb}} \text{aSb} \to_G^{S \to \text{ab}} \text{aabb}$ (*multiple steps*). So, $S \to_G^* \text{aabb}$

Def 12.16. Sentential Form: Given a phrase structure grammar $G := \{N, \Sigma, P, S\}$, we say that $s \in (\Sigma \cup N)^*$ is a **sentential form** of G, iff $S \to_G^* s$

Def 12.17. Sentence: A sentential form that does not contain nonterminals is called a sentence of G, we also say that G **accepts** s. We say that G **rejects** s, iff it is not a sentence of G.

Examples

1. "Article teacher Vi" is a sentential form

$$\begin{aligned} S &\to_G \text{NP Vi} \\ &\to_G \text{Article N Vi} \\ &\to_G \text{Article teacher Vi} \end{aligned}$$

2. "the teacher sleeps" is a sentence

$$\begin{aligned} S &\to_G^* \text{Article teacher Vi} \\ &\to_G \text{the teacher Vi} \\ &\to_G \text{the teacher sleeps} \end{aligned}$$

Def 12.18. Language Generation: The language $L(G)$ of G is the set of its sentences. We say that $L(G)$ is **generated** by G.

Def 12.19. Equivalent Grammars: We call two grammars **equivalent**, iff they have the same languages.

Def 12.20. Universal Grammar: A grammar G is said to be **universal** if $L(G) = \Sigma^*$

Def 12.21. Syntactic Analysis: Syntactic analysis (parsing / syntax analysis) is the process of analyzing a string of symbols, either in a formal or a natural language by means of a grammar.

> **Note**
>
> The shape of the grammar determines the size of its language

Def 12.22. Context-Sensitive: We call a grammar context-sensitive (or type 1), if the bodies of production rules have no less symbols than the heads.

Example: The language $\left\{a^{[n]}b^{[n]}c^{[n]}\right\}$ is accepted by

$$\begin{aligned} S &\to \text{a b c} \mid A \\ A &\to \text{a } A\ B \text{ c} \mid \text{a b c} \\ \text{c } B &\to B \text{ c} \\ \text{b } B &\to \text{b b} \end{aligned}$$

Def 12.23. Context-Free: We call a grammar context-free (or type 2), iff the heads have exactly one symbol.

Example: The language $\left\{a^{[n]}b^{[n]}\right\}$ is accepted by $S \to \text{ a } S \text{ b } \mid \epsilon$

Def 12.24. Regular: We call a grammar regular (or type 3, or REG), if additionally the bodies are empty or consist of a nonterminal, optionally followed by a terminal symbol.

Example: The language $\left\{a^{[n]}\right\}$ is accepted by $S \to S \text{ a } \mid \epsilon$

Note

By extension, a formal language L is called *context-sensitive / context-free / regular*, iff it is the language of a respective grammar. Context-free grammars are sometimes `CFGs` and context-free languages `CFLs`.

Observation: Natural languages are probably context sensitive but parsable in real time!

13 Term Languages and Abstract Grammars

Note

In most applications of symbolic AI, the formal languages are very strucutred.

Example (Arithmetic Expressions): Consider the CFG G and the G-derivation:

```
term   → num | var | sum | prod | neg
num    → digit*
digit  → 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 0
var    → 'x' num
sum    → '(' term '+' term ')'
prod   → '(' term '*' term ')'
neg    → '-(' term ')'

term → G -(term)
     → G -((term * term))
     → G -(((term + term) * num))
     → G -(((var + num) * digit digit))
     → G -(((x num + digit*) * 555))
     → G -(((x digit digit + 3) * 555))
     → G -(((x29 + 3) * 555))
```

G accepts the string $-(((X29 + 3) * 555))$ but not $-(X29 * 3($.

Note

We call such languages term languages

Observation: Strings in $L(G)$ are *well-bracketed* and can be split into sub-strings from $L(G)$.

Intuition: The parse trees are the primary objects for symbolic AI, the strings are just the technical I/O.

Def 13.1. Parse Tree: Let G be a CFG that accepts a string s. Then a **parse tree** is a node labeled, ordered tree P_s that represents the syntactic strucutre of s.

Example: the parse tree of "the teacher sleeps":

```
        S
       / \
     NP   Vi
    /  \    \
Article N   sleeps
   |    |
  the teacher
```

Note

There may be multiple derivations that accept a string s in a CFG. For example, $S \to$ NP Vi $\to$ Article N Vi $\cdots$ or $S \to$ NP Vi $\to$ NP sleeps $\cdots$, and so on.

Let G be CFG and $D := s \to_G^* t$ a G-derivation where t is a sentence of G. We define the parse tree P_D of D recursively:

- IF $D = s \to_G^p t$, then p is a lexical rule $s \to t$ and t consists of a single terminal. Then P_D is the tree whose root has label s with one child labeled with t.
- If $D = s \to_G^p \bar{s} \to_G^* t$, $\bar{s} \to_G^* t$, and $p = h \to a_1 \cdots a_n$, then the root of P_D has label h and it has children that are the parse trees for the subderivations for the a_i in $\bar{D}$.

Note

Term languages are ones that have enough structural characters so that the parse tree is obvious.

In SML we can program the same grammar as an abstract datatype:

```
datatype aterm =
    anum of int
  | avar of int
  | aneg of aterm
  | asum of aterm * aterm
  | aprod of aterm * aterm;
```

And we can express arithmetic expressions as SML expressions directly, e.g. $-(((X29 + 3) * 555))$:

```
- val ex = aneg (aprod (asum (avar 29, anum 3), anum 555));
val ex = aneg (aprod (asum (avar 29,anum 3),anum 555)) : aterm
```

Moreover, we can traverse the term recursively and print it as a string:

```
fun aprint (anum n) = Int.toString n
  | aprint (avar n) = "x" ^ Int.toString n
  | aprint (aneg t) = "-(" ^ aprint t ^ ")"
  | aprint (asum (t1, t2)) = "(" ^ aprint t1 ^ "+" ^ aprint t2 ^ ")"
  | aprint (aprod (t1, t2)) = "(" ^ aprint t1 ^ "*" ^ aprint t2 ^ ")";
```

Let's test that on our `ex`:

```
- aprint ex;
val it = "-(((x29+3)*555))" : string
```

We can also do more complex tasks via tree traversal. For example, we can evaluate `"-(((x29+3)*555))"`. To do that, we need to know the value of the variable $X29$. This is traditionally given by an assignment that assigns values to variables, e.g. $X1 \mapsto 3, X29 \mapsto 5, \cdots$ (a dictionary), which we can represent as a list of pairs in SML:

```
type assignment = (int * aterm) list
```

Then we can represent the evaluation function by recursion over the structure of the expression:

```
fun aeval (anum n, a:assignment) = n
|   aeval (aneg (n), a) = ~(aeval (n,a))
|   aeval (asum (s,t), a) = aeval (s,a) + aeval (t,a)
|   aeval (aprod (s,t), a) = aeval (s,a) * aeval (t,a);
```

In all cases, the assignment is just passed on to the recursive call. The only exception is the variable case, where we need to recursively find the right value from the assignment, so the final function would look like:

```
fun aeval (anum n, a) = n
  | aeval (aneg t, a) = ~ (aeval (t, a))
  | aeval (asum (s, t), a) = aeval (s, a) + aeval (t, a)
  | aeval (aprod (s, t), a) = aeval (s, a) * aeval (t, a)
  | aeval (avar n, []) = raise Fail "unbound variable"
  | aeval (avar n, (m, r) :: l) =
      if n = m then aeval (r, l) else aeval (avar n, l);
```

Let's first assign $X29 \mapsto 5$

```
val env = [(29, anum 5)];
```

Then test the function on our term:

```
- val ex = aneg (aprod (asum (avar 29, anum 3), anum 555));
val ex = aneg (aprod (asum (avar 29,anum 3),anum 555)) : aterm

- val result = aeval (ex, env);
val result = ~4440 : int
```

Note

The situation for arithmetic expressions in SML is typical for term languages.

- Inductive data types, one constructor per production rule
- Expressions as tree-structured data structures (no brackets needed)
- All computation on expressions via recursive tree traversal

Def 13.2. Tree Grammar: A tree grammar is a grammar where the result data type is trees not strings.

If we interpret the brackets as "parse tree constructors", then we do not have to worry about matching them. The string representation - with e.g. infix notations for operators - is just a derived / secondary artifact for storage / communication (implementation details).

Def 13.3. Abstract vs Concrete Grammars: The underlying tree grammar of a language is often called the **abstract grammar**; it captures the essence of the formal language (the constructors), and any derived (string) grammar - there may be multiple - is called a **concrete grammar**; they correspond to, e.g. the results of print functions.

Note

We are mostly interested in **abstract grammars** in symbolic AI

14 General Problem Solving

Def 14.1. Search Problem: A search problem $\Pi := \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{I}, \mathcal{G} \rangle$ consists of a set $\mathcal{S}$ of states, a set $\mathcal{A}$ of actions, and a transition model $\mathcal{T} : \mathcal{A} \times \mathcal{S} \to \mathcal{P}(\mathcal{S})$ that assigns to any action $a \in \mathcal{A}$ and state $s \in \mathcal{S}$ a set of successor states. Certain states in $\mathcal{S}$ are designated as goal (terminal) states ($\mathcal{G} \subseteq \mathcal{S}$ with $\mathcal{G} \neq \phi$) and initial states $\mathcal{I} \subseteq \mathcal{S}$.

Def 14.2. Action Application: We say that an action $a \in \mathcal{A}$ is applicable in state $s \in \mathcal{S}$, iff $\mathcal{T}(a, s) \neq \phi$ and that any $s' \in \mathcal{T}(a, s)$ is a result of applying action a to state s. We call $\mathcal{T}_a : \mathcal{S} \to \mathcal{P}(\mathcal{S})$ with $\mathcal{T}_a(s) := \mathcal{T}(a, s)$ the result relation for a and $\mathcal{T}_\mathcal{A} := \bigcup_{a \in \mathcal{A}} \mathcal{T}_a$ the result relation of Π.

Def 14.3. State Space: The graph $\langle \mathcal{S}, \mathcal{T}_\mathcal{A} \rangle$ is called the state space induced by Π.

Def 14.4. Solution: A solution for Π consists of a sequence $a_1, \cdots, a_n$ of actions such that for all $1 < i \leq n$:

- a_i is applicable to state s_{i-1} where $s_o \in \mathcal{I}$, and
- $s_i \in \mathcal{T}_{a_i}(s_{i-1})$, and $s_n \in \mathcal{G}$

Def 14.5. Cost Function: Often we add a cost function $c : \mathcal{A} \to \mathbb{R}_0^+$ that associates a step cost $c(a)$ to an action $a \in \mathcal{A}$. The cost of a solution is the sum of the step costs of its actions.

> **Note**
>
> - For deterministic environments, we have $|\mathcal{T}(a, s)| \leq 1$
> - For fully observable ones, we have $\mathcal{I} = \{s_0\}$

Def 14.6. Successor Function/State: In a search problem, $\mathcal{T}_a$ induces a partial function $S_a : \mathcal{S} \rightharpoonup \mathcal{S}$ whose natural domain is the set of states where a is applicable: $S_a(s) := s'$ if $\mathcal{T}_a = \{s'\}$ and undefined at s otherwise. We call S_a the **successor function** for a and $S_a(s)$ the **successor state** of s.

> **Note**
>
> - A search problem is called a **single-state problem** iff it is fully observable, deterministic, static, and discrete.
> - A search problem is called a **multi-state problem** iff it is partially observable.
> - A search problem is called a **contingency problem** iff the environment is non deterministic and the state space is unknown.

Def 14.7. Tree Search: Given a search problem $\Pi := \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{I}, \mathcal{G} \rangle$, the **tree search algorithm** consists of the simulated exploration of the state space $\langle \mathcal{S}, \mathcal{T}_\mathcal{A} \rangle$ in a search tree formed by successively expanding already explored states.

Def 14.8. Path Cost: We define the path cost of a node n in a search tree T to be the sum of the step costs on the path from n to the root of T.

The general structure of Tree Search algorithms is as follows:

```
TREE_SEARCH(start, INSERT):
    fringe ← empty list
    append start to fringe
    while fringe is not empty:
        node ← remove first element of fringe
        if is_goal(node):
            return node
        children ← expand(node)
        INSERT(fringe, children)
    return failure
```

Def 14.9. Strategy: A strategy is a function that picks a node from the fringe of a search tree

> **Note**
>
> Every algorithm has its own strategy, hence the implemention of the `INSERT` differs based on that strategy.

Def 14.10. Properties of Strategies:

- completeness: does it always find a solution if one exists?
- optimality: does it always find the optimal solution (least cost)?

- time complexity: number of nodes the algorithm explores (expands)
- space complexity: maximum number of nodes in memory

> **Note**
>
> Time and space complexity measured in terms of:
> - b: maximum branching factor of the search tree
> - d: minimal graph depth of a solution in the search tree
> - m: maximum graph depth of the search tree

Def 14.11. Breadth-First Search (BFS): The BFS strategy treats the fringe as a FIFO queue, i.e. successors go in at the end (back) of the fringe.

BFS Properties Analysis

- Time Complexity:

  assume we have a binary search tree ($b = 2$) and we found solution in depth $= 3$. In the 0th level we explored $2^0 = 1$ nodes. In the 1st level, $2^1 = 2$ nodes. In the 2nd level, $2^2 = 4$ nodes. And finally $2^3 = 8$ nodes in the third level. Number of nodes the algorithm explored will be $2^0 + 2^1 + 2^2 + 2^3$.

  - Generally, for arbitrary b, d, we have $1 + b + b^2 + \cdots + b^d$, which means, worst time complexity is $\mathcal{O}(b^d)$ (exponential in d).
- The space complexity is also $\mathcal{O}(b^d)$.
- BFS is complete if: (i) b is finite, and (ii) the state space is finite or has a solution.
- BFS is optimal if we have a uniform constant cost for all actions.

Def 14.12. Uniform-Cost Search (UCS): UCS is the strategy where the fringe is ordered by increasing path cost. (*expands least cost first*)

> **Note**
>
> UCS it is equivalent to BFS if all costs are equal

UCS Properties Analysis

- Time and space complexity $\approx \mathcal{O}(b^d)$
- UCS is complete if:
  - b is finite
  - all actions costs are $\geq \epsilon > 0$
  - state space is finite or has a solution.

Def 14.13. Depth-First Search (DFS): DFS is the strategy where the fringe is organized as a LIFO stack, i.e. successors go in at the front of the fringe.

Def 14.14. Backtracking: Every node that is pushed to the stack is called a **backtrack point**. Popping a non-goal node from the stack and continuing the search with the new top element is called **backtracking**. For this reason, the DFS algorithm is also referred to as a **backtracking search**.

DFS Properties Analysis

- Time complexity is $\mathcal{O}(b^m)$ (exponential in the maximum depth)
- Space complexity is linear. While BFS keeps track of all nodes at the current level in memoty, DFS needs to only keep the current path (from root to m), and also needs to keep the remaining unexplored sibling root node for each node along that path (to backtrack). This means, for each level, we store the siblings, i.e. b nodes. We explore until we hit the max depth m and in each level we store b. Hence we have a linear space $\mathcal{O}(b \cdot m)$
- DFS is complete if the search tree is finite and acyclic.
- DFS is not optimal.

Def 14.15. Depth-Limited Search (DLS): DLS is a DFS with a depth limit l. We treat all nodes at depth l as if they have no children.

DLS Properties Analysis

- Time complexity $\rightsquigarrow \mathcal{O}(b^l)$
- Space complexity $\rightsquigarrow \mathcal{O}(b \cdot l)$
- Not complete (solution maybe beyond chosen l)
- Not optimal

Def 14.16. Iterative Deepening Search (IDS): IDS is a DLS with an ever-increasing depth limit. We call the difference between successive depth limits the step size.

> Note
>
> - IDS solves the problem of choosing a good l in DLS.
> - IDS tries all values until either a solution is found (depth is returned to DLS, we call that the cutoff value) or we return failure.
> - IDS combines benefits of DFS and BFS.

IDS Properties Analysis

It is similar to DFS in memory requirements (assuimg finite and acyclic search tree).

- $\mathcal{O}(b \cdot d)$ when there is a solution
- $\mathcal{O}(b \cdot m)$ when there is no solution

It is similar to BFS in terms of optimality, completeness, and time complexity. Optimal for problems where all actions have the same cost. Complete if b is finite and the state space is finite or has a solution.

With regard to time complexity, consider IDS finds a solution at depth d, then:

- nodes at the very bottom (d) are visited only once in the final iteration.
- nodes at $d-1$ are visited twice, once in the search with depth $d-1$ and once in the search with depth d.
- nodes at $d-2$ are visited three times
- and so son, until root node ($d=0$) which is visited d times.

Generally, we have $(d)b^1 + (d-1)b^2 + (d-2)b^3 + \cdots + b^d$. However, the majority of nodes are at the max depth where there are b^d nodes. Those are only visited once and the extra cost of visiting shallower nodes does not significantly increase the overall count. Therefore, time complexity is bounded by b^d ($\mathcal{O}(b^d)$) because b^d dominates the total nodes count in a large search space.

Def 14.17. Graph Search: A graph search algorithm is a variant of a tree search algorithm that prunes nodes whose state has already been considrered (duplicate pruning), essentially using a DAG data strucutre.

The general structure of Graph Search algorithms is as follows:

```
GRAPH_SEARCH(start, INSERT):
    fringe ← empty list
    visited ← empty set
    append start to fringe
    add state(start) to visited
    while fringe is not empty:
        node ← remove first element of fringe
        if is_goal(node):
            return node
        children ← expand(node)
        for each child in children:
            if state(child) not in visited:
                add state(child) to visited
                INSERT(fringe, child)
    return failure
```

Def 14.18. Search Alrogrithm: We speak of a **search algorithm** when we do not want to distinguish whether it is a tree or a graph search algorithm - *difference considered as an implementation detail.*

Def 14.19. Informed: A search algorithm is called **informed**, iff it uses some form of external information to guide the search.

Def 14.20. Evaluation Function: An evaluation function assigns a desirability value to each node of the search tree. It is not part of the search problem, but must be added externally.

Def 14.21. Best-First Search: In best-first search, the fringe is a queue sorted in decreasing order of desirability.

Def 14.22. Heuristic: A heuristic is an evaluation function h on states that estimates the cost form n to the nearest goal state. We speak of **heuristic search** if the search algorithm uses a heuristic in some way.

Def 14.23. Greedy Search: A best-first search strategy where the fringe is organized as a queue sorted based on h value.

Greedy Search Properties

- Time and space complexity are both exponential in m (max depth) $\rightsquigarrow \mathcal{O}(b^m)$
- It is not optimal. But it is complete if the state space is finite.

Def 14.24. Heuristic Function: A heuristic funciton for search problem Π is a function $h : \mathcal{S} \to \mathbb{R}_0^+ \cup \{\infty\}$ so that $h(s) = 0 \quad \forall s \in \mathcal{G}$.

Def 14.25. Goal Distance Function: The function $h^* : \mathcal{S} \to \mathbb{R}_0^+ \cup \{\infty\}$, where $h^*(s)$ is the cost of a cheapest path from s to a goal state, or ∞ if no such path exists. (*the perfect heuristic which we do not know and cannot compute*)

Def 14.26. Admissible: For a search problem Π with states $\mathcal{S}$ and actions $\mathcal{A}$. We say that a heuristic h for Π is **admissible** if

$$h(s) \leq h^*(s) \quad \forall s \in \mathcal{S}$$

> **Note**
>
> Admissible heuristics never overestimates; our guess should be always optimistic or exact, never too high

Def 14.27. Consistent: For a search problem Π with states $\mathcal{S}$ and actions $\mathcal{A}$. We say that a heuristic h for Π is **consistent** if

$$h(s) - h(s') \leq c(a) \quad \forall s \in \mathcal{S}, a \in \mathcal{A}, s' \in \mathcal{T}(s, a)$$

> **Note**
>
> A consistent heuristic never drops faster than the real cost you pay when moving through the state space. Formally, for every transition $s \xrightarrow{a} s'$,
>
> $$h(s) \leq c(a) + h(s')$$
>
> This means the heuristic respects a triangle inequality with respect to the actual step costs. Consistency ensures that the estimated total cost along a path never decreases.

Def 14.28. A* Evaluation Function: Given a path cost function g and a heuristic h, the **A* evaluation function** is

$$f(n) \;=\; g(n) + h(n).$$

It estimates the total cost of a cheapest path from the start state to a goal via n.

Def 14.29. A* Search: A* search is the best-first search strategy that uses the A* evaluation function $f = g + h$ to order the fringe.

Theorem 14.1. A* Search with admissible heuristic is optimal

Def 14.30. Dominance: Let h_1 and h_2 be two admissible heuristic, we say that h_2 **dominates** h_1 if $h_2(s) \geq h_1(s) \quad \forall s \in \mathcal{S}$.

Theorem 14.2. If h_2 dominates h_1, then h_2 is better for search than h_1

Proof. If h_2 dominates h_1, then h_2 is closer to h^* than h_1 □

Def 14.31. Adversarial Search: An adversarial search problem is a search problem $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{I}, \mathcal{G} \rangle$, where:

- $\mathcal{S} = \mathcal{S}^{\text{Max}} \uplus \mathcal{S}^{\text{Min}} \uplus \mathcal{G}$
- $\mathcal{A} = \mathcal{A}^{\text{Max}} \uplus \mathcal{A}^{\text{Min}}$
- For $a \in \mathcal{A}^{\text{Max}}$, if $s \xrightarrow{a} s'$, then $s \in \mathcal{S}^{\text{Max}}$ and $s' \in (\mathcal{S}^{\text{Min}} \cup \mathcal{G})$
- For $a \in \mathcal{A}^{\text{Min}}$, if $s \xrightarrow{a} s'$, then $s \in \mathcal{S}^{\text{Min}}$ and $s' \in (\mathcal{S}^{\text{Max}} \cup \mathcal{G})$

together with a **game utility function** $u : \mathcal{G} \to \mathbb{R}$

Def 14.32. Strategy: Let Θ be an adversarial search problem, and let $X \in \{\text{Max}, \text{Min}\}$. A **strategy** for X is a function $\sigma^X : \mathcal{S}^X \to \mathcal{A}^X$ so that $a \in \mathcal{A}^X$ is applicable to $s \in \mathcal{S}^X$ whenever $\sigma^X(s) = a$.

Def 14.33. Optimal Strategy: A strategy is called **optimal** if it yields the best possible utility for X assuming perfect opponent play.

> **Note**
>
> Assumptions for Adversarial Games:
> - Two players acting in strictly alternating turns
> - Fully observable and deterministic game states
> - Finite state space so the game tree is finite
> - Terminal states have a real-valued utility
> - The game is zero-sum: Max maximizes the utility, Min minimizes the same utility

Def 14.34. Minimax: Let Θ be an adversarial search problem with utility function $u : \mathcal{G} \to \mathbb{R}$. The **minimax value** is the function $\hat{u} : \mathcal{S} \to \mathbb{R}$ defined by:

$$\hat{u}(s) = \begin{cases} u(s) & \text{if } s \in \mathcal{G}, \\ \max\limits_{s' \in \mathcal{T}(a,s)} \hat{u}(s') & \text{if } s \in \mathcal{S}^{\text{Max}}, \\ \min\limits_{s' \in \mathcal{T}(a,s)} \hat{u}(s') & \text{if } s \in \mathcal{S}^{\text{Min}}, \end{cases}$$

where a ranges over all actions applicable to s. The **minimax decision** at a root state r is any action whose successor state achieves $\hat{u}(r)$.

Def 14.35. Minimax Algorithm: The minimax algorithm is given by the following recursive function whose arguments are a node (state) and whether it is Max's turn or Min's.

```
function MINIMAX(node, maximizing):
    if TERMINAL(node):
        return UTILITY(node)

    if maximizing:
        best = -infinity
        for each child in SUCCESSORS(node):
            val = MINIMAX(child, false)
            if val > best:
                best = val
        return best

    else:
        best = +infinity
        for each child in SUCCESSORS(node):
            val = MINIMAX(child, true)
            if val < best:
                best = val
        return best
```

Def 14.36. Evaluation Function: In practice, full-depth minimax is infeasible because the search tree is too large. We therefore impose a fixed *depth limit / horizon* d and stop the search at all states $\{ s \in \mathcal{S} \mid \text{dp}(s) = d \}$, called *cut-off states*.

An evaluation function is a function $f : \mathcal{S} \to \mathbb{R}$ that assigns a numerical estimate of the true minimax value $\hat{u}(s)$ to any nonterminal cut-off state s. If a cut-off state is terminal, its utility $u(s)$ is used instead of $f(s)$.

Def 14.37. Alpha Value: For each node n in a minimax search tree, the alpha value $\alpha(n)$ is the highest Max-node utility that search has encountered on its path from the root to n.

Def 14.38. Beta Value: For each node n in a minimax search tree, the beta value $\beta(n)$ is the highest Min-node utility that search has encountered on its path from the root to n.

Def 14.39. Alpha-Beta Pruning: Alpha-beta pruning is a variant of minimax that avoids exploring branches that cannot change the minimax value.

- At a Max-node n, if a successor s is found with utility $\hat{u}(s) \geq \beta(n)$, then no remaining successors of n can affect its value and they are pruned.
- At a Min-node n, if a successor s is found with utility $\hat{u}(s) \leq \alpha(n)$, then no remaining successors of n can affect its value and they are pruned.

The algorithm returns the same minimax value as full search while expanding fewer nodes.

```
function ALPHABETA(node, alpha, beta, maximizing):
    if TERMINAL(node):
        return UTILITY(node)

    if maximizing:
        best = -infinity
        for each child in SUCCESSORS(node):
            val = ALPHABETA(child, alpha, beta, false)
            if val > best:
                best = val
            if best > alpha:
                alpha = best
            if beta <= alpha:
                break
        return best

    else:
        best = +infinity
        for each child in SUCCESSORS(node):
            val = ALPHABETA(child, alpha, beta, true)
            if val < best:
                best = val
            if best < beta:
                beta = best
            if beta <= alpha:
                break
        return best
```

15 Constraint Satisfaction Problems

Def 15.1. Constraint Satisfaction Problem (CSP): A triple $\gamma := \langle V, D, C \rangle$ where

- V is a finite set of variables,
- D is an V-indexed family of domains: $(D_v)_{v \in V}$
- C is the set of constraints. For a subset $\{v_1, \cdots, v_k\} \subseteq V$, a constraint $C_{\{v_1, \cdots, v_k\}} \subset D_{v_1} \times \cdots \times D_{v_k}$

Def 15.2. Variable Assignment: We call a partial function:

$$\varphi : V \rightharpoonup \bigcup_{v \in V} D_v$$

a **variable assignment** if $\varphi(v) \in D_v \quad \forall v \in \mathrm{dom}(\varphi)$

Def 15.3. Satisfying Assignment: a variable assignment φ **satisfies** a constraint $C_{\{v_1, \cdots, v_k\}}$ iff $(\varphi(v_1), \cdots, \varphi(v_k)) \in C_{\{v_1, \cdots, v_k\}}$.

Def 15.4. Consistent Assignment: a variable assignment φ is called **consistent** iff it satisfies all constraints in C.

Def 15.5. Legal Assignment: A value $d \in D_v$ is called **legal** for a variable $v \in V$ iff $v \mapsto d$ is a consistent assignemnt; otherwise, **illegal**.

Def 15.6. Conflicted: A variable with an illegal value under assignment φ is called **conflicted**.

Def 15.7. CSP Solution: A variable assignment that is total (i.e. function) and consistent is a **solution** for the CSP.

Def 15.8. Satisfiable: A CSP γ is called **satisfiable** iff it has a solution (a total variable assignemnt φ that satisfies all constraints).

Def 15.9. Discrete: We call a CSP **discrete** iff all of the variables have countable domains; otherwise, **continuous**.

Def 15.10. Boolean CSP: A discrete CSP is called **boolean** iff $|D_v| = 2 \quad \forall v \in V$.

> **Note**
>
> Discrete CSPs with domain size d and n variables has a search space of size d^n, so a naive solve (brute-force) is worst case $\mathcal{O}(d^n)$. In general, deciding solvability of a finite-domain CSP is NP-complete.

Def 15.11. Constraint Order (Arity): Let $\gamma = \langle V, D, C \rangle$ be a CSP. For a constraint

$$C_{\{v_1, \ldots, v_k\}} \subseteq D_{v_1} \times \cdots \times D_{v_k},$$

its **order** (or *arity*) is

$$\mathrm{ord}\big(C_{\{v_1, \ldots, v_k\}}\big) := k.$$

The order of γ is

$$\max_{C_{\{v_1, \ldots, v_k\}} \in C} k.$$

A constraint of order 1 is *unary*, order 2 is *binary*, and any constraint with order > 2 is *higher order*.

Def 15.12. Binary: A **binary CSP** is a CSP where each constraint is unary or binary.

Def 15.13. Constraint Graph: A binary CSP forms a graph called the **constraint graph** whose nodes are variables, and whose edges represent the constraints.

Def 15.14. Constraint Network: A constraint network is a CSP $\gamma := \langle V, D, C \rangle$ of order 2, hence, we write a unary constraint as C_v (representing $C_{\{v\}}$) and a binary constraint as C_{uv} (representing $C_{\{u,v\}}$). Note that $C_{uv} = C_{vu}$.

> **Note**
>
> The constraint graph where all constraints are binary is the undirected graph:
>
> $$\big\langle V, \big\{(u, v) \in V^2 \mid C_{uv} \neq D_u \times D_v\big\}\big\rangle$$

CSP as Search

We can induce a search problem $\Pi_\gamma := \langle \mathcal{S}_\gamma, \mathcal{A}_\gamma, \mathcal{T}_\gamma, \mathcal{I}_\gamma, \mathcal{G}_\gamma \rangle$ from the constraint network $\gamma := \langle V, D, C \rangle$.

We define the set of states $\mathcal{S}_\gamma$ as the set of all partial assignments:

$$\mathcal{S}_\gamma := \left\{ \varphi : V \rightharpoonup \bigcup_{v \in V} D_v \mid \varphi(v) \in D_v \quad \forall v \in \mathrm{dom}(\varphi) \right\}$$

We define goal states $\mathcal{G}_\gamma$ as the set of total and consistent assignments

$$\mathcal{G}_\gamma := \left\{ \varphi : V \to \bigcup_{v \in V} D_v \mid (\varphi(v) \in D_v \quad \wedge \quad (\varphi(u), \varphi(v)) \in C_{uv}) \quad \forall u, v \in V \right\}$$

The initial state is when we have no assignments:

$$\mathcal{I}_\gamma := \left\{ \varphi : V \rightharpoonup \bigcup_{v \in V} D_v \mid \mathrm{dom}(\varphi) = \emptyset \right\}$$

Actions represent assignments as pairs:

$$\mathcal{A}_\gamma := \{(v, d) \mid v \in V \wedge d \in D_v\}$$

Transition model, given a current state s and an action $a = (v, d)$ gives us the successor state s'.

$$\mathcal{T}_\gamma : \mathcal{A}_\gamma \times \mathcal{S}_\gamma \to \mathcal{P}(\mathcal{S}_\gamma)$$

Concretely: $\mathcal{T}_\gamma((v, d), s) := \{ s' \}$ where s' is the successor state (we are updating the variable assignment) which will evaluate as follows:

$$s'(w) := \begin{cases} d, & \text{if } w = v \\ s(w), & \text{if } w \in \mathrm{dom}(s) \text{ and } w \neq v \end{cases} \quad \text{and} \quad \mathrm{dom}(s') := \mathrm{dom}(s) \cup \{v\}.$$

Def 15.15. Backtracking Search: Backtracking search is the basic uninformed algorithm for solving CSPs. It is DFS with two improvements, namely:

1. One variable at a time
   - variable assignments are commutative, fixing the order saves us time!
   - i.e. starting from `state(WA=red)` → `state(WA=red, NT=green)` is the same as starting from `state(NT=green)` → `state(NT=green, WA=red)` [1]
   - We only need to consider assignments to a single variable at each step
2. Check constraints as you go
   - Consider only values which do not conflict previous assignments
   - That is not a goal test! We can think about it as incremental goal test.

The pseudocode for backtracking search is shown in Algorithm 1

Algorithm 1 Backtracking Search

```
function BACKTRACKING-SEARCH(csp)
    return RECURSIVE-BACKTRACKING({}, csp)
end function
function RECURSIVE-BACKTRACKING(assignment, csp)
    if assignment is complete then
        return assignment
    end if
    var ← SELECT-UNASSIGNED-VARIABLE(Variables[csp], assignment, csp)
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
        if value is consistent with assignment given Constraints[csp] then
            add {var = value} to assignment
            result ← RECURSIVE-BACKTRACKING(assignment, csp)
            if result ≠ failure then
                return result
            end if
            remove {var = value} from assignment
        end if
    end for
    return failure
end function
```

> **Note**
>
> Note that the exact behaviors of the function: `SELECT-UNASSIGNED-VARIABLE` and `ORDER-DOMAIN-VALUES` are not specified yet

Def 15.16. Forward Checking: Forward checking is a filtering (inference) technique that improves the general uninformed backtracking seach. It propagates information about illegal values. Whenever a variable $v \in V$ is assigned by $d \in D_v$ ($\varphi(v) = d$), we delete all values that are inconsistent with $\varphi(v)$ from every D_u for all variables u connected with v by a constraint.

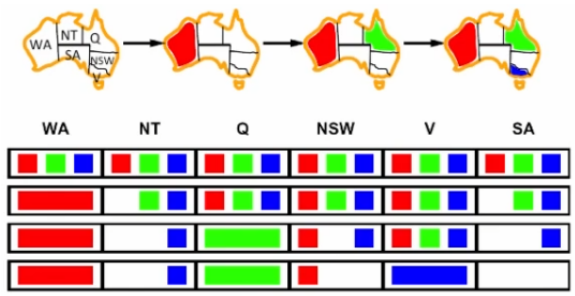
> **Note**
>
> The idea is to keep track of domains for unassigned variables and cross off bad options. In forward checking we cross off values that violate a constraint when added to the existing assignment (look ahead).

forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures. For example, (remember Australia map):

- Assume we chose to assign `WA=Red`
- Because `NT, SA` are neighbors, we remove `Red` from their domains

[1] I am referring to the known Australia map example mentioned in the lecture notes

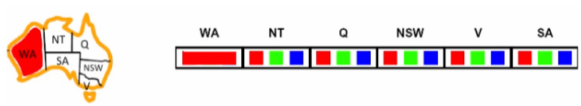
- Assume we pick next `Q=Green`
- Because `NT, SA, NSW` are neighbors, we remove `Green` from their domains
- At this point, forward checking thinks we are in good shape, but we are already doomed! *why?* Because `NT, SA` are neighbors and they both are left with only `Blue` in their domains!
- Assuming, next we choose `V=Green`, we cross off `Green` from `NSW, SA`, now `SA` is left with no colors, only then, we know we failed, so we backtrack.



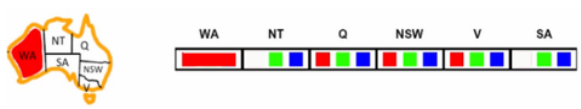
Def 15.17. Arc Consistency: Let $\gamma := \langle V, D, C \rangle$ be a constraint network, a variable $u \in V$ is **arc consistent** relative to another variable $v \in V$ if either $C_{uv} \notin C$, or for every value $d \in D_u$ there exist a value $d' \in D_v$ such that $(d, d') \in C_{uv}$. The constraint network γ is **arc consistent** if every variable $u \in V$ is **arc consistent** relative to every other variable $v \in V$.

> **Note**
>
> Note that when we are checking some arc $x \to y$ and find it inconsistent, we remove values from D_x to make it consistent. When doing so, we need to check every other arc $z \to x$ (where the *tail* of the arc we checked is the *head* of other arcs) even if it was checked before (because now we have fewer options, it is not necessarily consistent anymore!)

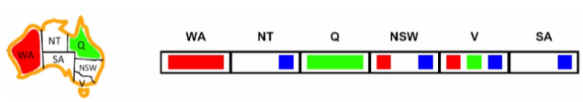
Example:



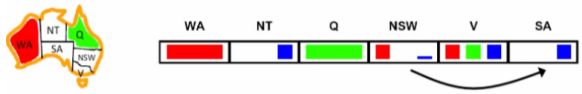
- Checking `NT -> WA` $\rightsquigarrow$ removing `Red` from the domain of `NT`
- Checking `SA -> WA` $\rightsquigarrow$ removing `Red` from the domain of `SA`
- All arcs are consistent now



Assume we pick next `Q=Green`, and therefore remove `Green` from `NT, NSW, SA`:

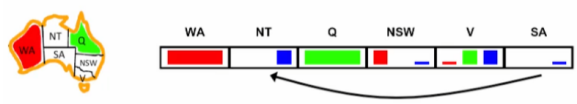


- Looking at `V -> NSW` $\rightsquigarrow$ consistent
- Looking at `SA -> NSW` $\rightsquigarrow$ consistent
- Looking at `NSW -> SA` $\rightsquigarrow$ not consistent! If we choose `NSW = Blue`, nothing I can choose in `SA` that would not violate the constraint!
- We make `NSW -> SA` consistent by removing `Blue` from the domain of `NSW`



- Now, we check again the arc where `NSW` was the head.
- Looking at `V -> NSW` $\rightsquigarrow$ not consistent

- Looking at `SA -> NSW` $\rightsquigarrow$ consistent
- Continue checking arcs. Assume looking at `SA -> NT` $\rightsquigarrow$ not consistent



- Now `SA` domain is empty, we know we failed, so we backtrack!
- Notice that we detect failure earlier than forward checking!

The pseudocode for the arc consistency algorithm (called `AC-3`) is shown in Algorithm 2

Algorithm 2 AC-3

```
function AC-3(csp)
    inputs: csp, a binary CSP with variables {X_1, X_2, ..., X_n}
    local variables: queue, a queue of arcs, initially all the arcs in csp
    while queue is not empty do
        (X_i, X_j) ← REMOVE-FIRST(queue)
        if REMOVE-INCONSISTENT-VALUES((X_i, X_j)) then
            for each X_k in Neighbors[X_i] do
                add (X_k, X_i) to queue
            end for
        end if
    end while
    return csp
end function
function REMOVE-INCONSISTENT-VALUES((X_i, X_j))
    removed ← false
    for each x in Domain[X_i] do
        if no value y in Domain[X_j] allows (x, y) to satisfy the constraint X_i ↔ X_j then
            delete x from Domain[X_i]
            removed ← true
        end if
    end for
    return removed
end function
```

- In the worst case, we have a fully connected constraint network of n variables, hence we will have a total of $n(n-1)$ arcs.
- For one arc $x \to y$, if the number of values each variable can take is d, to make this arc consistent, we have one variable that has d possibilities (*the tail*) and the *head* too (another d possibilities), and we want to make sure for everything in the *tail* there is some value in the *head* that we can assign. That requires $d \cdot d$ checks in the worst case $= d^2$.
- Moreover, it is not enough to check each arc once. We `enqueue` an already `dequeued` arc whenever we eliminate a value from the domain of the arc's *head*. In the worst case, we might eliminate all the values of a variable, i.e. we put an arc back d times.

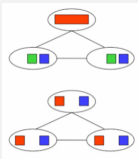
- Therefore, the total run time for `AC-3` is $\mathcal{O}((n(n-1)) \cdot d^2 \cdot d) = \mathcal{O}(n^2 d^3)$

Note

Notice that the run time $\approx$ polynomial and yet detecting all possible future problems is NP-Hard because of backtracking. Arc consistency does not guarantee finding a solution. After enforcing arc consistency, we might have:

- one solution left
- multiple solutions left
- no solutions left (and not know it!)

Moreover, the algorithm still runs inside backtracking search.



Def 15.18. Minimum Remaining Values: The **MRV** heuristic for backtracking search (*a.k.a Most Constrained Variable First*) selects the unassigned variable with the fewest legal values given the current partial assignment φ. Formally, it chooses the variable v that minimizes $\left|\{\, d \in D_v \mid \varphi \cup \{v \mapsto d\} \text{ is consistent} \,\}\right|$.

Note

MRV $\mapsto$ we want to fail early! Hence, MRV ordering is also called fail-fast ordering.

Def 15.19. Least Constraining Value: Given a variable v, the LCV heuristic chooses the least constraining value for v, i.e. the one that rules out the fewest values in the remaining variables. For the current partial assignment φ and a chosen variable v, we pick a value $d \in D_v$ that minimizes $|d' \in D_u \mid u \notin \text{dom}(\varphi), C_{uv} \in C, \text{ and } (d', d) \notin C_{uv}|$

Def 15.20. Independent Subproblems: Assume we have independent connected components of a constraint graph. We treat each component as a separate CSP since it has no constraints outside its variables.

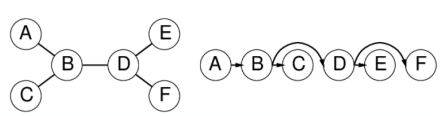
Remember, a naive solver (brute-force) solves a discrete CSPs with domain size d and n variables in $\mathcal{O}(d^n)$ (exponential in n). If that same problem has independent subproblems each with c variables, then we have $\frac{n}{c}$ subproblems where each one in the worst case is $\mathcal{O}(d^c)$. So the total time will be $\mathcal{O}(\frac{n}{c} d^c)$, that is, linear in n and exponential in c. That is very fast for small c.

Def 15.21. Decomposition: The process of decomposing a constraint network into components.

Def 15.22. Tree-Structured CSP: We call a CSP **tree-structured** iff its constraint graph is acyclic.

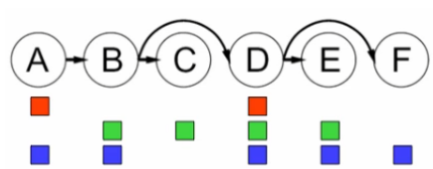
Theorem 15.1. A Tree-Structured CSP can be solved in $\mathcal{O}(nd^2)$

To solve a Tree-Structured CSP, we choose a variable as root and order the variables from root to leaves such that every node's parent precedes it in the ordering:



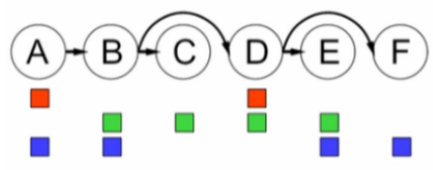
- Backward pass: For $i = n$ and down to 2, apply `Remove-Inconsistent-Values`$((\texttt{Parent}(X_i), X_i))$
- Forward pass: For $i = 1$ and up to n, assign X_i consistently with `Parent`(X_i)

For example, consider the following domains:

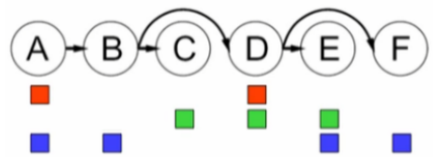


Backward Pass:

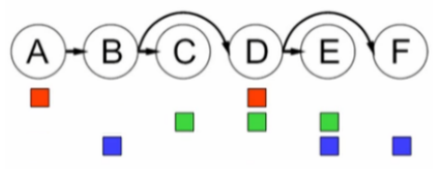
- Start with F, checking $D \to F \rightsquigarrow$ not consistent:



- Next is E, checking $D \to E \rightsquigarrow$ consistent.
- Next is D, checking $B \to D \rightsquigarrow$ consistent.
- Next is C, checking $B \to C \rightsquigarrow$ not consistent:



- Next is B, checking $A \to B \rightsquigarrow$ not consistent:



Forward Pass:

$\varphi(A) = \texttt{Red}$, $\varphi(B) = \texttt{Blue}$, $\varphi(C) = \texttt{Green}$, $\varphi(D) = \texttt{Green}$, $\varphi(E) = \texttt{Blue}$, $\varphi(F) = \texttt{Blue}$

Note

After the backward pass on a tree-structured CSP, every root-to-leaf arc becomes consistent, and no arc ever needs to be enqueued again. The key reason is the direction in which consistency is enforced. Starting from the leaves and moving upward, each arc $x \to y$ is processed only after all of x's children have been handled. When enforcing consistency on $x \to y$, we only remove values from the domain of the tail x. Since all arcs of the form $v \to x$ were already processed earlier, there will be no later step that removes values from the head y of any previously processed arc. In contrast to general AC algorithms, where pruning the head forces re-checking incoming arcs, the tree structure guarantees that once an arc's head has been finalized, it is never modified again. This is why no arc needs to be re-enqueued: domain reductions always flow upward toward the root, never back down.

Note

If root-to-leaf arcs are consistent, the forward pass will not backtrack and will go straight to the solution!

Def 15.23. Conditioning: Instantiate a variable, prune its neighbors' domains

Def 15.24. Cutset conditioning: Instantiate in all ways a set of variables such that the remaining constraint graph is a tree.

Note

A cutset of size c gives runtime $\mathcal{O}(d^c(n-c)d^2)$:

- Because we are left with a tree of size $n-c$, we have $\mathcal{O}((n-c)d^2)$
- Because we have to look at all possible assignments of the cutset, we have $\mathcal{O}(d^c)$

and that is very fast for small c!

16 Formal Systems

Def 16.1. Logical System: A **logical system** (or simply a **logic**) is a triple $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \vDash \rangle$, where

- $\mathcal{L}$ is a set of propositions
- $\mathcal{M}$ is a set of **models**,
- $\vDash$ is a relation $\vDash \subseteq \mathcal{M} \times \mathcal{L}$ called the **satisfaction relation**. We read $\mathcal{M} \vDash A$ as $\mathcal{M}$ **satisfies** A and correspondingly $\mathcal{M} \nvDash A$ as $\mathcal{M}$ **falsifies** A

Let $\langle \mathcal{L}, \mathcal{M}, \vDash \rangle$ be a logical system, $M \in \mathcal{M}$ a model and $A \in \mathcal{L}$ a proposition. Then we say that A is

- **satisfied** by M iff $M \vDash A$
- **satisfiable** iff A is satisfied by some model
- **unsatisfiable** iff A is not satisfiable
- **falsified** by M iff $M \nvDash A$
- **valid** or **unfalsifiable** (write $\vDash A$) iff A is satisfied by **every** model $M \in \mathcal{M}$
- **invalid** or **falsifiable** (write $\nvDash A$) iff A is not valid

Def 16.2. Derivation Relation ($\vdash$): Let $\mathcal{L}$ be a formal language. For any set of propositions $\mathcal{H} \subseteq \mathcal{L}$ (*called the context or hypotheses*) and any proposition $A \in \mathcal{L}$, we write $\mathcal{H} \vdash A$ to say that A is derivable from $\mathcal{H}$. We call a relation $\vdash \subseteq \mathcal{P}(\mathcal{L}) \times \mathcal{L}$ a **derivation relation** for $\mathcal{L}$, iff

- $\mathcal{H} \vdash A$, if $A \in \mathcal{H}$ (proof reflexive),
- $\mathcal{H} \vdash A$ and $(\mathcal{H}' \cup \{A\}) \vdash B$ imply $(\mathcal{H} \cup \mathcal{H}') \vdash B$ (proof transitive),
- $\mathcal{H} \vdash A$ and $\mathcal{H} \subseteq \mathcal{H}'$ imply $\mathcal{H}' \vdash A$ (monotonic admits weakening).

Def 16.3. Inference Rules: Let $\mathcal{L}$ be a formal language, then an **inference rule** over $\mathcal{L}$ is decidable $n+1$-ary relation on $\mathcal{L}$. Inference rules are traditionally written as:

$$\frac{A_1 \quad \dots \quad A_n}{C}\,\mathcal{N}$$

where $A_1, \cdots, A_n$ and C are schemata for words in $\mathcal{L}$ and $\mathcal{N}$ is a name. The A_i are called **assumptions** of $\mathcal{N}$, and C is called its **conclusion**.

Any $n+1$-tuple $\frac{a_1 \quad \dots \quad a_n}{c}$ in $\mathcal{N}$ is called an **application** of $\mathcal{N}$ and we say that we apply $\mathcal{N}$ to a set M of words with $a_1, \cdots, a_n \in M$ to obtain c.

Def 16.4. Axiom: An inference rule without assumptions is called an **axiom**

Def 16.5. Calculus: A **calculus** (or **inference system**) is a formal language $\mathcal{L}$ equipped with a set $\mathcal{C}$ of inference rules over $\mathcal{L}$.

Def 16.6. $\mathcal{C}$-derivation: Let $S := \langle \mathcal{L}, \mathcal{M}, \vDash \rangle$ be a logical system, and $\mathcal{C}$ a calculus for $\mathcal{L}$, then a $\mathcal{C}$-derivation of a proposition $P \in \mathcal{L}$ from a set $\mathcal{H} \subseteq \mathcal{L}$ of hypotheses (write $\mathcal{H} \vdash_{\mathcal{C}} P$) is a sequence $A_1, \cdots, A_m$ of propositions where:

- $A_m = P$,
- for all $1 \leq i \leq m$, either $A_i \in \mathcal{H}$, or,
- there is an inference rule $\frac{A_{l_1} \quad \dots \quad A_{l_k}}{A_i}\,\mathcal{N}$ in $\mathcal{C}$ with $l_j < i$ for all $j \leq k$

We can also see a $\mathcal{C}$-derivation as a **derivation tree**, where A_{l_j} are the children of the node A_i

Def 16.7. $\mathcal{C}$-refutation: Let $\mathcal{C}$ be a calculus, then a formula set Φ is called **$\mathcal{C}$-refutable** if there is a **$\mathcal{C}$-refutation**, i.e. a $\mathcal{C}$-derivation of a contradiction from Φ. The act of finding a **refutation** for Φ is called **refuting** Φ

Def 16.8. Derivation System: We call $\langle \mathcal{L}, \mathcal{M}, \vDash, \vdash \rangle$ a **derivation system**, iff $\langle \mathcal{L}, \mathcal{M}, \vDash \rangle$ is a logical system, and $\vdash$ a derivation relation for $\mathcal{L}$.

Assertion Let $\langle \mathcal{L}, \mathcal{M}, \vDash \rangle$ be a logical system, and $\mathcal{C}$ a calculus, then $\vdash_{\mathcal{C}}$ is a derivation relation and thus $\langle \mathcal{L}, \mathcal{M}, \vDash, \vdash_{\mathcal{C}} \rangle$ is a derivation system

Def 16.9. Formal System: Let $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \vDash \rangle$ be a logical system, and let $\mathcal{C}$ be a calculus over the language $\mathcal{L}$. Then the pair $\langle \mathcal{S}, \mathcal{C} \rangle$ is called a **formal system**.

> Note
>
> Let $\mathcal{C}$ be a calculus, then a $\mathcal{C}$-derivation $\emptyset \vdash_{\mathcal{C}} A$ is called a proof of A and if one exists (write $\vdash_{\mathcal{C}} A$) then A is called a $\mathcal{C}$-theorem.

Def 16.10. Admissible: An inference rule $\mathcal{J}$ is called **admissible** in a calculus $\mathcal{C}$, if the extension of $\mathcal{C}$ by $\mathcal{J}$ does not yield new theorems

Def 16.11. Derived Rules: An inference rule $\dfrac{A_1 \quad \dots \quad A_n}{C}\ \mathcal{N}$ is called **derived / derivable** in a calculus $\mathcal{C}$, if there is a $\mathcal{C}$-derivation $A_1, \cdots, A_n \vdash_{\mathcal{C}} C$.

> Note
>
> Derivable inference rules are admissible, but not the other way around.

Theorem 16.1. It is not true that every admissible inference rule is derivable

Proof.

1. Construct an empty calculus $\mathcal{C}_\emptyset$
2. Since we have no axioms and no inference rules, then the set of theorems is also empty $\mathrm{Th}(\mathcal{C}_\emptyset) = \emptyset$
3. Let $\mathcal{J}$ be an arbitrary inference rule
$$\frac{A_1 \quad \dots \quad A_n}{B}\ \mathcal{J}$$
4. Now we extend our empty calculus by this rule $\mathcal{C}_\emptyset \cup \mathcal{J}$
5. Since we have no axioms, the rule $\mathcal{J}$ has no *input* to operate on, therefore, the set of theorems stays empty $\mathrm{Th}(\mathcal{C}_\emptyset \cup \mathcal{J}) = \emptyset$
6. Thus, $\mathcal{J}$ is admissible.
7. Since $\mathcal{C}_\emptyset$ has no rule and no axioms, it is impossible to construt steps from $A_1, \cdots, A_n$ to B
8. Hence, $\mathcal{J}$ is not derivable
9. Since $\mathcal{J}$ is admissible but not derived, the statement *"every admissible inference rule is derivable"* is False, hence, it is not true that every admissible inference rule is derivable.

$\square$

17 Propositional Logic

Def 17.1. Propositional Logic: Propositional Logic (PL^0) is a logical system $\langle \mathcal{L}, \mathcal{M}, \models \rangle$ Where

- the formal language $\mathcal{L}$ is defined as the well-formed formulae of propositional logic: $\mathrm{wff}_0(\Sigma_{\mathrm{PL}^0}, \mathcal{V}_{\mathrm{PL}^0})$,
- the set $\mathcal{M}$ of models is given by the set $\mathcal{K}_0$ of total functions $\varphi : \mathcal{V}_0 \to \mathcal{D}_0$ (i.e., propositional variable assignments),
- and $\varphi \models A$ iff $\mathcal{I}_\varphi(A) = T$ (a proposition is satisfiable by model φ if the interpretation of it -under that φ- is true)

Hence, propositional logic is a logical system $\langle \mathrm{wff}_0(\Sigma_{\mathrm{PL}^0}, \mathcal{V}_{\mathrm{PL}^0}), \mathcal{K}_0, \models \rangle$

Def 17.2. Formulae of PL^0: The formulae of PL^0 are made up from

- propositional variables $\mathcal{V}_{\mathrm{PL}^0}$ (or just $\mathcal{V}_0$) : $\mathcal{V}_0 := \{P, Q, R, \cdots\}$ (countably infinite)
- propositional signature Σ_{PL^0} (or just Σ_0): $\Sigma_0 := \{\top, \bot, \neg, \vee, \wedge, \Rightarrow, \Leftrightarrow, \cdots\}$ (called *connectives*)

We define the set $\mathrm{wff}_0(\Sigma_{\mathrm{PL}^0}, \mathcal{V}_{\mathrm{PL}^0})$ of **well-formed propositional formulae (wffs)** as:

- propositional variables
- logical constants $\top$ and $\bot$
- negations $\neg A$
- conjunctions $A \wedge B$
- disjunctions $A \vee B$
- implications $A \Rightarrow B$, and
- equivalence (biimplications) $A \Leftrightarrow B$

where $A, B \in \mathrm{wff}_0(\Sigma_{\mathrm{PL}^0}, \mathcal{V}_{\mathrm{PL}^0})$ themselves.

Def 17.3. Grammar for PL^0: The set of propositional formulae $\{A \mid A \in \mathrm{wff}_0(\Sigma_{\mathrm{PL}^0}, \mathcal{V}_{\mathrm{PL}^0})\}$ is given by the following grammar:

$$\begin{array}{rcll} X & ::= & P \mid Q \mid R \mid \cdots & (\textit{propositional variables}) \\ A & ::= & X \mid \top \mid \bot \mid \neg A \mid A \wedge A \mid A \vee A \mid A \Rightarrow A \mid A \Leftrightarrow A & \end{array}$$

Def 17.4. Canonical Model: We call a logical system $\langle \mathcal{L}, \mathcal{M}, \models \rangle$ a **single-model** with **canonical model** M iff $\mathcal{M} = \{M\}$

Def 17.5. PL^0 Models: A model $\mathcal{M} := \langle \mathcal{D}_0, \mathcal{I} \rangle$ for PL^0 consists of

- the universe $\mathcal{D}_0 := \{T, F\}$ (truth values)
- the interpretation $\mathcal{I}$ that assigns truth values to essential connectives
  1. $\mathcal{I}(\neg) : \mathcal{D}_0 \to \mathcal{D}_0$; $T \mapsto F, F \mapsto T$
  2. $\mathcal{I}(\wedge) : \mathcal{D}_0 \times \mathcal{D}_0 \to \mathcal{D}_0$; $\langle \alpha, \beta \rangle \mapsto T$, iff $\alpha = \beta = T$
  3. $\mathcal{I}(\top) = T$
  4. $\mathcal{I}(\bot) = F$

> **Note**
>
> PL^0 is a single-model logical system with canonical model $\langle \mathcal{D}_0, \mathcal{I} \rangle$

Def 17.6. Variable Assignment: PL^0 uses a total variable assignment $\varphi : \mathcal{V}_0 \to \mathcal{D}_0$ that assigns truth values to propositional variables.

Def 17.7. Value Function: PL^0 uses a value function $\mathcal{I}_\varphi : \mathrm{wff}_0(\Sigma_{\mathrm{PL}^0}, \mathcal{V}_{\mathrm{PL}^0}) \to \mathcal{D}_0$ that assigns truth values to PL^0 formulae. It is defined as follows:

- $\mathcal{I}_\varphi(P) = \varphi(P)$
- $\mathcal{I}_\varphi(\neg A) = \mathcal{I}(\neg)(\mathcal{I}_\varphi(A))$

- $\mathcal{I}_\varphi(A \wedge B) = \mathcal{I}(\wedge)(\mathcal{I}_\varphi(A), \mathcal{I}_\varphi(B))$

Note

Value Function Example: Compute $\mathcal{I}_\varphi(P \vee Q)$ under the assignment $\varphi := \{P \mapsto F, Q \mapsto T\}$

$$\begin{aligned}
\mathcal{I}_\varphi(P \vee Q) &= \mathcal{I}_\varphi(\neg(\neg P \wedge \neg Q)) \\
\mathcal{I}_\varphi(\neg P) &= \mathcal{I}(\neg)(\mathcal{I}_\varphi(P)) \\
\mathcal{I}_\varphi(\neg Q) &= \mathcal{I}(\neg)(\mathcal{I}_\varphi(Q)) \\
\mathcal{I}_\varphi(\neg P \wedge \neg Q) &= \mathcal{I}(\wedge)(\mathcal{I}_\varphi(\neg P), \mathcal{I}_\varphi(\neg Q)) \\
\mathcal{I}_\varphi(P \vee Q) &= \mathcal{I}(\neg)(\mathcal{I}_\varphi(\neg P \wedge \neg Q)) \\
&= \mathcal{I}(\neg)(\mathcal{I}(\wedge)(\mathcal{I}_\varphi(\neg P), \mathcal{I}_\varphi(\neg Q))) \\
&= \mathcal{I}(\neg)(\mathcal{I}(\wedge)(\mathcal{I}(\neg)(\mathcal{I}_\varphi(P)), \mathcal{I}(\neg)(\mathcal{I}_\varphi(Q))))
\end{aligned}$$

We know that
- $\mathcal{I}_\varphi(P) = \varphi(P) = F$, and
- $\mathcal{I}_\varphi(Q) = \varphi(Q) = T$

Hence:

$$\begin{aligned}
&= \mathcal{I}(\neg)(\mathcal{I}(\wedge)(\mathcal{I}(\neg)(F), \mathcal{I}(\neg)(T))) \\
&= \mathcal{I}(\neg)(\mathcal{I}(\wedge)(T, F)) \\
&= \mathcal{I}(\neg)(F) \\
&= T
\end{aligned}$$

Def 17.8. Variable Occurrence: A function that maps a PL^0 formula to the set of variable occurred in that formula. It is defined as:

- $\mathrm{Var}(P) = \{P\}$
- $\mathrm{Var}(\top) = \emptyset$
- $\mathrm{Var}(\bot) = \emptyset$
- $\mathrm{Var}(\neg A) = \mathrm{Var}(A)$
- $\mathrm{Var}(A \circ B) = \mathrm{Var}(A) \cup \mathrm{Var}(B)$ where $\circ \in \{\wedge, \vee, \Rightarrow, \Leftrightarrow\}$

Def 17.9. Ground Formula: A PL^0 formula A is called **ground** iff $\mathrm{Var}(A) = \emptyset$

Note

Alternative Notation: write $[\![A]\!]_\varphi$ for $\mathcal{I}_\varphi(A)$ (and $[\![A]\!]$ if A is ground)

Def 17.10. Equivalent: Two formulae A and B are called **equivalent**, iff $\mathcal{I}_\varphi(A) = \mathcal{I}_\varphi(B)$ for all variable assignments φ

Def 17.11. PL^0 Identities: Table 4 shows the identities in PL^0

Table 4: PL^0 Identities

Name	**for** $\wedge$	**for** $\vee$
Idempotence	$\varphi \wedge \varphi = \varphi$	$\varphi \vee \varphi = \varphi$
Identity	$\varphi \wedge \top = \varphi$	$\varphi \vee \bot = \varphi$
Absorption 1	$\varphi \wedge \bot = \bot$	$\varphi \vee \top = \top$
Commutativity	$\varphi \wedge \psi = \psi \wedge \varphi$	$\varphi \vee \psi = \psi \vee \varphi$
Associativity	$\varphi \wedge (\psi \wedge \theta) = (\varphi \wedge \psi) \wedge \theta$	$\varphi \vee (\psi \vee \theta) = (\varphi \vee \psi) \vee \theta$
Distributivity	$\varphi \wedge (\psi \vee \theta) = (\varphi \wedge \psi) \vee (\varphi \wedge \theta)$	$\varphi \vee (\psi \wedge \theta) = (\varphi \vee \psi) \wedge (\varphi \vee \theta)$
Absorption 2	$\varphi \wedge (\varphi \vee \theta) = \varphi$	$\varphi \vee (\varphi \wedge \theta) = \varphi$
De Morgan rule	$\neg(\varphi \wedge \psi) = \neg\varphi \vee \neg\psi$	$\neg(\varphi \vee \psi) = \neg\varphi \wedge \neg\psi$
double negation	$\neg\neg\varphi = \varphi$	
Definitions	$\varphi \Rightarrow \psi \;=\; \neg\varphi \vee \psi$	$\varphi \Leftrightarrow \psi \;=\; (\varphi \Rightarrow \psi) \wedge (\psi \Rightarrow \varphi)$

Let $\mathcal{M} := \langle \mathcal{D}_0, \mathcal{I} \rangle$ be our model, then we say that a formula A is

- **true under** φ in $\mathcal{M}$, iff $\mathcal{I}_\varphi(A) = T$, (write $\mathcal{M} \models^\varphi A$)
- **falsifies** φ in $\mathcal{M}$, iff $\mathcal{I}_\varphi(A) = F$, (write $\mathcal{M} \not\models^\varphi A$)
- **satisfiable in** $\mathcal{M}$, iff $\mathcal{I}_\varphi(A) = T$ for some assignment φ
- **valid in** M, iff $\mathcal{M} \models^\varphi A$ for all variable assignments φ
- **falsifiable in** $\mathcal{M}$, iff $\mathcal{I}_\varphi(A) = F$ for some assignment φ, and
- **unsatisfiable in** $\mathcal{M}$, iff $\mathcal{I}_\varphi(A) = F$ for all variable assignments φ.

Note

Since PL^0 is a single-model logical system, we can omit the explicit reference to the canonical model and write $\varphi \models A$ to mean "A is true under φ", recovering the notation from $\mathcal{M} \models^\varphi A$.

Def 17.12. Entailment: We say that A **entails** B (write $A \models B$), iff $\mathcal{I}_\varphi(B) = T$ for all φ with $\mathcal{I}_\varphi(A) = T$ (i.e. all variable assignments) that makes A true also make B true

Theorem 17.1. $A \models B \Rightarrow A \wedge C \models B \wedge C$

Proof.

1. Assume that $A \models B$ ($\mathcal{H}_1$), we want to show that for all φ that makes $A \wedge C$ true also make $B \wedge C$ true
2. Assume there is some φ that makes $A \wedge C$ true $\rightsquigarrow \mathcal{I}_\varphi(A \wedge C) = T$ ($\mathcal{H}_2$)
3. This means that $\mathcal{I}_\varphi(A) = \mathcal{I}_\varphi(C) = T$
4. Given $\mathcal{H}_1$ we now know that our φ also make B true $\rightsquigarrow \mathcal{I}_\varphi(B) = T$
5. From 3 and 4 and by the definition of conjunction: $\mathcal{I}_\varphi(B \wedge C) = T$

□

Def 17.13. Soundness: Let $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ be a logical system and $\mathcal{H} \subseteq \mathcal{L}$ a set of hypotheses, then we call a calculus $\mathcal{C}$ over $\mathcal{L}$ **sound** (correct), iff $\mathcal{H} \models A$ whenever $\mathcal{H} \vdash_\mathcal{C} A$

Def 17.14. Completeness: Let $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ be a logical system and $\mathcal{H} \subseteq \mathcal{L}$ a set of hypotheses, then we call a calculus $\mathcal{C}$ over $\mathcal{L}$ **complete**, iff $\mathcal{H} \vdash_\mathcal{C} A$ whenever $\mathcal{H} \models A$

Def 17.15. Hilbert Calculus: The Hilbert calculus $\mathcal{H}^0$ is a calculus that consists of the following inference rules:

$$\frac{}{P \Rightarrow (Q \Rightarrow P)}\ K \qquad \frac{}{(P \Rightarrow (Q \Rightarrow R)) \Rightarrow ((P \Rightarrow Q) \Rightarrow (P \Rightarrow R))}\ S$$

$$\frac{A \Rightarrow B \quad A}{B}\ \text{MP} \qquad \frac{A}{[B/X](A)}\ \text{Subst}$$

Note

Subst inference rule is admissible but not derivable

Def 17.16. Hilbert Formal System: Using propositional logic and the Hilbert Calculus, we can build a simple formal system:

$$\langle \underbrace{\langle \mathcal{L}, \mathcal{M}, \models \rangle}_{\text{propositional logic}}, \underbrace{\mathcal{H}^0}_{\text{Hilbert Calculus}} \rangle$$

Example: $C \Rightarrow C$ is a $\mathcal{H}^0$ and here is its proof:

Proof. We show that $\emptyset \vdash_{\mathcal{H}^0} C \Rightarrow C$

1. axiom S gives us:
$$(P \Rightarrow (Q \Rightarrow R)) \Rightarrow ((P \Rightarrow Q) \Rightarrow (P \Rightarrow R))$$

2. We apply Subst with $[C/P], [C \Rightarrow C/Q], [C/R]$, we get:

$$(C \Rightarrow ((C \Rightarrow C) \Rightarrow C)) \Rightarrow ((C \Rightarrow (C \Rightarrow C)) \Rightarrow (C \Rightarrow C))$$

3. axiom K gives us:

$$P \Rightarrow (Q \Rightarrow P)$$

4. We apply Subst with $[C/P], [C \Rightarrow C/Q]$, we get:

$$C \Rightarrow ((C \Rightarrow C) \Rightarrow C)$$

5. We use MP on 4 and 2, we get:

$$(C \Rightarrow (C \Rightarrow C)) \Rightarrow (C \Rightarrow C)$$

6. axiom K gives us:

$$P \Rightarrow (Q \Rightarrow P)$$

7. We apply Subst with $[C/P], [C/Q]$, we get:

$$C \Rightarrow (C \Rightarrow C)$$

8. We apply MP on 7 and 5, we get:

$$C \Rightarrow C$$

□

Def 17.17. $\mathcal{ND}_0$**:** The **propositional natural deduction calculus** $\mathcal{ND}_0$ has inference rules for the introduction and elimination of connectives. $\mathcal{ND}_0$ rules are shown in figure 1

$$\frac{A \quad B}{A \wedge B}\ \wedge I \qquad \frac{A \wedge B}{A}\ \wedge E_l \qquad \frac{A \wedge B}{B}\ \wedge E_r$$

$$\frac{\begin{array}{c}[A]^1\\ \vdots \\ B\end{array}}{A \Rightarrow B}\ \Rightarrow I^1 \qquad \frac{A \Rightarrow B \quad A}{B}\ \Rightarrow E$$

$$\frac{}{A \vee \neg A}\ \mathcal{TND}$$

- - - -

Def 17.18. $\mathcal{ND}_0$ **Derivation Tree:** Given a set $\mathcal{H} \subseteq \mathrm{wff}_0(\Sigma_{\mathrm{PL}^0}, \mathcal{V}_{\mathrm{PL}^0})$ of assumptions and a conclusion C, we write $\mathcal{H} \vdash_{\mathcal{ND}_0} C$, iff there is an $\mathcal{ND}_0$ derivation tree whose leaves are in $\mathcal{H}$

Def 17.19. Hypothetical Reasoning: Notice that the rule $(\Rightarrow I)$ proves $A \Rightarrow B$ by exhibiting an $\mathcal{ND}_0$ derivation D (*depicted by the three vertical dots*) of B from a **local hypothesis** A. $(\Rightarrow I)$ then **discharges** the local hypothesis (*get rid of A, which can only be used in D*) and concludes $A \Rightarrow B$. This mode of reasoning is called **hypothetical reasoning** (proof by local hypothesis).

Note

The $\mathcal{TND}$ axiom in the $\mathcal{ND}_0$ calculus is called *the law/principle of excluded middle/third* (in latin *tertium non datur*) is only acceptable in **classical logic**, intuitionistic/constructive logic/mathematics does not affirm that law. (That applies for the derived rule $(\neg E)$ too)

$$\frac{}{\Gamma, A \vdash A}\ \mathrm{Ax} \qquad \frac{\Gamma \vdash B}{\Gamma, A \vdash B}\ \mathrm{weaken} \qquad \frac{}{\Gamma \vdash A \vee \neg A}\ \mathcal{TND}$$

$$\frac{\Gamma \vdash A \qquad \Gamma \vdash B}{\Gamma \vdash A \wedge B}\ \wedge I \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A}\ \wedge E_l \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B}\ \wedge E_r$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash A \vee B}\ \vee I_l \qquad \frac{\Gamma \vdash A \vee B \qquad \Gamma, A \vdash C \qquad \Gamma, B \vdash C}{\Gamma \vdash C}\ \vee E \qquad \frac{\Gamma \vdash B}{\Gamma \vdash A \vee B}\ \vee I_r$$

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \Rightarrow B}\ \Rightarrow I \qquad \frac{\Gamma \vdash A \Rightarrow B \qquad \Gamma \vdash A}{\Gamma \vdash B}\ \Rightarrow E$$

$$\frac{\Gamma, A \vdash \bot}{\Gamma \vdash \neg A}\ \neg I \qquad \frac{\Gamma \vdash \neg\neg A}{\Gamma \vdash A}\ \neg E$$

$$\frac{\Gamma \vdash \neg A \qquad \Gamma \vdash A}{\Gamma \vdash \bot}\ \bot I \qquad \frac{\Gamma \vdash \bot}{\Gamma \vdash A}\ \bot E$$

Figure 2: Propositional Sequent Style Natural Deduction Calculus

Note

The sequent style of $\mathcal{ND}_0$ calculus ($\mathcal{ND}^0_{\vdash}$) is shown in figure 2

Example: Figures 3 to 6 show different $\mathcal{ND}_0$ proof styles for the following propositional formula:

$$(A \vee B) \wedge (A \Rightarrow C) \wedge (B \Rightarrow C) \Rightarrow C$$

Let $\Gamma = (A \vee B) \wedge ((A \Rightarrow C) \wedge (B \Rightarrow C))$.

$$\dfrac{\dfrac{\dfrac{}{\Gamma \vdash \Gamma}\,\text{Ax}}{\Gamma \vdash A \vee B}\,\wedge E_l \qquad \dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma \vdash \Gamma}\,\text{Ax}}{\Gamma, A \vdash \Gamma}\,\text{weaken}}{\Gamma, A \vdash (A \Rightarrow C) \wedge (B \Rightarrow C)}\,\wedge E_r}{\Gamma, A \vdash A \Rightarrow C}\,\wedge E_l \qquad \dfrac{}{\Gamma, A \vdash A}\,\text{Ax}}{\Gamma, A \vdash C}\,\Rightarrow E \qquad \dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\Gamma \vdash \Gamma}\,\text{Ax}}{\Gamma, B \vdash \Gamma}\,\text{weaken}}{\Gamma, B \vdash (A \Rightarrow C) \wedge (B \Rightarrow C)}\,\wedge E_r}{\Gamma, B \vdash B \Rightarrow C}\,\wedge E_r \qquad \dfrac{}{\Gamma, B \vdash B}\,\text{Ax}}{\Gamma, B \vdash C}\,\Rightarrow E}{\dfrac{\Gamma \vdash C}{\vdash \Gamma \Rightarrow C}\,\Rightarrow I}\,\vee E$$

Figure 3: Gentzen Sequent Style Proof

Let $\Gamma = (A \vee B) \wedge ((A \Rightarrow C) \wedge (B \Rightarrow C))$.

$$\dfrac{\dfrac{[\Gamma]^1}{A \vee B}\,\wedge E_l \qquad \dfrac{\dfrac{\dfrac{[\Gamma]^1}{(A \Rightarrow C) \wedge (B \Rightarrow C)}\,\wedge E_r}{A \Rightarrow C}\,\wedge E_l \qquad [A]^2}{C}\,\Rightarrow E \qquad \dfrac{\dfrac{\dfrac{[\Gamma]^1}{(A \Rightarrow C) \wedge (B \Rightarrow C)}\,\wedge E_r}{B \Rightarrow C}\,\wedge E_r \qquad [B]^2}{C}\,\Rightarrow E}{\dfrac{C}{\Gamma \Rightarrow C}\,\Rightarrow I^1}\,\vee E^2$$

Figure 4: Gentzen-Prawitz Style Proof

Step	**Formula**	**Rule Applied**
(1)	$(A \vee B) \wedge (A \Rightarrow C) \wedge (B \Rightarrow C)$	Assumption
(2)	$A \vee B$	$\wedge E_l$ (on 1)
(3)	$(A \Rightarrow C) \wedge (B \Rightarrow C)$	$\wedge E_r$ (on 1)
(4)	$A \Rightarrow C$	$\wedge E_l$ (on 3)
(5)	$B \Rightarrow C$	$\wedge E_r$ (on 3)
(6)	A	Assumption (Case 1)
(7)	C	$\Rightarrow E$ (on 4 and 6)
(8)	B	Assumption (Case 2)
(9)	C	$\Rightarrow E$ (on 5 and 8)
(10)	C	$\vee E$ (on 2, 7 and 9)
		Discharges assumptions 6 & 8
(11)	$((A \vee B) \wedge (A \Rightarrow C) \wedge (B \Rightarrow C)) \Rightarrow C$	$\Rightarrow I$ (on 1 and 10)
		Discharges assumption 1

Figure 5: Linearized (Tabular) Style Proof

1	$(A \lor B) \land ((A \Rightarrow C) \land (B \Rightarrow C))$	Assumption
2	$A \lor B$	$\land E_l$ (on 1)
3	$(A \Rightarrow C) \land (B \Rightarrow C)$	$\land E_r$ (on 1)
4	$A \Rightarrow C$	$\land E_l$ (on 3)
5	$B \Rightarrow C$	$\land E_r$ (on 3)
6	$\quad A$	Assumption
7	$\quad C$	$\Rightarrow$ E (on 4, 6)
8	$\quad B$	Assumption
9	$\quad C$	$\Rightarrow$ E (on 5, 8)
10	C	$\lor$E (on 2, 7, 9)
11	$((A \lor B) \land ((A \Rightarrow C) \land (B \Rightarrow C))) \Rightarrow C$	$\Rightarrow$ I (on 1–10)

Figure 6: Fitch Style Proof

Def 17.20. First-Order Signature: A First-Order (Logic) Signature (FOL Signature) is a tuple $\Sigma := \langle \Sigma^f, \Sigma^p \rangle$ where:

- $\Sigma^f := \bigcup_{k \in \mathbb{N}} \Sigma^f_k$ of **function constants** (or **terms**), where members of Σ^f_k denote the k-ary functions on **individuals**
- $\Sigma^p := \bigcup_{k \in \mathbb{N}} \Sigma^p_k$ of **predicate constants**, where member of Σ^p_k denote k-ary relations among individuals
- Σ^f_k and Σ^p_k are pairwise disjoint, countable sets of symbols for each $k \in \mathbb{N}$.
- A 0-ary **function constant** refers to a single **individual**, therefore, we call it an **individual constant**

Def 17.21. Predicate Logic Without Quantifiers PL^{nq}: Given a FOL Signature Σ, the **formulae** of PL^{nq} are given by the following grammar:

$$\begin{array}{rcl} f^k & \in & \Sigma^f_k \\ p^k & \in & \Sigma^f_p \\ t & := & f^0 \mid f^k(t_1, \cdots, t_k) \\ A & := & p^k(t_1, \cdots, t_k) \mid \neg A \mid A \land A \end{array}$$

Def 17.22. Well-Formed Terms: We denote the set of all well-formed terms over a FOL Signature Σ with $\text{wff}_\iota(\Sigma)$, and the closed ones with $\text{cwff}_\iota(\Sigma)$

Def 17.23. Well-Formed Formulae: We denote the set of all well-formed formulae over a FOL Signature Σ with $\text{wff}_o(\Sigma)$, and the closed ones with $\text{cwff}_o(\Sigma)$

Note

The Greek letter ι (read *iota*) stands for individuals (objects), however, o (read *omicron*) refers to propositions that can be true of false

Def 17.24. PL^{nq} Universe: PL^{nq} uses the universe $\mathcal{D}_{\text{PL}^{\text{nq}}} := \mathcal{D}_0 \cup \mathcal{D}_\iota$ where $\mathcal{D}_0 = \{T, F\}$ is the set of truth values and $\mathcal{D}_\iota \neq \emptyset$ is a non-empty set of individuals

Def 17.25. Interpretation in PL^{nq}: The interpretation function in PL^{nq} assigns values to constants:

1. We use the interpretation from PL^0
   - $\mathcal{I}(\neg) : \mathcal{D}_0 \to \mathcal{D}_0$; $T \mapsto F, F \mapsto T$
   - $\mathcal{I}(\land) : \mathcal{D}_0 \times \mathcal{D}_0 \to \mathcal{D}_0$; $\langle \alpha, \beta \rangle \mapsto T$, iff $\alpha = \beta = T$
   - $\mathcal{I}(\top) = T$
   - $\mathcal{I}(\bot) = F$
2. We interpret individual constants as individuals: $\mathcal{I} : \Sigma^f_0 \to \mathcal{D}_\iota$
3. We interpret function constants as functions: $\mathcal{I} : \Sigma^f_{k \neq 0} \to \mathcal{D}_\iota^{k \neq 0} \to \mathcal{D}_\iota$

4. We interpret predicate constants as relations: $\mathcal{I} : \Sigma_k^p \to \mathcal{P}(\mathcal{D}_\iota^k)$

Note

This definition naturally includes propositions as a special case. By defining $\mathcal{I} : \Sigma_k^p \to \mathcal{P}(\mathcal{D}_\iota^k)$, we see that for $k = 0$:

$$\mathcal{D}_\iota^0 = \{\langle\rangle\} \implies \mathcal{P}(\mathcal{D}_\iota^0) = \{\emptyset, \{\langle\rangle\}\} \cong \{F, T\}$$

Thus, a 0-ary predicate constant is interpreted as a truth value (a proposition), just like in propositional logic.

Def 17.26. Value Function: The value function assigns values to formulae:

1. $\mathcal{I}(f(A_1, \cdots, A_k)) := \mathcal{I}(f)(\mathcal{I}(A_1), \cdots, \mathcal{I}(A_k))$
2. $\mathcal{I}(p(A_1, \cdots, A_k)) := T,$ iff $\langle \mathcal{I}(A_1), \cdots, \mathcal{I}(A_k)\rangle \in \mathcal{I}(p)$
3. and Just as in PL^0:
   - $\mathcal{I}(\neg A) := \mathcal{I}(\neg)(\mathcal{I}(A))$
   - $\mathcal{I}(A \wedge B) := \mathcal{I}(\wedge)(\mathcal{I}(A), \mathcal{I}(B))$

Theorem 17.2. $\mathrm{PL}^{\mathrm{nq}}$ is **isomorphic** to PL^0

Theorem 17.3. Unsatisfiability Theorem. $\mathcal{H} \vDash A \Leftrightarrow \mathcal{H} \cup \{\neg A\}$ is unsatisfiable.

Proof.

1. Assume $H \vDash A$
   - For any φ with $\varphi \vDash H$ we have $\varphi \vDash A$ and thus $\varphi \nvDash (\neg A)$
2. Assume $\mathcal{H} \cup \{\neg A\}$ is unsatisfiable
   - For any φ with $\varphi \vDash \mathcal{H}$ we have $\varphi \nvDash (\neg A)$ and thus $\varphi \vDash A$

□

18 Machine-Oriented Calculi for Propositional Logic

Note

From Theorem 17.3, we observe that entailment can be tested via satisfiablity

Def 18.1. Theorem Proving: Given a formal system $\langle \mathcal{S}, \mathcal{C} \rangle$, where $\mathcal{C}$ is a calculus and $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ is a logical system, the task of **theorem proving** is the task of determining whether $\mathcal{H} \vdash_{\mathcal{C}} C$ for a **conjecture** $C \in \mathcal{L}$ and hypotheses $\mathcal{H} \subseteq \mathcal{L}$

Def 18.2. Automated Theorem Proving: **ATP** is the automation of theorem proving.

Def 18.3. Test Calculus: For a given conjecture A and hypotheses $\mathcal{H}$, a **test calculus** $\mathcal{T}$ tries to derive a refutation $\mathcal{H}, \neg A \vdash_{\mathcal{T}} \bot$ instead of $\mathcal{H} \vdash A$, where $\neg A$ is unsatisfiable iff A is valid and $\bot$ an unsatisfiable proposition.

Def 18.4. Atomic Formulae: A formula is called **atmoic** (or an **atom**) if it does not contain logical constants, otherwise, **complex**

Def 18.5. Labeled Formula: Let $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ be a logical system, $A \in \mathcal{L}$ a formula, L a **label set**, and $\alpha \in L$ a **label**, then we call a pair $\langle A, \alpha \rangle$ a **labeled formula** and write it as A^α. For a set Φ of propositions we use $\Phi^\alpha := \{A^\alpha \mid A \in \Phi\}$

Note

If the label set is $\mathcal{D}_0 := \{T, F\}$, we call a labeled formula A^T **positive** and A^F **negative**

Note

Let $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ be a logical system, and A^α a labeled formula. Then we say that $M \in \mathcal{M}$ **satisfies** A^α (write $M \models A^\alpha$), iff $\alpha = T$ and $M \models A$ or $\alpha = F$ and $M \not\models A$

Def 18.6. Literal: Let $\mathcal{S} := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ be a logical system, $A \in \mathcal{L}$ is an atomic formula, and $\alpha \in \{T, F\}$, then we call the labeled formula A^α a **literal**.

Def 18.7. Opposite Literal: For a literal A^α, we call the literal A^β with $\alpha \neq \beta$ the **opposite literal**.

Def 18.8. CNF: A formula is in **conjunctive normal form (CNF)** if it is $\top$ or a conjunction of disjunctions of literals, i.e. if it is of the form:

$$\bigwedge_{i=1}^{n} \bigvee_{j=1}^{m_i} l_{ij} := \underbrace{\overbrace{(l_{1,1} \vee \cdots \vee l_{1,m_1})}^{\text{Clause } i=1 \text{ has } m_1 \text{ literals}} \wedge \overbrace{(l_{2,1} \vee \cdots \vee l_{2,m_2})}^{\text{Clause } i=2 \text{ has } m_2 \text{ literals}} \wedge \cdots \wedge \overbrace{(l_{n,1} \vee \cdots \vee l_{n,m_n})}^{\text{Clause } i=n \text{ has } m_n \text{ literals}}}_{\text{There are } n \text{ total clauses}}$$

Def 18.9. DNF: A formula is in **disjunctive normal form (DNF)** if it is $\bot$ or a disjunction of conjunctions of literals, i.e. if it is of the form:

$$\bigvee_{i=1}^{n} \bigwedge_{j=1}^{m_i} l_{ij} := \underbrace{\overbrace{(l_{1,1} \wedge \cdots \wedge l_{1,m_1})}^{\text{Term } i=1 \text{ has } m_1 \text{ literals}} \vee \overbrace{(l_{2,1} \wedge \cdots \wedge l_{2,m_2})}^{\text{Term } i=2 \text{ has } m_2 \text{ literals}} \vee \cdots \vee \overbrace{(l_{n,1} \wedge \cdots \wedge l_{n,m_n})}^{\text{Term } i=n \text{ has } m_n \text{ literals}}}_{\text{There are } n \text{ total terms}}$$

Note

$\top$ is in CNF because it represents the **empty conjunction**. (The semantic identity for conjunction is T, so we use the constant $\top$ where $\mathcal{I}(\top) = T$). $\bot$ is in DNF because it represents the **empty disjunction**. (The semantic identity for disjunction is F, so we use the constant $\bot$ where $\mathcal{I}(\bot) = F$).

Def 18.10. Tableau Calculus: A **tableau calculus** is a test calculus that analyzes a labeled formula in a tree to determine satisfiability, its branches correspond to valuations (models)

Def 18.11. Saturated Tableau: We call a tableau **saturated** iff no application of a rule adds new material to any branch.

Def 18.12. Closed and Open Branches: A tableau branch is **closed** if it ends with $\bot$. Otherwise, the branch is **open**.

Note

We sometimes decorate **open** branches with a $\square$ symbol to visually indicate they are finished and non-contradictory.

Def 18.13. Closed Tableau: A tableau is **closed** iff every one of its branches is closed. Otherwise, the tableau is **open**.

Note

It is crucial to distinguish between the **outcome** of a proof and the **completion** of the process:

- **Closed vs. Open:** This refers to the *logical status*. A tableau is **closed** if a contradiction is found on **every** branch. It is **open** if at least one branch remains consistent.
- **Saturated vs. Unsaturated:** This refers to the *construction status*. A tableau is **saturated** if no more rules can be applied to any branch.

Note: A **closed** tableau is rarely **saturated**. Once a specific branch closes, we stop applying rules to **that branch** for efficiency, even if formulas on that branch could still be expanded.

Note

Open branches in a saturated tableaux yield satisfiying assignments

Def 18.14. $\mathcal{T}_0$**:** The Propositional Tableau Calculus $\mathcal{T}_0$ has the inference rules shown in figure 7

$$\frac{(A \wedge B)^T}{\begin{array}{c} A^T \\ B^T \end{array}}\ \mathcal{T}_0\wedge \qquad \frac{(A \wedge B)^F}{A^F \mid B^F}\ \mathcal{T}_0\vee$$

$$\frac{(\neg A)^T}{A^F}\ \mathcal{T}_0\neg^T \qquad \frac{(\neg A)^F}{A^T}\ \mathcal{T}_0\neg^F$$

$$\frac{\begin{array}{c} A^\alpha \\ \vdots \\ A^\beta \quad (\alpha \neq \beta) \end{array}}{\bot}\ \mathcal{T}_0\bot$$

$$\frac{(A \Rightarrow B)^T}{A^F \mid B^T} \qquad \frac{(A \Rightarrow B)^F}{\begin{array}{c} A^T \\ B^F \end{array}} \qquad \frac{\begin{array}{c} A^T \\ (A \Rightarrow B)^T \end{array}}{B^T}$$

$$\frac{(A \vee B)^T}{A^T \mid B^T} \qquad \frac{(A \vee B)^F}{\begin{array}{c} A^F \\ B^F \end{array}}$$

$$\frac{(A \iff B)^T}{\begin{array}{c} A^T \\ B^T \end{array} \mid \begin{array}{c} A^F \\ B^F \end{array}} \qquad \frac{(A \iff B)^F}{\begin{array}{c} A^T \\ B^F \end{array} \mid \begin{array}{c} A^F \\ B^T \end{array}}$$

Figure 7: Propositional Tableau Calculus $\mathcal{T}_0$

Def 18.15. $\mathcal{T}_0$**-Theorem:** A is a $\mathcal{T}_0$-theorem ($\vdash_{\mathcal{T}_0} A$), iff there is a closed tableau with A^F at the root.

Def 18.16. : A labeled formula A^α is **valid under** φ, iff $\mathcal{I}_\varphi(A) = \alpha$

Def 18.17. Satisfiable Tableau: A tableau $\mathcal{T}$ is **satisfiable**, iff there is a satisfiable branch $\mathcal{B}$ in $\mathcal{T}$, i.e. if the set of

formulae on $\mathcal{B}$ is satisfiable.

Theorem 18.1. $\mathcal{T}_0$ is sound, i.e. $\Phi \subseteq \mathrm{wff}_0(\Sigma_{\mathrm{PL}^0}, \mathcal{V}_{\mathrm{PL}^0})$ is valid if there is a closed tableau $\mathcal{T}$ for Φ^F.

Theorem 18.2. $\mathcal{T}_0$ is complete, i.e. if $\Phi \subseteq \mathrm{wff}_0(\Sigma_{\mathrm{PL}^0}, \mathcal{V}_{\mathrm{PL}^0})$ is valid, then there is a closed tableau $\mathcal{T}$ for Φ^F.

Lemma 18.3. $\mathcal{T}_0$ terminates, i.e. every $\mathcal{T}_0$ tableau becomes saturated after finitely many rule applications.

Corollary 18.4. $\mathcal{T}_0$ induces a decision procedure for validity in PL^0.

Proof.

- By Lemma 18.3, it is decidable whether $\vdash_{\mathcal{T}_0} A$
- By Theorem 18.1 and Theorem 18.2, $\vdash_{\mathcal{T}_0} A$ iff A is valid

□

Example: Figure 8 shows a $\mathcal{T}_0$ proof of the formula $(A \vee B) \wedge (A \Rightarrow C) \wedge (B \Rightarrow C) \Rightarrow C$

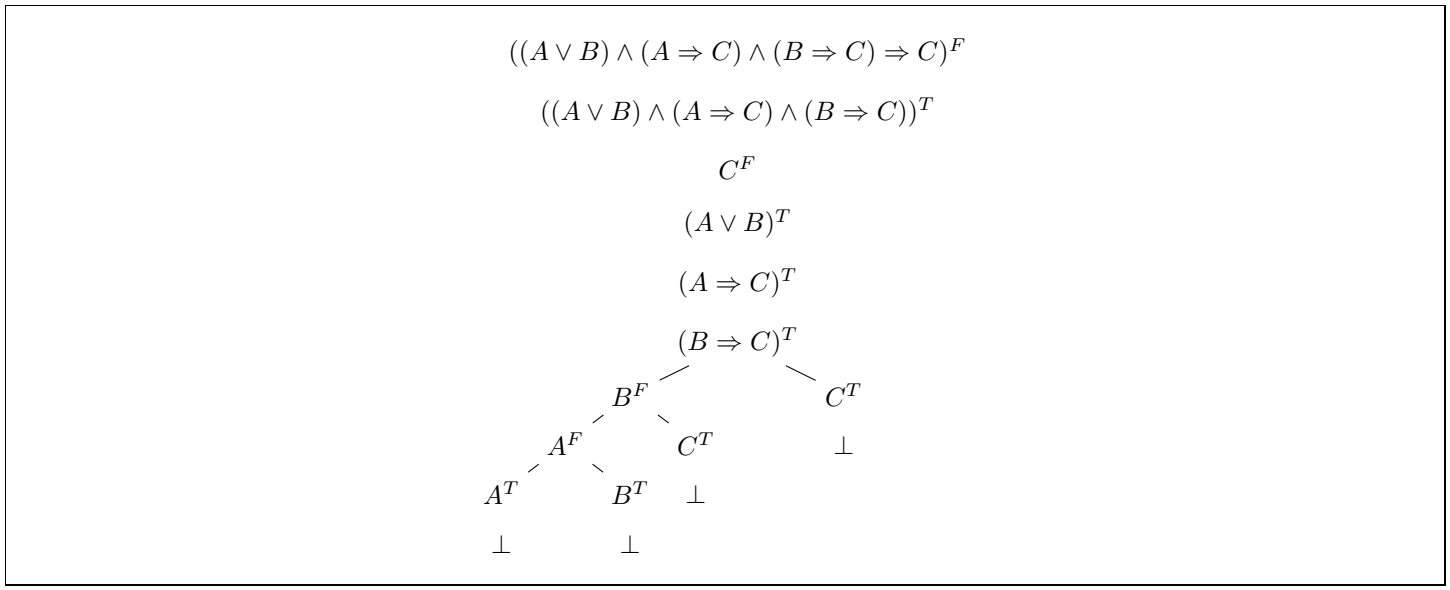


Figure 8: $\mathcal{T}_0$ Proof Example

Def 18.18. Clause: A **clause** is a disjunction $l_1^{\alpha_1} \vee \cdots \vee l_n^{\alpha_n}$ of literals. We use $\square$ for the *"empty"* disjunction (no disjuncts) and call it the **empty clause**. A clause with exactly one literal is called a **unit clause**.

Def 18.19. Clause Set: We often write a **clause set** $\{C_1, \cdots, C_n\}$ as $C_1; \cdots; C_n$, and write $S; T$ for the union of the clause sets S and T, and $S; C$ for the extension of S by a clause C.

Def 18.20. $\mathcal{R}_0$: The propositional resolution calculus $\mathcal{R}_0$ operates on clause sets via a single inference rule:

$$\frac{P^T \vee A \qquad P^F \vee B}{A \vee B}\ \mathcal{R}_0$$

This rule allows to add the **resolvant** $(A \vee B)$ to a clause set which contains the two clauses $P^T \vee A$ and $P^F \vee B$. The literals P^T and P^F are called **cut literals**.

Note

Modus Ponens can be viewed as a specific instance of the $\mathcal{R}_0$ rule. Given the premises $P \to Q$ and P, we first convert the implication to its clausal form $\neg P \vee Q$. Mapping this to the $\mathcal{R}_0$ definition:

- Let the first clause be $P^T \vee \bot$.
- Let the second clause be $P^F \vee Q$.

The application of the rule then results in:

$$\frac{P^T \vee \bot \qquad\qquad P^F \vee Q}{Q}\ \mathcal{R}_0$$

which simplifies to the standard Modus Ponens conclusion Q.

Def 18.21. Resolution Refutation: Let S be a clause set, then we call an $\mathcal{R}_0$-derivation of the empty clause $\square$ from S **$\mathcal{R}_0$-refutation** and write $\mathcal{D} : S \vdash_{\mathcal{R}_0} \square$.

Def 18.22. CNF Calculus: $\mathcal{R}_0$ operates only on clause sets (CNF), to transform formulae into CNF we use the **propositional CNF calculus** (CNF_0) which is shown in figure 9.

$$\frac{C \vee (A \vee B)^T}{C \vee A^T \vee B^T}\ \vee^T \qquad \frac{C \vee (A \vee B)^F}{(C \vee A^F) \wedge (C \vee B^F)}\ \vee^F$$

$$\frac{C \vee \neg A^T}{C \vee A^F}\ \neg^T \qquad \frac{C \vee \neg A^F}{C \vee A^T}\ \neg^F$$

- -

$$\frac{C \vee (A \Rightarrow B)^T}{C \vee A^F \vee B^T}\ \Rightarrow^T \qquad \frac{C \vee (A \Rightarrow B)^F}{(C \vee A^T) \wedge (C \vee B^F)}\ \Rightarrow^F$$

$$\frac{C \vee (A \wedge B)^T}{(C \vee A^T) \wedge (C \vee B^T)}\ \wedge^T \qquad \frac{C \vee (A \wedge B)^F}{C \vee A^F \vee B^F}\ \wedge^F$$

Figure 9: Propositional CNF Calculus

Note

We write $\mathrm{CNF}_0(A^\alpha)$ for the set of all clauses derivable from A^α via CNF_0

Example: We want to prove using $\mathcal{R}_0$.

$$(p \Rightarrow q \Rightarrow r) \Rightarrow (p \Rightarrow q) \Rightarrow p \Rightarrow r$$

We label it with F, then we transform it to CNF using CNF_0:

$$\cfrac{\cfrac{\cfrac{((p \Rightarrow (q \Rightarrow r)) \Rightarrow ((p \Rightarrow q) \Rightarrow (p \Rightarrow r)))^F}{(p \Rightarrow (q \Rightarrow r))^T \wedge ((p \Rightarrow q) \Rightarrow (p \Rightarrow r))^F}\ \Rightarrow^F}{(p \Rightarrow (q \Rightarrow r))^T \wedge (p \Rightarrow q)^T \wedge (p \Rightarrow r)^F}\ \Rightarrow^F}{(p \Rightarrow (q \Rightarrow r))^T \wedge (p \Rightarrow q)^T \wedge p^T \wedge r^F}\ \Rightarrow^F$$

Then, expanding the T terms:

$$\cfrac{\cfrac{\cfrac{(p \Rightarrow (q \Rightarrow r))^T \wedge (p \Rightarrow q)^T \wedge p^T \wedge r^F}{(p \Rightarrow (q \Rightarrow r))^T \wedge (p^F \vee q^T) \wedge p^T \wedge r^F}\ \Rightarrow^T}{(p^F \vee (q \Rightarrow r)^T) \wedge (p^F \vee q^T) \wedge p^T \wedge r^F}\ \Rightarrow^T}{(p^F \vee q^F \vee r^T) \wedge (p^F \vee q^T) \wedge p^T \wedge r^F}\ \Rightarrow^T$$

The final clause set is $\Phi := \{p^F \vee q^F \vee r^T, p^F \vee q^T, p^T, r^F\}$

Resolution Proof:

1	$p^F \vee q^F \vee r^T$	initial
2	$p^F \vee q^T$	initial
3	p^T	initial
4	r^F	initial
5	$p^F \vee q^F$	resolve 1.3 with 4.1
6	q^F	resolve 3.1 with 5.1
7	p^F	resolve 2.2 with 6.1
7	$\square$	resolve 3.1 with 7.1

A resolution proof can be represented in a proof tree as shown in figure 10 or in a graph as shown in figure 11

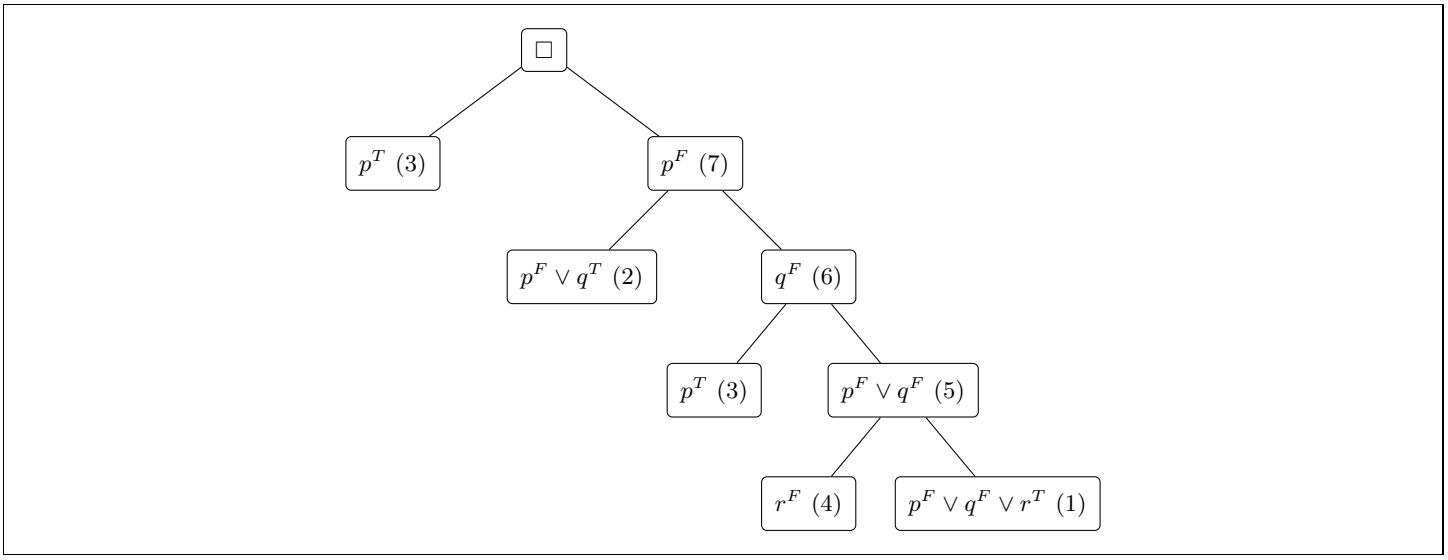


Figure 10: Resolution Proof Tree

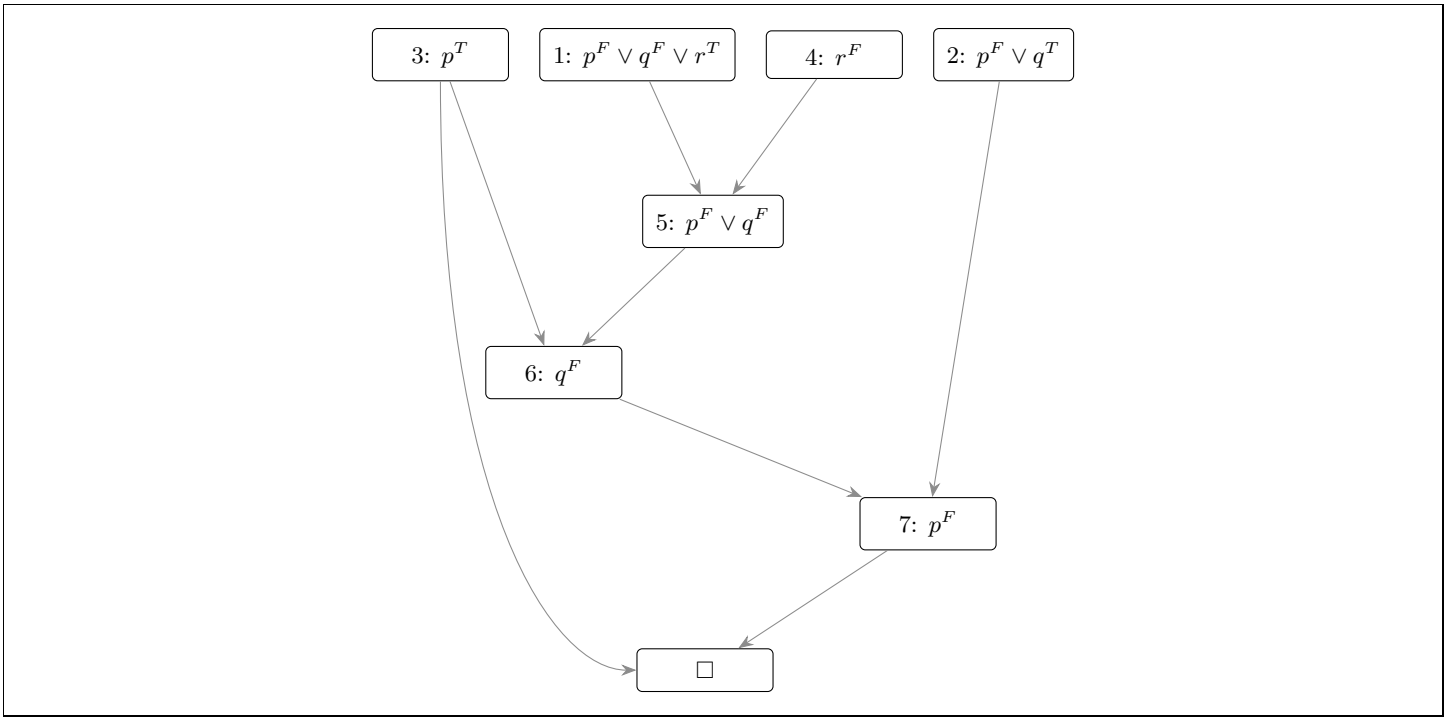


Figure 11: Resolution Proof Graph

Def 18.23. Clause Set Simplification: Let Δ be a clause set, $l \in \Delta$ is a unit clause, and Δ' be Δ where

- all clauses $l \vee C$ have been removed, and
- all clauses $\bar{l} \vee C$ have been shortened to C.

Then Δ is satisfiable iff Δ' is.

We call Δ' the **clause set simplification** of Δ w.r.t. l.

Note: as long as we have unit clauses, we can apply clause set simplification recursively, if we can derive the empty clause, we have a proof!

Example: Let $\Delta := \left\{p^F \vee q^F \vee r^T, p^F \vee q^T, p^T, r^F\right\}$

- Choose a unit clause, say p^T
- Remove any clause with p^T and shorten any clause with $p^F \vee C$ to just C
- $\Delta' := \left\{q^F \vee r^T, q^T, r^F\right\}$
- Choose a unit clause, say q^T
- Remove any clause with q^T and shorten any clause $q^F \vee C$ to just C
- $\Delta'' := \left\{r^T, r^F\right\}$
- Choose any unit clause, say r^T
- Remove any clause with r^T and shorten any clause $r^F \vee C$ to just C
- $\Delta''' := \{\square\}$

Using clause set simplification, we derived the empty clause without even resolving!

Note

If we apply clause set simplification to a satisfiable clause set, we can read off the model that satisfies the clause set (and we will end up with an empty set of clauses, not the the set containing the empty clause). For example, consider $\Delta := \{p^T, q^T \vee r^T, r^F\}$

- Choose a unit clause, say p^T
- $\Delta' := \{q^T \vee r^T, r^F\}$
- Choose a unit clause, say r^F
- $\Delta'' := \{q^T\}$
- Choose a unit clause, say q^T
- $\Delta''' := \emptyset$

The unit clauses we chose on the way correspond to the model φ that satisfies the clause set, namely $\varphi := \{p \mapsto T, r \mapsto F, q \mapsto T\}$. Which makes sense since $p \wedge (q \vee r) \wedge \neg r$ is indeed satisfiable under φ.

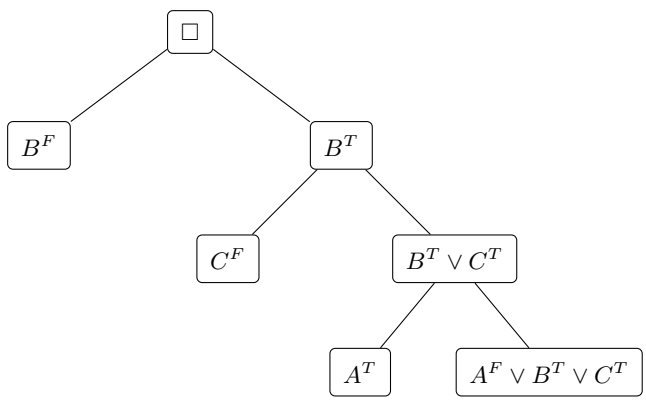
Example: Resolution in the real world

Consider we have a knowledge base $\mathrm{KB} := \{A \Rightarrow B \vee C, A, \neg B\}$, we want to check if $\mathrm{KB} \models C$. $\mathrm{KB} \models C$ iff $\mathrm{KB} \cup \{\neg C\}$ is unsatisfiable.

First we need to compute $\mathrm{CNF}_0(A \Rightarrow B \vee C)$

- $\mathrm{CNF}_0(A \Rightarrow B \vee C) = \neg A \vee B \vee C$

$\mathrm{KB} \cup \{\neg C\} := \{\neg A \vee B \vee C, A, \neg B, \neg C\} := \{A^F \vee B^T \vee C^T, A^T, B^F, C^F\}$



Since $\mathrm{KB} \cup \{\neg C\}$ is unsatisfiable, $\mathrm{KB} \models C$

Def 18.24. SAT: Given a propositional formula A, the **Boolean Satisfiablity Problem** $\langle A, V \subset \mathcal{V}_{\mathrm{PL}^0}\rangle$ is to decide whether or not A is satisfiable. We denote the class of all these problems with **SAT**. **SAT** problems are NP-Complete.

Def 18.25. : The tools that address SAT are commonly referred to as **SAT Solvers**.

Theorem 18.5. Given any constraint network $\gamma := \langle V, D, C\rangle$, we can in low-order polynomial time construct a CNF formula $A(\gamma)$ that is satisfiable iff γ is solvable.

SAT $\to$ CSP:

Given a SAT instance $S = \langle A, V\rangle$, we define a CSP instance $\gamma := \langle V', D, C\rangle$ and bijections f, f^{-1} mapping satisfying assignments of S to solutions of γ and vice versa:

- we define $V' := V$.
- we define $D := \bigcup_v D_v$ where $D_v := \{T, F\} \quad \forall v \in V'$
- the contraints C consist of a single higher-order constraint which is simply the formula $\rightsquigarrow C := \{A\}$
- The bijections f, f^{-1} are identity mappings because solutions are identical

CSP $\to$ SAT:

Given a CSP instance $\gamma := \langle V, D, C\rangle$, we define a SAT instance $S := \langle V', A\rangle$ and bijections f, f^{-1}:

Variables: For each $v \in V$ and each value $a \in D_v$, we introduce a new propositional variable $P_{v,a}$, V' contains all those variables:

$$V' := \{P_{v,a} \mid v \in V \wedge a \in D_v\}$$

Formula: The formula A is defined as the conjunction of three sets of constraints:

$$A := \Phi_{\text{at-least-one}} \wedge \Phi_{\text{at-most-one}} \wedge \Phi_{\text{consistent}}$$

- **At-least-one assignment**: Ensures that each variable v is assigned at least one value from its domain D_v:

$$\Phi_{\text{at-least-one}} = \bigwedge_{v \in V} \left(\bigvee_{a \in D_v} P_{v,a} \right)$$

- **At-most-one assignment**: Ensures that a variable v cannot be assigned more than one value by forbidding all pairs of distinct values:

$$\Phi_{\text{at-most-one}} = \bigwedge_{v \in V} \bigwedge_{\substack{a,b \in D_v \\ a \neq b}} (\neg P_{v,a} \vee \neg P_{v,b})$$

- **Consistency**: Ensures only assignments that satisfy the CSP constraints C_{vw} are allowed by forbidding inconsistent pairs (a, b):

$$\Phi_{\text{consistent}} = \bigwedge_{\{v,w\} \subseteq V} \bigwedge_{(a,b) \notin C_{vw}} (\neg P_{v,a} \vee \neg P_{w,b})$$

Bijections:

- f maps a solution α of the CSP to a satisfying assignment φ for the SAT instance.

$$f(\alpha) = \varphi, \text{ such that } \forall v \in V, \forall a \in D_v : \varphi(P_{v,a}) = \begin{cases} T & \text{if } \alpha(v) = a \\ F & \text{otherwise} \end{cases}$$

- f^{-1} maps a satisfying assignment φ back to a solution α

$$f^{-1}(\varphi) = \alpha \text{ such that } \forall v \in V : \alpha(v) = a \Leftrightarrow \varphi(P_{v,a}) = T$$

Def 18.26. UR: Unit Resolution (UR) is a test calculus consisting of the following inference rule:

$$\frac{C \vee P^{\alpha} \qquad P^{\beta} \qquad \alpha \neq \beta}{C} \text{ UR}$$

It is a restricted form of $\mathcal{R}_0$ where at least one of the two clauses is a unit clause.

Def 18.27. UP: Unit Propagation (UP) is an iterative clause set simplification technique triggering a chain reaction of simplifications using only UR.

Def 18.28. Splitting: The process of picking a variiable p and branching the search into two cases: one where p is assumed to be T, and one where p is assumed to be F, resulting in a search tree.

Def 18.29. DPLL: The **Davis Putnam (Logemann-Loverland) Procedure (DPLL)** is a SAT solver called on a clause set Δ and the empty assignments ϵ. It is the standard algorithm foe deciding the satisfiablity of a clause set:

1. Perform Unit Propagation until no more unit clauses exists.
2. Check if the clause set is empty (satisfiable) or contains an empty clause (unsatisfiable)
3. If neither, perform splitting on a variable and repeat.

The pseudocode for DPLL algorithm is shown in Algorithm 3 and Algorithm 4.

Algorithm 3 DPLL

```
function DPLL(Δ, φ)                                   ▷ Returns assignment φ or unsatisfiable
    Δ' ← copy of Δ
    φ' ← φ
    while Δ' contains a unit clause C = P^α do        ▷ Unit Propagation
        φ' ← φ' ∪ {P ↦ α}
        Δ' ← SIMPLIFY(Δ', P^α)
    end while
    if □ ∈ Δ' then                                    ▷ Termination Test: Contradiction
        return unsatisfiable
    end if
    if Δ' = ∅ then                                    ▷ Termination Test: Success
        return φ'
    end if
    P ← SELECT-UNASSIGNED-VARIABLE(Δ', φ')            ▷ Splitting Rule
    φ'' ← φ' ∪ {P ↦ T}
    Δ'' ← SIMPLIFY(Δ', P^T)
    res ← DPLL(Δ'', φ'')
    if res ≠ unsatisfiable then
        return res
    end if
    φ'' ← φ' ∪ {P ↦ F}
    Δ'' ← SIMPLIFY(Δ', P^F)
    return DPLL(Δ'', φ'')
end function
```

Algorithm 4 Simplify

```
function SIMPLIFY(Δ, P^α)
    Δ' ← ∅
    for each clause C ∈ Δ do
        if P^α ∈ C then
            continue                                  ▷ Remove any clause that has P^α
        else if P^β ∈ C then
            C' ← C \ {P^β}                            ▷ Remove P^β from the clause and keep the rest
            Δ' ← Δ' ∪ {C'}
        else
            Δ' ← Δ' ∪ {C}                             ▷ Clause does not contain the variable P at all
        end if
    end for
    return Δ'
end function
```

DPLL Example: Let $\Delta := \{p^T \vee q^T \vee r^F, p^F \vee q^F, p^T \vee q^F, r^T\}$

- We start with the empty assignments $\varphi := \emptyset$
- We see a unit clause r^T, we assign r to T, hence $\varphi := \{r \mapsto T\}$
- We apply UP $\rightsquigarrow \Delta' := \{p^T \vee q^T, p^F \vee q^F, p^T \vee q^F\}$
- No unit clauses $\rightsquigarrow$ apply splitting:
- Choose any variable and branch, say we choose p

Branch 1: Assume p^T
We copy the assignment $\varphi_1 := \{r \mapsto T\}$ and we copy the clause set Δ_1'

- we treat p^T as a unit clause, hence $\varphi_1 := \{r \mapsto T, p \mapsto T\}$
- we apply UP on Δ_1'
- $\Delta_1'' := \{q^F\}$
- we see a unit clause q^F, we assign, $\varphi_1 := \{r \mapsto T, p \mapsto T, q \mapsto F\}$
- we apply UP on Δ''
- $\Delta_1''' := \emptyset$
- we return satisfiable with φ_1

Branch 2: Assume p^F
We copy the assignment $\varphi_2 := \{r \mapsto T\}$ and we copy the clause set Δ_2'

- we treat p^F as a unit clause, hence $\varphi_2 := \{r \mapsto T, p \mapsto F\}$
- we apply UP on Δ_2'
- $\Delta_2'' := \{q^T, q^F\}$
- we choose a unit clause, say q^T, we assign, $\varphi_2 := \{r \mapsto T, p \mapsto F, q \mapsto T\}$
- we apply UP on Δ_2''
- $\Delta_2''' := \{\square\}$
- we return unsatisfiable for this branch.

The DPLL procedure can be visualized as a search tree as shown in figure 12

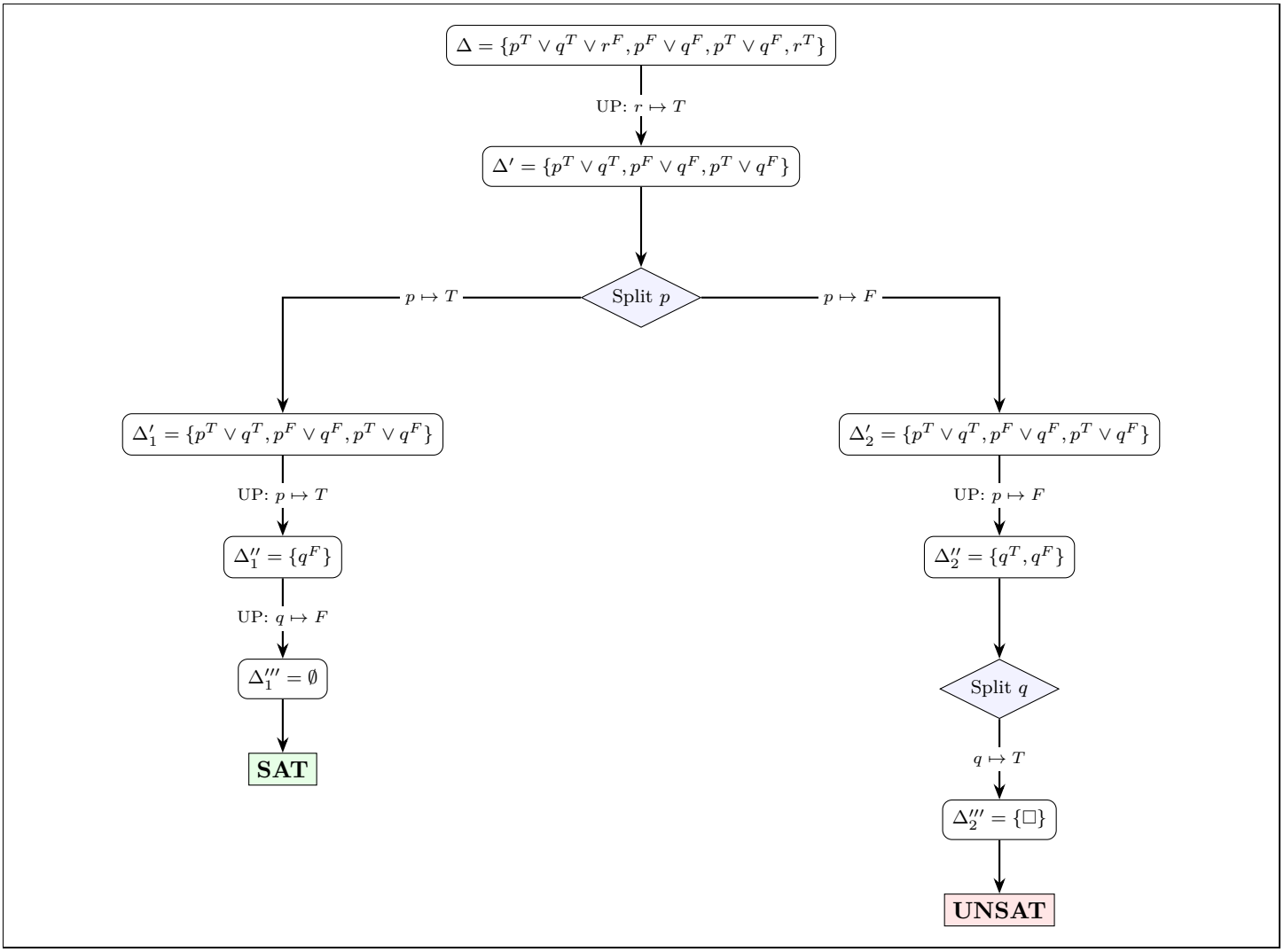


Figure 12: DPLL Search Tree

Theorem 18.6. UP is sound

Theorem 18.7. UR is refutation-sound (*since* $\mathcal{R}_0$ *is*)

Theorem 18.8. UR is not refutation-complete

Proof Sketch: UR makes only limited inference, as long as there are unit clauses. It does not guarantee to infer everything that can be inferred

Theorem 18.9. If DPLL returns unsatisfiable on Δ, then $\Delta \vdash_{\mathcal{R}_0} \square$ with a $\mathcal{R}_0$-refutation

Observation: we can read of a resolution proof from a DPLL trace. From a (top-down) DPLL tree, we generate a (bottom-up) resolution proof as shown in figure 13.

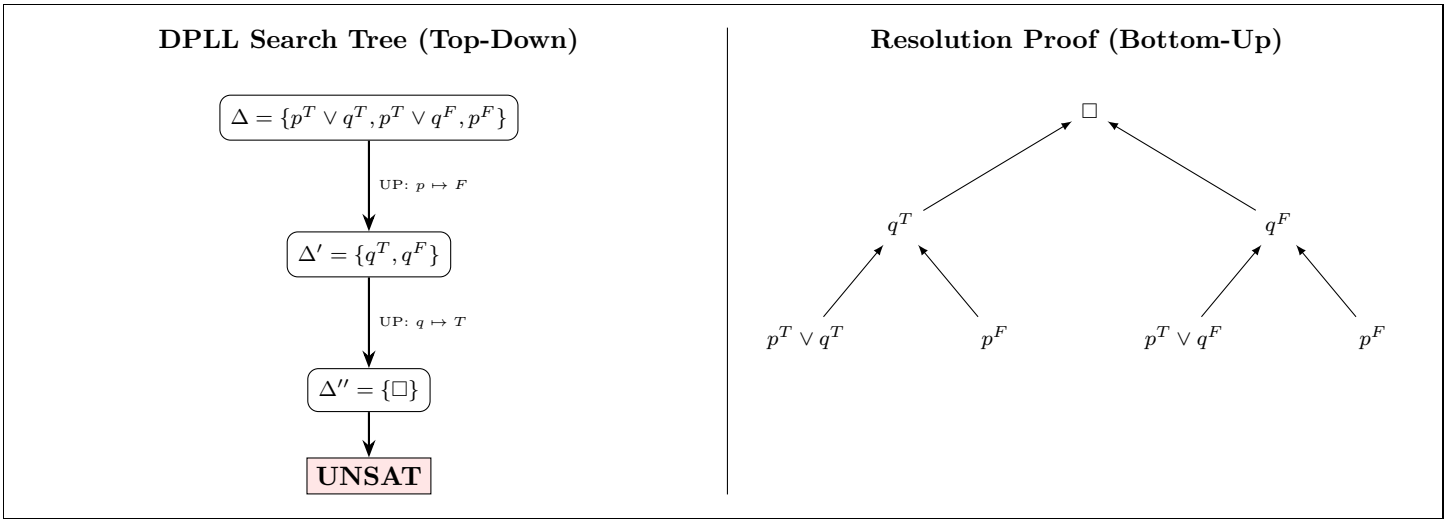


Figure 13: DPLL vs Resolution

19 First-Order Predicate Logic

Def 19.1. PL^1: First-Order (Predicate) Logic PL^1 is a formal system extensively used in mathematics, philosophy, linguistics, and computer science. It combines PL^0 with the ability to quantify over individuals

Def 19.2. First-Order Signature: A First-Order Signature is a tuple $\Sigma := \langle \Sigma_1, \Sigma_0 \rangle$ where $\Sigma_1 := \Sigma^f \cup \Sigma^p \cup \Sigma^{sk}$:

- $\Sigma^f := \bigcup_{k \in \mathbb{N}} \Sigma^f_k$ of **function constants** (or **terms**), where members of Σ^f_k denote the k-ary functions on **individuals**
- $\Sigma^p := \bigcup_{k \in \mathbb{N}} \Sigma^p_k$ of **predicate constants**, where member of Σ^p_k denote k-ary relations among individuals
- $\Sigma^{sk} := \bigcup_{k \in \mathbb{N}} \Sigma^{sk}_k$ of **Skolem constants** that act as witness constructors
- All are pairwise disjoint, countable sets of symbols for each $k \in \mathbb{N}$.

We also assume a set of **individual variables** $V_\iota := \{X, Y, Z, \cdots\}$ which we can quantify over them.

> **Note**
>
> Def 19.2 is the full FOL Signature. In PL^{nq}, we did not explicitly define Σ_0 (assumed to be there) and we did not need Σ^{sk} since no quantification over individuals was allowed. That is why in Def 17.20 we only mentioned what PL^{nq} actually uses of the full FOL Signature

> **Note**
>
> In PL^{nq}, we defined terms and formulae in Def 17.22 and Def 17.23 as follows:
> - We denote the set of all well-formed terms over a FOL Signature Σ with $\mathrm{wff}_\iota(\Sigma)$, and the closed ones with $\mathrm{cwff}_\iota(\Sigma)$
> - We denote the set of all well-formed formulae over a FOL Signature Σ with $\mathrm{wff}_o(\Sigma)$, and the closed ones with $\mathrm{cwff}_o(\Sigma)$

Def 19.3. PL^1 Terms: The set $\mathrm{wff}_\iota(\Sigma_1, V_\iota)$ of **well-formed terms** over the signature Σ_1 and variables V_ι is defined by the following grammar:

$$\begin{array}{rcl} f^k & \in & \Sigma^f_k \cup \Sigma^{sk}_k \\ X & \in & V_\iota \\ t & ::= & X \mid f^k(t_1, \cdots, t_k) \end{array}$$

Def 19.4. PL^1 Propositions: The set $\mathrm{wff}_\circ(\Sigma_1, V_\iota)$ of **well-formed formulae** over the signature Σ_1 and variables V_ι is defined by the following grammar:

$$\begin{array}{rcl} p^k & \in & \Sigma^p_k \\ t_i & \in & \mathrm{wff}_\iota(\Sigma_1, V_\iota) \\ X & \in & V_\iota \\ A & ::= & p^k(t_1, \cdots, t_k) \mid \top \mid \neg A \mid A \wedge A \mid \forall X.A \end{array}$$

Def 19.5. Universal Quantifier: The **universal quantifier** $\forall$ is a binding operator used to express $\forall X.A$, i.e. a formula A holds for all possible values of an individual variable $X \in V_\iota$

> **Note**
>
> Notice that we only need $\top, \wedge, \neg$ as connectives because the others $(\bot, \vee, \Rightarrow, \Leftrightarrow)$ are defined via **abbreviations**:
> - $\bot := \neg\top$
> - $A \vee B := \neg(\neg A \wedge \neg B)$
> - $A \Rightarrow B := \neg A \vee B$
> - $A \Leftrightarrow B := (A \Rightarrow B) \wedge (B \Rightarrow A)$

Def 19.6. Existential Quantifier: The **existential quantifier** $\exists$ is a binding operator defined as an **abbreviation** for a negated universal statement: $\exists X.A :\equiv \neg(\forall X.\neg A)$. It allows us to express that at least one individual satisfies the formula A.

Note

Remember: Def 17.8 , Def 17.9 for PL^0. We have a similar notion of free variables in PL^1 that help us define closed/ground PL^1 formulae.

Def 19.7. Canonical Forms of Formulae: A formula is said to be in:

- **Prenex Normal Form (PNF)** iff it consists of a sequence of quantifiers (called the *prefix*) over a quantifier-free formula (called the *matrix*). Example: $\forall X \exists Y (P(X) \vee Q(Y))$
- **Skolem Normal Form (SNF)**, iff it is in PNF and does not contain existential quantifiers. Example: $\forall X (P(X) \vee Q(f(X)))$
- **Negation Normal Form (NNF)**, iff negations are only literals. Example: $(\neg P \vee Q) \wedge (R \vee \neg S)$
- **Conjunctive Normal Form (CNF)**, iff it is a conjunction of disjunctions of literals. Example: $(P \vee Q \vee \neg R) \wedge (\neg P \vee S)$
- **Disjunctive Normal Form (DNF)**, iff it is a disjunction of conjunctions of literals. Example: $(P \wedge Q \wedge \neg R) \vee (\neg P \wedge S)$

Def 19.8. Free Variables: An occurrence of a variable X is **free** in a formula A if it is not within the scope of a binder. We define the set of free variables in A ($\mathrm{free}(A)$) recursively as follows:

- **For Terms** ($t \in \mathrm{wff}_\iota(\Sigma_1, V_\iota)$)
  - $\mathrm{free}(X) = \{X\}$
  - $\mathrm{free}(f^k(t_1, \cdots, t_k)) := \bigcup_{i=1}^k \mathrm{free}(t_i)$
- **For Formulae** ($A \in \mathrm{wff}_\circ(\Sigma_1, V_\iota)$)
  - $\mathrm{free}(p^k(t_1, \cdots, t_k)) := \bigcup_{i=1}^k \mathrm{free}(t_i)$
  - $\mathrm{free}(\top) := \emptyset$
  - $\mathrm{free}(\neg A) := \mathrm{free}(A)$
  - $\mathrm{free}(A \wedge B) := \mathrm{free}(A) \cup \mathrm{free}(B)$
  - $\mathrm{free}(\forall X.A) := \mathrm{free}(A) \setminus \{X\}$

Def 19.9. Bound Variables: An occurrence of a variable X is **bound** if it occurs within the scope of a quantifier. We define the set of bound variables $\mathrm{BVar}(A)$ as the set of all variables that are *captured* by a binder:

- **For Terms** ($t \in \mathrm{wff}_\iota(\Sigma_1, V_\iota)$)
  - $\mathrm{BVar}(t) := \emptyset$ (*variables in terms are always free until a formula provides a binder*)
- **For Formulae** ($A \in \mathrm{wff}_\circ(\Sigma_1, V_\iota)$)
  - $\mathrm{BVar}(p^k(t_1, \cdots, t_k)) := \emptyset$
  - $\mathrm{BVar}(\neg A) := \mathrm{BVar}(A)$
  - $\mathrm{BVar}(A \wedge B) := \mathrm{BVar}(A) \cup \mathrm{BVar}(B)$
  - $\mathrm{BVar}(\forall X.A) := \mathrm{BVar}(A) \cup \{X\}$

Def 19.10. Ground Formulae: A PL^1 formula A is called **ground/closed** iff $\mathrm{free}(A) := \emptyset$. A closed formula is called a sentence. We denote the set of all **ground** terms with $\mathrm{cwff}_\iota(\Sigma_1, V_\iota)$ and the set of all sentences with $\mathrm{cwff}_\circ(\Sigma_1, V_\iota)$

Def 19.11. Alphabetical Variants: Bound variables can be **renamed**; specifically, if $\forall X.B$ is a sub-formula of A, let A' be the result of replacing $\forall X.B$ in A with $\forall Y.B'$, where B' is obtained from B by replacing all occurrences of $X \in \mathrm{free}(B)$ with a new variable Y that does not occur in A. We call A' an **alphabetical variant** of A (and vice versa).

Def 19.12. PL^1 Universe: PL^1 uses the universe $\mathcal{D}_{\mathrm{PL}^1} := \mathcal{D}_0 \cup \mathcal{D}_\iota$ where $\mathcal{D}_0 = \{T, F\}$ is the set of truth values and $\mathcal{D}_\iota \neq \emptyset$ is a non-empty set of individuals

Def 19.13. Interpretation in PL^1: The interpretation function in PL^1 assigns values to constants:

1. We use the interpretation from PL^0
   - $\mathcal{I}(\neg) : \mathcal{D}_0 \to \mathcal{D}_0$; $T \mapsto F, F \mapsto T$
   - $\mathcal{I}(\wedge) : \mathcal{D}_0 \times \mathcal{D}_0 \to \mathcal{D}_0$; $\langle \alpha, \beta \rangle \mapsto T$, iff $\alpha = \beta = T$

- $\mathcal{I}(\top) = T$
- $\mathcal{I}(\bot) = F$

2. We interpret individual constants as individuals: $\mathcal{I} : \Sigma_0^f \to \mathcal{D}_\iota$
3. We interpret function constants as functions: $\mathcal{I} : \Sigma_{k\neq 0}^f \to \mathcal{D}_\iota^{k\neq 0} \to \mathcal{D}_\iota$
4. We interpret predicate constants as relations: $\mathcal{I} : \Sigma_k^p \to \mathcal{P}(\mathcal{D}_\iota^k)$

Def 19.14. PL[1] Value Function: Given a model $\langle \mathcal{D}, \mathcal{I} \rangle$, the value function $\mathcal{I}_\varphi$ is recursively defined:

1. $\mathcal{I}_\varphi : \mathrm{wff}_\iota(\Sigma_1, V_\iota) \to \mathcal{D}_\iota$ assigns values to terms
   - $\mathcal{I}_\varphi(X) := \varphi(X)$, and
   - $\mathcal{I}_\varphi(f(A_1, \cdots, A_k)) := \mathcal{I}(f)(\mathcal{I}_\varphi(A_1), \cdots, \mathcal{I}_\varphi(A_k))$
2. $\mathcal{I}_\varphi : \mathrm{wff}_\circ(\Sigma_1, V_\iota) \to \mathcal{D}_\circ$ assigns values to formulae
   - $\mathcal{I}_\varphi(\top) = \mathcal{I}(\top) = T$
   - $\mathcal{I}_\varphi(\neg A) := \mathcal{I}(\neg)(\mathcal{I}_\varphi(A))$
   - $\mathcal{I}_\varphi(A \wedge B) := \mathcal{I}(\wedge)(\mathcal{I}_\varphi(A), \mathcal{I}_\varphi(B))$
   - $\mathcal{I}_\varphi(p(A_1, \cdots, A_k)) := T$, iff $\langle \mathcal{I}_\varphi(A_1), \cdots, \mathcal{I}_\varphi(A_k) \rangle \in \mathcal{I}(p)$
   - $\mathcal{I}_\varphi(\forall X.A) := T$, iff $\mathcal{I}_{\varphi,[a/X]}(A) = T$ for all $a \in \mathcal{D}_\iota$

Def 19.15. Assignment Extension: Let $\varphi : V_\iota \to \mathcal{D}_\iota$ be a **variable assignment** and $a \in \mathcal{D}_\iota$. The **extension** of φ by a for X, denoted by $\varphi, [a/X]$ is a variable assignment defined as:

$$\varphi, [a/X](Y) := \begin{cases} a & \text{if } Y = X \\ \varphi(Y) & \text{if } Y \neq X \end{cases}$$

Equivalently, viewing the assignment as a set of ordered pairs:

$$\varphi, [a/X] = \{(Y, \varphi(Y)) \mid Y \in V_\iota \setminus \{X\}\} \cup \{(X, a)\}$$

This assignment coincides with φ for all variables except X, where it maps to the individual a.

Def 19.16. Expression Language: Let signature Σ and variables $\mathcal{V}$ be disjoint sets and G a grammar whose vocabulary is $\Sigma \cup \mathcal{V}$, then we call $\langle \Sigma, \mathcal{V}, G \rangle$ an **expression language**, and denote the language of G with $\mathrm{wfe}(\Sigma, \mathcal{V})$: the set of **well-formed expressions**.

> Note
>
> For PL[1], we have:
>
> $$\mathrm{wfe}(\Sigma_1, \mathcal{V}_\iota) := \mathrm{wff}_\circ(\Sigma_1, V_\iota) \cup \mathrm{wff}_\iota(\Sigma_1, V_\iota)$$

Def 19.17. Support: Let $f : A \to B$ be a function. The **support** of f, denoted by $\mathrm{supp}(f)$, is the set of all elements in the domain A for which the function does not return a "default" or "identity" value.

Def 19.18. Substitution: Let $\mathrm{wfe}(\Sigma, \mathcal{V})$ be an expression language, then we call $\sigma : \mathcal{V} \to \mathrm{wfe}(\Sigma, \mathcal{V})$ a **substitution** iff the support $\mathrm{supp}(\sigma) := \{X \mid (X, A) \in \sigma \wedge X \neq A\}$ of σ is finite. We denote the **empty substitution** with ϵ.

Def 19.19. : We can **discharge** a variable X from a substitution σ by setting $\sigma_{-X} := \sigma, [X/X]$

Def 19.20. : Let σ be a substitution, then we call $\mathrm{intro}(\sigma) := \bigcup_{X \in \mathrm{supp}(\sigma)} \mathrm{free}(\sigma(X))$ the set of variables **introduced** by σ.

Example: consider the formula $A := \forall X.p(X, Y)$ and the substitution $\sigma := \{[Z/Y]\}$:

- $\mathrm{supp}(\sigma) = \{Y\}$
- $\mathrm{free}(\sigma(Y)) = \mathrm{free}(Z) = \{Z\}$
- $\mathrm{intro}(\sigma) = \mathrm{free}(\sigma(Y)) = \{Z\}$

Def 19.21. Substitution Application: We define substitution application by:

- $\sigma(c) = c$ for $c \in \Sigma$

- $\sigma(X) = A$ iff $X \in \mathcal{V}$ and $(X, A) \in \sigma$
- $\sigma(f(A_1, \cdots, A_n)) = f(\sigma(A_1), \cdots, \sigma(A_n))$
- $\sigma(\forall X.A) = \forall X.\sigma_{-X}(A)$

Note

The concept of support is useful to handle finiteness in systems that are infinite. For example, in PL[1], the set of variables $\mathcal{V}_\iota$ is infinite. However, when we apply a substitution, we don't want to deal with an infinite number of changes. By requiring the support of a substitution to be finite, we ensure that only a managable, countable number of variables are being modified, while the infinite rest of them simply stay the same (mapping to their *"identity"* value).

Substitution Example: consider the formula $g(x, y, h(z))$, and we want to apply the substitution $\sigma := \{x \mapsto a, y \mapsto f(b), z \mapsto a\}$ we get the formula $g(a, f(b), h(a))$

Note

Usually we use another notation for writing the substitution $\sigma := \{x \mapsto a, y \mapsto f(b), z \mapsto a\}$ which is $[a/x], [f(b)/y], [a/z]$ where $[A/X]$ (read as A *for* X i.e. substitute X with A) $\equiv X \mapsto A \equiv (X, A) \in \sigma$

Def 19.22. Substitution Extension: Let σ be a substitution, then we denote the **extension** of σ with $[A/X]$ by $\sigma, [A/X]$ and define it as $\{(Y, B) \in \sigma \mid Y \neq X\} \cup \{(X, A)\}$.

Def 19.23. Capture-Avoiding Substitution Application: Let σ be a substitution, A a formula, and A' an alphabetical variant of A, such that $\text{intro}(\sigma) \cap \text{BVar}(A') = \emptyset$. Then we define **capture-avoiding substitution application** via $\sigma(A) := \sigma(A')$

Note

We said $\sigma(\forall X.A) = \forall X.\sigma_{-X}(A)$ but this can lead to **variable capture**, i.e. $[f(X)/Y](\forall X.p(X, Y))$ would evaluate to $\forall X.p(X, f(X))$ where the second occurrence of X is now bound whereas it was free before.
The solution for this is to rename the bound variable X in $\forall X.p(X, Y)$ before applying the substitution and there where we need capture-avoiding substitution application

Example: we want to apply $\sigma := \{[f(X)/Y]\}$ to the formula $A = \forall X.p(X, Y)$

- $\text{intro}(\sigma) \cap \text{BVar}(A) = \{X\} \cap \{X\} = \{X\}$
- Hence, we need to find an alphabetical variant A' for A, say $A' = \forall Z.p(Z, Y)$
- Now $\text{intro}(\sigma) \cap \text{BVar}(A') = \{X\} \cap \{Z\} = \emptyset$
- Hence, we define $\sigma(A) = \sigma(A')$
- $\sigma(A') = \sigma(\forall Z.p(Z, Y)) = \forall Z.\sigma_{-Z}(p(Z, Y))$
- $[f(X)/Y](p(Z, Y)) = p(Z, f(X))$
- $\sigma(A) = \sigma(A') = \forall Z.p(Z, f(X))$

Lemma 19.1. Let A and B be terms, then $\mathcal{I}_\varphi([B/X](A)) = \mathcal{I}_\psi(A)$, where $\psi = \varphi, [\mathcal{I}_\varphi(B)/X]$.

Proof. By induction on the depth of A:

1. depth $= 0$

   Then A is either a variable or a constant. If A is variable, say Y, we have two cases $Y = X$ and $Y \neq X$:

   (a) $A = Y = X$:

   then $\mathcal{I}_\varphi([B/X](A)) = \mathcal{I}_\varphi([B/X](X)) = \mathcal{I}_\varphi(B) = \psi(X) = \mathcal{I}_\psi(X) = \mathcal{I}_\psi(A)$

   (b) $A = Y \neq X$:

   then $\mathcal{I}_\varphi([B/X](A)) = \mathcal{I}_\varphi([B/X](Y)) = \mathcal{I}_\varphi(Y) = \varphi(Y) = \psi(Y) = \mathcal{I}_\psi(Y) = \mathcal{I}_\psi(A)$

   (c) A is a constant $\rightsquigarrow$ analogous to 1.b

2. depth > 0

   then $A = f(A_1, \cdots, A_n)$ and we have

$$\begin{array}{rcl} \mathcal{I}_\varphi([B/X](A)) & = & \mathcal{I}(f)(\mathcal{I}_\varphi([B/X](A_1)), \cdots, \mathcal{I}_\varphi([B/X](A_n))) \\ & = & \mathcal{I}(f)(\mathcal{I}_\psi(A_1), \cdots, \mathcal{I}_\psi(A_n)) \\ & = & \mathcal{I}_\psi(A) \end{array}$$

   by induction hypothesis

□

Lemma 19.2. Let A and B be propositions, then $\mathcal{I}_\varphi([B/X]A) = \mathcal{I}_\psi(A)$, where $\psi = \varphi, [\mathcal{I}_\varphi(B)/X]$.

Proof. By induction on the number of n connectives and quantifiers in A:

1. $n = 0$

   then A is an atomic formula, and we can argue like in the induction step of the Lemma 19.1

2. $n > 0$ and $A = \neg B$ or $A = C \circ D$

   Here we argue like in the induction step of Lemma 19.1 as well

3. $n > 0$ and $A = \forall Y.C$ where (WLOG) $X \neq Y$ (otherwise, rename)
   (a) then $\mathcal{I}_\psi(A) = \mathcal{I}_\psi(\forall Y.C) = T$, iff $\mathcal{I}_{\psi,[a/Y]}(C) = T$ for all $a \in \mathcal{D}_\iota$
   (b) But $\mathcal{I}_{\psi,[a/Y]}(C) = \mathcal{I}_{\varphi,[a/Y]}([B/X](C)) = T$, by induction hypothesis.
   (c) So $\mathcal{I}_\psi(A) = \mathcal{I}_\varphi(\forall Y.[B/X](C)) = \mathcal{I}_\varphi([B/X](\forall Y.C)) = \mathcal{I}_\varphi([B/X](A))$

□

Def 19.24. $\mathcal{ND}_1$: **The First-Order Natural Deduction Calculus** extends $\mathcal{ND}_0$ by the following four inference rule:

$$\frac{A}{\forall X.A}\ \forall I^* \qquad\qquad \frac{\forall X.A}{[B/X](A)}\ \forall E$$

* means that A does not depend on any hypothesis in which X is free

$$\frac{[B/X](A)}{\exists X.A}\ \exists I \qquad\qquad \frac{\exists X.A \qquad \begin{array}{c}[[c/X](A)]^1 \\ \vdots \\ C\end{array} \qquad c \in \sum_0^{sk} \text{new}}{C}\ \exists E^1$$

Note

If our knowledge base has loves(Anna, Anna), using $\exists I$ we can infer several things, e.g.:

- $\exists X.\text{loves}(X, X)$
- $\exists X.\text{loves}(X, \text{Anna})$
- $\exists X.\text{loves}(\text{Anna}, X)$
- $\exists X \exists Y.\text{loves}(X, Y)$

Hence, you might see the rule written in the following notation:

$$\frac{A(\cdots c \cdots c \cdots)}{\exists X.A(\cdots c \cdots X \cdots)}\ \exists I$$

to emphasize that we do not need to replace all instances of the terms c with the variable X (but we can!)

Notes on $\forall I$

If we are trying to introduce something of the form $\forall X.A$, conceptually, this is a tricky thing to do! Because how could we prove that A holds for everything! It is not practical to go through every individuals! Therefore, we need a clever way of doing that.

Let's compare between two types of reasoning that involve generalization, first, let's see the following one:

Step	Formula	Rule Applied
(1)	Blonde(Anna)	Assumption
(2)	Blonde(Anna) $\vee$ Brown(Anna)	$\vee I_l$ (on 1)
(3)	Blonde(Anna) $\Rightarrow$ (Blonde(Anna) $\vee$ Brown(Anna))	$\Rightarrow I^1$ (on 1, 2) $\rightsquigarrow$ Discharge (1)
(4)	$\forall X.$(Blonde(X) $\Rightarrow$ (Blonde(X) $\vee$ Brown(X)))	$\forall I$ (on 3)

That was a **valid** reasoning. Now let's see another one that looks *(similar)*:

Step	Formula	Rule Applied
(1)	Blonde(Anna)	Assumption
(2)	Tall(Anna)	Assumption
(3)	Blonde(Anna) $\wedge$ Tall(Anna)	$\wedge I$ (on 1,2)
(4)	Tall(Anna) $\Rightarrow$ (Blonde(Anna) $\wedge$ Tall(Anna))	$\Rightarrow I^2$ (on 2,3) $\rightsquigarrow$ Discharge (2)
(5)	$\forall X.$(Tall(X) $\Rightarrow$ (Blonde(X) $\wedge$ Tall(X)))	$\forall I$ (on 4)

That was invalid reasoning in an obvious way! Just because someone is tall, that doesn't mean they are both tall and blonde!

The difference is in the way the individuals "Anna" was used. In the incorrect proof, we assumed Anna is blonde, then we assumed something further about her (being tall), under that new assumption we inferred she's both tall and blonde and then we inferred something further to conclude everyone is like that!

In the correct one, however, we derived the formula from one assumption about Anna and Anna did not appear anywhere before that. Anna was just an *arbitrary* name.

Therefore, the rule might also be written in a different notation:

$$\frac{A(\cdots c \cdots c \cdots)}{\forall X.A(\cdots X \cdots X \cdots)}\ \forall I$$

where c is arbitrary and $X \notin A$

Def 19.25. $\mathcal{ND}^1_{\vdash}$**:** The inference rules of the **First-Order Sequent Style Natural Deduction Calculus** consist of those from $\mathcal{ND}^0_{\vdash}$ plus the following quantifiers rules:

$$\frac{\Gamma \vdash A \quad X \notin \text{free}(\Gamma)}{\Gamma \vdash \forall X.A}\ \forall I \qquad \frac{\Gamma \vdash \forall X.A}{\Gamma \vdash [B/X](A)}\ \forall E$$

$$\frac{\Gamma \vdash [B/X](A)}{\Gamma \vdash \exists X.A}\ \exists I \qquad \frac{\Gamma \vdash \exists X.A \quad \Gamma, [c/X]A \vdash C \quad c \in \Sigma_0^{\text{sk}}\text{new}}{\Gamma \vdash C}\ \exists E^1$$

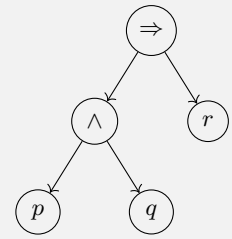
Def 19.26. $\mathrm{PL}^1_=$: We extend PL^1 with a new logical constant for **equality** $= \in \Sigma^p_2$ and fix its interpretation to $\mathcal{I}(=) := \{(x, x) \mid x \in \mathcal{D}_\iota\}$. We call the extended logic **First-Order Logic with Equality**.

Def 19.27. Position: A position is a finite sequence of natural numbers $p = \langle i_1, i_2, \ldots, i_n \rangle$ that describes a path from the root of a well-formed expression to a specific sub-node. For an expression A, the subexpression at position p is denoted by $A|_p$. We write $C[A]_p$ to indicate that the expression C contains the subexpression A at position p. If A is an atomic formula, we call p an **occurrence** of A in C.

Note

Example: consider $C := (p \wedge q) \Rightarrow r$

- Position $\langle 1 \rangle$ is the subexpression $(p \wedge q)$
- Position $\langle 1, 1 \rangle$ is the occurrence of the atom p
- Position $\langle 2 \rangle$ is the occurrence of the atom r



Def 19.28. Replacement: Let p be a position, then $[A/p]C$ is the expression obtained from C by **replacing** the subexpression at p by A

Def 19.29. $\mathcal{ND}^1_=$: The First-Order Logic with Equality Natural Deduction Calculus extends $\mathcal{ND}_1$ by the following two inference rule:

$$\frac{}{A = A}\ = I \qquad\qquad \frac{A = B \quad C[A]_p}{[B/p]C}\ = E$$

Note

equivalence ($\Leftrightarrow$) behaves exactly as equality ($=$)

Def 19.30. : $\Leftrightarrow I$ is derivable and $\Leftrightarrow E$ is admissible in $\mathcal{ND}_1$:

$$\frac{}{A \Leftrightarrow A}\ \Leftrightarrow I \qquad\qquad \frac{A \Leftrightarrow B \quad C[A]_p}{[B/p]C}\ \Leftrightarrow E$$

Figure 14 shows all inference rules of the natural deduction calculus, i.e. the union of $\mathcal{ND}_0$, $\mathcal{ND}_1$, $\mathcal{ND}^1_=$, and equivalence rules

$$\frac{A \quad B}{A \wedge B}\ \wedge I \qquad \frac{A \wedge B}{A}\ \wedge E_l \qquad \frac{A \wedge B}{B}\ \wedge E_r$$

$$\frac{\begin{array}{c}[A]^1\\ \vdots \\ B\end{array}}{A \Rightarrow B}\ \Rightarrow I^1 \qquad \frac{A \Rightarrow B \quad A}{B}\ \Rightarrow E$$

$$\frac{}{A \vee \neg A}\ \mathcal{TND}$$

$$\frac{}{A \Leftrightarrow A}\ \Leftrightarrow I \qquad \frac{A \Leftrightarrow B \quad C[A]_p}{[B/p]C}\ \Leftrightarrow E$$

$$\frac{A}{A \vee B}\ \vee I_l \qquad \frac{B}{A \vee B}\ \vee I_r$$

$$\frac{A \vee B \quad \begin{array}{c}[A]^1\\ \vdots \\ C\end{array} \quad \begin{array}{c}[B]^1\\ \vdots \\ C\end{array}}{C}\ \vee E^1$$

$$\frac{\begin{array}{c}[A]^1\\ \vdots \\ C\end{array} \quad \begin{array}{c}[A]^1\\ \vdots \\ \neg C\end{array}}{\neg A}\ \neg I^1 \qquad \frac{\neg\neg A}{A}\ \neg E$$

$$\frac{\neg A \quad A}{\bot}\ \bot I \qquad \frac{\bot}{A}\ \bot E$$

$$\frac{A}{\forall X.A}\ \forall I^* \qquad \frac{\forall X.A}{[B/X](A)}\ \forall E$$

* means that A does not depend on any hypothesis in which X is free

$$\frac{[B/X](A)}{\exists X.A}\ \exists I \qquad \frac{\exists X.A \quad \begin{array}{c}[[c/X](A)]^1\\ \vdots \\ C\end{array} \quad c \in \sum_0^{sk} \text{new}}{C}\ \exists E^1$$

$$\frac{}{A = A}\ = I \qquad \frac{A = B \quad C[A]_p}{[B/p]C}\ = E$$

Figure 14: All Natural Deduction Rules

Example Proofs

Figure 15 shows a linearized $\mathcal{ND}_1$ proof of the formula:

$$((\exists X.f(X)) \Rightarrow g(a)) \Rightarrow (\forall X.(f(X) \Rightarrow g(a)))$$

Step	Formula	Rule Applied
(1)	$(\exists X.f(X)) \Rightarrow g(a)$	Assumption
(2)	$f(b)$	Assumption
(3)	$\exists X.f(X)$	$\exists I$ (on 2)
(4)	$g(a)$	$\Rightarrow E$ (on 1,3)
(5)	$f(b) \Rightarrow g(a)$	$\Rightarrow I^2$ (on 2, 4) $\rightsquigarrow$ Discharge (2)
(6)	$\forall X.(f(X) \Rightarrow g(a))$	$\forall I$ (on 5)
(7)	$((\exists X.f(X)) \Rightarrow g(a)) \Rightarrow (\forall X.(f(X) \Rightarrow g(a)))$	$\Rightarrow I^1$ (on 1, 6) $\rightsquigarrow$ Discharge (1)

Figure 15: $\mathcal{ND}_1$ Linearized Style Example Proof

The same formula proof in Gentzen-Prawitz Style is shown in figure 16:

$$\dfrac{\dfrac{\dfrac{\dfrac{[(\exists X.f(X)) \Rightarrow g(a)]^1 \quad \dfrac{[f(b)]^2}{\exists X.f(X)}\,\exists I}{g(a)}\,\Rightarrow E}{f(b) \Rightarrow g(a)}\,\Rightarrow I^2}{\forall X.(f(X) \Rightarrow g(a))}\,\forall I}{((\exists X.f(X)) \Rightarrow g(a)) \Rightarrow (\forall X.(f(X) \Rightarrow g(a)))}\,\Rightarrow I^1$$

Figure 16: $\mathcal{ND}_1$ Gentzen-Prawitz Style Example Proof

Figure 17 shows an example Sequent-Style proof for the validity of the sequent:

$$(\exists X.(f(X) \wedge g(X))) \vdash (\exists X.f(X) \wedge \exists X.g(X))$$

Let $\Phi = \exists X.(f(X) \wedge g(X))$.

$$\dfrac{\dfrac{}{\Phi \vdash \exists X.(f(X) \wedge g(X))}\,\text{Ax} \quad \dfrac{\dfrac{\dfrac{\dfrac{}{\Phi, f(a) \wedge g(a) \vdash f(a) \wedge g(a)}\,\text{Ax}}{\Phi, f(a) \wedge g(a) \vdash f(a)}\,\wedge E_l}{\Phi, f(a) \wedge g(a) \vdash \exists X.f(X)}\,\exists I \quad \dfrac{\dfrac{\dfrac{}{\Phi, f(a) \wedge g(a) \vdash f(a) \wedge g(a)}\,\text{Ax}}{\Phi, f(a) \wedge g(a) \vdash g(a)}\,\wedge E_r}{\Phi, f(a) \wedge g(a) \vdash \exists X.g(X)}\,\exists I}{\Phi, f(a) \wedge g(a) \vdash \exists X.f(X) \wedge \exists X.g(X)}\,\wedge I \quad \begin{array}{l} a \in \Sigma_0^{sk} \\ \text{new} \end{array}}{\Phi \vdash \exists X.f(X) \wedge \exists X.g(X)}\,\exists E$$

Figure 17: $\mathcal{ND}^1_{\vdash}$ Example Proof

Excursion

Def 19.31. Integer Division (Euclidean Division): For any integers a (the **dividend**) and b (the **divisor**) where $b \neq 0$, there exist unique integers q (the **quotient**) and r (the **remainder**) such that: $a = b \cdot q + r \quad \text{and} \quad 0 \leq r < |b|$

Def 19.32. Quotient: The result q obtained from integer division. It represents how many full times the divisor b fits into the dividend a.

Def 19.33. Remainder: The amount r "left over" after integer division. If $r = 0$, then $b \mid a$ (the divisor divides the dividend).

Def 19.34. Set of Divisors: For a given integer $n \in \mathbb{Z}$, the **set of divisors** is the set of all integers that stand in the divides relation to n. Formally: $\text{Div}(n) := \{d \in \mathbb{Z} \mid d \mid n\}$

Def 19.35. Prime Number: A natural number $p \in \mathbb{N}$ is **prime** iff $p > 1$ and its only positive divisors are 1 and itself. Formally: $p \in \mathbb{P} :\equiv p > 1 \wedge \forall d \in \mathbb{N}.((d \mid p) \Rightarrow d = 1 \vee d = p)$

Def 19.36. Composite Number: A natural number $n \in \mathbb{N}$ is **composite** iff there exists at least one divisor other than 1 and itself. Formally: n is composite $:\equiv n > 1 \wedge \exists d \in \mathbb{N}.(1 < d < n \wedge d \mid n)$

Def 19.37. Prime Factor: An integer p is a **prime factor** of $n \in \mathbb{Z}$ iff $p \in \mathbb{P}$ and $p \in \text{Div}(n)$ (i.e., p is a prime number that stands in the divides relation to n).

Def 19.38. Prime Factor Multiset: For any $n \in \mathbb{N}, n > 1$, the **prime factor multiset** $\mathcal{M}(n)$ over the domain $\mathbb{P}$ is the multiset where the multiplicity $m(p)$ of each prime p is its exponent in the prime factorization. The product of all elements in the multiset equals n: $\prod_{p \in \mathbb{P}} p^{m(p)} = n$.

Def 19.39. Prime Factorization: The representation of a natural number $n > 1$ as its prime factor multiset. By the Fundamental Theorem of Arithmetic, this multiset is unique.

Note

Example: Prime Factorization of 12 To find the prime factor multiset, we use **Trial Division**. We repeatedly divide by the smallest possible prime number ($p \in \{2, 3, 5, \dots\}$) until the quotient reaches 1.

1. **Trial with** $p = 2$: $12 \div 2 = 6$. Record $m(2) = 1$. Target is 6.
2. **Trial with** $p = 2$ **again**: $6 \div 2 = 3$. Update $m(2) = 2$. Target is 3.
3. **Trial with** $p = 2$: $3 \div 2 = 1$ with remainder 1. Since $2 \nmid 3$, we move to the next prime, $p = 3$.
4. **Trial with** $p = 3$: $3 \div 3 = 1$. Record $m(3) = 1$. Target is 1. Terminate.

Final Multiset: The multiset is $\mathcal{M}(12) = \{2, 2, 3\}$, where $m(2) = 2$ and $m(3) = 1$. **Final Notation:** $12 = 2^2 \cdot 3$.

Def 19.40. Coprime: Two integers $a, b \in \mathbb{Z}$ are **coprime** iff the intersection of their prime factor multisets is empty. Formally: $\mathcal{M}(a) \cap \mathcal{M}(b) = \emptyset$, where $\cap$ for multisets is defined by $m_\cap(x) = \min(m_a(x), m_b(x))$.

Def 19.41. Greatest Common Divisor (GCD): For two natural numbers $a, b \in \mathbb{N}$, the **Greatest Common Divisor**, denoted $\gcd(a, b)$, is the largest positive integer that divides both a and b. If $\gcd(a, b) = 1$, then a and b are coprime.

Method 1: Prime Factorization This method identifies the shared "building blocks" of two numbers. The GCD is the product of the elements in the multiset intersection of their prime factor multisets.

Example: Find gcd$(24, 36)$

1. Find prime factor multisets: $\mathcal{M}(24) = \{2, 2, 2, 3\}$ and $\mathcal{M}(36) = \{2, 2, 3, 3\}$.
2. Find the multiset intersection: $\mathcal{M}(24) \cap \mathcal{M}(36) = \{2, 2, 3\}$ (using the minimum multiplicity for each prime).
3. Multiply the elements: $2 \times 2 \times 3 = 12$.
4. Result: $\gcd(24, 36) = 12$.

Method 2: Successive Division This method uses the remainder from integer division. We repeatedly replace the larger number with the remainder until the remainder is 0. The last non-zero remainder is the GCD.

Example: Find gcd$(48, 18)$

1. $48 \div 18$: quotient 2, remainder 12. $(48 = 18 \cdot 2 + 12)$
2. $18 \div 12$: quotient 1, remainder 6. $(18 = 12 \cdot 1 + 6)$
3. $12 \div 6$: quotient 2, remainder 0. $(12 = 6 \cdot 2 + 0)$

The last non-zero remainder is **6**, so $\gcd(48, 18) = 6$.

Def 19.42. Even Integer: An integer $n \in \mathbb{Z}$ is **even** iff there exists an integer $k \in \mathbb{Z}$ such that $n = 2k$.

Def 19.43. Odd Integer: An integer $n \in \mathbb{Z}$ is **odd** iff there exists an integer $k \in \mathbb{Z}$ such that $n = 2k + 1$.

Lemma 19.3. $\forall n, m \in \mathbb{Z}, \neg(2n + 1 = 2m)$ (an odd number can never equal an even number)

Lemma 19.4. $\forall n, m \in \mathbb{N} \setminus \{1\}.\mathcal{M}(nm) \equiv \mathcal{M}(mn)$ (The prime factor multiset of nm is identical to that of mn, as $\mathcal{M}(n) \uplus \mathcal{M}(m) \equiv \mathcal{M}(m) \uplus \mathcal{M}(n)$.)

Lemma 19.5. $\forall n \in \mathbb{N} \setminus \{1\}, \forall p \in \mathbb{P}. |\mathcal{M}(n \cdot p)| = |\mathcal{M}(n)| + 1$ (Multiplying a number by a prime p increases the total count of prime factors by exactly one, as $\mathcal{M}(np) \equiv \mathcal{M}(n) \uplus \{p\}$.)

Lemma 19.6. $\forall n \in \mathbb{N} \setminus \{1\}, \forall k \in \mathbb{N}. \left|\mathcal{M}(n^k)\right| = k \cdot |\mathcal{M}(n)|$ (The total count of prime factors of n^k is k times the count of prime factors of n. Specifically, for $k = 2$, the count **doubles**: $\left|\mathcal{M}(n^2)\right| = 2 \cdot |\mathcal{M}(n)|$.)

Def 19.44. Rational Number: A real number $x \in \mathbb{R}$ is **rational** iff it can be expressed as a ratio of two integers. Formally: $x \in \mathbb{Q} :\equiv \exists p, q \in \mathbb{Z}.(q \neq 0 \wedge x = \frac{p}{q})$

Def 19.45. Irrational Number: A real number $x \in \mathbb{R}$ is **irrational** iff it is not rational. Formally: $x \in \mathbb{I} :\equiv x \in \mathbb{R} \setminus \mathbb{Q}$

Theorem 19.7. $\sqrt{2} \notin \mathbb{Q}$ (irrational).

Proof. Suppose for the sake of contradiction that $\sqrt{2}$ is rational.

1. Then there exist $p, q \in \mathbb{N}$ such that $\sqrt{2} = \frac{p}{q}$.
2. Squaring both sides gives $2 = \frac{p^2}{q^2}$, which implies:
$$2q^2 = p^2$$
3. Let $|\mathcal{M}(n)|$ be the total count of prime factors of n. We analyze the parity of the counts on both sides of the equation:
   - **Right Side (p^2):** By the Scaling Lemma (lemma 19.6), $\left|\mathcal{M}(p^2)\right| = 2 \cdot |\mathcal{M}(p)|$. This count is even.
   - **Left Side ($2q^2$):** By the Increment Lemma (lemma 19.5), $\left|\mathcal{M}(2 \cdot q^2)\right| = \left|\mathcal{M}(q^2)\right| + 1$.
   - Since $\left|\mathcal{M}(q^2)\right|$ is $2 \cdot |\mathcal{M}(q)|$ (even), then $\left|\mathcal{M}(q^2)\right| + 1$ is odd.
4. This implies an even number of prime factors equals an odd number of prime factors.
5. By the Parity Non-Equivalence Lemma (lemma 19.3), $\forall n, m.\neg(2n = 2m + 1)$.

This is a contradiction. Therefore, the assumption that $\sqrt{2} \in \mathbb{Q}$ must be false. Thus, $\sqrt{2}$ is irrational. □

We can do real mathematics with $\mathcal{ND}^1_=$, figure 18 shows an $\mathcal{ND}^1_=$ proof by contradiction that $\sqrt{2}$ is irrational:

Step	Formula	Rule Applied
(1)	$\forall n, m \in \mathbb{Z}, \neg(2n+1 = 2m)$	Lemma 19.3
(2)	$\forall n \in \mathbb{N} \setminus \{1\}, \forall k \in \mathbb{N}. \left\lvert\mathcal{M}(n^k)\right\rvert = k \cdot \lvert\mathcal{M}(n)\rvert$	Lemma 19.6
(3)	$\forall n \in \mathbb{N} \setminus \{1\}, \forall p \in \mathbb{P}. \lvert\mathcal{M}(n \cdot p)\rvert = \lvert\mathcal{M}(n)\rvert + 1$	Lemma 19.5
(4)	$\forall x.\mathrm{irr}(x) \Leftrightarrow \neg(\exists p, q.x = \frac{p}{q})$	Def 19.45
(5)	$\mathrm{irr}(\sqrt{2}) \Leftrightarrow \neg(\exists p, q.\sqrt{2} = \frac{p}{q})$	$\forall E$ (on 4)
(6)	$\neg\mathrm{irr}(\sqrt{2})$	Assumption
(7)	$\neg\neg(\exists p, q.\sqrt{2} = \frac{p}{q})$	$\Leftrightarrow E$ (on 5,6)
(8)	$\exists p, q.\sqrt{2} = \frac{p}{q}$	$\neg E$ (on 7)
(9)	$\sqrt{2} = \frac{p}{q}$	Ax (follows from 6)
(10)	$2 = \frac{p^2}{q^2} \rightsquigarrow 2q^2 = p^2$	Arithmetic (on 9)
(11)	$\left\lvert\mathcal{M}(p^2)\right\rvert = 2 \cdot \lvert\mathcal{M}(p)\rvert$	$\forall E$ (on 2)
(12)	$q^2 \in \mathbb{N} \wedge 2 \in \mathbb{P} \Rightarrow \left\lvert\mathcal{M}(q^2 \cdot 2)\right\rvert = \left\lvert\mathcal{M}(q^2)\right\rvert + 1$	$\forall E$ (on 3)
(13)	$q^2 \in \mathbb{N} \wedge 2 \in \mathbb{P}$	Ax, Def 19.35
(14)	$\left\lvert\mathcal{M}(2q^2)\right\rvert = \left\lvert\mathcal{M}(q^2)\right\rvert + 1$	$\Rightarrow E$ (on 12, 13)
(15)	$\left\lvert\mathcal{M}(q^2)\right\rvert = 2 \cdot \lvert\mathcal{M}(q)\rvert$	$\forall E$ (on 2)
(16)	$\left\lvert\mathcal{M}(2q^2)\right\rvert = 2 \cdot \lvert\mathcal{M}(q)\rvert + 1$	$= E$ (on 14, 15)
(17)	$\left\lvert\mathcal{M}(p^2)\right\rvert = \left\lvert\mathcal{M}(p^2)\right\rvert$	$= I$
(18)	$\left\lvert\mathcal{M}(2q^2)\right\rvert = \left\lvert\mathcal{M}(p^2)\right\rvert$	$= E$ (on 10,17)
(19)	$2 \cdot \lvert\mathcal{M}(q)\rvert + 1 = \left\lvert\mathcal{M}(p^2)\right\rvert$	$= E$ (on 16,18)
(20)	$2 \cdot \lvert\mathcal{M}(q)\rvert + 1 = 2 \cdot \lvert\mathcal{M}(p)\rvert$	$= E$ (on 11,19)
(21)	$\neg(2\,\lvert\mathcal{M}(q)\rvert + 1 = 2\,\lvert\mathcal{M}(p)\rvert)$	$\forall E$ (on 1)
(22)	$\perp$	$\perp I$ (on 20,21)
(23)	$\perp$	$\exists E^6$ (on 22)
(24)	$\neg\neg\mathrm{irr}(\sqrt{2})$	$\neg I$ (on 23)
(25)	$\mathrm{irr}(\sqrt{2})$	$\neg E$ (on 24)

Figure 18: $\mathcal{ND}^1_=$ Proof of $\sqrt{2} \notin \mathbb{Q}$

20 Automated Theorem Proving in First-Order Logic

Def 20.1. $\mathcal{T}_1$**:** The **Standard First-Order Tableau Calculus** extends $\mathcal{T}_0$ with the following inference rules

$$\frac{(\forall X.A)^T \quad C \in \mathrm{cwff}_\iota(\Sigma)}{([C/X](A))^T}\ \mathcal{T}_1\forall \qquad\qquad \frac{(\forall X.A)^F \quad c \in \Sigma_0^{sk}\mathrm{new}}{([c/X]A)^F}\ \mathcal{T}_1\exists$$

Note

Problem: The rule $\mathcal{T}_1\forall$ displays a case of *"don't know indeterminism"*: to find a refutation we have to guess a formula C from the infinite set $\mathrm{cwff}_\iota(\Sigma)$.
For proof search, this means $\mathcal{T}_1\forall$ is infinitely branching in general

Def 20.2. $\mathcal{T}_1^f$**:** The **Free Variable Tableau Calculus** extends $\mathcal{T}_0$ with the following inference rules

$$\frac{(\forall X.A)^T \quad Y\mathrm{new}}{([Y/X](A))^T}\ \mathcal{T}_1^f\forall \qquad\qquad \frac{(\forall X.A)^F \quad \mathrm{free}(\forall X.A) = \{X^1, \cdots, X^k\} \quad f \in \Sigma_k^{sk}\mathrm{new}}{([f(X^1, \cdots, X^k)/X]A)^F}\ \mathcal{T}_1^f\exists$$

and generalizes its cut rule $\mathcal{T}_0\bot$ to:

$$\frac{\begin{array}{l} A^\alpha \\ \vdots \\ B^\beta \qquad (\alpha \neq \beta) \qquad \sigma(A) = \sigma(B) \end{array}}{\bot : \sigma}\ \mathcal{T}_1^f\bot$$

$\mathcal{T}_1^f$ **instantiates** the **whole tableau** by σ

Def 20.3. : Derivable rules in $\mathcal{T}_1^f$:

$$\frac{(\exists X.A)^F \quad Y\mathrm{new}}{([Y/X](A))^F} \qquad\qquad \frac{(\exists X.A)^T \quad free(\forall X.A) = \{X^1, \cdots, X^k\} \quad f \in \Sigma_k^{sk}\mathrm{new}}{([f(X^1, \cdots, X^k)/X](A))^T}$$

Combining $\mathcal{T}_0$ and $\mathcal{T}_1^f$ we get the inference rules shown in figure 19:

$$\frac{(A \wedge B)^T}{\begin{array}{c} A^T \\ B^T \end{array}}\ \mathcal{T}_0\wedge \qquad\qquad \frac{(A \wedge B)^F}{A^F \mid B^F}\ \mathcal{T}_0\vee$$

$$\frac{(\neg A)^T}{A^F}\ \mathcal{T}_0\neg^T \qquad\qquad \frac{(\neg A)^F}{A^T}\ \mathcal{T}_0\neg^F$$

$$\frac{\begin{array}{l} A^\alpha \\ \vdots \\ A^\beta \quad (\alpha \neq \beta) \end{array}}{\bot}\ \mathcal{T}_0\bot$$

$$\frac{(\forall X.A)^T \quad Y\text{new}}{([Y/X](A))^T}\ \mathcal{T}_1^f\forall \qquad\qquad \frac{(\forall X.A)^F \quad \text{free}(\forall X.A) = \{X^1, \cdots, X^k\} \quad f \in \Sigma_k^{sk}\text{new}}{([f(X^1, \cdots, X^k)/X]A)^F}\ \mathcal{T}_1^f\exists$$

$$\frac{\begin{array}{l} A^\alpha \\ \vdots \\ B^\beta \quad (\alpha \neq \beta) \quad \sigma(A) = \sigma(B) \end{array}}{\bot : \sigma}\ \mathcal{T}_1^f\bot$$

$$\frac{(A \Rightarrow B)^T}{A^F \mid B^T} \qquad\qquad \frac{(A \Rightarrow B)^F}{\begin{array}{c} A^T \\ B^F \end{array}} \qquad\qquad \frac{\begin{array}{c} A^T \\ (A \Rightarrow B)^T \end{array}}{B^T}$$

$$\frac{(A \vee B)^T}{A^T \mid B^T} \qquad\qquad \frac{(A \vee B)^F}{\begin{array}{c} A^F \\ B^F \end{array}}$$

$$\frac{(A \iff B)^T}{\begin{array}{c} A^T \\ B^T \end{array} \mid \begin{array}{c} A^F \\ B^F \end{array}} \qquad\qquad \frac{(A \iff B)^F}{\begin{array}{c} A^T \\ B^F \end{array} \mid \begin{array}{c} A^F \\ B^T \end{array}}$$

$$\frac{(\exists X.A)^F \quad Y\text{new}}{([Y/X](A))^F} \qquad\qquad \frac{(\exists X.A)^T \quad free(\forall X.A) = \{X^1, \cdots, X^k\} \quad f \in \Sigma_k^{sk}\text{new}}{([f(X^1, \cdots, X^k)/X](A))^T}$$

Figure 19: $\mathcal{T}_0 \cup \mathcal{T}_1^f$

Example Proof

$$[(\forall X.P(X) \Rightarrow Q(X)) \Rightarrow ((\forall X.P(X)) \Rightarrow (\forall X.Q(X)))]^F$$

$$[\forall X.P(X) \Rightarrow Q(X)]^T$$

$$((\forall X.P(X)) \Rightarrow (\forall X.Q(X)))^F$$

$$[\forall X.P(X)]^T$$

$$[\forall X.Q(X)]^F$$

$$[P(Y)]^T$$

$$[Q(c)]^F$$

$$[P(Z) \Rightarrow Q(Z)]^T$$

$$\begin{array}{cc} [P(Z)]^F & [Q(Z)]^T \\ \bot : [Y/Z] & [Q(Y)]^T \\ & \bot : [c/Y] \end{array}$$

Figure 20: $\mathcal{T}_1^f$ Proof Example

All $\mathcal{T}_1^f$ rules need to be applied once except for $\mathcal{T}_1^f\forall$. The number of applications of $\mathcal{T}_1^f\forall$ to $\forall X.A$ is called it's **multiplicity**.

Theorem 20.1. $\mathcal{T}_1^f$ is only complete with unbounded multiplicities

> **Note**
>
> That is why validity in PL[1] is undecidable

For example, see the proof in figure 21:

$$[(P(a) \vee P(b)) \Rightarrow \exists X.P(X)]^F$$

$$[P(a) \vee P(b)]^T$$

$$[\exists X.P(X)]^F$$

$$[P(Y)]^F \xrightarrow{[a/Y]} [P(a)]^F$$

$$\begin{array}{cc} [P(a)]^T & [P(b)]^T \\ \bot : [a/Y] & \\ & [P(Z)]^F \xrightarrow{[b/Z]} [P(b)]^F \\ & \bot : [b/Z] \end{array}$$

2nd application of $\mathcal{T}_1^f\forall$

Figure 21: $\mathcal{T}_1^f\forall$ Multiplicity Example

Def 20.4. Unification: For given terms A_1, A_2, **unification** is the problem of finding a substitution σ (called **unifier**), such that $\sigma(A_1) = \sigma(A_2)$

Def 20.5. Unification Problem: We call a formula $A^1 =^? B^1 \wedge \cdots \wedge A^n =^? B^n$ a **unification problem** iff $A^i, B^i \in \mathrm{wff}_\iota(\Sigma_1, V_\iota)$

> **Note**
>
> We consider unification problems as sets of equations. $\wedge$ here is associative, commutative, and idempotent. We consider equations as two-element multisets, $=^?$ here is commutative

Def 20.6. : A substitution is called a **unifier** for a unification problem $\mathcal{E}$ (and thus $\mathcal{E}$ is **unifiable**), iff it is a (simultaneous) unifier for all pairs in $\mathcal{E}$

Def 20.7. : The solution for a unification problem $U(A_1, A_2)$ is the set of unifiers $\{\sigma \mid \sigma(A_1) = \sigma(A_2)\}$. For example, let $\mathcal{E} := f(X) =^? f(g(Y))$, solutions are, e.g.

- $[g(a)/X], [a/Y]$
- $[g(g(a))/X], [g(a)/Y]$
- $[g(Z)/X], [Z/Y]$
- $[g(Y)/X]$

Def 20.8. : Let σ and θ be substitutions and $W \subseteq \mathcal{V}_\iota$, we say that a substitution σ is **more general** than θ on W (write as $\sigma \leq \theta[W]$), iff there is a substitution ρ, such that $\theta = \rho \circ \sigma[W]$, where $\sigma = \rho[W]$, iff $\sigma(X) = \rho(X)$ for all $X \in W$

- The substitution $\sigma = [g(Z)/X, Z/Y]$ is more general than $\theta = [g(a)/X, a/Y]$ because there exists $\rho = [a/Z]$ such that:

  $$\theta = \rho \circ \sigma$$

  This identity $\theta = \rho \circ \sigma$ is verified by applying the composition to the variables in the domain:

  - $(\rho \circ \sigma)(X) = \rho(\sigma(X)) = \rho(g(Z)) = g(a)$
  - $(\rho \circ \sigma)(Y) = \rho(\sigma(Y)) = \rho(Z) = a$

  Since the mapping for each variable matches θ, we confirm that θ is a specialization of σ via ρ.

- Likewise, σ is more general than $\theta' = [g(g(a))/X, g(a)/Y]$ via $\rho' = [g(a)/Z]$, since:

  $$\theta' = \rho' \circ \sigma$$

  where $(\rho' \circ \sigma)(X) = g(g(a))$ and $(\rho' \circ \sigma)(Y) = g(a)$.

Def 20.9. MGU: σ is called a **most general unifier (mgu)** of A_1, A_2, iff it is minimal in $U(A_1, A_2)$ **with respect to** the more general ordering ($\leq$) restricted to the variables occurring in the problem: $W = \mathrm{free}(A_1) \cup \mathrm{free}(A_2)$.

Def 20.10. : We call a pair $A =^? B$ **solved** in a unification problem $\mathcal{E}$, iff A is a variable X, $\mathcal{E} = X =^? A \wedge \mathcal{E}'$, and $X \notin (\mathrm{free}(A) \cup \mathrm{free}(\mathcal{E}'))$. We call a unification problem $\mathcal{E}$ a **solved form**, iff all its pairs are **solved**.

Lemma 20.2. Solved forms are of the form $X^1 =^? B^1 \wedge \cdots \wedge X^n =^? B^n$ where X^i are distinct and $X^i \notin \mathrm{free}(B^j)$

Def 20.11. $\mathcal{E}_\sigma$: Any substitution $\sigma = [B^1/X^1], \cdots, [B^n/X^n]$ induces a **solved unification problem**

$$\mathcal{E}_\sigma := (X^1 =^? B^1 \wedge \cdots \wedge X^n =^? B^n)$$

Lemma 20.3. If $\mathcal{E} = X^1 =^? B^1 \wedge \cdots \wedge X^n =^? B^n$ is a solved form, then $\mathcal{E}$ has the unique MGU $\sigma_\epsilon := [B^1/X^1], \cdots, [B^n/X^n]$.

Def 20.12. $\mathcal{U}$: The inference system $\mathcal{U}$ consists of the following inference rules:

$$\frac{\mathcal{E} \wedge f(A^1, \cdots, A^n) =^? f(B^1, \cdots, B^n)}{\mathcal{E} \wedge A^1 =^? B^1 \wedge \cdots \wedge A^n =^? B^n}\ \mathcal{U}_{dec} \qquad \frac{\mathcal{E} \wedge A =^? A}{\mathcal{E}}\ \mathcal{U}_{triv}$$

$$\frac{\mathcal{E} \wedge X =^? A \quad X \notin free(A) \quad X \in free(\mathcal{E})}{[A/X](\mathcal{E} \wedge X =^? A)}\ \mathcal{U}_{elim} \qquad \frac{\mathcal{E} \wedge f(X^1, \cdots, X^n) =^? g(Y^1, \cdots, Y^n)}{\bot}\ \mathcal{U}\bot$$

Unification Example : Let $\mathcal{E} := f(g(X,X),h(a)) =^? f(g(a,Z),h(Z))$

$$\begin{array}{c}
f(g(X,X),h(a)) =^? f(g(a,Z),h(Z)) \\ \hline
g(X,X) =^? g(a,Z) \wedge h(a) =^? h(Z) \quad \mathcal{U}_{dec} \\ \hline
X =^? a \wedge X =^? Z \wedge h(a) =^? h(Z) \quad \mathcal{U}_{dec} \\ \hline
X =^? a \wedge X =^? Z \wedge a =^? Z \quad \mathcal{U}_{dec} \\ \hline
X =^? a \wedge a =^? Z \wedge a =^? Z \quad \mathcal{U}_{elim} \\ \hline
X =^? a \wedge Z =^? a \wedge a =^? a \quad \mathcal{U}_{elim} \\ \hline
X =^? a \wedge Z =^? a \quad \mathcal{U}_{triv}
\end{array}$$

The final $\mathcal{E}$ is in solved form since all the pairs are solved. Hence $\mathcal{E}$ has the unique MGU $\sigma_\epsilon := [a/X],[a/Z]$.

Unification Example : Let $\mathcal{E} := S_1(g(f(x),g(x,y)),y) =^? S_1(g(z,v),f(w))$

$$\begin{array}{c}
S_1(g(f(x),g(x,y)),y) =^? S_1(g(z,v),f(w)) \\ \hline
g(f(x),g(x,y)) =^? g(z,v) \wedge y =^? f(w) \quad \mathcal{U}_{dec} \\ \hline
z =^? f(x) \wedge v =^? g(x,y) \wedge y =^? f(w) \quad \mathcal{U}_{dec} \\ \hline
z =^? f(x) \wedge v =^? g(x,y) \wedge y =^? f(w) \quad \mathcal{U}_{elim} \\ \hline
z =^? f(x) \wedge v =^? g(x,f(w)) \wedge y =^? f(w) \quad \mathcal{U}_{elim}
\end{array}$$

The final equation $z =^? f(x) \wedge v =^? g(x,f(w)) \wedge y =^? f(w)$ is in solved form.
The MGU is $\{z \to f(x), v \to g(x,f(w)), y \to f(w)\}$.

Unification Example : Let $\mathcal{E} := S_2(g(f(x),g(x,u)),f(y),z) =^? S_2(g(g(g(u,v),f(w)),f(c)),f(g(u,v)),f(c))$ (not unifiable)

$$\begin{array}{c}
S_2(g(f(x),g(x,u)),f(y),z) =^? S_2(g(g(g(u,v),f(w)),f(c)),f(g(u,v)),f(c)) \\ \hline
g(f(x),g(x,u)) =^? g(g(g(u,v),f(w)),f(c)) \wedge f(y) =^? f(g(u,v)) \wedge z =^? f(c) \quad \mathcal{U}_{dec} \\ \hline
f(x) =^? g(g(u,v),f(w)) \wedge g(x,u) =^? f(c) \wedge f(y) =^? f(g(u,v)) \wedge z =^? f(c) \quad \mathcal{U}_{dec} \\ \hline
\bot \quad \mathcal{U}\bot
\end{array}$$

Lemma 20.4. $\mathcal{U}$ is sound: $\mathcal{E} \vdash_{\mathcal{U}} \mathcal{F}$ implies $U(\mathcal{F}) \subseteq U(\mathcal{E})$

Lemma 20.5. $\mathcal{U}$ is complete: $\mathcal{E} \vdash_{\mathcal{U}} \mathcal{F}$ implies $U(\mathcal{E}) \subseteq U(\mathcal{F})$

Lemma 20.6. $\mathcal{U}$ is confluent: the order of derivations does not matter

Corollary 20.7. First-Order unification is unitary: i.e. MGUs are unique up to renaming of introduced variables

Lemma 20.8. $\mathcal{U}$ is terminating; any $\mathcal{U}$--derivation is finite

Def 20.13. : We call an equational problem $\mathcal{E}$ **$\mathcal{U}$-reducible**, iff there is a $\mathcal{U}$-step $\mathcal{E} \vdash_{\mathcal{U}} \mathcal{F}$ from $\mathcal{E}$

Lemma 20.9. If $\mathcal{E}$ is unifiable but not solved, then it is $\mathcal{U}$-reducible

Corollary 20.10. First-Order unification is decidable in PL[1]

Recall: in $\mathcal{T}_1^f$, the rule $\mathcal{T}_1^f\bot$ instantiates the whole tableau. The problem is that there might be more than one $\mathcal{T}_1^f\bot$ opportunity on a branch and choosing which one makes a difference. For example, in the following proof, if we choose to close first on the right branch, the tableau does not close! However, choosing the other $\mathcal{T}_1^f\bot$ in the left branch allows closure!

$$(\exists X.(p(a) \wedge p(b) \Rightarrow p(X)) \wedge (q(b) \Rightarrow q(X)))^F$$

$$((p(a) \wedge p(b) \Rightarrow p(Y)) \wedge (q(b) \Rightarrow q(Y)))^F$$

$(p(a) \wedge p(b) \Rightarrow p(Y))^F$	$(q(b) \Rightarrow q(Y))^F$
$p(a)^T$	$q(b)^T$
$p(b)^T$	$q(Y)^F$
$p(Y)^F$	
$\bot : [a/Y]$	

Therefore, when implementing $\mathcal{T}_1^f$, there are two ways of systematic search, namely **backtracking search** back over $\mathcal{T}_1^f\bot$ opportunities and saturate without $\mathcal{T}_1^f\bot$ and find **spanning matings**.

Def 20.14. Spanning Mating: Let $\mathcal{T}$ be a $\mathcal{T}_1^f$ tableau, then we call a unification problem $\mathcal{E} := A_1 =?B_1 \cdots A_n =?B_n$ a **mating** for $\mathcal{T}$, iff A_i^T and B_i^F occur in the same branch in $\mathcal{T}$. We say taht $\mathcal{E}$ is a **spanning mating**, if $\mathcal{E}$ is unifiable and every branch $\mathcal{B}$ of $\mathcal{T}$ contains A_i^T and B_i^F for some i.

Theorem 20.11. A $\mathcal{T}_1^f$-tableau with spanning mating induces a closed $\mathcal{T}_1$ tableau.

Def 20.15. CNF_1: The **First-Order CNF Calculus** CNF_1 extends CNF_0 by the following rules:

$$\frac{(\forall X.A)^T \vee C \quad Z \notin (\mathrm{free}(A) \cup \mathrm{free}(C))}{([Z/X](A))^T \vee C}\ \forall^T$$

$$\frac{(\forall X.A)^F \vee C \quad \mathrm{free}(\forall X.A) = \{X_1, \cdots, X_k\} \quad f \in \sum_k^{sk} \mathrm{new}}{([f(X_1, \cdots, X_k)/X](A))^F \vee C}\ \forall^F$$

Def 20.16. : The following inference rules are derived in CNF_1:

$$\frac{(\exists X.A)^T \vee C \quad \mathrm{free}(\exists X.A) = \{X_1, \cdots, X_k\} \quad f \in \sum_k^{sk} \mathrm{new}}{([f(X_1, \cdots, X_k)/X](A))^T \vee C}\ \exists^T$$

$$\frac{(\exists X.A)^F \vee C \quad Z \notin (\mathrm{free}(A) \cup \mathrm{free}(C))}{([Z/X](A))^F \vee C}\ \exists^F$$

Def 20.17. : $\mathrm{CNF}_0 \cup \mathrm{CNF}_1$ rules are shown in figure 22

$$\frac{C \vee (A \vee B)^T}{C \vee A^T \vee B^T} \vee^T \qquad \frac{C \vee (A \vee B)^F}{(C \vee A^F) \wedge (C \vee B^F)} \vee^F$$

$$\frac{C \vee \neg A^T}{C \vee A^F} \neg^T \qquad \frac{C \vee \neg A^F}{C \vee A^T} \neg^F$$

$$\frac{(\forall X.A)^T \vee C \quad Z \notin (\text{free}(A) \cup \text{free}(C))}{([Z/X](A))^T \vee C} \forall^T$$

$$\frac{(\forall X.A)^F \vee C \quad \text{free}(\forall X.A) = \{X_1, \cdots, X_k\} \quad f \in \sum_k^{sk} \text{new}}{([f(X_1, \cdots, X_k)/X](A))^F \vee C} \forall^F$$

$$\frac{C \vee (A \Rightarrow B)^T}{C \vee A^F \vee B^T} \Rightarrow^T \qquad \frac{C \vee (A \Rightarrow B)^F}{(C \vee A^T) \wedge (C \vee B^F)} \Rightarrow^F$$

$$\frac{C \vee (A \wedge B)^T}{(C \vee A^T) \wedge (C \vee B^T)} \wedge^T \qquad \frac{C \vee (A \wedge B)^F}{C \vee A^F \vee B^F} \wedge^F$$

$$\frac{(\exists X.A)^T \vee C \quad \text{free}(\exists X.A) = \{X_1, \cdots, X_k\} \quad f \in \sum_k^{sk} \text{new}}{([f(X_1, \cdots, X_k)/X](A))^T \vee C} \exists^T$$

$$\frac{(\exists X.A)^F \vee C \quad Z \notin (\text{free}(A) \cup \text{free}(C))}{([Z/X](A))^F \vee C} \exists^F$$

Figure 22: $\text{CNF}_0 \cup \text{CNF}_1$

Def 20.18. $\mathcal{R}_1$**:** The **First-Order Resolution Calculus** $\mathcal{R}_1$ is a test calculus that manipulates formulae in CNF, therefore, it operates on clause sets, and it has the following two inference rules:

$$\frac{A^T \vee C \quad B^F \vee D \quad \sigma = \text{MGU}(A, B)}{(\sigma(C)) \vee (\sigma(D))} \mathcal{R}_1 \qquad \frac{A^\alpha \vee B^\beta \vee C \quad \sigma = \text{MGU}(A, B)}{(\sigma(A)) \vee (\sigma(C))} \mathcal{R}_1$$

Def 20.19. : $\mathcal{R}_0 \cup \mathcal{R}_1$ inference rules are shown in figure 23

$$\frac{P^T \vee A \qquad P^F \vee B}{A \vee B} \mathcal{R}_0$$

$$\frac{A^T \vee C \quad B^F \vee D \quad \sigma = \text{MGU}(A, B)}{(\sigma(C)) \vee (\sigma(D))} \mathcal{R}_1 \qquad \frac{A^\alpha \vee B^\beta \vee C \quad \sigma = \text{MGU}(A, B)}{(\sigma(A)) \vee (\sigma(C))} \mathcal{R}_1$$

Figure 23: $\mathcal{R}_0 \cup \mathcal{R}_1$

Example 1: We want to prove the following using $\mathcal{R}_1$:

$$\exists X.\forall Y.\exists Z.\exists W.((\neg P(Z) \wedge \neg R(b, a)) \vee \neg R(a, b) \vee R(W, a) \vee (P(Y) \wedge R(X, b)))$$

We label it with F and use CNF_1:

$$[\exists X.\forall Y.\exists Z.\exists W.((\neg P(Z) \wedge \neg R(b, a)) \vee \neg R(a, b) \vee R(W, a) \vee (P(Y) \wedge R(X, b)))]^F$$

We apply $\exists^F$ where

$$X1 \notin \text{free}(\forall Y.\exists Z.\exists W.((\neg P(Z) \wedge \neg R(b, a)) \vee \neg R(a, b) \vee R(W, a) \vee (P(Y) \wedge R(X, b)))) = \{X\}$$

We get:

$$[\forall Y.\exists Z.\exists W.((\neg P(Z) \wedge \neg R(b,a)) \vee \neg R(a,b) \vee R(W,a) \vee (P(Y) \wedge R(X1,b)))]^F$$

We apply $\forall^F$ where:

$$\text{free}(\forall Y.\exists Z.\exists W.((\neg P(Z) \wedge \neg R(b,a)) \vee \neg R(a,b) \vee R(W,a) \vee (P(Y) \wedge R(X1,b)))) = \{X1\} \quad |\{X1\}| = 1$$

Hence, we invent a new $f(X1) \in \Sigma_1^{sk}$ and apply $[f(X1)/Y]$, we get:

$$[\exists Z.\exists W.((\neg P(Z) \wedge \neg R(b,a)) \vee \neg R(a,b) \vee R(W,a) \vee (P(f(X1)) \wedge R(X1,b)))]^F$$

We apply $\exists^F$ where:

$$Z1 \notin \text{free}(\exists W.((\neg P(Z) \wedge \neg R(b,a)) \vee \neg R(a,b) \vee R(W,a) \vee (P(f(X1)) \wedge R(X1,b)))) = \{Z, X1\}$$

We get:

$$[\exists W.((\neg P(Z1) \wedge \neg R(b,a)) \vee \neg R(a,b) \vee R(W,a) \vee (P(f(X1)) \wedge R(X1,b)))]^F$$

We apply $\exists^F$ where:

$$W1 \notin \text{free}(((\neg P(Z1) \wedge \neg R(b,a)) \vee \neg R(a,b) \vee R(W,a) \vee (P(f(X1)) \wedge R(X1,b)))) = \{Z1, W, X1\}$$

We get:

$$[((\neg P(Z1) \wedge \neg R(b,a)) \vee \neg R(a,b) \vee R(W1,a) \vee (P(f(X1)) \wedge R(X1,b)))]^F$$

We apply $\vee^F$, we get:

$$[\neg P(Z1) \wedge \neg R(b,a)]^F \wedge [\neg R(a,b)]^F \wedge [R(W1,a)]^F \wedge [(P(f(X1)) \wedge R(X1,b))]^F$$

We apply $\wedge^F$, we get:

$$[\neg P(Z1)]^F \vee [\neg R(b,a)]^F \wedge [\neg R(a,b)]^F \wedge [R(W1,a)]^F \wedge [P(f(X1))]^F \vee [R(X1,b)]^F$$

We apply $\neg^F$:

$$[P(Z1)]^T \vee [R(b,a)]^T \wedge [R(a,b)]^T \wedge [R(W1,a)]^F \wedge [P(f(X1))]^F \vee [R(X1,b)]^F$$

The final clause set is:

$$\left\{[P(Z1)]^T \vee [R(b,a)]^T, [R(a,b)]^T, [R(W1,a)]^F, [P(f(X1))]^F \vee [R(X1,b)]^F\right\}$$

We start the $\mathcal{R}_1$-refutation:

1. $[P(Z1)]^T \vee [R(b,a)]^T$
2. $[R(a,b)]^T$
3. $[R(W1,a)]^F$
4. $[P(f(X1))]^F \vee [R(X1,b)]^F$
5. Resolve 2 with 4 under $\sigma = [a/X1] \rightsquigarrow [P(f(a))]^F$
6. Resolve 1 with 5 under $\sigma = [a/Z1] \rightsquigarrow [R(b,a)]^T$
7. Resolve 3 with 6 under $\sigma = [b/W1] \rightsquigarrow \{\square\}$

A trick to make life easier

When asked to prove something using $\mathcal{R}_1$, we need first to transform to CNF using CNF_1. Instead of applying the CNF_1 rules, we do the following steps:

1. Negate the formula
2. Push negation inside
3. Eliminate $\forall$ and $\exists$:
   - If you are eliminating a $\forall$, leave the universally quantified variable as it is in the formula
   - If you are eliminating a $\exists$, Skolemize:

- If before the $\exists$ there is a $\forall$, we Skolemize by new function $f \in \Sigma_k^{sk}$ whose parameters are the universally quantified variables before $\exists$
- If no $\forall$ is before the $\exists$, we Skolemize with a new constant $c \in \Sigma_0^{sk}$

Same Example (now with the trick): We want to prove the following using $\mathcal{R}_1$:

$$\exists X.\forall Y.\exists Z.\exists W.((\neg P(Z) \wedge \neg R(b,a)) \vee \neg R(a,b) \vee R(W,a) \vee (P(Y) \wedge R(X,b)))$$

We negate the formula:

$$\neg(\exists X.\forall Y.\exists Z.\exists W.((\neg P(Z) \wedge \neg R(b,a)) \vee \neg R(a,b) \vee R(W,a) \vee (P(Y) \wedge R(X,b))))$$

We push negation inside:

$$\forall X.\exists Y.\forall Z.\forall W.\neg((\neg P(Z) \wedge \neg R(b,a)) \vee \neg R(a,b) \vee R(W,a) \vee (P(Y) \wedge R(X,b)))$$

We eliminate $\forall W.$:

$$\forall X.\exists Y.\forall Z.\neg((\neg P(Z) \wedge \neg R(b,a)) \vee \neg R(a,b) \vee R(W,a) \vee (P(Y) \wedge R(X,b)))$$

We eliminate $\forall Z.$:

$$\forall X.\exists Y.\neg((\neg P(Z) \wedge \neg R(b,a)) \vee \neg R(a,b) \vee R(W,a) \vee (P(Y) \wedge R(X,b)))$$

We Skolemize $\exists Y.$:

$$\forall X.\neg((\neg P(Z) \wedge \neg R(b,a)) \vee \neg R(a,b) \vee R(W,a) \vee (P(f(X)) \wedge R(X,b)))$$

We eliminate $\forall X.$

$$\neg((\neg P(Z) \wedge \neg R(b,a)) \vee \neg R(a,b) \vee R(W,a) \vee (P(f(X)) \wedge R(X,b)))$$

We continue by starting applying CNF_0 rules as above.

Example 2: We want to prove the following using $\mathcal{R}_1$:

$$\exists X.\forall Y.\exists W.\exists Z.\neg((R(Z,Y) \vee \neg P(Z)) \wedge (\neg Q(d) \vee P(c)) \wedge (Q(d) \vee \neg P(c)) \wedge (\neg R(Z,Y) \vee \neg P(W) \vee \neg Q(X)) \wedge P(c))$$

We negate:

$$\neg(\exists X.\forall Y.\exists W.\exists Z.\neg((R(Z,Y) \vee \neg P(Z)) \wedge (\neg Q(d) \vee P(c)) \wedge (Q(d) \vee \neg P(c)) \wedge (\neg R(Z,Y) \vee \neg P(W) \vee \neg Q(X)) \wedge P(c)))$$

We push negation inside:

$$\forall X.\exists Y.\forall W.\forall Z.\neg\neg((R(Z,Y) \vee \neg P(Z)) \wedge (\neg Q(d) \vee P(c)) \wedge (Q(d) \vee \neg P(c)) \wedge (\neg R(Z,Y) \vee \neg P(W) \vee \neg Q(X)) \wedge P(c))$$

We eliminate $\forall Z.$

$$\forall X.\exists Y.\forall W.\neg\neg((R(Z,Y) \vee \neg P(Z)) \wedge (\neg Q(d) \vee P(c)) \wedge (Q(d) \vee \neg P(c)) \wedge (\neg R(Z,Y) \vee \neg P(W) \vee \neg Q(X)) \wedge P(c))$$

We eliminate $\forall W.$

$$\forall X.\exists Y.\neg\neg((R(Z,Y) \vee \neg P(Z)) \wedge (\neg Q(d) \vee P(c)) \wedge (Q(d) \vee \neg P(c)) \wedge (\neg R(Z,Y) \vee \neg P(W) \vee \neg Q(X)) \wedge P(c))$$

We Skolemize $\exists Y.$

$$\forall X.\neg\neg((R(Z,f(X)) \vee \neg P(Z)) \wedge (\neg Q(d) \vee P(c)) \wedge (Q(d) \vee \neg P(c)) \wedge (\neg R(Z,f(X)) \vee \neg P(W) \vee \neg Q(X)) \wedge P(c))$$

We eliminate $\forall X.$

$$\neg\neg((R(Z,f(X)) \vee \neg P(Z)) \wedge (\neg Q(d) \vee P(c)) \wedge (Q(d) \vee \neg P(c)) \wedge (\neg R(Z,f(X)) \vee \neg P(W) \vee \neg Q(X)) \wedge P(c))$$

This is just:

$$[\neg((R(Z,f(X)) \vee \neg P(Z)) \wedge (\neg Q(d) \vee P(c)) \wedge (Q(d) \vee \neg P(c)) \wedge (\neg R(Z,f(X)) \vee \neg P(W) \vee \neg Q(X)) \wedge P(c))]^F$$

Using $\neg^F$

$$[(R(Z, f(X)) \vee \neg P(Z)) \wedge (\neg Q(d) \vee P(c)) \wedge (Q(d) \vee \neg P(c)) \wedge (\neg R(Z, f(X)) \vee \neg P(W) \vee \neg Q(X)) \wedge P(c)]^T$$

Using $\wedge^T$

$$(R(Z, f(X)) \vee \neg P(Z))^T \wedge (\neg Q(d) \vee P(c))^T \wedge (Q(d) \vee \neg P(c))^T \wedge (\neg R(Z, f(X)) \vee \neg P(W) \vee \neg Q(X))^T \wedge (P(c))^T$$

We are in CNF, we start the proof:

1	$(R(Z, f(X)))^T \vee (P(Z))^F$	initial
2	$(Q(d))^F \vee (P(c))^T$	initial
3	$(Q(d))^T \vee (P(c))^F$	initial
4	$(R(Z, f(X)))^F \vee (P(W))^F \vee (Q(X))^F$	initial
5	$(P(c))^T$	initial
6	$(Q(d))^T$	3, 5
7	$(R(c, f(X)))^T$	1, 5 $[c/Z]$
8	$(P(W))^F \vee (Q(X))^F$	4, 7 $[c/Z]$
9	$(Q(X))^F$	5, 8 $[c/W]$
10	$\square$	6, 9 $[d/X]$

Example 3: Prove the following using $\mathcal{R}_1$:

$$\forall X \forall Y \forall Z \exists Z \exists U \exists V \exists W ((P(X, Y) \Rightarrow (P(Z, a) \Rightarrow R(a))) \Rightarrow ((P(U, V) \wedge P(W, a)) \Rightarrow R(a)))$$

Negate:

$$\neg(\forall X \forall Y \forall Z \exists Z \exists U \exists V \exists W ((P(X, Y) \Rightarrow (P(Z, a) \Rightarrow R(a))) \Rightarrow ((P(U, V) \wedge P(W, a)) \Rightarrow R(a))))$$

Push negation inside:

$$\exists X \exists Y \exists Z \forall Z \forall U \forall V \forall W \neg((P(X, Y) \Rightarrow (P(Z, a) \Rightarrow R(a))) \Rightarrow ((P(U, V) \wedge P(W, a)) \Rightarrow R(a)))$$

Eliminate $\forall Z \forall U \forall V \forall W$

$$\exists X \exists Y \exists Z \neg((P(X, Y) \Rightarrow (P(Z, a) \Rightarrow R(a))) \Rightarrow ((P(U, V) \wedge P(W, a)) \Rightarrow R(a)))$$

Skolemize:

$$\neg((P(x, y) \Rightarrow (P(z, a) \Rightarrow R(a))) \Rightarrow ((P(U, V) \wedge P(W, a)) \Rightarrow R(a))) \quad x, y, z \in \Sigma_0^{sk}$$

This is just:

$$((P(x, y) \Rightarrow (P(z, a) \Rightarrow R(a))) \Rightarrow ((P(U, V) \wedge P(W, a)) \Rightarrow R(a)))^F$$

Apply $\Rightarrow^F$

$$(P(x, y) \Rightarrow (P(z, a) \Rightarrow R(a)))^T \wedge ((P(U, V) \wedge P(W, a)) \Rightarrow R(a))^F$$

Apply $\Rightarrow^T$

$$(P(x, y))^F \vee (P(z, a) \Rightarrow R(a))^T \wedge ((P(U, V) \wedge P(W, a)) \Rightarrow R(a))^F$$

Apply $\Rightarrow^F$

$$(P(x, y))^F \vee (P(z, a) \Rightarrow R(a))^T \wedge (P(U, V) \wedge P(W, a))^T \wedge (R(a))^F$$

Apply $\wedge^T$

$$(P(x, y))^F \vee (P(z, a) \Rightarrow R(a))^T \wedge (P(U, V))^T \wedge (P(W, a))^T \wedge (R(a))^F$$

Apply $\Rightarrow^T$

$$(P(x, y))^F \vee (P(z, a))^F \vee (R(a))^T \wedge (P(U, V))^T \wedge (P(W, a))^T \wedge (R(a))^F$$

We are in CNF, we start the proof:

1	$(P(x,y))^F \vee (P(z,a))^F \vee (R(a))^T$	initial
2	$(P(U,V))^T$	initial
3	$(P(W,a))^T$	initial
4	$(R(a))^F$	initial
5	$(P(z,a))^F \vee (R(a))^T$	1,2 $[x/U],[y/V]$
6	$(R(a))^T$	3,5 $[z/W]$
7	$\square$	4,6

Example 4: Let's just see an example where we need to convert to CNF (no proof required)

$$\forall X.((\forall Y.Q(Y) \Rightarrow P(X,Y)) \Rightarrow (\exists Y.P(Y,X)))$$

First, we transform $(Q(Y) \Rightarrow P(X,Y))$ $\qquad A \Rightarrow B \rightsquigarrow \neg A \vee B$

$$\forall X.((\forall Y.\neg Q(Y) \vee P(X,Y)) \Rightarrow (\exists Y.P(Y,X)))$$

Then we transform the outer implication:

$$\forall X.(\neg(\forall Y.\neg Q(Y) \vee P(X,Y)) \vee (\exists Y.P(Y,X)))$$

We push negation:

$$\forall X.((\exists Y.Q(Y) \wedge \neg P(X,Y)) \vee (\exists Y.P(Y,X)))$$

Since we have two **different** $\exists Y$, we rename one to keep them distinct:

$$\forall X.((\exists Y.Q(Y) \wedge \neg P(X,Y)) \vee (\exists Z.P(Z,X)))$$

We Skolemize $\exists Z$ and $\exists Y$:

$$\forall X.((Q(g(X)) \wedge \neg P(X,g(X))) \vee (P(f(X),X)))$$

We eliminate the $\forall X$:

$$(Q(g(X)) \wedge \neg P(X,g(X))) \vee (P(f(X),X))$$

We distribute $\vee$ over $\wedge$

$$(Q(g(X)) \vee P(f(X),X)) \wedge (\neg P(X,g(X)) \vee P(f(X),X))$$

The final clause set is: $\{Q(g(X)) \vee P(f(X),X), \neg P(X,g(X)) \vee P(f(X),X)\}$

Def 20.20. Horn Clause: A **Horn clause** is a clause with **at most** one **positive** literal

> **Note**
>
> In Logic Programming Languages (e.g. PROLOG), PL[1] Resolution is the main machine behind the language.
> **Observations**:
> - A rule in PROLOG $H:-B_1,\cdots,B_n$ correspond to $H^T \vee B_1^F \cdots B_n^F$, i.e. a Horn clause
> - A goal set (query) $?-G_1,\cdots,G_n$ corresponds to $G_1^F \cdots G_n^F$
> - A fact F corresponds to the unit clause F^T

Def 20.21. Horn Logic: Let $S := \langle \mathcal{L}, \mathcal{M}, \models \rangle$ be a logical system. A formal system $\langle S, \mathcal{C} \rangle$ is called a **Horn Logic** if:

- The language $\mathcal{L}$ is the set of Horn clauses.
- The calculus $\mathcal{H}$ is defined by the rules of Modus Ponens (MP), Conjunction Introduction ($\wedge I$), and Substitution (Subst).

Def 20.22. Answer Substitution: A logic program (set of Horn clauses) P **entails** a query Q with **answer substitution** σ, iff there is an $\mathcal{H}$-derivation D of Q from P and σ is the combined substitution of the Subst instances in D

Example:

```
human(leibniz).
human(sokrates).
greek(sokrates).
fallible(X):-human(X).
```

Query: ?-fallible(X), greek(X).

We have three facts, one rule, and one query. They translate to the following clause set:

1. human(leibniz)^T
2. human(sokrates)^T
3. greek(sokrates)^T
4. fallible$(X)^T \vee$ human$(X)^F$ (human$(X) \Rightarrow$ fallible(X))
5. fallible$(X)^F \vee$ greek$(X)^F$ (Start Resolution)
6. Resolve (1) with (4) under [leibniz/X] $\rightsquigarrow$ fallible(leibniz)^T
7. Resolve (5) with (6) under [leibniz/X] $\rightsquigarrow$ greek(leibniz)^F
8. greek(leibniz)^F cannot be resolved further; the program backtracks to try another substitution.
9. Resolve (2) with (4) under [sokrates/X] $\rightsquigarrow$ fallible(sokrates)^T
10. Resolve (5) with (9) under [sokrates/X] $\rightsquigarrow$ greek(sokrates)^F
11. Resolve (3) with (10) $\rightsquigarrow \{\square\}$
12. The program returns T and the answer substitution $\sigma :=$ [sokrates/X].

Forward and Backward Chaining:

Other than querying (Backchaining), PROLOG derives facts given the set of facts initially written (Forward Chaining):

Backchaining Example:

```
has_motor(c).
has_wheels(c,4).
car(X) :- has_motor(X), has_wheels(X,4).
```

Query: ?-car(X).

We have two facts, one rule, and one query. They translate to the following clause set:

1. m$(c)^T$
2. w$(c,4)^T$
3. c$(X)^T \vee$ m$(X)^F \vee$ w$(X,4)^F$ (m$(X) \wedge$ w$(X,4) \Rightarrow$ c(X))
4. c$(X)^F$ (Start Resolution)
5. Resolve (3) with (4) $\rightsquigarrow$ m$(X)^F \vee$ w$(X,4)^F$
6. Resolve (5) with (1) under [c/X] $\rightsquigarrow$ w$(c,4)^F$
7. Resolve (6) with (2) $\rightsquigarrow \{\square\}$
8. The program returns T and the answer substitution $\sigma :=$ [c/X].

Forward Chaining Example:

```
has_motor(c).
has_wheels(c,4).
car(X) :- has_motor(X),
          has_wheels(X,4).
```

We have two facts and one rule, which translate to:

1. $m(c)^T$
2. $w(c,4)^T$
3. $m(X) \wedge w(X,4) \Rightarrow car(X)$

PROLOG can derive the fact car(c) using the $\mathcal{H}$ calculus:

$$\dfrac{\dfrac{\dfrac{}{m(c)}\,Ax \quad \dfrac{}{w(c,4)}\,Ax}{m(c) \wedge w(c,4)}\,\wedge I \quad \dfrac{\dfrac{}{m(X) \wedge w(X,4) \Rightarrow car(X)}\,Ax}{m(c) \wedge w(c,4) \Rightarrow car(c)}\,\text{Subst [c/X]}}{\text{car}(c)}\,MP$$

21 Logic-Based Knowledge Representation

Def 21.1. Semantic Network: A semantic network is a directed graph for representing knowledge:

- nodes represent **objects** and **concepts** (classes of objects)
- edges (called links) represent relations between these

Def 21.2. Inclusion: We call links labeled by *isa* inclusion links or isa links (inclusion of concepts)

Def 21.3. Instance: We call links labeled by *inst* instance links or inst links (concept memership)

Def 21.4. Relations in Semantic Nets: We call all links labels **except** isa and inst in a semantic network **relations**.

Def 21.5. Inference in Semantic Networks: Let N be a semantic network and R a relation in N such that $A \xrightarrow{isa} B \xrightarrow{R} C$ or $A \xrightarrow{inst} B \xrightarrow{R} C$, then we can derive a relation $A \xrightarrow{R} C$ in N. The process of deriving new concepts and relation from existing ones is called **inference** and concepts/relation that are only available via inference are called **implicit** in a semantic network.

Example: blue are derived relations, red is not allowed:

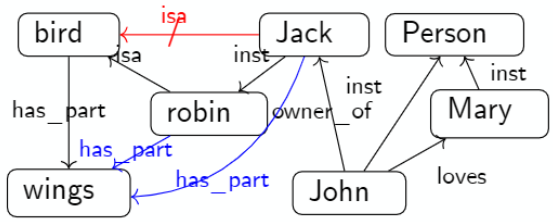


Figure 24: Inference in semantic networks

Def 21.6. TBox: We call the subgraph of a semantic network N spanned by the isa links and relations between **concepts** the **terminology** (or **TBox**, or **Isa Hierarchy**).

Def 21.7. ABox: We call the subgraph of a semantic network N spanned by the inst links and relations between **objects** the **assertions** (or **ABox**).

Example:

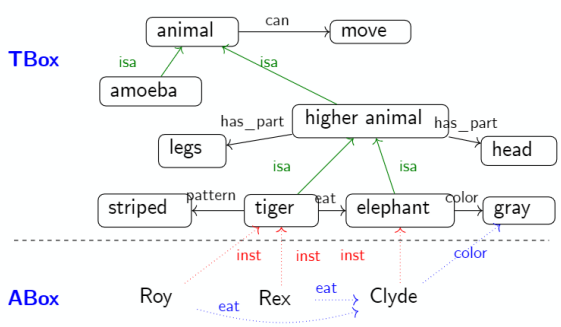


Figure 25: TBox and ABox in semantic networks

Def 21.8. $\mathbf{PL^0_{DL}}$: We use propositional logic as a set description language. We define $\mathrm{PL}^0_{\mathrm{DL}}$ by the following grammar for the $\mathrm{PL}^0_{\mathrm{DL}}$ concepts (formulae):

$$\mathcal{L} ::= C \mid \top \mid \bot \mid \overline{\mathcal{L}} \mid \mathcal{L} \sqcap \mathcal{L} \mid \mathcal{L} \sqcup \mathcal{L} \mid \mathcal{L} \sqsubseteq \mathcal{L} \mid \mathcal{L} \equiv \mathcal{L}$$

$\mathrm{PL}^0_{\mathrm{DL}}$ is formed from:

- atomic formulae
- concept intersection ($\sqcap$)
- concept complement ($\overline{\cdot}$)

- concept union ($\sqcup$)
- concept subsumption ($\sqsubseteq$), and equivalence ($\equiv$)

Def 21.9. Set-Theoretic Semantics of $\mathbf{PL^0_{DL}}$: Let $\mathcal{D}$ be a given set (domain of discourse) and $\varphi : \mathcal{V}_{\mathrm{PL}^0} \to \mathcal{P}(\mathcal{D})$. We define the valuation $[\![\cdot]\!]$ as:

- $[\![P]\!] := \varphi(P)$ (for atomic concepts)
- $[\![\top]\!] := \mathcal{D}$ and $[\![\bot]\!] := \emptyset$
- $[\![\overline{A}]\!] := \mathcal{D} \setminus [\![A]\!]$
- $[\![A \sqcap B]\!] := [\![A]\!] \cap [\![B]\!]$
- $[\![A \sqcup B]\!] := [\![A]\!] \cup [\![B]\!]$
- $[\![A \sqsubseteq B]\!]$ is satisfied iff $[\![\overline{A}]\!] \cup [\![B]\!] = \mathcal{D}$ which is satisfied iff $[\![A]\!] \subseteq [\![B]\!]$
- $[\![A \equiv B]\!]$ is satisfied iff $[\![A]\!] = [\![B]\!]$

Note

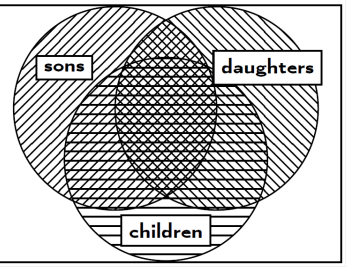
The triple $\langle \mathrm{PL}^0_{\mathrm{DL}}, \mathcal{S}, [\![\cdot]\!] \rangle$, where $\mathcal{S}$ is the class of possible domains, forms a logical system.

Def 21.10. Concept Axiom: A concept axiom is a $\mathrm{PL}^0_{\mathrm{DL}}$ formula A that is assumed to be true in the world.

Def 21.11. Set-Theoretic Semantics of Axioms: A is true in domain of discourse $\mathcal{D}$ iff $[\![A]\!] = \mathcal{D}$

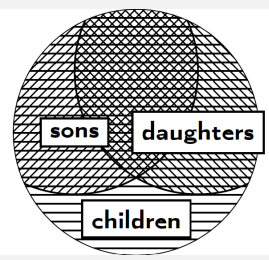
Note

Any concept we add to our world is just a set, for example, here is a world with three concepts (no concept axioms yet):

concepts	Set Semantics
child daughter son	

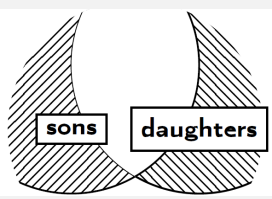
Looks a mess? try to add concept axioms. Here is how it looks like when adding:

- son $\sqsubseteq$ child
- daughter $\sqsubseteq$ child



We add two more concept axioms:

- $\overline{\text{son} \sqcap \text{daughter}}$
- child $\sqsubseteq$ son $\sqcup$ daughter



The set-theoretic semantics is compatible with the regular semantics of propositional logic, therefore we have the same propositional identities:

Table 5: $\mathrm{PL}^0_{\mathrm{DL}}$ Identities

Name	**for $\sqcap$**	**for $\sqcup$**
Idempotence	$\varphi \sqcap \varphi = \varphi$	$\varphi \sqcup \varphi = \varphi$
Identity	$\varphi \sqcap \top = \varphi$	$\varphi \sqcup \bot = \varphi$
Absorption 1	$\varphi \sqcap \bot = \bot$	$\varphi \sqcup \top = \top$
Commutativity	$\varphi \sqcap \psi = \psi \sqcap \varphi$	$\varphi \sqcup \psi = \psi \sqcup \varphi$
Associativity	$\varphi \sqcap (\psi \sqcap \theta) = (\varphi \sqcap \psi) \sqcap \theta$	$\varphi \sqcup (\psi \sqcup \theta) = (\varphi \sqcup \psi) \sqcup \theta$
Distributivity	$\varphi \sqcap (\psi \sqcup \theta) = (\varphi \sqcap \psi) \sqcup (\varphi \sqcap \theta)$	$\varphi \sqcup (\psi \sqcap \theta) = (\varphi \sqcup \psi) \sqcap (\varphi \sqcup \theta)$
Absorption 2	$\varphi \sqcap (\varphi \sqcup \theta) = \varphi$	$\varphi \sqcup (\varphi \sqcap \theta) = \varphi$
De Morgan rule	$\overline{\varphi \sqcap \psi} = \overline{\varphi} \sqcup \overline{\psi}$	$\overline{\varphi \sqcup \psi} = \overline{\varphi} \sqcap \overline{\psi}$
double negation	$\overline{\overline{\varphi}} = \varphi$	

Def 21.12. Translation to $\mathbf{PL}^1$: We define the translation of $\mathrm{PL}^0_{\mathrm{DL}}$ expressions into PL^1 using two functions: a recursive mapping FOL_x for concept expressions and a top-level mapping FOL for axioms/formulae.

Recursive mapping for Concepts:

$\mathrm{FOL}_x(p) := p(X)$
$\mathrm{FOL}_x(\overline{A}) := \neg\mathrm{FOL}_x(A)$
$\mathrm{FOL}_x(A \sqcap B) := \mathrm{FOL}_x(A) \wedge \mathrm{FOL}_x(B)$
$\mathrm{FOL}_x(A \sqcup B) := \mathrm{FOL}_x(A) \vee \mathrm{FOL}_x(B)$
$\mathrm{FOL}_x(A \sqsubseteq B) := \mathrm{FOL}_x(A) \Rightarrow \mathrm{FOL}_x(B)$
$\mathrm{FOL}_x(A \equiv B) := \mathrm{FOL}_x(A) \Leftrightarrow \mathrm{FOL}_x(B)$

Mapping for Formulae (calls FOL_x): $\mathrm{FOL}(A) := \forall X.\, \mathrm{FOL}_x(A)$

Example: Let's translate the formula: *"an undergrad is a person who is not a graduate"*. In $\mathrm{PL}^0_{\mathrm{DL}}$, we can express this by:

$$\text{undergrad} \equiv \text{person} \sqcap \overline{\text{graduate}}$$

$$\begin{aligned}
\mathrm{FOL}(\text{undergrad} \equiv \text{person} \sqcap \overline{\text{graduate}}) &= \forall X.\mathrm{FOL}_x(\text{undergrad} \equiv \text{person} \sqcap \overline{\text{graduate}}) \\
\mathrm{FOL}_x(\text{undergrad} \equiv \text{person} \sqcap \overline{\text{graduate}}) &= \mathrm{FOL}_x(\text{undergrad}) \Leftrightarrow \mathrm{FOL}_x(\text{person} \sqcap \overline{\text{graduate}}) \\
\mathrm{FOL}_x(\text{undergrad}) &= \text{undergrad}(X) \\
\mathrm{FOL}_x(\text{person} \sqcap \overline{\text{graduate}}) &= \mathrm{FOL}_x(\text{person}) \wedge \mathrm{FOL}_x(\overline{\text{graduate}}) \\
&= \text{person}(X) \wedge \neg\mathrm{FOL}_x(\text{graduate}) \\
&= \text{person}(X) \wedge \neg\text{graduate}(X) \\
\mathrm{FOL}(\text{undergrad} \equiv \text{person} \sqcap \overline{\text{graduate}}) &= \forall X.(\text{undergrad}(X) \Leftrightarrow (\text{person}(X) \wedge \neg\text{graduate}(X)))
\end{aligned}$$

Def 21.13. Ontology: An **ontology** consists of a formal system $\langle \mathcal{L}, \mathcal{M}, \models, \mathcal{C} \rangle$ with concept axioms about:

- individuals: concrete entities in a domain of discourse (instances of concepts)
- concepts: particular collections of individuals that share properties and aspects
- relations: ways in which individuals can be related to one another

Note

An **ontology** is a representation of the types, properties, and interrelationships of the entities that really exist for a particular domain of discourse

Note

- Semantic networks are ontologies
- $\mathrm{PL}^0_{\mathrm{DL}}$ is an ontology format that is *formal* but *weak*
- PL^1 is an ontology format that is *formal* and *expressive*

Def 21.14. Description Logic: A description logic (DL) is a formal system for talking about collections of objects and their relations that is at least as expressive as PL^0 with Set-Theoretic Semantics and offers individuals and relations.

Def 21.15. DL Components: A description logic has the following four components:

- a formal language $\mathcal{L}$ with logical constants $\sqcap, \bar{\cdot}, \sqcup, \sqsubseteq$,and $\equiv$
- a Set-Theoretic Semantics $\langle \mathcal{D}, [\![\cdot]\!] \rangle$
- a translation into PL^1 that is compatible with $\langle \mathcal{D}, [\![\cdot]\!] \rangle$
- a calculus for $\mathcal{L}$ that induces a decision procedure for $\mathcal{L}$-satisfibility

Def 21.16. Concept Definition: Let $\mathcal{D}$ be a description logic with concepts $\mathcal{C}$ Then a **concept definition** is a pair $c = C$ where c is a new concept name and $C \in \mathcal{C}$ is a $\mathcal{D}$-formula. Example: mother = woman $\sqcap$ has_child

Def 21.17. Recursive Concept Definition: A concept definition $c = C$ is called **recursive**, iff c occurs in C.

Def 21.18. Acyclic TBox: A TBox is a finite set of concept definitions and concept axioms. It is called **acyclic**, iff it does not contain recursive definitions.

Def 21.19. : A formula A is called **normalized** w.r.t. a TBox $\mathcal{T}$, iff it does not contain concepts defined in $\mathcal{T}$.

Def 21.20. : Ontology systems employ three main kinds of inference / reasoning:

- Consistency test: is a concept definition satisfiable
- Subsumption test: does a concept subsume another?
- Instance test: is an individual an example of a concept?

Def 21.21. Consistent Concept: We call a concept C consistent, iff there is no concept A with both $C \sqsubseteq A$ and $C \sqsubseteq \overline{A}$

Def 21.22. Inconsistent Concept: A concept C is called inconsistent, iff $[\![C]\!] = \emptyset$ for all $\mathcal{D}$.

For example, consider we have the following TBox in $\mathrm{PL}^0_{\mathrm{DL}}$:

- $\text{man} = \text{person} \sqcap \text{has_Y}$
- $\text{woman} = \text{person} \sqcap \overline{\text{has_Y}}$
- $\text{hermaphrodite} = \text{man} \sqcap \text{woman}$

the concept hermaphrodite is inconsistent since $[\![\text{hermaphrodite}]\!] = \emptyset$ for all $\mathcal{D}$

> **Note**
>
> We test satisfiability using tableaux, resolution, DPLL in $\mathrm{PL}^0_{\mathrm{DL}}$

Def 21.23. Subsumption: A subsumes B (modulo a set $\mathcal{A}$ of concept axioms), iff $[\![B]\!] \subseteq [\![A]\!]$ for all interpretations $\mathcal{D}$ that satisfy $\mathcal{A}$. Equivalently, iff $\mathcal{A} \sqsubseteq B \sqsubseteq A = T$

> **Note**
>
> Subsumption tests reduce to consistency tests because in PL^0, $\mathcal{A} \Rightarrow (A \Rightarrow B)$ is valid iff $\mathcal{A} \wedge A \wedge \neg B$ is inconsistent.

Example: if we want to know whether $\text{man} \sqsubseteq \text{person}$ in the previous TBox, we check the consistency of $\text{man} \wedge \neg\text{person}$. $\text{person} \wedge \text{has_Y} \wedge \neg\text{person}$ is inconsistent, hence $\text{man} \sqsubseteq \text{person}$.

Def 21.24. Instance Test: An instance test computes, given an ontology, whether an individual is a member of a given concept.

> **Note**
>
> This is not something that we can do in $\mathrm{PL}^0_{\mathrm{DL}}$ because it is a TBox only system.

Def 21.25. $\mathcal{ALC}$ Syntax: The formulae of $\mathcal{ALC}$ are given by the following grammar:

$$F_{\mathcal{ALC}} ::= C \mid \top \mid \bot \mid F_{\mathcal{ALC}} \sqcap F_{\mathcal{ALC}} \mid F_{\mathcal{ALC}} \sqcup F_{\mathcal{ALC}} \mid \exists R.F_{\mathcal{ALC}} \mid \forall R.F_{\mathcal{ALC}}$$

> **Note**
>
> - PL^0 is not expressive enough
> - PL^1 is too expressive and not decidable
> - Middle ground $\rightsquigarrow \mathcal{ALC}$
> - $\mathcal{ALC}$ is a simple description logic
> - It is more expressive than PL^0 but weaker than PL^1
> - It allows us to quantify only over finite sets (unline PL^1 where we quantify over V_ι which is infinite)
> - Restricted quantification: the quantified variables in $\mathcal{ALC}$ only range over values that can be reached via binary relations such as `has_child`

Def 21.26. Role: A role represent a binary relation (like in PL^1)

Syntax Examples:

- $\text{person} \sqcap (\exists\text{has_child.student})$

  This means the set of persons that have a child which is a student, i.e. parents of students

- $\text{person} \sqcap (\exists\text{has_child}.\exists\text{has_child.student})$

  grandparents of students

- person $\sqcap$ ($\exists$has_child.$\exists$has_child.student $\sqcup$ teacher)

  grandparents of students or teachers

- person $\sqcap$ ($\forall$has_child.student)

  parents whose children are **all** students

- person $\sqcap$ ($\forall$has_child.$\exists$has_child.student)

  grandparents, whose children **all** have **at least** one child that is a student

- car $\sqcap$ $\exists$has_parts.$\exists$made_in.$\overline{\text{EU}}$

  cars that have at least one part that has not been made in the EU

- student $\sqcap$ $\forall$audit_course.graduatelevelcourse

  students that **all** the courses they audit are graduate level courses

- house $\sqcap$ $\forall$has_parkig.off_street

  houses with only off-street parking

- student $\sqcap$ $\forall$audit_course.($\exists$has_tutorial.$\top$ $\sqsubseteq$ $\forall$has_TA.woman) remember $A \sqsubseteq B \equiv \overline{A} \sqcup B$

  students that only audit courses that either have no tutorials or tutorials that are TAed by women

Def 21.27. Concept Definition in $\mathcal{ALC}$: A concept definition is a pair consisting of a new concept name (the defineiendum) and an $\mathcal{ALC}$ formula (the definiens). Concepts that are not defineienda are called **primitive**

Note

We extend the $\mathcal{ALC}$ grammar from Def 21.25 by the following:

$$\text{CD}_{\mathcal{ALC}} ::= C = F_{\mathcal{ALC}}$$

So it becomes:

$$\begin{aligned} F_{\mathcal{ALC}} &::= C \mid \top \mid \bot \mid F_{\mathcal{ALC}} \sqcap F_{\mathcal{ALC}} \mid F_{\mathcal{ALC}} \sqcup F_{\mathcal{ALC}} \mid \exists R.F_{\mathcal{ALC}} \mid \forall R.F_{\mathcal{ALC}} \\ \text{CD}_{\mathcal{ALC}} &::= C = F_{\mathcal{ALC}} \end{aligned}$$

Concept definitions Examples:

- man = person $\sqcap$ $\exists$has_chrom.Y_chrom

  meaning, a man is a perosn who has at lease one Y chromosome. `person` here is a primitive concept

- woman = person $\sqcap$ $\forall$has_chrom.$\overline{\text{Y_chrom}}$

- mother = woman $\sqcap$ $\exists$has_child.person

- father = man $\sqcap$ $\exists$has_child.person

- grandparent = person $\sqcap$ $\exists$has_child.(mother $\sqcup$ father)

Def 21.28. : We call an $\mathcal{ALC}$ formula φ **normalized** w.r.t. a set of concept definitions, iff all concepts occuring in φ are primitive.

Def 21.29. Normalization: Given a set $\mathcal{D}$ of concept definitions, **normalization** is the process of replacing in an $\mathcal{ALC}$ formula φ all occurences of defineienda in $\mathcal{D}$ with their definientia.

Example: normalization of `grandparent`:

- grandparent = person $\sqcap$ $\exists$has_child.(mother $\sqcup$ father)

- grandparent = person $\sqcap$ $\exists$has_child.(woman $\sqcap$ $\exists$has_child.person $\sqcup$ man $\sqcap$ $\exists$has_child.person)

- grandparent = person$\sqcap$$\exists$has_child.(person$\sqcap$$\forall$has_chrom.$\overline{\text{Y_chrom}}$$\sqcap$$\exists$has_child.person$\sqcup$person$\sqcap$$\exists$has_chrom.Y_chrom$\sqcap$ $\exists$has_child.person)

Note

- Normalization results can be exponential.
- Normalization need not to terminate on cyclic TBoxes

Def 21.30. $\mathcal{ALC}$ Semantics: $\mathcal{ALC}$ Semantics is an extension of the Set-Theoretic Semantics of PL^0.

Def 21.31. $\mathcal{ALC}$ Interpretation: A **model** for $\mathcal{ALC}$ is a pair $\langle \mathcal{D}, [\![\cdot]\!] \rangle$, where $\mathcal{D}$ is a non-empty set (domain of discourse) and $[\![\cdot]\!]$ is the interpretation such that:

- $[\![\top]\!] = \mathcal{D}, \quad [\![\bot]\!] = \emptyset, \quad [\![r]\!] \subseteq \mathcal{D} \times \mathcal{D}$
- $[\![\overline{\varphi}]\!] = \overline{[\![\varphi]\!]} = \mathcal{D} \setminus [\![\varphi]\!]$
- $[\![\varphi \sqcap \psi]\!] = [\![\varphi]\!] \cap [\![\psi]\!]$
- $[\![\varphi \sqcup \psi]\!] = [\![\varphi]\!] \cup [\![\psi]\!]$
- $[\![\exists R.\varphi]\!] = \{x \in \mathcal{D} \mid \exists y.\, \langle x, y\rangle \in [\![R]\!] \wedge y \in [\![\varphi]\!]\}$
- $[\![\forall R.\varphi]\!] = \{x \in \mathcal{D} \mid \forall y.\, \langle x, y\rangle \in [\![R]\!] \Rightarrow y \in [\![\varphi]\!]\}$

Def 21.32. $\mathcal{ALC}$ to PL^1 Translation: The translation of $\mathcal{ALC}$ into PL^1 extends the one from Def 21.12 by the following quantifier rules:

$\mathrm{FOL}_x(\forall R.\varphi) := (\forall Y.R(X,Y) \Rightarrow \mathrm{FOL}_(\varphi))$
$\mathrm{FOL}_x(\exists R.\varphi) := (\exists Y.R(X,Y) \wedge \mathrm{FOL}_y(\varphi))$

Def 21.33. $\mathcal{ALC}$ Identities:

1. $\overline{\exists R.\varphi} = \forall R.\overline{\varphi}$
2. $\forall R.(\varphi \sqcap \psi) = \forall R.\varphi \sqcap \forall R.\psi$
3. $\overline{\forall R.\varphi} = \exists R.\overline{\varphi}$
4. $\exists R.(\varphi \sqcup \psi) = \exists R.\varphi \sqcup \exists R.\psi$

Proof of (1):

$$
\begin{aligned}
[\![\overline{\exists R.\varphi}]\!] &= \overline{[\![\exists R.\varphi]\!]} = \mathcal{D} \setminus [\![\exists R.\varphi]\!] \\
&= \mathcal{D} \setminus \{x \in \mathcal{D} \mid \exists y.\, \langle x, y\rangle \in [\![R]\!] \wedge y \in [\![\varphi]\!]\} \\
&= \{x \in \mathcal{D} \mid \neg(\exists y.\, \langle x, y\rangle \in [\![R]\!] \wedge y \in [\![\varphi]\!])\} \\
&= \{x \in \mathcal{D} \mid \forall y.\neg(\langle x, y\rangle \in [\![R]\!] \wedge y \in [\![\varphi]\!])\} && \neg(A \wedge B) = \neg A \vee \neg B = A \Rightarrow \neg B \\
&= \{x \in \mathcal{D} \mid \forall y.\, \langle x, y\rangle \in [\![R]\!] \Rightarrow y \notin [\![\varphi]\!]\} \\
&= \{x \in \mathcal{D} \mid \forall y.\, \langle x, y\rangle \in [\![R]\!] \Rightarrow y \in [\![\overline{\varphi}]\!]\} \\
&= [\![\forall R.\overline{\varphi}]\!]
\end{aligned}
$$

Def 21.34. NNF in $\mathcal{ALC}$: An $\mathcal{ALC}$ formula is in Negation Normal Form (NNF), iff complement ($\bar{\cdot}$) is only applied to primitive concept.

Example: $\overline{\exists R.(\forall S.e \sqcap \overline{\forall S.d})} \rightsquigarrow \forall R.\overline{\forall S.e \sqcap \overline{\forall S.d}} \rightsquigarrow \forall R.\overline{\forall S.e} \sqcup \overline{\overline{\forall S.d}} \rightsquigarrow \forall R.\exists S.\overline{e} \sqcup \forall S.d$

Def 21.35. : We define the **ABox Assertions** for $\mathcal{ALC}$:

- $a : \varphi$ — role assertions (a is a φ, i.e. instance of φ)
- a R b — a stands in relation R to b

assertions make up the ABox in $\mathcal{ALC}$

Def 21.36. : We extend $\mathcal{ALC}$ semantics by:

- $[\![a : \varphi]\!] = T \Leftrightarrow [\![a]\!] \in [\![\varphi]\!]$
- $[\![a \text{ R } b]\!] = T \Leftrightarrow ([\![a]\!], [\![b]\!]) \in [\![R]\!]$

Def 21.37. : We extend the translation of $\mathcal{ALC}$ to PL[1] by the following:

$\mathrm{FOL}(a : \varphi) := \mathrm{FOL}_a(\overline{\varphi})$
$\mathrm{FOL}(a \ \mathrm{R} \ b) := \mathrm{R}(a, b)$

Note

The complete $\mathcal{ALC}$ semantics:

- $[\![\top]\!] = \mathcal{D}, \quad [\![\bot]\!] = \emptyset, \quad [\![r]\!] \subseteq \mathcal{D} \times \mathcal{D}$
- $[\![\overline{\varphi}]\!] = \overline{[\![\varphi]\!]} = \mathcal{D} \setminus [\![\varphi]\!]$
- $[\![\varphi \sqcap \psi]\!] = [\![\varphi]\!] \cap [\![\psi]\!]$
- $[\![\varphi \sqcup \psi]\!] = [\![\varphi]\!] \cup [\![\psi]\!]$
- $[\![\exists R.\varphi]\!] = \{x \in \mathcal{D} \mid \exists y. \langle x, y\rangle \in [\![R]\!] \wedge y \in [\![\varphi]\!]\}$
- $[\![\forall R.\varphi]\!] = \{x \in \mathcal{D} \mid \forall y. \langle x, y\rangle \in [\![R]\!] \Rightarrow y \in [\![\varphi]\!]\}$
- $[\![a : \varphi]\!] = T \Leftrightarrow [\![a]\!] \in [\![\varphi]\!]$
- $[\![a \ \mathrm{R} \ b]\!] = T \Leftrightarrow ([\![a]\!], [\![b]\!]) \in [\![R]\!]$

The complete translation:
$\mathrm{FOL}_x(p) := p(X)$
$\mathrm{FOL}_x(\overline{A}) := \neg\mathrm{FOL}_x(A)$
$\mathrm{FOL}_x(A \sqcap B) := \mathrm{FOL}_x(A) \wedge \mathrm{FOL}_x(B)$
$\mathrm{FOL}_x(A \sqcup B) := \mathrm{FOL}_x(A) \vee \mathrm{FOL}_x(B)$
$\mathrm{FOL}_x(A \sqsubseteq B) := \mathrm{FOL}_x(A) \Rightarrow \mathrm{FOL}_x(B)$
$\mathrm{FOL}_x(A \equiv B) := \mathrm{FOL}_x(A) \Leftrightarrow \mathrm{FOL}_x(B)$
$\mathrm{FOL}_x(\forall R.\varphi) := (\forall Y. R(X, Y) \Rightarrow \mathrm{FOL}_y(\varphi))$
$\mathrm{FOL}_x(\exists R.\varphi) := (\exists Y. R(X, Y) \wedge \mathrm{FOL}_y(\varphi))$

For Formulae:
$\mathrm{FOL}(A) := \forall X. \mathrm{FOL}_x(A)$
$\mathrm{FOL}(a : \varphi) := \mathrm{FOL}_a(\overline{\varphi})$
$\mathrm{FOL}(a \ \mathrm{R} \ b) := \mathrm{R}(a, b)$

Note

We can also interpret inverse of roles:

- If $[\![\exists R.\varphi]\!] = \{x \in \mathcal{D} \mid \exists y. \langle x, y\rangle \in [\![R]\!] \wedge y \in [\![\varphi]\!]\}$
  Then $[\![\exists R^{-1}.\varphi]\!] = \{x \in \mathcal{D} \mid \exists y. \langle y, x\rangle \in [\![R]\!] \wedge y \in [\![\varphi]\!]\}$
- Similarly, if $[\![\forall R.\varphi]\!] = \{x \in \mathcal{D} \mid \forall y. \langle x, y\rangle \in [\![R]\!] \Rightarrow y \in [\![\varphi]\!]\}$
  Then $[\![\forall R^{-1}.\varphi]\!] = \{x \in \mathcal{D} \mid \forall y. \langle y, x\rangle \in [\![R]\!] \Rightarrow y \in [\![\varphi]\!]\}$

For example, assume we have a simple $\mathcal{D} := \{\text{Alice}, \text{Bob}, \text{Mary}, \text{Charlie}, \text{Ben}, \text{David}\}$
And we have a role has_child $:= \{\langle \text{Alice}, \text{Bob}\rangle, \langle \text{Alice}, \text{Mary}\rangle, \langle \text{David}, \text{Ben}\rangle, \langle \text{David}, \text{Charlie}\rangle\}$
Additionally, we have three concepts:

- student $:= \{\text{Bob}, \text{Charlie}, \text{Ben}\}$
- professor $:= \{\text{David}\}$
- person $:= \top$

Let's examine the following concepts:

- $\forall \text{has_child}.\text{student} \rightsquigarrow \{x \in \mathcal{D} \mid \forall y. \langle x, y\rangle \in [\![\text{has_child}]\!] \Rightarrow y \in [\![\text{student}]\!]\} \rightsquigarrow \{\text{David}\}$
- $\forall \text{has_child}^{-1}.\text{professor} \rightsquigarrow \{x \in \mathcal{D} \mid \forall y. \langle y, x\rangle \in [\![\text{has_child}]\!] \Rightarrow y \in [\![\text{professor}]\!]\} \rightsquigarrow \{\text{Charlie}, \text{Ben}\}$
- $\exists \text{has_child}.\text{student} \rightsquigarrow \{x \in \mathcal{D} \mid \exists y. \langle x, y\rangle \in [\![\text{has_child}]\!] \wedge y \in [\![\text{student}]\!]\} \rightsquigarrow \{\text{Alice}, \text{David}\}$
- $\exists \text{has_child}^{-1}.\text{professor} \rightsquigarrow \{x \in \mathcal{D} \mid \exists y. \langle y, x\rangle \in [\![\text{has_child}]\!] \wedge y \in [\![\text{professor}]\!]\} \rightsquigarrow \{\text{Charlie}, \text{Ben}\}$

Def 21.38. $\mathcal{T}_{\mathcal{ALC}}$: The $\mathcal{ALC}$ tableau calculus $\mathcal{T}_{\mathcal{ALC}}$ acts on assertions $x : \varphi$ and $x \ \mathrm{R} \ y$ where φ is a normalized $\mathcal{ALC}$ concept in NNF with the following inference rules

$$\frac{x : \varphi \sqcap \psi}{x : \varphi \quad x : \psi}\ \mathcal{T}_{\sqcap} \qquad \frac{x : \varphi \sqcup \psi}{x : \varphi \mid x : \psi}\ \mathcal{T}_{\sqcup}$$

$$\frac{x : \forall R.\varphi \quad xRy}{y : \varphi}\ \mathcal{T}_{\forall} \qquad \frac{x : \exists R.\varphi}{xRy \quad y : \varphi}\ \mathcal{T}_{\exists}$$

$$\frac{x : \varphi \quad x : \overline{\varphi}}{\bot}\ \mathcal{T}_{\bot}$$

Note

The $\forall$ rule states that if an individual x belongs to $\forall R.\varphi$, which is the set:

$$\{x \in \mathcal{D} \mid \forall y.\, \langle x, y\rangle \in [\![R]\!] \Rightarrow y \in [\![\varphi]\!]\}$$

and we know that some y is related to x, i.e. $\langle x, y\rangle \in R$, then that y must belong to the concept φ.
For example, assume `Alice` : $\forall$`has_child.student`, that means:

$$\texttt{Alice} \in \{x \in \mathcal{D} \mid \forall y.\, \langle x, y\rangle \in [\![\texttt{has_child}]\!] \Rightarrow y \in [\![\texttt{student}]\!]\}$$

If `Alice` has a child `Bob` $\rightsquigarrow \langle \texttt{Alice}, \texttt{Bob}\rangle \in \texttt{has_child}$
Then `Bob` must be a `student` $\rightsquigarrow \texttt{Bob} \in [\![\texttt{student}]\!]$

Note

The $\exists$ rule states that if x belongs to the concept $\exists R.\varphi$, which is the set:

$$\{x \in \mathcal{D} \mid \exists y.\, \langle x, y\rangle \in [\![R]\!] \wedge y \in [\![\varphi]\!]\}$$

then we infer that there must be at least a y related to x via R ($\langle x, y\rangle \in R$) and that y must belong to the concept φ
For example: assume `Alice` : $\exists$`has_pet.dog` that means:

$$\texttt{Alice} \in \{x \in \mathcal{D} \mid \exists y.\, \langle x, y\rangle \in [\![\texttt{has_pet}]\!] \wedge y \in [\![\texttt{dog}]\!]\}$$

We can safely infer `Alice has_pet y:dog` and `y:dog`

$\mathcal{T}_{\mathcal{ALC}}$ for consistency checking

- We convert the concept to NNF
- We initialize the tableau with $x : \varphi$ for an arbitrary x
- We apply rules and expand
- We check if any branch remains open
  - if all branches close $\rightsquigarrow \varphi$ is consistent (unsatisfiable)
  - if at least one open branch remains $\rightsquigarrow \varphi$ is consistent (satisfiable)

Example Check consistency of: $\forall$has_child.man $\sqcap$ $\exists$has_child.$\overline{\text{man}}$

1	$x : \forall$has_child.man $\sqcap$ $\exists$has_child.$\overline{\text{man}}$	initial
2	$x : \forall$has_child.man	$\mathcal{T}_{\sqcap}$ on 1
3	$x : \exists$has_child.$\overline{\text{man}}$	$\mathcal{T}_{\sqcap}$ on 1
4	$x$has_child$y$	$\mathcal{T}_{\exists}$ on 3
5	$y : \overline{\text{man}}$	$\mathcal{T}_{\exists}$ on 3
6	y : man	$\mathcal{T}_{\forall}$ on 2, 5
7	$\bot$	$\mathcal{T}_{\bot}$ on 5,6

Example: Show satisfiability of: $\forall$has_child.(ugrad $\sqcup$ grad) $\sqcap$ $\exists$has_child.$\overline{\text{ugrad}}$

$$x : \forall \text{has_child}.(\text{ugrad} \sqcup \text{grad}) \sqcap \exists \text{has_child}.\overline{\text{ugrad}}$$

$$x : \forall \text{has_child}.(\text{ugrad} \sqcup \text{grad})$$

$$x : \exists \text{has_child}.\overline{\text{ugrad}}$$

$$x \text{ has_child } y$$

$$y : \overline{\text{ugrad}}$$

$$y : (\text{ugrad} \sqcup \text{grad})$$

$$y : \text{ugrad} \qquad\qquad y : \text{grad}$$

$$\bot \qquad\qquad \square \text{ (open)}$$

Since there is an open branch, we have a model that satisfy the concept, namely:
x has a child y that is graduate student.

Lemma 21.1. If φ satisfiable, then $\mathcal{T}_{\mathcal{ALC}}$ terminates on $x : \varphi$ with open branch

Lemma 21.2. open branches of a saturated tableau for φ induces models for φ

Theorem 21.3. $\mathcal{T}_{\mathcal{ALC}}$ terminates

Theorem 21.4. The SAT problem for $\mathcal{ALC}$ is in PSPACE

Theorem 21.5. The SAT problem for $\mathcal{ALC}$ is PSPACE-Complete

Theorem 21.6. The $\mathcal{ALC}$ SAT problem is in EXPTIME

22 STRIPS Planning

Def 22.1. STRIPS: The Stanford Research Institute Problem Solver (STRIPS) is a formal planning language that represents states and actions using propositional logic.

> Note
>
> - STRIPS utilizes propositional variables as atomic formulae to define world states.
> - **Preconditions:** A conjunction of atoms that must be satisfied to execute an action.
> - **Effects:** A conjunction of literals (additions and deletions) specifying how the state is modified by an action.
> - **Goals:** A conjunction of atoms defining the conditions required for a problem to be considered solved.

Def 22.2. STRIPS Task: A **STRIPS Task** is a quadruple $\langle \mathcal{F}, \mathcal{A}, \mathcal{I}, \mathcal{G} \rangle$ where:

- $\mathcal{F}$ is a finite set of **facts** which are atomic propositions in PL^0 or PL^{nq}. E.g. $\text{clean(room)}, \text{robot_at(door)}$
- $\mathcal{A}$ is a finite set of **actions**.
- An action $a \in \mathcal{A}$ is a triple $a := \langle \text{pre}_a, \text{add}_a, \text{del}_a \rangle$ of subsets of $\mathcal{F}$.
  - pre_a are the **preconditions** of a
  - add_a is a list of facts that become true when a is successfully applied
  - del_a is a list of facts that must be deleted after applying a

  For example, we represent action $\text{drive}(x, y)$ as $\langle \{\text{at}(x)\}, \{\text{at}(y), \text{visited}(y)\}, \{\text{at}(x)\} \rangle$
- $\mathcal{I} \subseteq \mathcal{F}$ is the initial state (facts that are true initially), e.g. initially we are at Sydney $\rightsquigarrow \{\text{at(Sy)}, \text{visited(Sy)}\}$
- $\mathcal{G} \subseteq \mathcal{F}$ is the goal state, e.g. *agent must start from Sydney, visit all cities, then go back to Sydney* $\rightsquigarrow \{\text{at(Sy)}\} \cup \{\text{visited}(x) \mid x \in \{\text{Sy}, \text{Ad}, \text{Br}, \text{Pe}, \text{Da}\}\}$

Def 22.3. STRIPS Task Solution: A solution to a STRIPS task is called a plan

Def 22.4. STRIPS as Search: A plan for a STRIPS task $\Pi := \langle \mathcal{F}, \mathcal{A}, \mathcal{I}, \mathcal{G} \rangle$ is a solution to an induced search problem $\Theta_\Pi := \langle \mathcal{S}_\Pi, \mathcal{A}_\Pi, \mathcal{T}_\Pi, \mathcal{I}_\Pi, \mathcal{G}_\Pi \rangle$, where:

- $\mathcal{S}_\Pi := \mathcal{P}(\mathcal{F})$ is the set of all possible states
- $\mathcal{A}_\Pi := \mathcal{A}$
- $\mathcal{I}_\Pi := \mathcal{I}$
- $\mathcal{G}_\Pi := \{s \in \mathcal{S}_\Pi \mid \mathcal{G} \subseteq s\}$ (Goal satisfaction via set inclusion)
- $\mathcal{T}_\Pi : \mathcal{A}_\Pi \times \mathcal{S}_\Pi \to \mathcal{P}(\mathcal{S}_\Pi)$ is the transition model defined as:

$$\mathcal{T}_\Pi(a, s) = \begin{cases} \{(s \setminus \text{del}_a) \cup \text{add}_a\} & \text{if } \text{pre}_a \subseteq s \\ \emptyset & \text{otherwise} \end{cases}$$

Def 22.5. Partially Ordered Plan (POP): Let $\Pi := \langle \mathcal{F}, \mathcal{A}, \mathcal{I}, \mathcal{G} \rangle$ be a STRIPS task, then a **partially ordered plan** $P := \langle V, E \rangle$ is a labeled DAG where the nodes in V are the steps (actions), where we have two special steps:

- **start**: This is the step that has no preconditions and its effects are simply $\mathcal{I}$
- **finish**: This is the step that has no effects and its preconditions are simply $\mathcal{G}$

An edge $e := \langle S, T \rangle \in E$ defines the relation between steps. We have two types of constrainst:

- **causal links**: Written as $S \xrightarrow{p} T$. It means step S is performed to achieve fact p because step T needs p to be true (precondition of T)
- **temporal constraints**: Written as $S \prec T$. It means S must happen before T

Additionally, we define an **open condition** to represent a *missing link*. If a step T has precondition p, but no causal link $S \xrightarrow{p} T$ exists yet, the plan is incomplete.

Def 22.6. Possibly Intervening: Let P be a partially ordered plan, then we call a step U **possibly intervening** in a causal link $S \xrightarrow{p} T$, iff $P \cup \{S \prec U, U \prec T\}$ is acyclic.

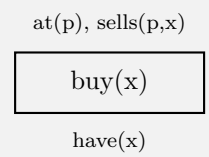
Def 22.7. Achieved Precondition: A precondition is **achieved** iff it is the effect of an earlier step and no possibly intervening step undoes it.

Def 22.8. Complete Plan: A partially ordered plan is called **complete** iff every precondition of every step is achieved.

Def 22.9. Partial Order Planning: The process of computing complete and acyclic partially ordered plans for a given planning task.

Note

In diagrams, we often write STRIPS actions into boxes with preconditions above and effects below.



Note

- A causal link $S \xrightarrow{p} T$ can be represented by a direct arrow between the effects (p) of S and the preconditions (p) of T
- Temporal constraints can be denoted via dashed arrows

Def 22.10. Clobbering: In a partially ordered plan, a step C **clobbers** a causal link $L := S \xrightarrow{p} T$, iff it destroys the condition p achieved by L. We sometimes refer to C as a **threat**.

Note

C is a threat if:
- C is possibly intervening in $S \xrightarrow{p} T$.
- $p \in \text{del}_C$ (the action C deletes the required fact).

Def 22.11. : If C clobbers $S \xrightarrow{p} T$ in a partially ordered plan P, then we can solve the conflict by either:

- **demotion**: add a temporal constraint $C \prec S$ to P, or
- **promotion**: add $T \prec C$ to P

Note

Note that we should always make sure that we have an acyclic graph. If our plan already has constraint $S \prec C$, then we cannot use demotion ($C \prec S$) because that would create $S \prec C \prec S$. Similarly, if the plan already has $C \prec T$, we cannot use promotion ($T \prec C$) because that would create $C \prec T \prec C$

Def 22.12. Totally Ordered Plan: A partially ordered plan is **totally ordered** iff for every pair of distinct steps $S, T \in V$, either $S \prec T$ or $T \prec S$ is in the set of constraints.

Def 22.13. Linearization: A **linearization** of a partially ordered plan P is a totally ordered plan that is consistent with all constraints in P.

Note

A common point of confusion is equating a complete plan with a totally ordered one. In Partial Order Planning, these are distinct concepts:

- **Completeness is about Logic:** A plan is complete when every "why" is answered (every precondition has a causal link) and every "conflict" is resolved (no threats).
- **Total Ordering is about Time:** A plan is totally ordered only when every action is forced into a single sequence (a straight line).

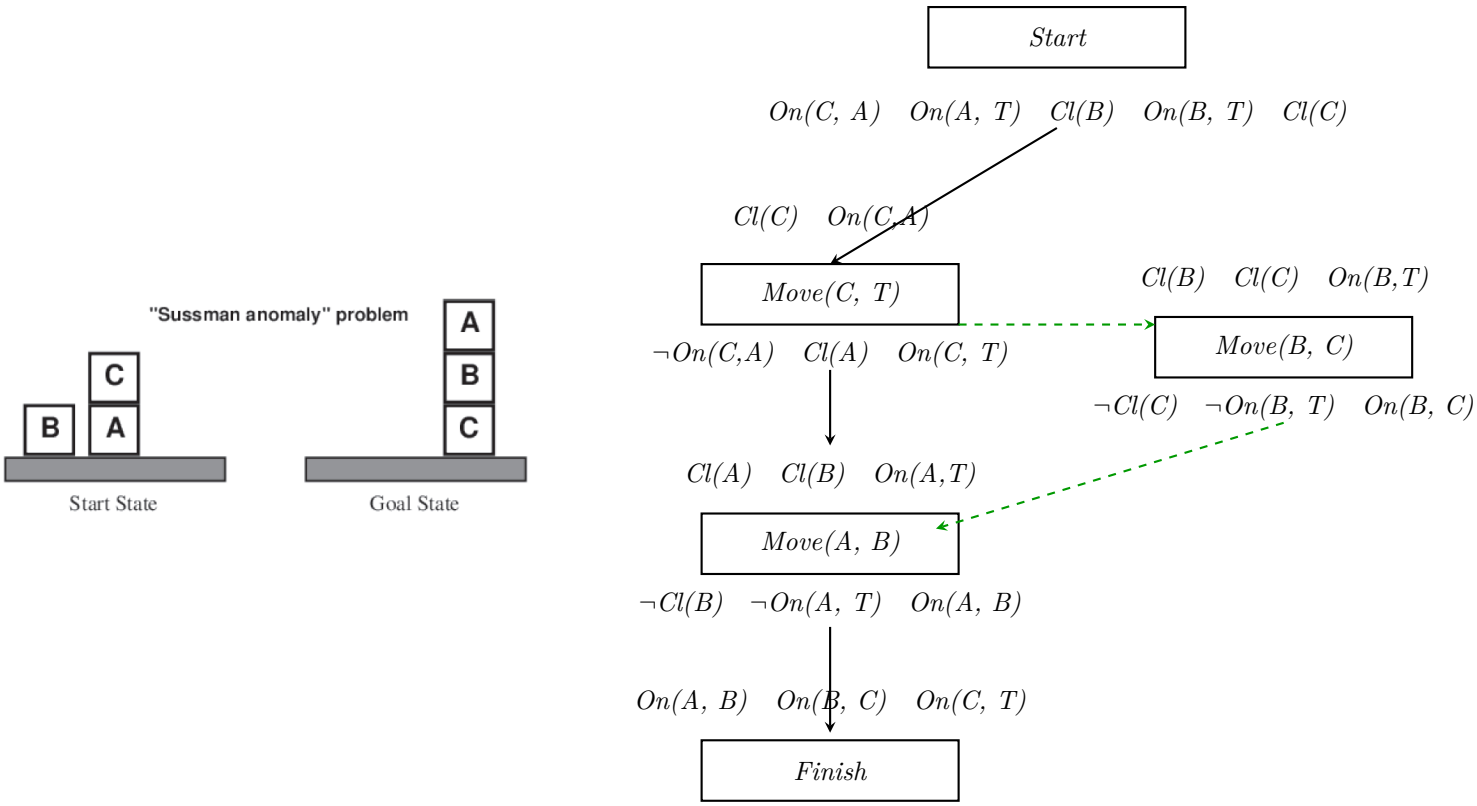
A complete plan **might not** be totally ordered. For example, if I need to "Buy Milk" and "Buy Bread" before I "Go Home," a complete plan will have causal links for both, but it won't care if I buy milk first or bread first. The actions remain "floating" relative to each other.

We only require a total ordering (a linearization) at the moment of execution, but the planning process is finished as soon as the plan is complete.

Example: Consider the following STRIPS task:

- We have three blocks $B := \{A, B, C\}$ and we have a table T.
- Fact: $\text{on}(X, Y)$ for $X \in B$ and $Y \in B \cup T$ to say that one block is on top of the other or is it at the table
- Fact: $\text{cl}(X)$ to say that nothing is on top of X
- Initial state: C is clear and on top of A which is on top of table. B is clear and on top of table
- Goal state: A is clear and on top of B which is on top of C which is on top of table
- Action: $\text{move}(X, Y)$ to move X to be on top of Y $(X, Y \in B)$
  - Pre: $\text{cl}(Y), \text{cl}(X), \text{on}(X, Z) \quad Z \in B \cup T$
  - Add: $\text{cl}(Z)(\text{if } Z \neq T), \text{on}(X, Y)$
  - Del: $\text{cl}(Y), \text{on}(X, Z)$
- Action: $\text{move}(X, T)$ to move X to be on top of T
  - Pre: $\text{cl}(X), \text{on}(X, Y) \quad X, Y \in B$
  - Add: $\text{cl}(Y), \text{on}(X, T)$
  - Del: $\text{on}(X, Y)$

Here is the totally ordered plan to solve the problem:



Def 22.14. Heuristic: Let $\Pi := \langle \mathcal{F}, \mathcal{A}, \mathcal{I}, \mathcal{G} \rangle$ be a STRIPS task and $\Theta_\Pi := \langle \mathcal{S}_\Pi, \mathcal{A}_\Pi, \mathcal{T}_\Pi, \mathcal{I}_\Pi, \mathcal{G}_\Pi \rangle$ is the induced search problem, A **heuristic function** or **heuristic** for Π is a function $h : \mathcal{S}_\Pi \to \mathbb{N} \cup \{\infty\}$ so that $h(s) = 0$ whenever $s \in \mathcal{G}_\Pi$

> **Note**
>
> We use $\mathbb{N}$ (unlike Def 14.24 where we used $\mathbb{R}_0^+$) because actions are discrete and we assume unit cost of 1 (we are just counting steps now)

Def 22.15. Perfect Heuristic: Let $\Pi := \langle \mathcal{F}, \mathcal{A}, \mathcal{I}, \mathcal{G} \rangle$ be a STRIPS task and $\Theta_\Pi := \langle \mathcal{S}_\Pi, \mathcal{A}_\Pi, \mathcal{T}_\Pi, \mathcal{I}_\Pi, \mathcal{G}_\Pi \rangle$ is the induced search problem, the **perfect heuristic** h^* assigns every $s \in \mathcal{S}_\Pi$ the length of the shortest path from s to a goal state $g \in \mathcal{G}_\Pi$, or ∞ if no such path exists.

> **Note**
>
> Different from Def 14.25 (cost vs length)

Def 22.16. Admissible: A heuristic h for a STRIPS task Π is **admissible** if $h(s) \leq h^*(s) \quad \forall s \in \mathcal{S}_\Pi$

Def 22.17. Relaxed Planning Task: Let Π be a STRIPS task. A **relaxed planning task** Π' is a modification of Π such that the set of possible plans for Π is a subset of the set of possible plans for Π'.

Def 22.18. Relaxed Heuristic: A **relaxed heuristic** h' for a STRIPS task Π is the perfect heuristic of an associated relaxed planning task Π'. That is:

$$h(s) = h^*_{\Pi'}(s)$$

> **Note**
>
> By definition, any solution to the original task Π remains a valid solution in the relaxed task Π'. Because Π' is "easier" (less constrained), its perfect heuristic value $h^*_{\Pi'}$ provides a lower bound on the true length h^*, ensuring that relaxed heuristics are always admissible.

Def 22.19. Only-Adds Relaxation: We drop pre_a and del_a for all $a \in \mathcal{A}$ resulting in a relaxed planning task where an action is just $a := \langle \text{add}_a \rangle$

Note

Example: Consider a goal $G := \{p, q\}$ and an action a that adds p, q but requires r. In the original task, I must first find a way to achieve r. In the only-adds relaxation, I can apply a immediately in the initial state to reach the goal in a single step, because the requirement r has been dropped.

ATTENTION! Search uses the real (un-relaxed) Π. The relaxation is applied **only** within the call to $h(s)$

Def 22.20. Delete Relaxation: Let $\Pi := \langle \mathcal{F}, \mathcal{A}, \mathcal{I}, \mathcal{G} \rangle$ be a STRIPS task, the **delete relaxation** of Π is the task $\Pi^+ := \langle \mathcal{F}, \mathcal{A}^+, \mathcal{I}, \mathcal{G} \rangle$ where $\mathcal{A}^+ := \{a^+ \mid a \in \mathcal{A}\}$ with $\text{pre}_{a^+} := \text{pre}_a$, and $\text{add}_{a^+} := \text{add}_a$, and $\text{del}_{a^+} := \emptyset$

Def 22.21. Relaxed Plan: Let $\Pi := \langle \mathcal{F}, \mathcal{A}, \mathcal{I}, \mathcal{G} \rangle$ be a STRIPS task and $\Theta_\Pi := \langle \mathcal{S}_\Pi, \mathcal{A}_\Pi, \mathcal{T}_\Pi, \mathcal{I}_\Pi, \mathcal{G}_\Pi \rangle$ is the induced search problem. A **relaxed plan** for a state $s \in \mathcal{S}_\Pi$ is a plan for $\langle \mathcal{F}, \mathcal{A}, s, \mathcal{G} \rangle^+$. A **relaxed plan** for $\mathcal{I}$ is called a **relaxed plan** for Π

Note

Relaxation is often used to mean delete relaxation by default

Def 22.22. Relaxed Plan Existence Problem: By PlanEx$^+$, we denote the problem of deciding, given a STRIPS task $\Pi := \langle \mathcal{F}, \mathcal{A}, \mathcal{I}, \mathcal{G} \rangle$, whether or not there exists a relaxed plan for Π.

Note

This is easier than PlanEx for general STRIPS. Algorithm 5 decides PlanEx$^+$. The algorithm terminates after at most $|\mathcal{F}|$ iterations, and thus runs in polynomial time (PlanEx$^+$ is in P).

Algorithm 5 Deciding PlanEx$^+$ (Relaxed Plan Existence)

```
var F := I
while G ⊈ F do
    F' := F ∪ ⋃_{a∈A:pre_a⊆F} add_a
    if F' = F then
        return "unsolvable"
    end if
    F := F'
end while
return "solvable"
```

Def 22.23. Optimal Relaxed Plan: Let $\Pi := \langle \mathcal{F}, \mathcal{A}, \mathcal{I}, \mathcal{G} \rangle$ be a STRIPS task and $\Theta_\Pi := \langle \mathcal{S}_\Pi, \mathcal{A}_\Pi, \mathcal{T}_\Pi, \mathcal{I}_\Pi, \mathcal{G}_\Pi \rangle$ is the induced search problem. An **optimal relaxed plan** for a state $s \in \mathcal{S}_\Pi$ is an optimal plan for $\langle \mathcal{F}, \mathcal{A}, \{s\}, \mathcal{G} \rangle^+$.

Def 22.24. h^+**:** Let $\Pi := \langle \mathcal{F}, \mathcal{A}, \mathcal{I}, \mathcal{G} \rangle$ be a STRIPS task and $\Theta_\Pi := \langle \mathcal{S}_\Pi, \mathcal{A}_\Pi, \mathcal{T}_\Pi, \mathcal{I}_\Pi, \mathcal{G}_\Pi \rangle$ is the induced search problem. The **ideal delete relaxation heuristic** h^+ for Π is a function $h^+ : \mathcal{S}_\Pi \to \mathbb{N} \cup \{\infty\}$ where for any $s \in \mathcal{S}_\Pi$, $h^+(s)$ is the length of an optimal relaxed plan for s if a relaxed plan for s exists. $h^+(s) = \infty$ otherwise.

Lemma 22.1. Let $\Pi := \langle \mathcal{F}, \mathcal{A}, \mathcal{I}, \mathcal{G} \rangle$ be a STRIPS task and $\Theta_\Pi := \langle \mathcal{S}_\Pi, \mathcal{A}_\Pi, \mathcal{T}_\Pi, \mathcal{I}_\Pi, \mathcal{G}_\Pi \rangle$ is the induced search problem. For a state $s \in \mathcal{S}_\Pi$, if $\langle a_1, \cdots, a_n \rangle$ is a plan for $\Pi_s := \langle \mathcal{F}, \mathcal{A}, \{s\}, \mathcal{G} \rangle$, then $\langle a_1^+, \cdots, a_n^+ \rangle$ is a plan for Π^+

Theorem 22.2. h^+ is admissible.

Example

Consider the following STRIPS task $\Pi := \langle \mathcal{F}, \mathcal{A}, \mathcal{I}, \mathcal{G} \rangle$:

- $\mathcal{F} := \{\text{truck}(x) \mid x \in \{A, B, C, D\}\} \cup \{\text{pack}(x) \mid x \in \{A, B, C, D, T\}\}$

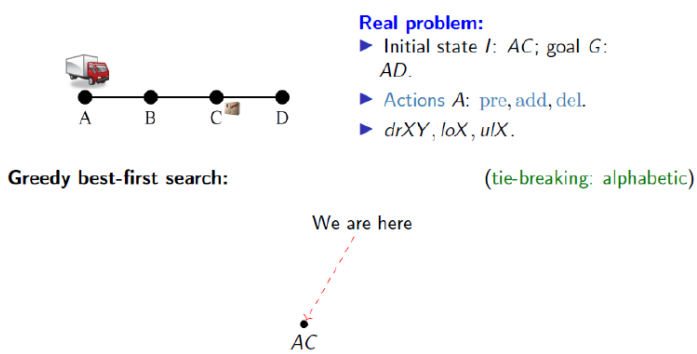
  i.e. we are representing where is the truck via $\text{truck}(x)$ and where is the pack via $\text{pack}(x)$ where $\text{pack}(T)$ means the packet is in the truck
- $\mathcal{I} := \{\text{truck}(A), \text{pack}(C)\}$ initially, the truck is at A and the packet is at C
- $\mathcal{G} := \{\text{truck}(A), \text{pack}(D)\}$ we want to move the packet to D and go back to A
- $\mathcal{A} \rightsquigarrow$ we write action $a := \langle \text{pre}_a, \text{add}_a, \text{del}_a \rangle$ as $a := \text{pre}_a \Rightarrow \text{add}_a, \neg\text{del}_a$
  - $\text{drive}(x, y) := \text{truck}(x) \Rightarrow \text{truck}(y), \neg\text{truck}(x)$
  - $\text{load}(x) := \text{truck}(x), \text{pack}(x) \Rightarrow \text{pack}(T), \neg\text{pack}(x)$
  - $\text{unload}(x) := \text{truck}(x), \text{pack}(T) \Rightarrow \text{pack}(x), \neg\text{pack}(T)$
- Relaxed Actions:
  - $\text{drive}(x, y) := \text{truck}(x) \Rightarrow \text{truck}(y)$
  - $\text{load}(x) := \text{truck}(x), \text{pack}(x) \Rightarrow \text{pack}(T)$
  - $\text{unload}(x) := \text{truck}(x), \text{pack}(T) \Rightarrow \text{pack}(x)$

For convenience:

- we represent a state $\text{truck}(X), \text{pack}(Y)$ by writing XY
- actions are shortened to $drXY, loX, ulX$

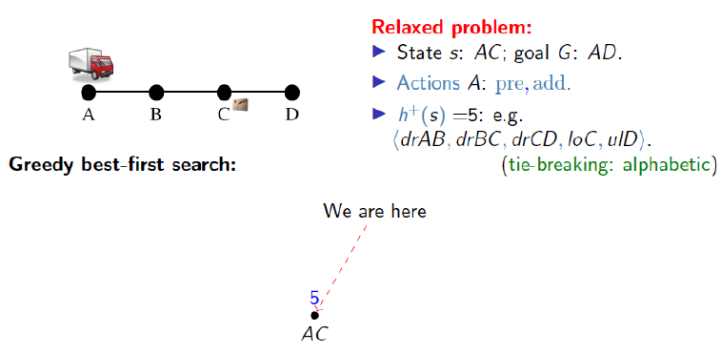
Let's see how to relax during search:

Initially, we are in the real problem in the initial state AC and the goal is AD.

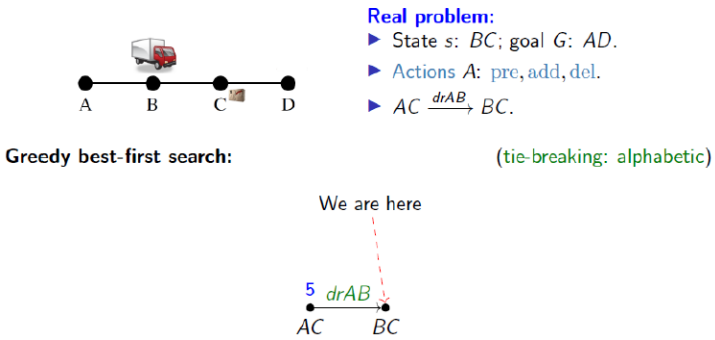


We go the relaxed world to calculate h^+ of the current state. We see that we would reach the goal if we apply

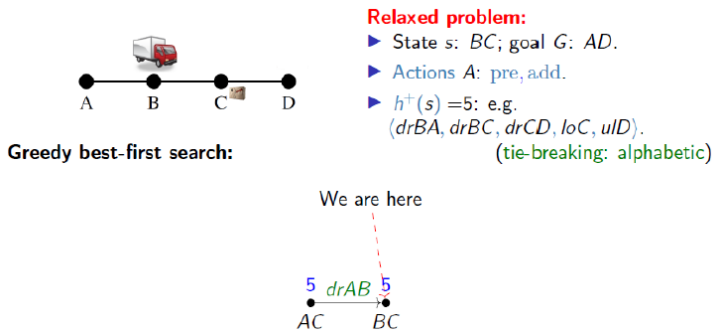
$\langle drAB, drBC, loC, drCD, ulD \rangle$ (we do not need to go back to A, since no deletions, we are still there). So $h^+(AC) = |\langle drAB, drBC, loC, drCD, ulD \rangle| = 5$



We go back to the real problem, we know the only action we can take is $drAB$ (the only possible successor state is BC), so we go there:

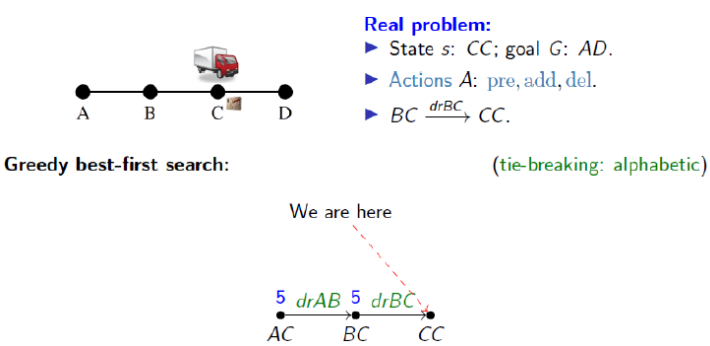


We go the relaxed world to calculate $h^+(BC)$. We see that we would reach the goal if we apply $\langle drBC, loC, drCD, ulD, drBA\rangle$, so $h^+(BC) = 5$

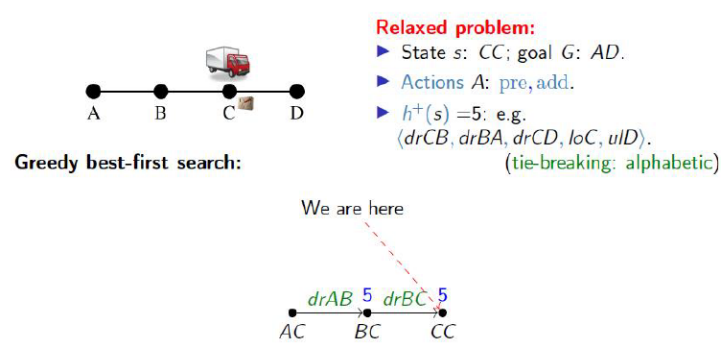


we go back to the real world, we know that two actions are applicable $drBA, drBC$ (two possible successor states: AC, CC).

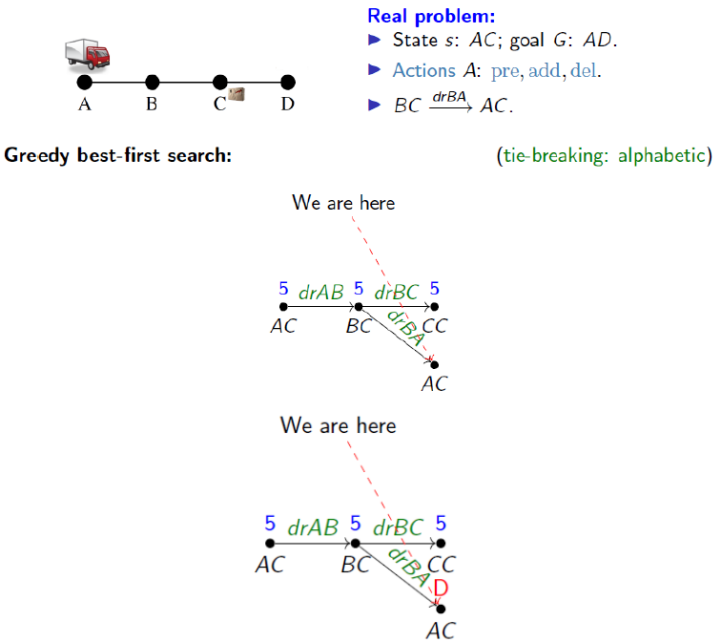
- we go first to CC:



we go to the relaxed world to calculate $h^+(CC)$:

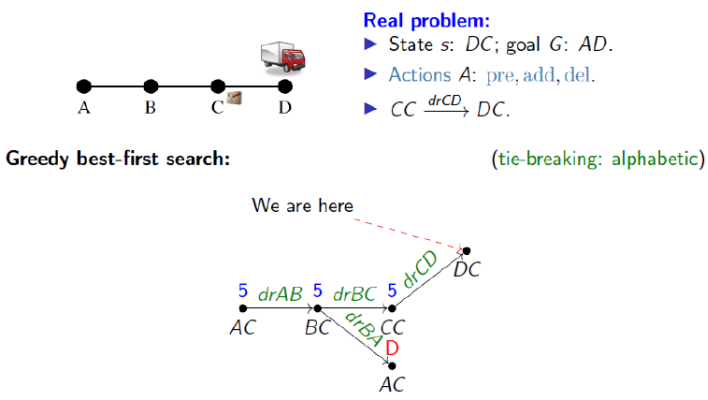


- next we go to AC, we figure that it has been explored, hence we mark it as duplicate:

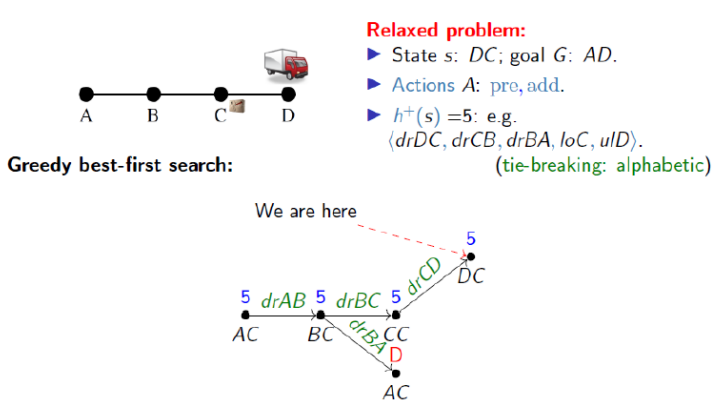


Back to real, we are at CC, we have three possible successors, we can move to D (DC), we can unload (CT), we can move to B (BC)

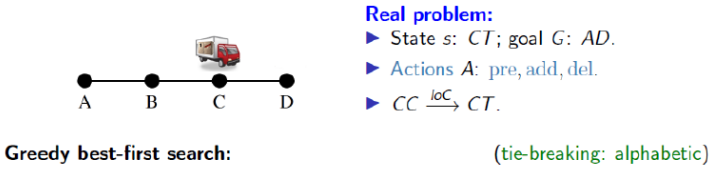
- Let's see DC:

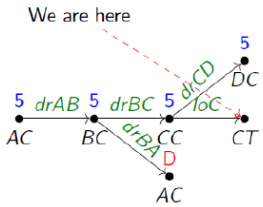


We relax to calculate:

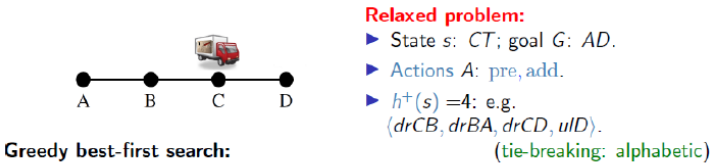


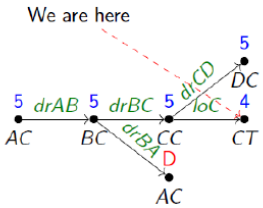
- Let's see CT:





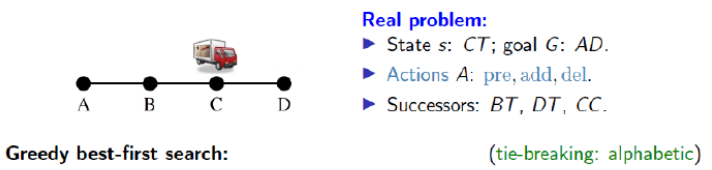
We relax to calculate: notice that $h^+(CT) = 4$ because we can do $drCD, ulD, drCB, drBA$

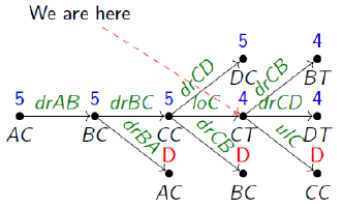




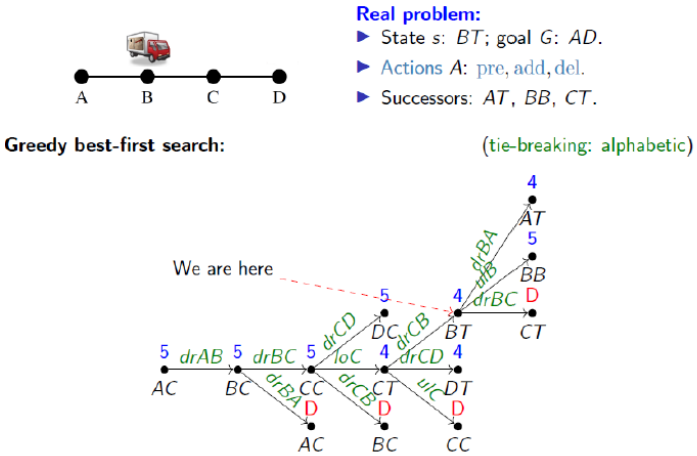
- Let's see BC: we mark as duplicate

Next we can expand CT with h^+ of 4 or DC with h^+ of 5. We choose CT. From there, we have three successors: we can unload (CC, duplicate), we can move to D (DT), we can move to B (BT), we relax to calculate h^+ of each:

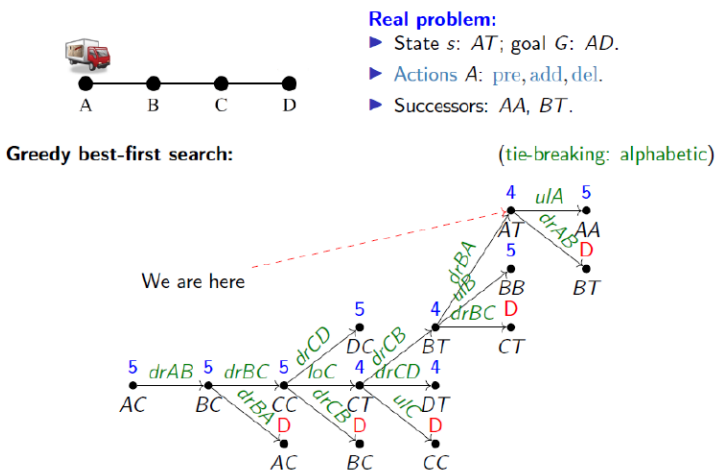




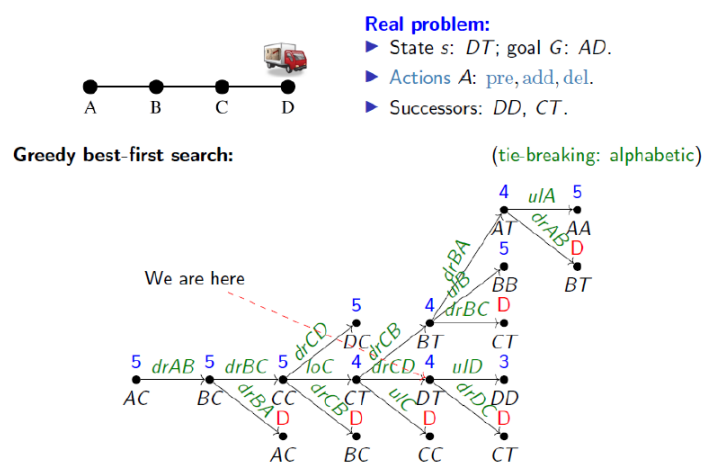
We expand next BT (break ties alphabetically), from there we can unload (BB), we can move to A (AT), we can move to C (CT, duplicate). We calculate h^+ of each:



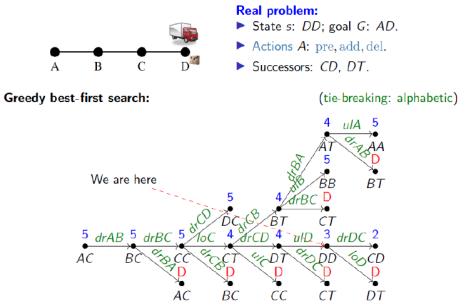
We expand AT first, we have two successors, we can move to B (BT, duplicate), or we can unload (AA):

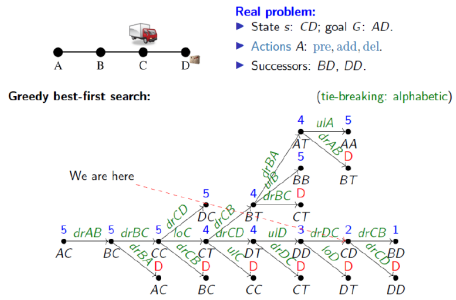


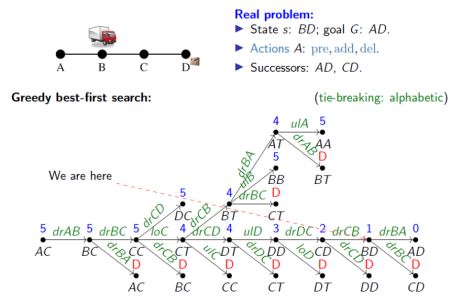
We can expand next AA or BB with h^+ of 5 or expand DT with h^+ of 4 (remember when we expanded CT we had two successors to investigate, BT and DT and we broke ties alphabetically). We choose to expand DT:



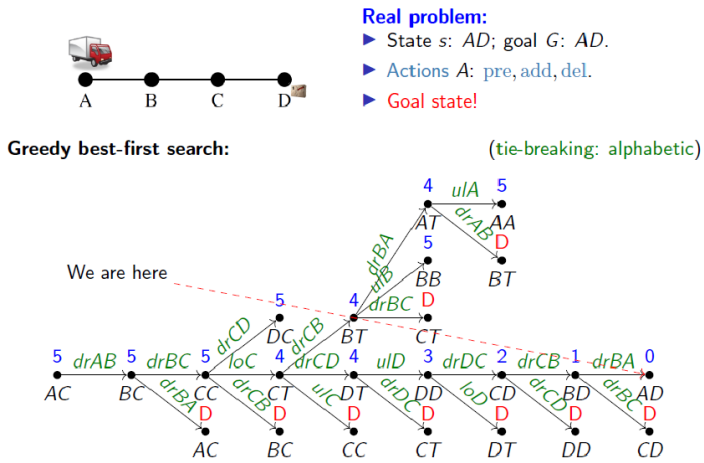
We expand next DD, CD, and BD respectively:







We go to AD only to figure out that this is a goal state! (Truck at A, pack at D).



To sum up the steps that we found:

$$\text{drive}(A,B) \rightsquigarrow \text{drive}(B,C) \rightsquigarrow \text{load}(C) \rightsquigarrow \text{drive}(C,D) \rightsquigarrow \text{unload}(D) \rightsquigarrow \text{drive}(D,C) \rightsquigarrow \text{drive}(C,B) \rightsquigarrow \text{drive}(B,A)$$